OCR PROGRAMMING COURSEWORK

ROBERT STARCK

MAY 2025

Table of Contents

Analysis 7
Introduction 7
Problem Identification 8
Computational Methods 10
Summary table 12
Stakeholders 13
Summary table 14
Research Existing Software 15
Summary table 23
Features of Proposed Solution 24
Limitations of the Proposed Solution 25
Requirements 26
Hardware Requirements 26
Software Requirements 26
File Requirements 26
Success Criteria 28
Design 32
Introduction 32
Problem Decomposition 33
Required Files and Entity Relationship Diagrams (ERDs) 34
Initial data files with justification 34
Entity Relationship Diagram 35
Linear Search 36
Top-Down-Design - Linear Search 36
Pseudocode – Linear Search 37
Binary Search 38
Top-Down Design – Binary Search 38
Pseudocode – Binary Search 39
Searching 40
Top-Down Design Searching 40
Pseudocode – Searching 41
Loading 42
Top Down Design – Loading 42
Pseudocode – Loading 44
Create Files 47
Top-Down Design - Create Files 47

Pseudocode – Create Files 52
Class Diagram (Unified Modelling Language (UML)) 73
Files Super Class 74
Top-Down Design – Files Class 74
Pseudocode – Files Class 75
Child Class 77
Explanation of design generalization 77
Top-Down Design – Child Class 78
Pseudocode – Child Class 80
Write Design 84
Top-Down Design – Write Design 84
Pseudocode – Write Design 85
Cursor Grid 86
Top-Down Design – Cursor Grid 86
Pseudocode – Cursor Grid 88
Create Menu 90
Top-Down Design – Create Menu 90
Pseudocode – Create Menu 91
Log In Menu 92
Top-Down Design – Log in Menu 92
Pseudocode – Log in Menu 93
Student's Menu 95
Top-Down Design – Students Menu 95
Pseudocode – Students Menu 97
Teacher's Menu 102
Top-Down Design – Teachers Menu 102
Pseudocode – Teachers Menu 103
Main 104
Top-Down Design – Main 104
Pseudocode – Main 105
Usability Features 107
Data Dictionary 108
Test Data Plan 127
Test 1 127
Test 2 128
Test 3 129
Post Development Data 130

Iterations 131
Evidence Explanations 131
Iteration 0 – Library Declarations 133
Aim 133
Iteration 0.1 – Linear Search 134
Code 134
Testing 135
Iteration 0.2 – Binary Search 141
Code 141
Testing 143
Iteration 0.3 – Search Script 148
Code 148
Testing 149
Iteration 0.4 – Loading 151
Code 151
Testing 155
Iteration 0.5 – Log In 160
Code 160
Testing 163
Iteration 0.6 – Write Design 164
Code 164
Testing 165
Iteration 0.7 – Cursor Grid 168
Code 168
Testing 171
Iteration 0.8 – Create Menu 175
Code 175
Testing 176
Review 180
Iteration 1 – Creating Initial Data 181
Aim 181
Code 182
Testing 205
Review 218
Iteration 2 – Class Structure 219
Aim 219
Code 220

Super Class 220
Child Class 224
Testing 229
Review 239
Iteration 4 – Student's Menu 240
Aim 240
Code 241
Testing 247
Review 253
Iteration 5 – Teacher's Menu 254
Aim 254
Code 255
Testing 256
Review 257
Iteration 6 – Main Program 258
Aim 258
Code 259
Testing 261
Review 263
Evaluation 264
Testing to Inform Evaluation 264
Success of the Solution 268
Evaluation of solution against success criteria for client 268
Evaluation of solution against success criteria for programmer 275
Evidence of usability features 277
Maintenance and Future Development 278
Future Development of Usability features 278
Maintenance issues and limitations 279
Appendix A – Final Code 280
`unit attendancProcedures;` 280
`UNIT BinarySearch;` 286
`UNIT BubbleSort;` 288
`unit classProcedures;` 289
`PROGRAM CreateFiles;` 294
`UNIT CursorGrid;` 318
`unit FileProcedures;` 321
`UNIT InsertionSort;` 325

UNIT LinearSearch; 326
unit loading; 327
unit LogInProcedures; 331
unit LogInScreen; 336
PROGRAM Main; 339
unit registerProcedures; 342
UNIT Search; 348
unit Sorting; 349
unit studentClassProcedures; 350
unit studentProcedures; 355
UNIT StudentsMenu; 361
unit teacherProcedures; 367
UNIT TeachersMenu; 372
unit timeTableProcedures; 373
UNIT waitTime; 378
UNIT writeDesign; 380
Extra Projects 381
Project 1 – Neural Network training using backpropagation 382
Project 2 – Genetic algorithm 387
Project 3 – Reinforcement Learning using Q learning 395

Analysis

Introduction

My Client, Mrs Elliot, is a computer science teacher at Shenley Brook End School, https://www.sbeschool.org.uk. She is also a form tutor for the sixth form at Shenley. She is both my computer science teacher and form tutor. She is currently getting frustrated with the current computer system used to take and manage registers and attendance.

One of the many tasks that she must do repeatedly, as a form tutor and teacher, is to look at the attendance for every student in her form and compare it with the standards set by the school. If it is subpar, below 95%, she needs to have a word with that student. However, she has the issue of not being able to check which lessons were missed for that student and if it was an error on the teacher's behalf, and she does not like how the student also can't see which lessons they got marked as absent for. She also would like to be able to sort by attendance to make it easier to see who she needs to have a word with as this is not currently and option. She also does not like how she cannot amend her registers past a specific point in time if they do need to be amended, and sometimes she also forgets to do the register so having a reminder pop up would make her life much easier as well. So, she has asked me to design a new register system to replace the current system, Arbor (https://login.arbor.sc).

Problem Identification

One of the tasks my client must do is review the attendance of all the students in her form. To do this she must first log in to the existing system and into the main menu. Next, she needs to search for the student's name in the search bar at the top. Then she needs to go to the attendance menu on the left and then into the submenu attendance menu, which will give a brief overview of the attendance of that student. Alternatively, she can go into the "marks by date" to get a timetable for the entire year for that one student with all their marks for each lesson on each day displayed. These are marked by a tick for present or characters for absences. She often does not remember which each character stands for and must hover each one every time to remind her of what they all mean. Another option she has is to download an attendance certificate which is generated by the current system to display a brief overview of the attendance at the top and more in-depth information throughout the rest of the document. She likes the brief overview of the statistics in the attendance menu and does not want that to change. However, she does not like the format of the marks by date view. One of the issues is that she cannot choose the date range; she would like to have this option. She also does not like the format in which the data of the attendance is currently displayed. Firstly, she does not like the ticks and would prefer a forward slash "/". She also does not like the letters to display other marks as she can never remember them and has asked for a key on the side and a simpler set of characters to be used so she does not have to keep hovering over them to find out what they mean. This feature will help her out by shortening the amount of time she has to spend combing through all the data and complicated symbols whilst trying to extract the required data that she needs. She also likes the attendance certificate as it allows her to print it off and it gives her all the information in one place. She has also requested that there is a new menu in which she can see all the attendance for all students in her form. She would also prefer it if it was ordered lowest attendance to highest attendance and colour coded with red being below 92%, orange from 92% to 97% and green from 97% to 100%. This feature will save her a lot of time as she does not have to individually search up each person in her form and then navigate through all the menus to find the attendance of that one student.

Currently, in order to take the attendance register, my client must go to her timetable, click on the current lesson, without validation to check if it is the right lesson, and then click on "Take register" to start taking the register.

For getting to the register, she would like the system to know automatically what lesson is about to take place and open the register for that class as she sometimes does not remember what lesson it is and sometimes opens the incorrect register eg, wrong date. She has also requested that the system will automatically fill double lesson registers with one register instead of having to do it twice as she forgets to redo the register while in the middle of teaching. Both features provide quality of life features for my client by making the system automate more and reduce more opportunities for human error which she is struggling with now.

Once she starts taking the register, she does have the option to mark all students as present or absent and then change individual students' records as required. When marking a student as absent it brings up another window to confirm, but this does not happen when marking a student as present. My client would like the new system to hold some of the features in their entirety, build on some others and add some more. As my client likes the "mark all as absent or present" feature, so this will be retained. However, she would like a shorthand to mark a student as "absent with a reason", in which case it asks her for a reason, or mark "absent without reason" and no dialogue box to pop up asking for a reason for absence. She would like to retain the current system of marking students as present. Theses changes

would provide an easier interface to use and make it faster for her to input all the data, thereby making it a better solution than the existing one.

The current system allows students to login and on their homepage which displays their overall attendance including authorised and unauthorised absences. However, with the current system there is no way to see where they have had those absences ie which lessons they missed. My client would like the new system to allow students to enter an area where they can see all previous lessons in a timetable form, but colour coded to display the attendance mark that they got for each lesson. My client would also prefer if there was also an option to jump to the lessons missed so that, if it was a while ago, the students don't have to spend long periods of time looking for that lesson. This will help speed up the process for reviewing attendance, as it will be faster if the student can have review their own timetables beforehand and note any discrepancies.

In summary my client has identified the following issues:

- Issues in entering the wrong register, wrong class, wrong day etc...
- Inefficient methods to take the register, unnecessary dialogue boxes
- Inefficient methods to view students' attendance
- An inability for students to look at their attendance in detail

By streamlining all these aspects, my client will be able to ensure that each register taken is accurate and correct for the required lesson and with minimal time spent undertaking this task. This will therefore allow more time for teaching and improved data quality for auditing and benchmarking purposes for the school.

Computational Methods

Sequencing is the computational method to describe the order of execution of tasks in a specific order to complete a task. A repetition of sequence is called iteration. In my coursework my client must, while taking the register, go through each student in turn alphabetically and mark whether they are present or not, this is sequence. And because this is repeated for all students until all students are registered, this is also an example of iteration. This is amenable to computer use due to the computational methods at use. Another example where sequence is used is when my client wants to review the attendance of each student. To view the attendance the registers must first be taken and then saved to the file, this is another example of sequence. But because she must review the attendance for all students this process must be repeated until all students in her form have had their attendance reviewed, it is also iteration. This task is also amenable to computer use due to the computational methods used to complete this task. My client also when choosing menus must go through them in a specific order to get to the section that she needs to be in to complete a specific task. This is an example of sequence and because of this, this task is amenable to computer use.

Another computational method is selection which is the process of a computer choosing a specific task based on a specific condition. In my coursework my client must choose what register mark to give each student, and different register marks lead to different outcomes, different pieces of code run. This is an example of selection. This is amenable to computer use due to the use of selection. Another example is when going through the menus, my client must make a choice on what section she wants to go into. And depending on the choice she makes different operations run, this is also an example of selection. This is also amenable to computer use due to the use of computational methods. Again, in my course work my client must also make choice on who she needs to have words with about their attendance, and various levels of attendance means different outcomes which is, again, selection. This is amenable to computer use as well due to the use of the computational method, selection. Another example of selection is the login, currently the current system checks the data that is input and then compares it against the stored data, and if it matches it lets them into that account, and then depending on the account privileges certain features will be locked or unlocked. This is another example of sequencing and that makes it amenable to computer use.

Decomposition is the breaking down of a complex task into smaller simpler tasks to make completing the main task easier. In my coursework my client breaks down one task into smaller sub tasks, one such example of this is when taking the register, she must look at each individual name then check if they are there and give them a mark. This is an example of decomposition and makes this task amenable to computer use. Another task she also must do is check students' attendance, this is currently done by searching a student's name, selecting them from the list of people that show up, then going into their attendance. This is another example of decomposition and makes this task also amenable to computer use.

Abstraction is the computational method which describes the action of leaving out certain details deliberately to make programming the program easier. Currently, in my coursework, my client ignores certain bits of data, for example when taking the register, she doesn't need to see their overall attendance, or what register marks they got previously as there is no need and it will be more useless information cluttering the screen. This is an example of abstractions and makes this task amenable to computer use. Another task where she uses abstraction is when looking at the list of people in her form whose attendance she must check and talk to people about. When doing this she ignores the

picture of them and most other information other than name and their attendance. This is again an example of abstraction and makes this task as well amenable to computer use.

Data storage describes how and where the data that the program needs is stored, for example: cloud or local storage. In my coursework my client inputs the data where it is then stored on the cloud so that other members of the school can access it, this is an example of data storage. This is amenable to computer use due to the computational methods at play here. My client must also be able to store lots of registers in one place which is easy to access from any device in the school. This is another example of data storage. This task is also amenable to computer use due to the computational method. Currently the files needed are, information about students, classes, registers, timetables, and teachers. These files are all linked together to help avoid redundancy and reduce the amount of space wasted. This is another example of data storage and is thus amenable to computer use.

Summary table

Computational Method	Definition	Tasks amenable to this computational method
Sequencing	Sequencing is the order in which tasks are completed to ensure the task is done correctly	• GO through each student in turn when taking the register. • To check the attendance of a student a register must be taken first for that attendance to be correct
Iteration	Iteration is the looping of a sequence to get it done multiple times	• When taking the register my client must fill in a register mark for each student • She must go through and find each student in her form's attendance
Selection	Selection is the choosing of a task based on whether a condition is met or not	• If the user that logs in has teacher privileges the system will give them access to more functions • Choosing a register mark when doing the register
Decomposition	Decomposition is the breaking down of a large task into smaller sub tasks	• Taking the register • Checking students' attendance
Abstraction	Is the removal of unnecessary data	• Removing certain bits of information from the students when taking the register • Checking students' attendance
Data Storage	Data storage is how and where the data is stored	• Stores data on the cloud currently • All files are linked together to reduce redundancy

Stakeholders

My primary stakeholder during development will be my client, Mrs Elliot. Currently her task is to teach computer science, taking the register at the start of each lesson. This is done by selecting the current class on her timetable and then going through each student and marking down whether they are there or not. She also must manage her forms attendance, which is done by individually searching up each student in her form and checking her attendance manually and determining if it is subpar (below 95%). The proposed solution will benefit her by allowing her to change the marks of students in her form if she feels it is incorrect. This will save her time as she doesn't have to email the sixth form team and wait for it to be changed, she also forgets to do this sometimes which causes issues with the data not being correct which means the school is not fulfilling its legal obligation to keep accurate records of students' attendance. The proposed solution will also benefit her as while taking the lesson she can use the shorthand that will be implemented to make taking the register easier these include short hands to mark everyone as present or absent with no reason, drastically reducing the time taken to take the register and increase teaching time. The system before will output the register to the client so she can check before it is saved to the system. While checking the attendance of her form the proposed solution will provide an option to view all students in her form with their attendance next to each person, and they will be ordered from lowest to highest attendance and will be colour coded to make it easier to see. This cuts down on the time taken to do this be lowering the number of inputs to one which is just a simple select at the start to view the attendance. The proposed solution is appropriate to her needs as it cuts down on the time taken to do the register at the start of each lesson increasing the amount of teaching time and reducing the time to search though all the students on her form to see if she needs to talk to anyone with low attendance.

Another stakeholder after development will be students. The students will be a secondary stakeholder. Students will also be directly interacting with the system to check out their own attendance and which lessons were missed. Currently their task is to check their own attendance and email the sixth form if they believe there are any issues. This is currently done by them having to guess when they missed the lesson or ask their form tutor to check for them which takes time out of the other things they need to do. The proposed solution will benefit them by providing a way for them to check their own attendance and know what lessons they missed. The students will select to go into the attendance view where they will get their timetable headered with each week start and end date and then each lesson is colour coded to match the register mark that they got for that register. They can also change the view to just displayed when they have been marked absent, on what day and on what lesson. This helps my client by making the student more able to review their attendance on their own first before she must deal with it, saving time in her day to do other more important tasks.

Pastoral leaders would be a tertiary client as they would use the output of the system. Currently they take the information about the attendance of the students and must forward it on to the government and do other data manipulation with it. The current system has no straightforward way to change the data if it does need to be change and the extremely limited validation can only be done by a few people so there is low chance that the data will be validated in time. The proposed solution will benefit them as it will be providing them more accurate data as if the students can validate their own attendance and ask to get it amended sooner it will help improve the validity of the data that is received by the school management team.

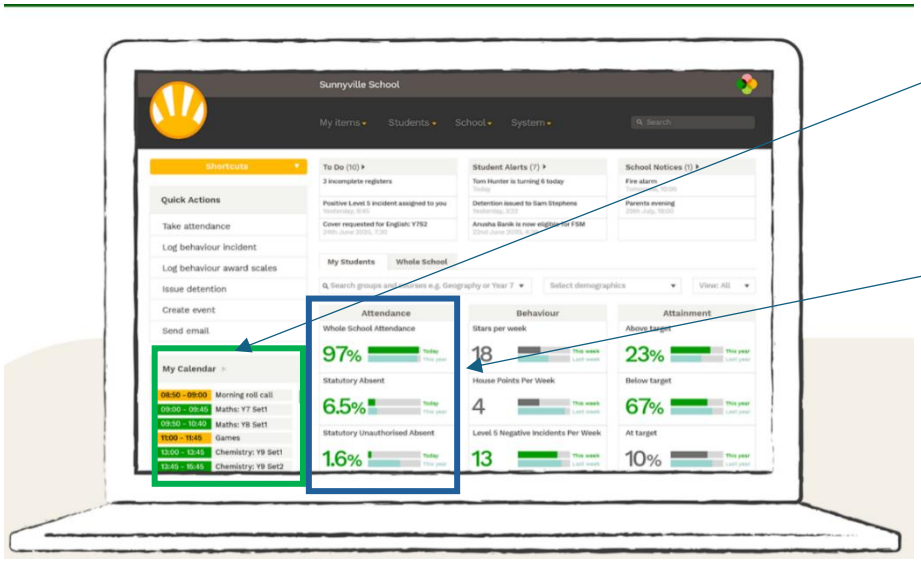
Summary table

Stakeholder	Description	How will they use the solution	Why is it appropriate
My client (primary)	• Taking the register • Managing her form's attendance	• They will use the new functions to cut down on the amount of time they spend filling in the empty dialogue boxes. • They will use the new view to quickly see her whole forms' attendance in one place.	• It is appropriate as it reduces the time spent to take the register. • It cuts down on the time taken to check her forms attendance
Students (primary)	• Email attendance team to get attendance marks fixed	• They will use the solution to more easily see where they have missed attendance marks so they can include that in their email to the team	• It is appropriate to their needs as it cuts down on the time, they need to get their attendance fixed because they no longer must guess when they were absent
Pastoral leaders (tertiary)	• Review all student's attendance and format it to send it off to the government	• They will not directly use the system, but they will use the output of the system to get more accurate data to send off to the government.	• It is appropriate to their needs as it saves them time on trying to fix students attendance. • Get more accurate data to send off

Research Existing Software

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

One system I am choosing to research is the current system Arbor https://arbor-education.com.



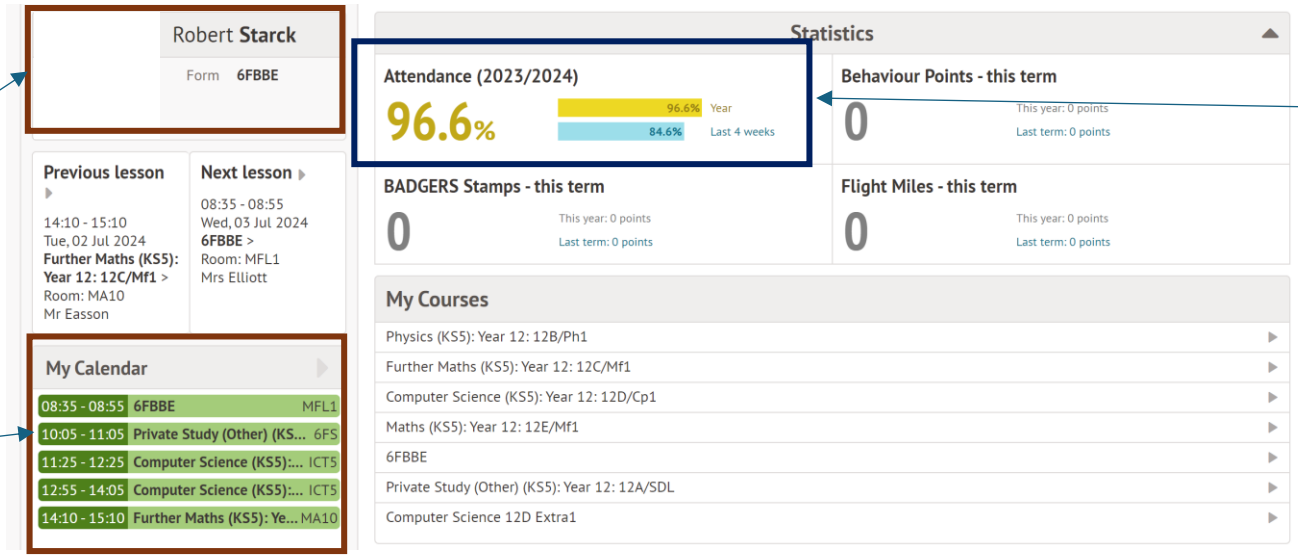
Preview of the calendar for that day

Overview of the attendance for the whole school

Arbor is used among many schools for their attendance and register system, it has features to automatically calculate a student's attendance, their behaviour and attainment. It also has functions to show to do lists and view their timetable to name a couple of functions. They have included data about each student's attendance, there behaviour and what grades they get in what subjects. They also store the data about the student including name, date of birth etc. Another piece of information that they store is the student's timetable.

Student profile overview, name, form and picture

Students timetable displaying lessons for that day, and the option to view entire timetable



Overview of attendance for the student and is all the student can see on their attendance

The data that is output by Arbor is overall attendance for the year and attendance for the last 4 weeks. The number of badger stamps received in the year and the last term, same with behaviour points and Flight miles. It also outputs the course they do and their calendar for that day. It also states their last lesson and my next lesson above it. Aswell as displaying their Name, form, and their school picture in the top left which I have blanked out.

Arbor is however missing on the student's side, for them to get more information about their attendance, for example where they missed lessons. It also does state where those behaviour points where from and expects the student to just know. It is also missing load of shorthand's for teachers that would make their life much easier, such as when taking the register, the ability to mark without reason without having to enter a separate window. Also, the ability to see attendance for all students with their form, meaning they must manually search each student's name and then navigate to where their attendance percentage is.

With respects to my coursework and what my client as asked me for, I am not going to touch on either the attainment or the behaviour part of Arbor. I will however be focusing entirely on the attendance and register systems in Arbor. I will keep the how the attendance is displayed in the student profile on their dashboard as it provides an easy way to quickly view their attendance percentage. But I will be adding functionality to the student side to allow them to see where they have missed lesson, this will be done so that my client doesn't always have to search for each student whenever they think they have gotten an incorrect and find out where it is for them, they can just do it on their own saving my client time. For the teacher side I will be adding a view for them where it

displays all the students in that teacher's form with the student's attendance next to their name, it will be ordered by attendance in ascending order, so lowest attendance at the top and highest attendance at the bottom. It will also be colour coded to make visualising who the form tutor needs to talk to easier and faster for her. I will also make changes to the register form, giving more options to use in the main menu rather than making submenus which popup all the time and waste time if there is nothing to fill in.

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Another system I looked at was Inventry, https://inventry.co.uk/products/sign-in-solutions/.



Log in for staff to access features assigned to them, such as taking registers

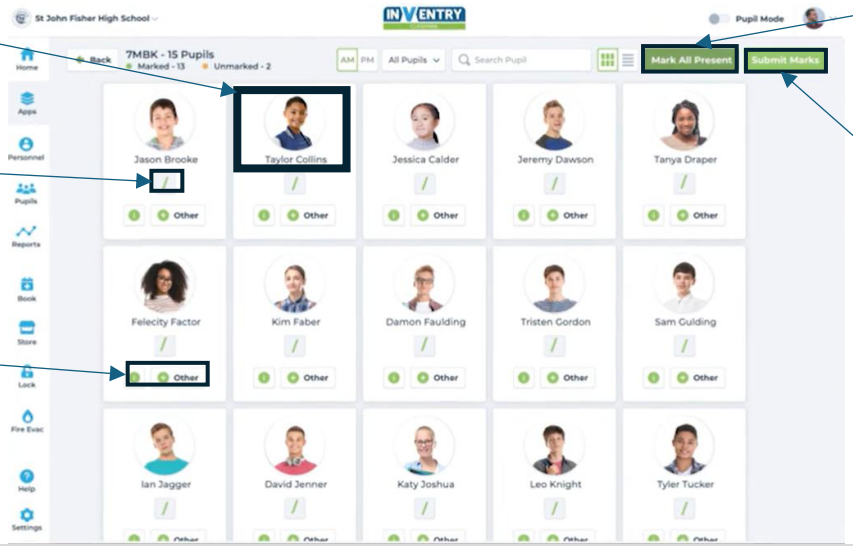
Option for students to log in

Inventry is also used a registration system and attendance management system which has a specific system for schools, which is what I will be researching. Inventry has functions to mark all students as present and a separate function which leads into all other options, such as late or not present. You can also click into students to see their personal details, or a club overview including what clubs they are in and their attendance for that specific club.

Information about the student

Mark assigned to the student

Give register marks, such as present, absent, late etc...



Option to mark all students as present

Submit register, save the register and push the changes to the data storeW

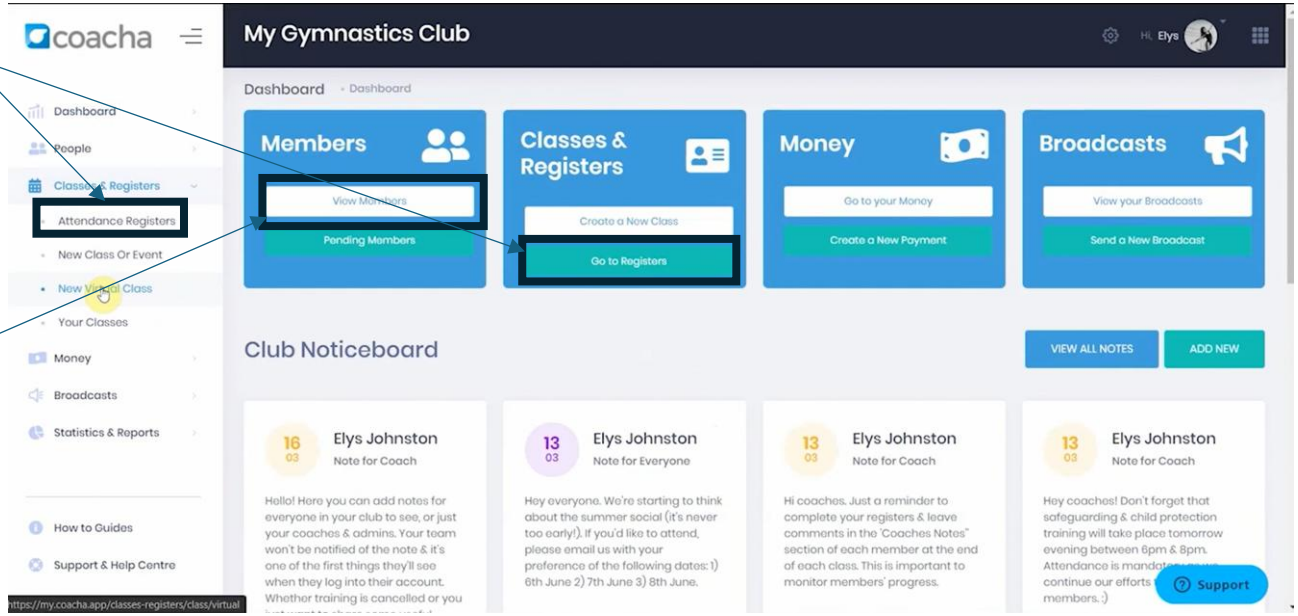
The data that is output to the teacher is a grid of the student's names, pictures and then the option to mark them present, or any other mark, which opens a new window in which the teacher can enter what mark they should get. If the teacher clicks into a specific student the teacher can also see all the information about the student such as name, date of birth etc... There is also a class view where all the classes for that teacher are listed.

From this system I am going to keep the mark all as present as that is a useful feature that my client has requested in the proposed solution. I will not use the grid format that Inventry use as my client would prefer it in a list view to make it easier to just go down one each time and mark it easier. I will also keep the submit marks as that allows my client another chance to check over it. I will also leave out the button for other as my client has stated that she doesn't want another dialogue box to have to fill in.

Another system I also investigated is called coacha, https://www.coacha.co.uk/Features/Classes-and-Registers.

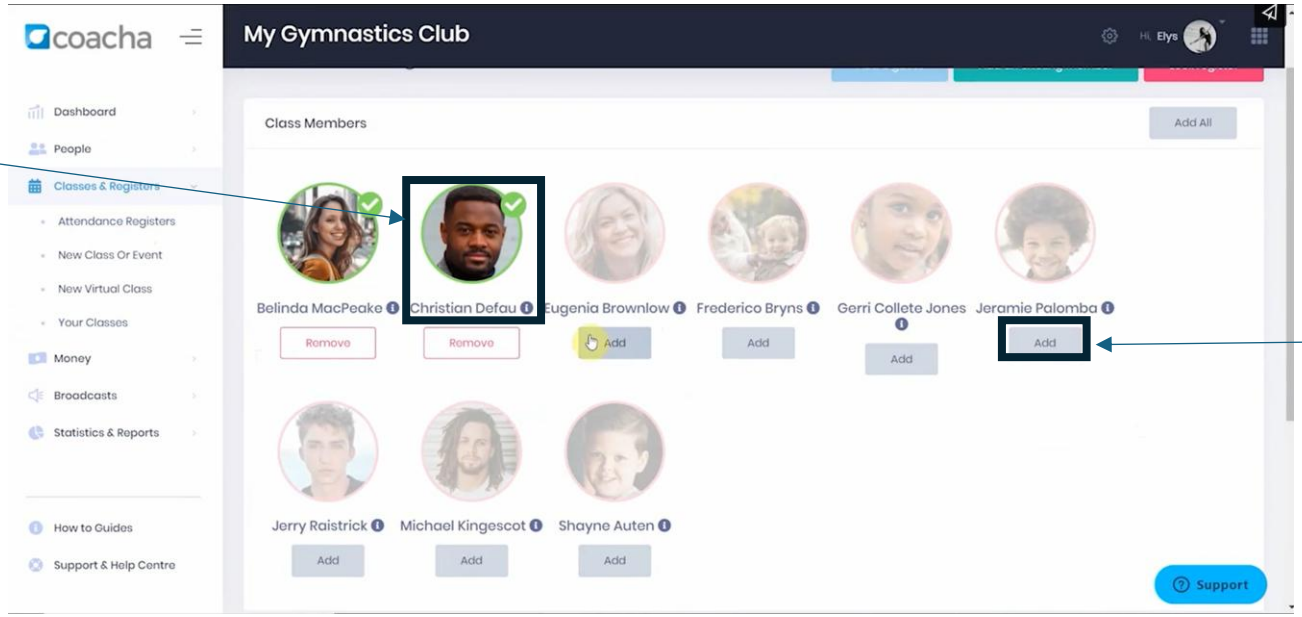
Ability to view registers and see past registers

Teacher can see all members and information about them



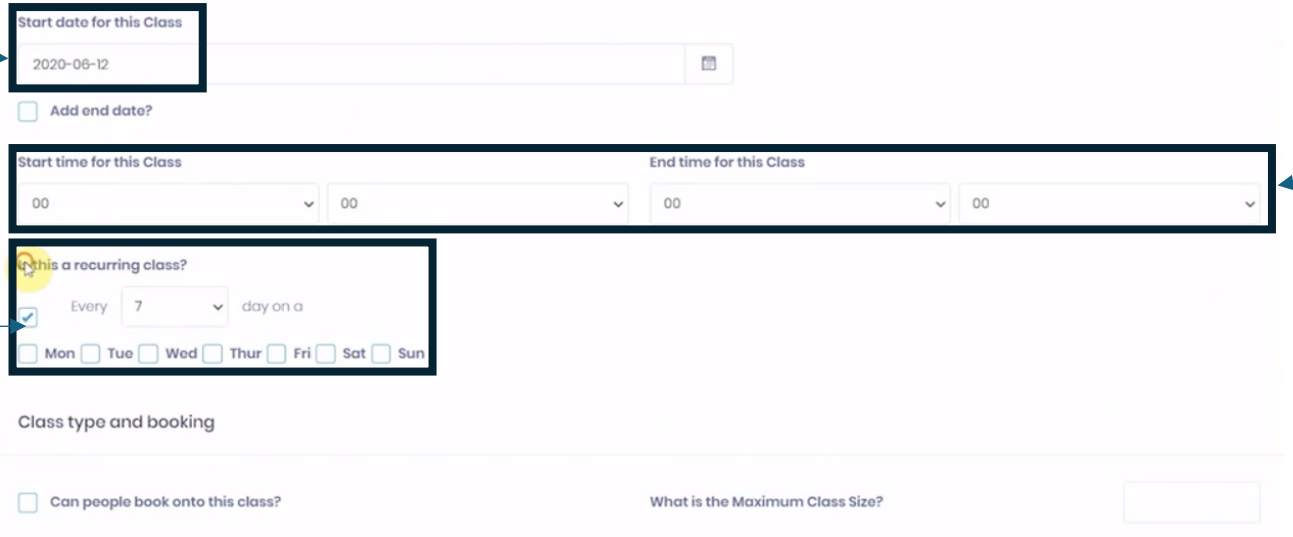
Coacha has a lot of useful features and functions, such a way to manage members, which classes are available and each of those classes' registers, money, and broadcasting information. Coacha has functions to mark students as present but not as late or absent with a reason.

Information about the student, such as picture and name



Ability to add people to a class

Setting the start date of the class



Setting the start and end time of the class

Setting if it is a recuring class and when it is set to reoccur

The data that is outputted is a grid of the student's profile picture with the option to sign them in below using the add button. There are also options to add students to class elsewhere in the system and to create classes. The data required for that is Class name and description, start date, whether it is recurring or not and the timings for the class. To then add students to a class there is another grid of pictures of people with the option to add them below by clicking a button below their picture labelled add.

From this software I will keep the singular button to say that people are attending as quick to do and does not have any other windows. I am going to leave out the profile picture in the register as when there are more people in a class it becomes hard to see everyone, also my client has stated that they would prefer the register without them. I also do not need to add any classes so I will also be leaving that feature out as that is not what my coursework focuses on.

Summary table

Software name features wanted justification

Software name	Features wanted	Justification
Arbor	• How attendance is displayed on the student's dashboard • How the register is laid out for the teacher • The attendance certificate • The Timetable view for attendance • The overview of attendance for a student	• It is short compact; gives all the details it needs to so there is no need to change it. • This is in a nice list format which makes it easy to scroll down and see all the names. • Effective way to see all the data about one student's attendance in one place. • Straightforward way to see where the student may have been absent. • Also, nice and shot overview of a specific student's attendance
Inventry	• Mark all as present button	• This makes it much faster to complete a register if you know everyone is present in the class as you only must press one button and then submit the register
Coacha	• Button to say people are attending with no extra windows	• This reduces the number of windows that you need to use which much faster than if you have an extra dialogue box which you plan to leave empty

Features of Proposed Solution

Feature	Description	Justification
Log in page	This is to restrict different people with various levels of access to only see certain features and perform certain tasks	This is so people with not the correct levels of access cannot mess with the data and cause inconsistencies in the data
Display register	This will display a list of names to the teacher when they go to take a register	This is so the teacher can call out the names and give different students the correct mark
Select different register marks	This will display a selection of characters next to each student in the register and allow the teacher to select a mark to give the student	This is so the teacher can give different register marks to different student based on whether they are present, late, or absent
Display teacher timetable	This will display the teacher timetable and highlight the current lesson that is going on	This is so the teacher can easily select the correct register to go in to and take at the correct time
Search for student	This will allow the teacher to search for a specific student by name to get an overview of their attendance	This is so, if necessary, the teacher can individually look up the student's attendance in detail and help them locate missed lessons
Display form attendance	This will display a list of all students in that teachers form with their attendance next to their name. It will also be ordered by attendance from lowest to highest and colour coded.	This makes it easier for the teacher to check each student in her form attendance quickly and all in one place. And it is colour coded to show who she needs to talk to about their attendance
attendance certificate	This will display an entire overview of one student's attendance in depth including percentage attendance, and a table of all register marks	This provides an entire overview of a student's attendance in one place which makes it easier for the teacher to scan over to see where things may have been marked incorrectly
Attendance timetable (teacher)	This will display a table with lessons number as a heading and date along the side with each section of the table containing the register mark for that lesson, with it being only between specific date set by the teacher	This help to identify where a student has missed lessons or marked as late, in a concise format which can be looked over quickly
Attendance overview (teacher)	This will display some stats about a student's attendance	This will just provide a nice overview for the teacher to quickly look at and see if they need to look deeper into where the student missed lessons
Attendance overview (student)	This will display an overview of the student's attendance, which is just their overall percentage attendance on the student's home page	This is so the student can quickly see their attendance and if it needs to be investigated
Attendance timetable (student)	This will display a table with lessons number as a heading and date along the side with each section of the table containing the register mark for that lesson, with it being only between specific date set by the student	This is so the student can look through on their own and see where they may have missed lessons before the teacher has to check for them saving time for the teacher

Limitations of the Proposed Solution

One limitation I have is that my current solution will have no networking functionality so it is only able to be run on one computer at any one time and any updates to files will be stored locally on my client's pc. This is because it is a prototype and because I am focused on the main functionality of the system rather than methods to make it a viable commercial product which will only happen after the prototype is working correctly.

Another limitation of the system is that it will have minimal user interface development. This is due to during development I am focused on functionality over user interface development as this is a prototype I am developing rather than a complete system currently. The UI design will be straight forward and simple to reduce the amount of time I spend on it, so I can focus on functionality.

The fact that the system will only have a keyboard as an input to the system is another limitation as it will have limited user intractability. Again, this is because I am developing a prototype, and my focus is the core functionality of the system and making sure that is working correctly before I add more ways for the user to interact with the system.

I also will not be developing any APIs to allow my system to communicate with other systems such as emailing systems. This is another limitation as it means that anything the software cannot do cannot be easily transferred over to another system. Once again this is because I am developing a prototype not a complete solution. This means I am more focused on developing standalone functionality for the program rather than how it can utilise other systems as well. Once functionality has proven to be robust this feature can be added.

Another limitation of the proposed solution is the size of the initial data that I have is much smaller than the data that will end up being used in the fully completed system. This is again because I am developing a prototype and focusing on the functionality and not focused on incorporating all the data which would come after the prototype is approved by the client and all functionality works.

Requirements

Hardware Requirements

The hardware requirements to run the system is limited, all my client needs are a pc, a monitor, keyboard, and mouse. The mouse is only needed to initially run the application. Then the keyboard is used to operate the rest of the program as the system runs in a command window and a mouse is not supported in mou. And the monitor is used to display the output. The hardware requirements need to be low as the computer she uses at school computers which are not particularly good. Such as they do not have graphics cards and have old CPUs which can means my program has to be designed with this in mind. It also has limited RAM so I cannot use up too much memory considering the OS and other programs.

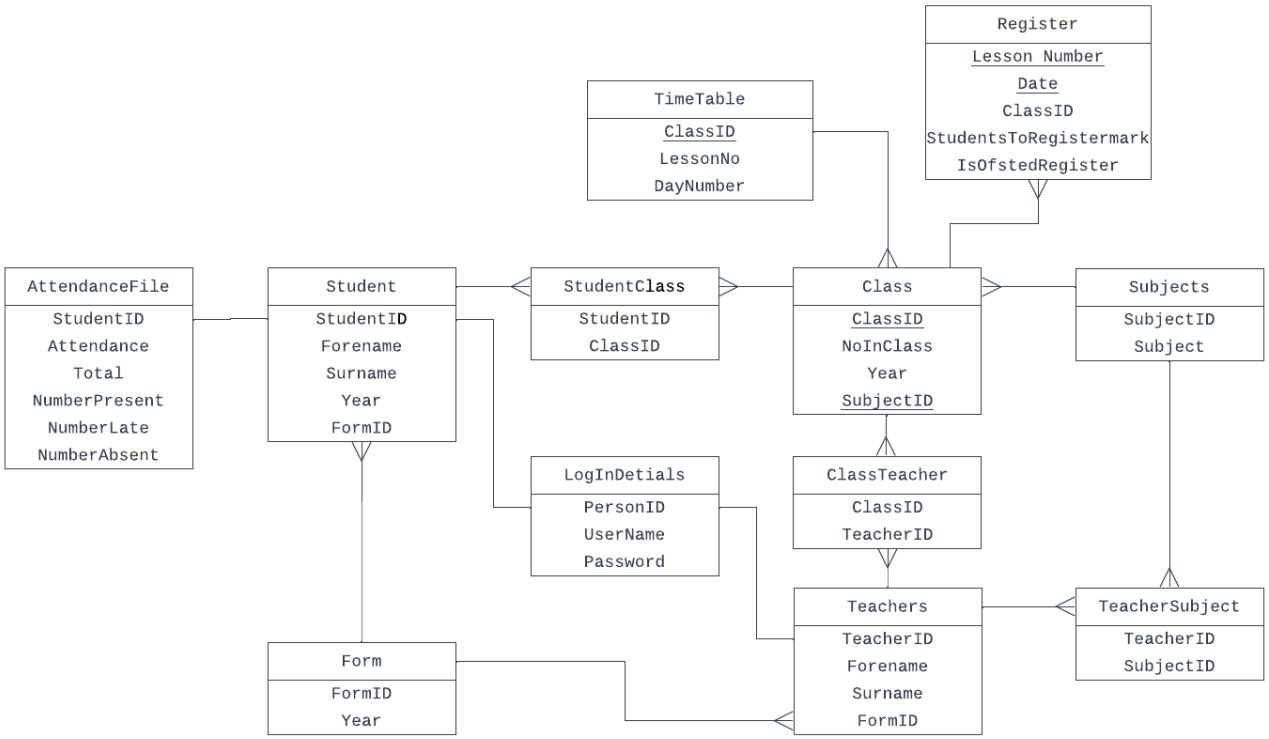
Software Requirements

The system requires a windows operating system as the program has been compiled into a .exe, but by using packages such as wine and winetrix (a Linux package used to emulate windows software in Linux) it is possible to run it on some Linux distributions. They would not need the ide as the idea is used to compile the program and is not needed to run it. It must be able to be run on windows as in school, where my client will run the program, the OS is only windows. My client would need the program to be in a .exe so windows can run it because all computers and laptops in school only run windows which require an exe.

File Requirements

- The data files required by the software are:
- The student file is the file that is used to store all the data bout the students including their name, StudentID and their attendance.
- The timetable file contains the information about the timetable, it keeps track of every class and when they all are using a combination of lesson number and day number to generate a table.
- The class file stores the information about every unique class including the class id and the subject id to make every class easily identifiable. It also stores the number of students in that class to make searching for those students easier
- Stored in the teacher file is all the information about the teachers such as their id and name.
- Stored in the subjects file is all the information about each subject, its name, and that subject id.
- The login details file stores all the information about each person's logins and is ordered by username to make searching it easier.
- The form file is a file that stores all the information about any form/tutor groups in the school.
- The register file stores all the information about every register like when it was taken and who was there with what register mark.
- The attendance file contains all the information that the program needs to calculate a student's attendance and stores that number.

All these files are linked together using many to one relationship as you cannot have a many to many relationships, so it is broken up into a resolver file with a many to one relationship. There is also someone to one relationship between the student and the Login details file and the teacher's files.



Data requirements, my client needs to store data about, normalised link relational system.

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Success Criteria

Feature	Description	Measurable success criteria
Log in page	This will ask for a username and password, and depending on the status logins in depends on the tasks offered to the user	This is measurable as it should either out put a menu for a teacher or a menu for the student. When a teacher logs in the functions will be: View Attendance Take a Register And if it was a student that logs in the list of functions will be: Attendance attendance percent: ...% Timetable
Display register	This will display a list of student's names in that class with a choice of marks next to each student name	This is measurable as it should return all the students in the class with the options to mark them here or not This will display the names in a format of Name1 - [present] [absent with reason] [absent no reason] [late] Name2 - [present] [absent with reason] [absent no reason] [late]... And there will be the same number of names as students in that class which can be check by outputting the number of students in that class from the class file. With a mark all as present, mark all as absent and a submit button at the bottom
Select different register marks	This will allow the user to move the cursor between each student and each option and selecting the option just by moving on to the next student. And them submitting by pressing enter	The end display will look like Name1 - [present] [absent with reason] [absent no reason] [late] Name2 - [present] [absent with reason] [absent no reason] [late] Name3 - [present] [absent with reason] [absent no reason] [late]... [Mark All Present] [Mark All Absent] [Submit Register]

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

<table>
<tr><th>Feature</th><th>Description</th><th>Measurable success criteria</th></tr>
<tr><td>Display teacher timetable</td><td>This will display the teachers timetable and highlight the current lesson that it is. The timetable will be displayed with day along the top with date above that and then lesson numbers down the sides</td><td>
<table>
<tr><td></td><td>01/07/24</td><td>02/07/2024</td><td>03/07/2024</td><td>04/07/2024</td><td>05/07/2024</td></tr>
<tr><td></td><td>Monday</td><td>Tuesday</td><td>Wednesday</td><td>Thursday</td><td>Friday</td></tr>
<tr><td>1</td><td>9AT1/Ch</td><td>9AT2/Ch</td><td>9AT1/Ch</td><td>9BT1/Ch</td><td></td></tr>
<tr><td>2</td><td>9AT1/Ch</td><td>9AT1/Bi</td><td></td><td>9BT1/Ch</td><td></td></tr>
<tr><td>3</td><td></td><td>9AT1/Bi</td><td>9BT1/Bi</td><td></td><td>9AT1/Ch</td></tr>
<tr><td>4</td><td>9BT1/Ch</td><td></td><td>9AT1/Bi</td><td>9BT2/Bi</td><td></td></tr>
<tr><td>5</td><td>9BT2/Bi</td><td></td><td></td><td>9AT2/Ch</td><td></td></tr>
</table>
The data will look something like this with classes and the highlighted one is the current lesson that they need to do the register for
</td></tr>
<tr><td>Search for student</td><td>There will be a search bar provided that can be typed into and will display all students which are close to what they have typed in, they can then navigate down to each student and then select the student</td><td>Search for name: R.
Robert Cole
Rita Gamble
Rose Roberts
Robert Flores…
This will refine as more letters are typed in, the selected one can then be selected by pressing enter and you will enter the overview for that student</td></tr>
<tr><td>Display form attendance</td><td>This will display a list of all students in that teachers form with their attendance next to their name and colour coded to highlight the students that need talking to. This will also be ordered from lowest attendance to highest attendance</td><td>Wendell Hulslander – 75%
Marylouise Brooks – 83%
Rose Roberts – 92%
Tracy Boyd – 95%
Velma Morris – 99.8%
This is what an example set of data should look like when looking through the teachers for attendance</td></tr>
</table>

<table>
<tr><th>Feature</th><th>Description</th><th>Measurable success criteria</th></tr>
<tr><td>attendance certificate</td><td>This will display all information about the student’s attendance. Including percentages and a table of where they missed lesson.</td><td>Name: Wendell Hulslander
Attendance: 75%
Authorised absences attendance: 89%
Unauthorised absences: 14
Lates: 12
<table><tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td>05/07/24</td><td>/\</td><td>L</td><td>A</td><td>L</td><td>/\</td></tr><tr><td>04/07/24</td><td>L</td><td>A</td><td>A</td><td>/\</td><td>/\</td></tr><tr><td>03/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>L</td></tr><tr><td>02/07/24</td><td>L</td><td>/\</td><td>A</td><td>/\</td><td>/\</td></tr><tr><td>01/07/24</td><td>L</td><td>L</td><td>A</td><td>/\</td><td>A</td></tr></table>...
IT should show the students name, attendance, attendance with authorised absences, unauthorised absences, and late marks. It should then also have a table and all marks in that year.</td></tr>
<tr><td>Attendance timetable (teacher)</td><td>This will provide a timetable for the student with all register marks for a ser period</td><td>Name: Erica Hernandez
<table><tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td>05/07/24</td><td>/\</td><td>L</td><td>A</td><td>L</td><td>/\</td></tr><tr><td>04/07/24</td><td>L</td><td>A</td><td>/\</td><td>/\</td><td>/\</td></tr><tr><td>03/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>L</td></tr><tr><td>02/07/24</td><td>L</td><td>/\</td><td>A</td><td>/\</td><td>/\</td></tr><tr><td>01/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>A</td></tr></table>...
This is a display of all register marks between two time periods set by the teacher when entering this view</td></tr>
<tr><td>Attendance overview (teacher)</td><td>This will bring up a display giving the name of the student, overall attendance, Attendance with authorised absences, Unauthorised absences, and late marks</td><td>Name: Ann Lavoie
Attendance: 85%
Authorised absences attendance: 93%
Unauthorised absences: 4
Lates: 2</td></tr>
</table>

<table>
<tr><th>Feature</th><th>Description</th><th>Measurable success criteria</th></tr>
<tr><td>Attendance overview (student)</td><td>This will give the student a quick overview of all their attendance statistics, which is the same as what the teacher will see. It will display overall attendance, attendance with authorised absences, Unauthorised absences, and late marks</td><td>Attendance: 89%
Authorised absences attendance: 96.4%
Unauthorised absences: 1
Lates: 0</td></tr>
<tr><td>Attendance timetable (student)</td><td>This will give the student a week-by-week view of all their register marks over the entire year which they can cycle through to see when they have missed lessons or were marked as late</td><td>This will display a 5-day week from Monday to Friday with lesson numbers as headings and it will display all the register marks that have been marked down that week. So, from my example data one example could look like this.
<table><tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td>01/07/24</td><td>/\</td><td>L</td><td>A</td><td>/\</td><td>/\</td></tr><tr><td>02/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td></tr><tr><td>03/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>L</td></tr><tr><td>04/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td></tr><tr><td>05/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>A</td></tr></table></td></tr>
<tr><td>Timetable (student)</td><td>This will give the student a week-by-week view of their timetable and their classes. This is so they can cross reference what register marks they got for which class</td><td>This will display a 5-day week from Monday to Friday with lesson numbers as headings and it will display the class that the student has at that time. So, from my example data one example could look like this.
<table><tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td>01/07/24</td><td>9AT1/Bi</td><td>9AT1/Bi</td><td>9D/Dt1</td><td>9D/Dt1</td><td>9AAX/Ma</td></tr><tr><td>02/07/24</td><td>9AAX/Ma</td><td>9ABY/En</td><td>9B/Fr1</td><td>9C/Pe1</td><td>9C/Pe1</td></tr><tr><td>03/07/24</td><td>9BAY/Lf</td><td>9A/Dr1</td><td>9ABY/En</td><td>9B/Fr1</td><td>9B/Fr1</td></tr><tr><td>04/07/24</td><td>9D/Dt1</td><td>9AAX/Ma</td><td>9A/Dr1</td><td>9AT1/Bi</td><td>9ABY/En</td></tr><tr><td>05/07/24</td><td>9ABY/En</td><td>9C/Pe1</td><td>9D/Dt1</td><td>9AAX/Ma</td><td>9A/Dr1</td></tr></table></td></tr>
</table>

Design

Introduction

The design of my solution should be simple and user friendly on the front end while maintaining a strong structure which can be used to easily expand the programme later. Both layout and user interactions should be simple and self-explanatory, this is to avoid complications and reduce the time the user must spend either searching through menus or working out how to input the data. Repeated delays in processes like this will cause frustration as it takes valuable time out of the user, which could be used to do other tasks. Ambiguous input methods can cause errors which can interfere with the integrity of the data and cause issues when it comes to analysing the data.

The overarching theme in the coursework will be consistent with itself, the user will always be interacting with the programme in the same way, by keyboard entry. The use of only the keyboard allows for simple entry methods which allows for menus to be navigated through faster and more easily than a mouse would allow. Being able to complete tasks fast is paramount to my solution as it will allow the user to quickly view the information without spending too much time either trying to find it or having it in an uneasily readable format.



Problem Decomposition

The order of development will allow both my client and I to have a clear understanding of what is being done when and why. It will also allow her to query any concerns during development.

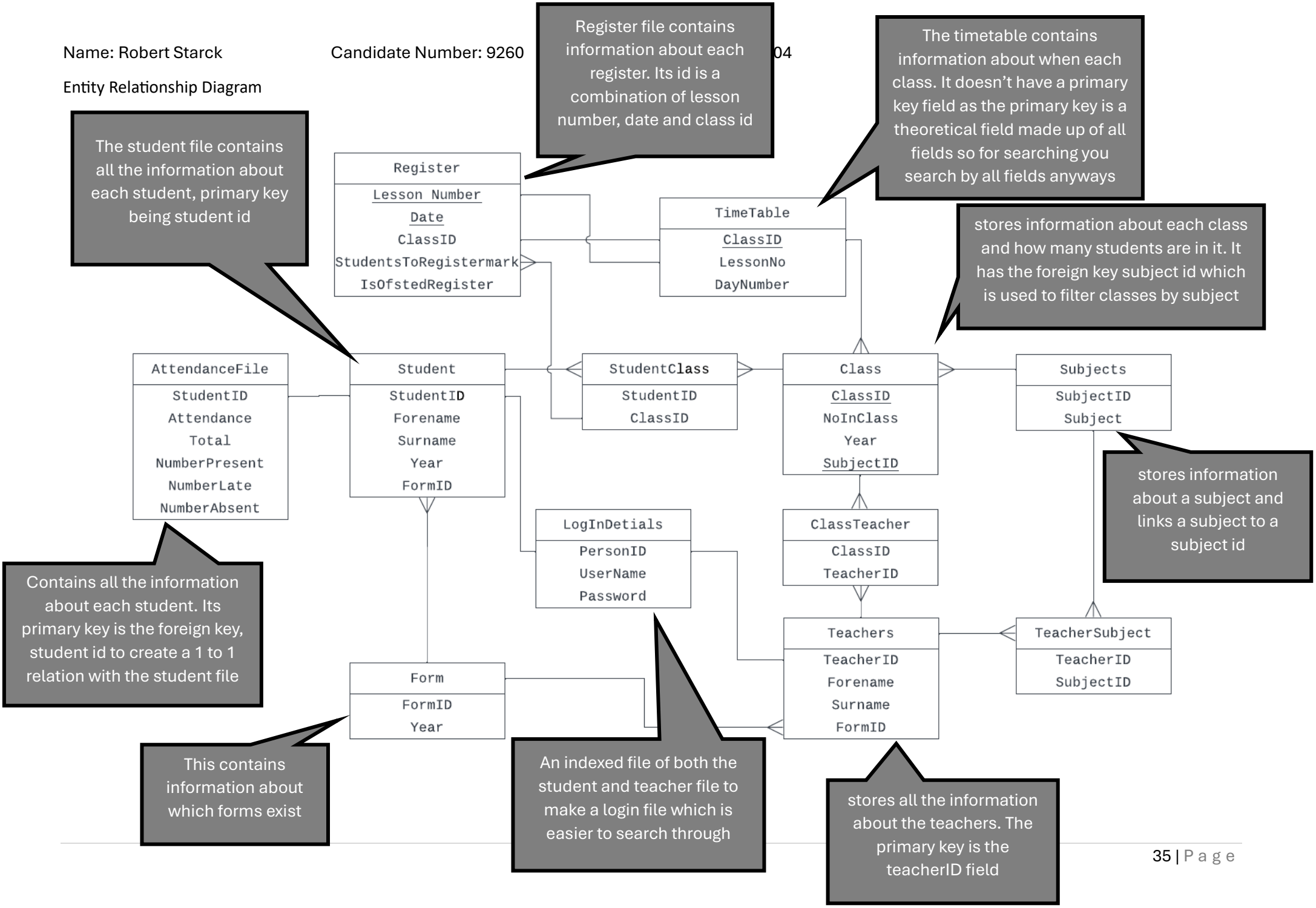
Iteration Number	Feature	Justification for order
0	Generic Functions	This iteration will contain generic procedures such as sorts and searches of arrays. Any other extra information units such as the progress bar and loading screens. This must be the first thing to be done as it is used in future iterations, and it allows time for my client to make any changes to what they want
1	Creating initial data	All initial data must be generated before the main programme can do its tasks as all procedures in the programme rely on displaying, adding or updating the data. Thus, all the files and the data contained in them must be generated first
2	Class Structure	To create a strong structure throughout the programme I decided to have each file as a class so I can attach my own methods to it. This must also be done before the main programme so that the main programme can take advantage of this structure.
3	Student Menu	The students need a base menu which they can use to access the data which my client decides that they needed to be able to see. This will directly connect to the output of the login menu script
4	Teacher Menu	The teachers will also need their own menu to view, and add/change data as required, due the teacher menu having far more tasks to achieve io decided to do this after the student menu. This will also be a directly connected to the output of the login menu
5.1	Login Menu	Before accessing any data, the user must log into an account to ensure that they can only see data they are permitted to access. Therefore, this section should be developed before the menus for each user
5.2	Main program	This will be the main program which pulls everything together and would be the program that the user will run

Required Files and Entity Relationship Diagrams (ERDs)

Initial data files with justification

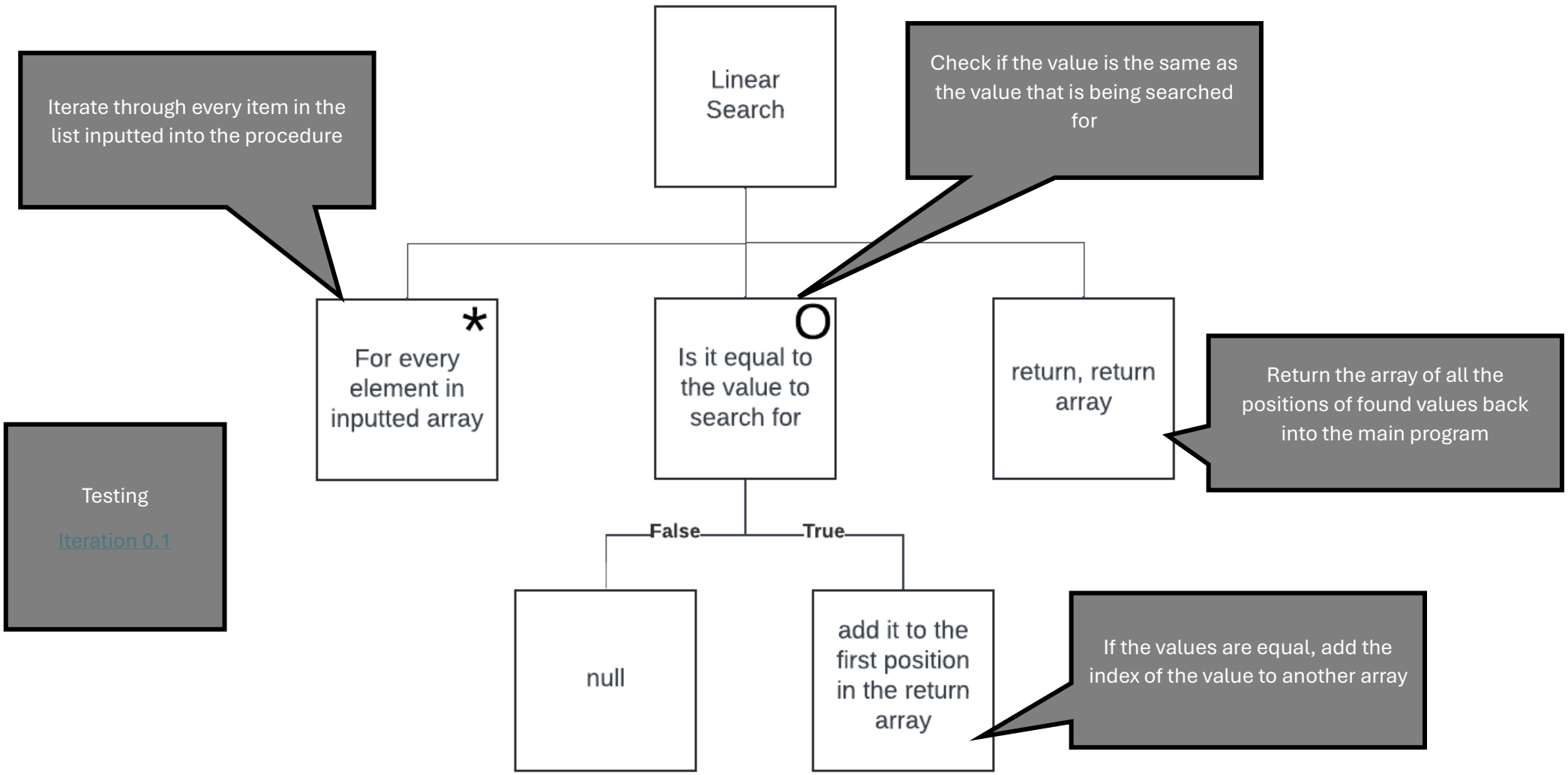
File Name	Justification
Timetable file	This file is required so that all the data about when which lessons are. And can therefore be used to store data about when registers where taken and feedback the data in a more readable format.
Student File	This file is required so that the data about each student can be stored and searched through as well
Attendance file	This file stores all the data about each student and their attendance, which is the main area of focus for my programme
Teacher File	Stores data about the teachers so that the teachers have an area to log into and perform teacher only actions such as taking the register.
Login details file	The log in details file is required so that searching for usernames is faster as all available logins are in one file and sorted by username so a binary search can be done on the data to increase search speeds
Form File	The form file is necessary to keep track of which student is in what form and thus give the form tutor access to the attendance data of that student in a more convenient manner
Class File	Includes the data about all the classes and what subject is taught in each class. This is required so that students can be part of a class, and so registers can be taken
Subject File	This is required so data about which subjects are available and so that classes can be filtered out by subject
Register File	This stores all the data about all registers. This is critical as to manage attendance you need to have registers to get the data to calculate the attendance from.
Student Class Resolver File	This resolves the many-to-many issue with the student and class file, where multiple students can be in 1 class and each student can be in multiple classes.
Teacher Class Resolver File	This is required to solve the issue of the many-to-many relation between the teacher and class file.
Teacher Subject Resolver File	This resolver file is required to resolve the subject file and teacher file, breaking the many-to-many relation into 2 1 to many relations

Entity Relationship Diagram



Linear Search

Top-Down-Design - Linear Search



Pseudocode – Linear Search

```
UNIT LinearSearch
  INTERFACE

  USES CRT, SYSUTILS

  PROCEDURE LinearSearchFunction(low, high, searchValue, UserArray, var returnArray, var numberDone)
    BEGIN
      i = -1
      a = 0
                // iterate through each element until en
      REPEAT
        i = i+1
        numberDone = numberDone +1
                  // add all positions for values matching the search value to a different array
        if userArray[i] = searchValue THEN
        begin
          returnArray[a] = i
          a = a + 1
        end
      UNTIL (i = high)
    END

  END.
```

Declares this section as a unit. So all procedures are made public and usable by other programs as long as it is declared in the use of the other programs

Declare counting variables for indexing the array with

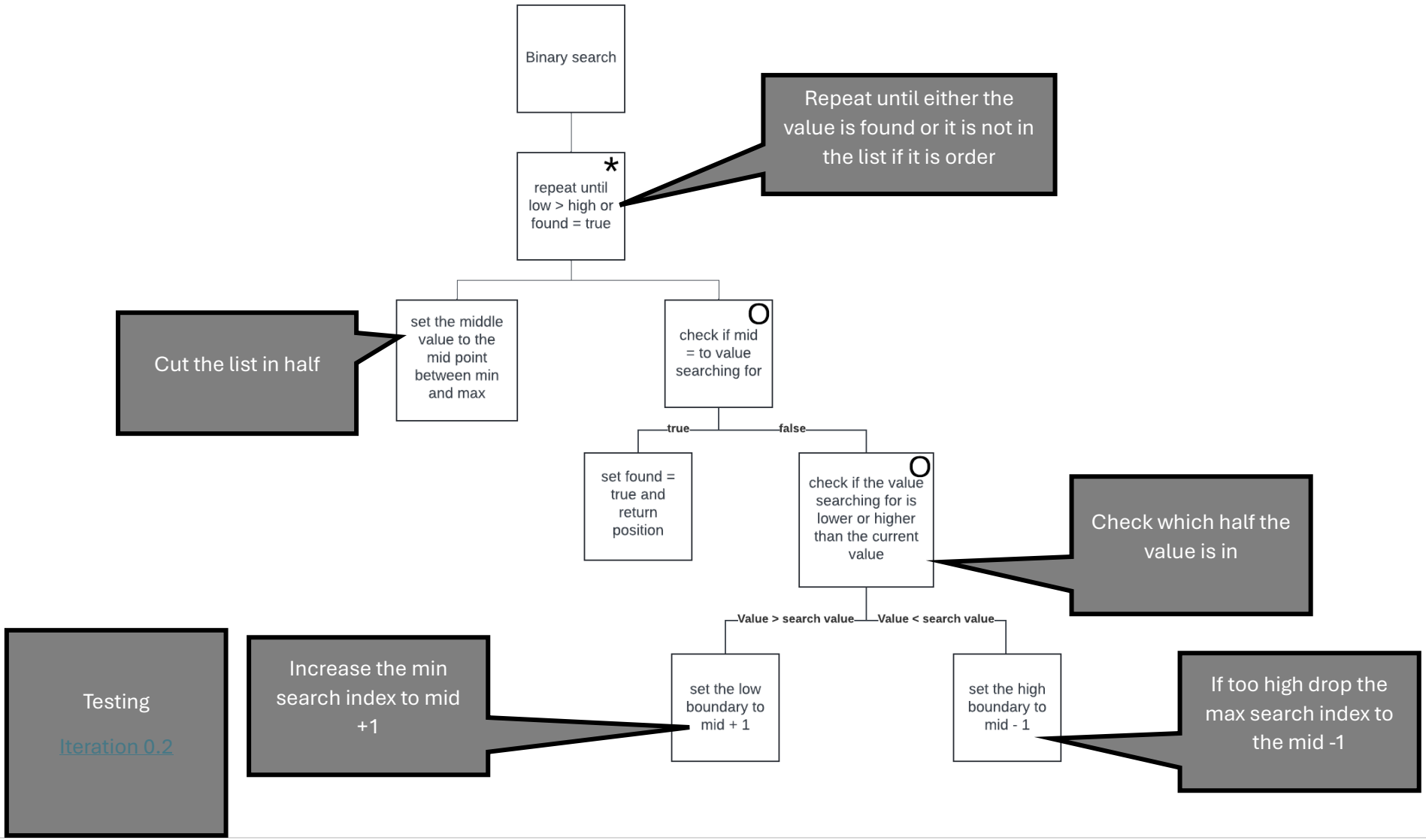
Testing the value in the array with the value that is being searched for

Add the index to an array, which in this procedure is a pointer to an array in the main program.

Keep iterating until the end is marked, which is set by the programmer

Binary Search

Top-Down Design – Binary Search



Pseudocode – Binary Search

```
UNIT BinarySearch

  USES CRT, SYSUTILS

  PROCEDURE BinarySearchFunction(UserArray, var returnArray, low, high, searchValue)
    BEGIN
      REPEAT
        mid = (low+high) DIV 2
        IF UserArray[mid] != searchValue THEN
          BEGIN
                IF UserArray[mid] > searchValue THEN
                  BEGIN
                    low = mid +1
                  END
                ELSE
                  BEGIN
                    high = mid -1
                  END
              END
        ELSE
          BEGIN
            returnArray[0] = mid
            shouldEnd = true
          END
      UNTIL (shouldEnd == true) OR (low > high)

    END

  END.
```

Start with mid as middle of array

Test if value is not found

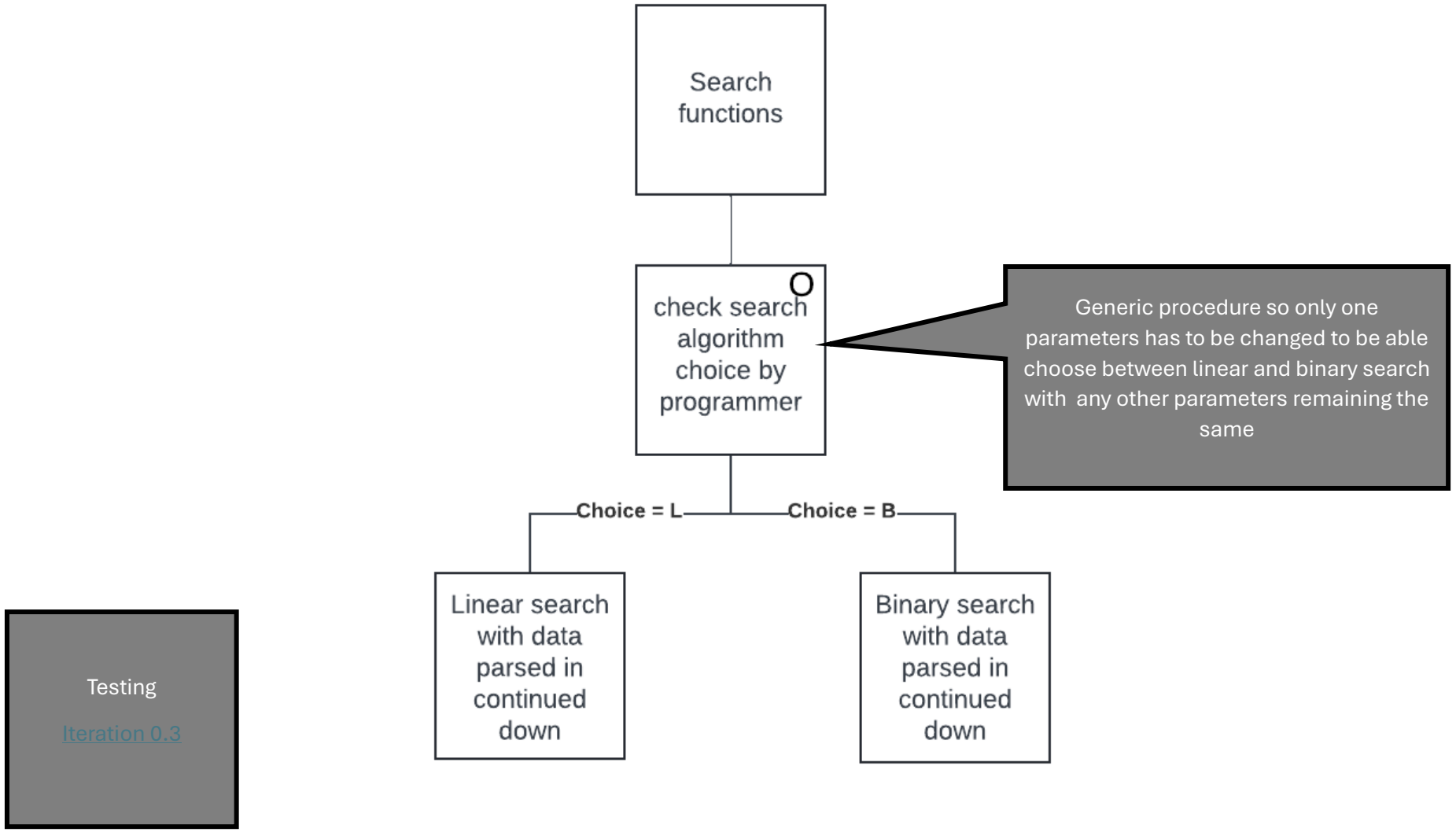
Find if the item of data is above or below the current middle point

If value is found, set value to return array

Repeat until found or not in array

Searching

Top-Down Design Searching



Pseudocode – Searching

```
UNIT Search

  USES CRT, SYSUTILS, BinarySearch, LinearSearch

  PROCEDURE SearchFunctions(SearchType, x, y, findValue, UserArray, var returnArray, var numberDone)
    BEGIN
      IF SearchType == 'L' THEN
        LinearSearch.LinearSearchFunction(x,y,FindValue,UserArray,returnArray,numberDone)
      IF SearchType == 'B' THEN
        BinarySearch.BinarySearch ction(UserArray,ReturnArray,x,y,FindValue)
    END

  END.
```

Uses the two search algorithm units

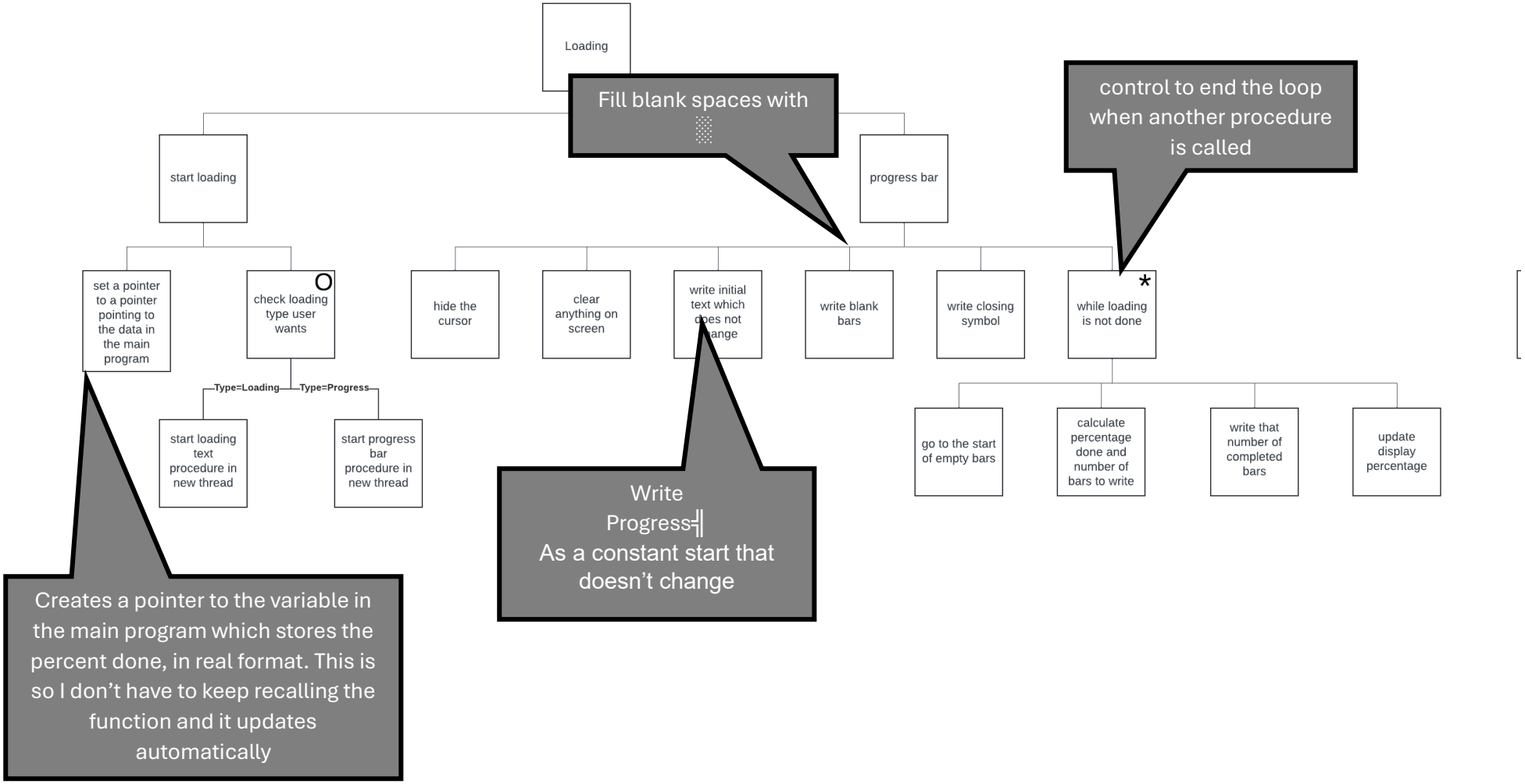
All data is parsed in to this one procedure and is then manipulated

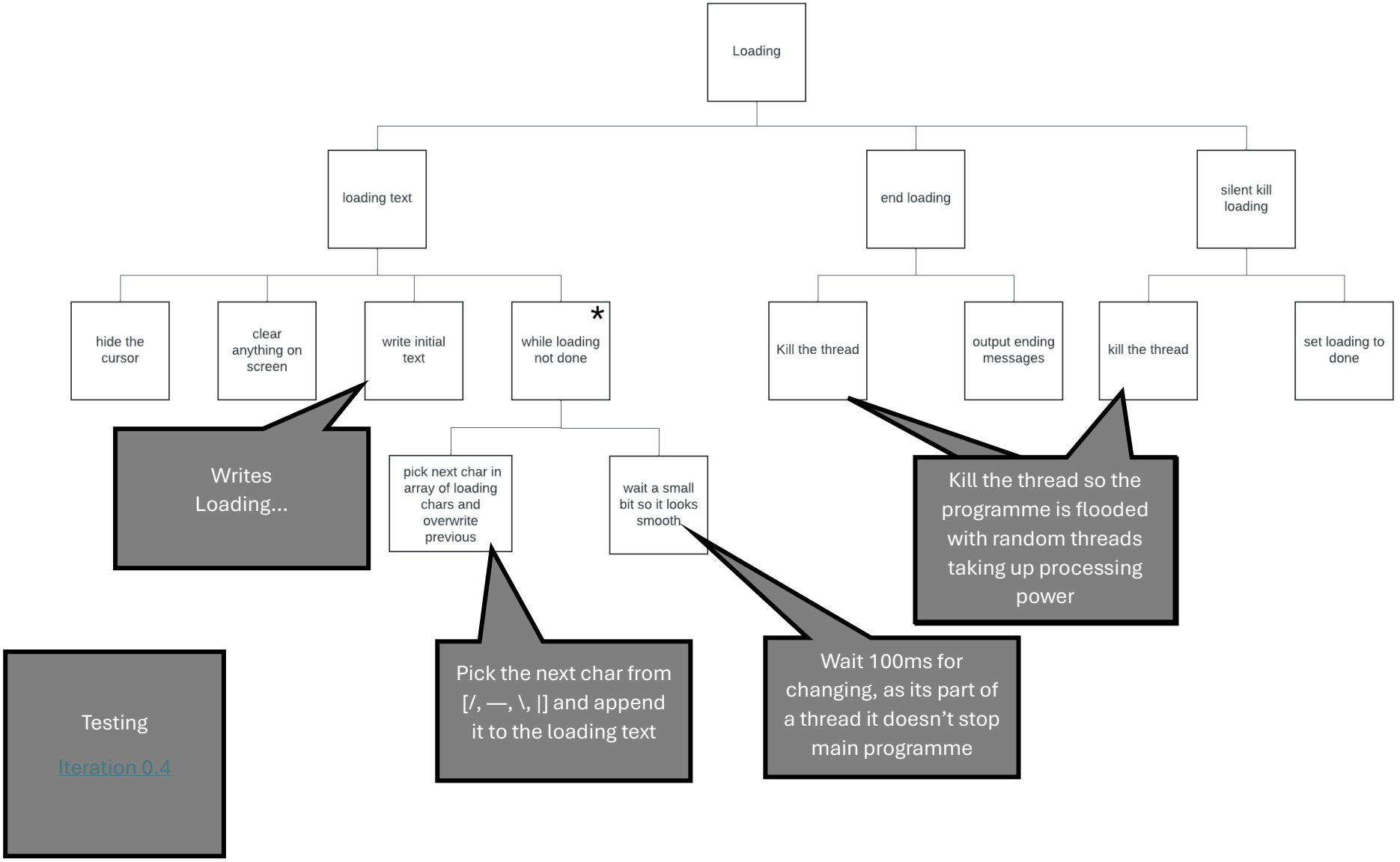
Choice between linear search and binary search, with all data automatically being parsed down

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Loading

Top Down Design – Loading




Loading
loading text
end loading
silent kill loading
hide the cursor
clear anything on screen
write initial text
while loading not done
*
Kill the thread
output ending messages
kill the thread
set loading to done
pick next char in array of loading chars and overwrite previous
wait a small bit so it looks smooth
Writes Loading...
Kill the thread so the programme is flooded with random threads taking up processing power
Pick the next char from [/, —, \, |] and append it to the loading text
Wait 100ms for changing, as its part of a thread it doesn't stop main programme
Testing
Iteration 0.4

Pseudocode – Loading

```
unit loading

const LoadingSymbols : ARRAY[1..4] OF CHAR = ('/',#196,'\','|')

function loadingText(pointer)
  begin
    HideCursor
    a = 1
    ClrScr
    while (!loadingDone) DO
      Begin
        WRITELN('Loading... ' + LoadingSymbols[a])
        a = a+1
        if a > 4 THEN
          a = 1
        Delay(100)
        GoToXY(1,1)
        DelLine
      end

    end
```

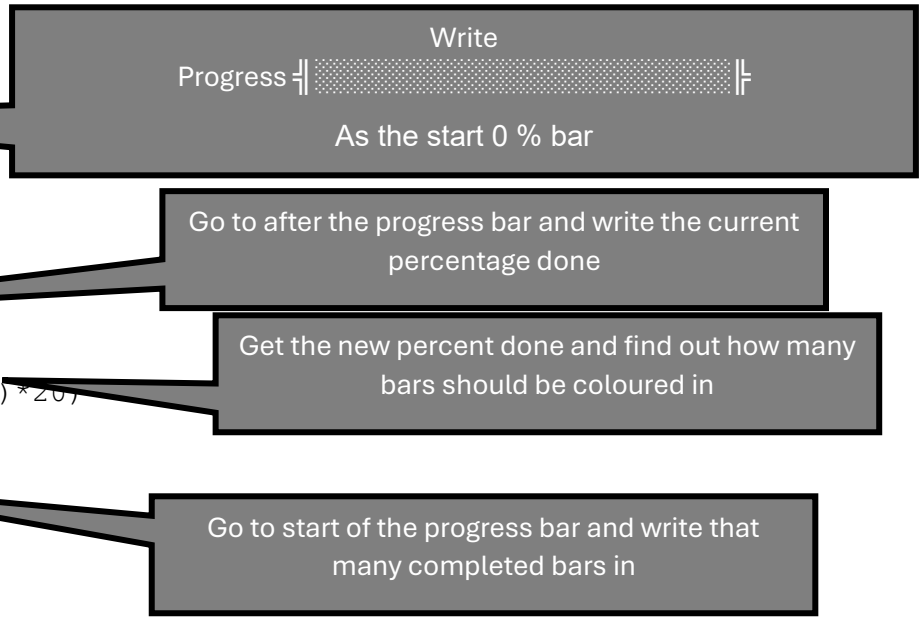
Unit so other programmes can use its functions

Declare the array for the spinning characters

Write Loading... with next spinning char

Increase and loop a so it goes between 1 and 4 for each char

```
function loadingBar(pointer) : LONGINT
  begin
    HideCursor
    ClrScr
    Write('Progress ',#185)
    percentDone = 0.0*100
    for c= 0 TO 20 DO
      write(#176)
    write(#204)
    completeBars = 0
    while !loadingDone DO
      begin
        gotoxy(33,1)
        Write(percentDone:4:0,'%')
        percentDone = (percentPtr^)*100
        completeBars = floor((percentPtr^)*20)
        gotoxy(11,1)
        for d = 0 to completeBars DO
          begin
            write(#219)
          end
    end
 end
```



```
procedure StartLoading(loadingType, var percentIn)
  begin
    startRunTime = Time
    loadingDone = false
    percentPtr = @percentIn
    if loadingType = 'L' THEN
      VPBeginThread(loadingText,1,nil)
    else if loadingType = 'P' THEN
      VPBeginThread(loadingBar,1,nil)
  End

procedure EndLoading
  begin
    KillThread(1)
    loadingDone = true
    WRITELN('Done!☺')
    endRunTime = Time
    Delay(500)
    ClrScr
    writeln('Time elapsed: ',TimeToStr(endRunTime - startRunTime))
    ShowCursor
    end

procedure silentKillLoading
  begin
    KillThread(1)
    loadingDone = true
    ClrScr
    ShowCursor
  end
end.
```

Set start run time to current real world time

Create a pointer to the memory address where the percent done is stored in the main programme

Choose the loading type and begin a new separate thread with the loading procedure running in that new thread so the program runs these two procedures in parallel so the loading is accurate to current progress time

Kill the thread with the loading procedure to save resources

Write time elapsed since start

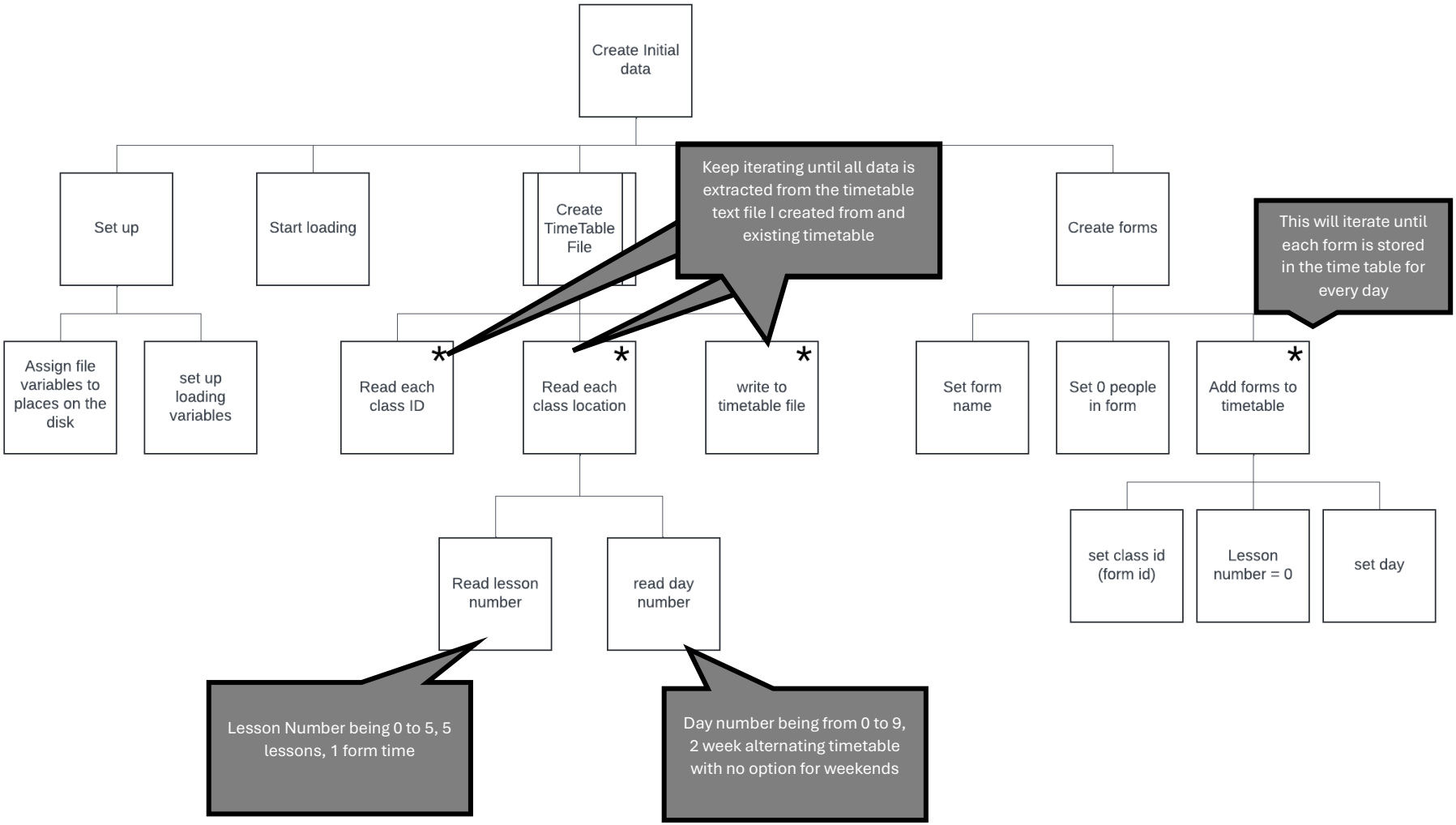
Write Kill the thread with the loading procedure to save resources

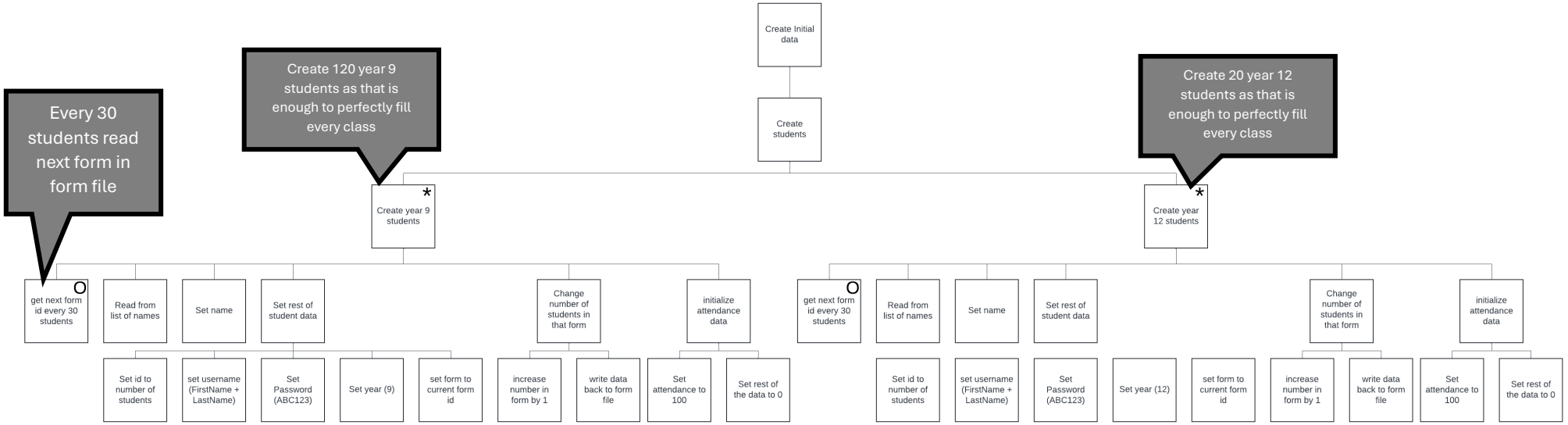
Clear up any changes in the background so the user doesn't see the changes

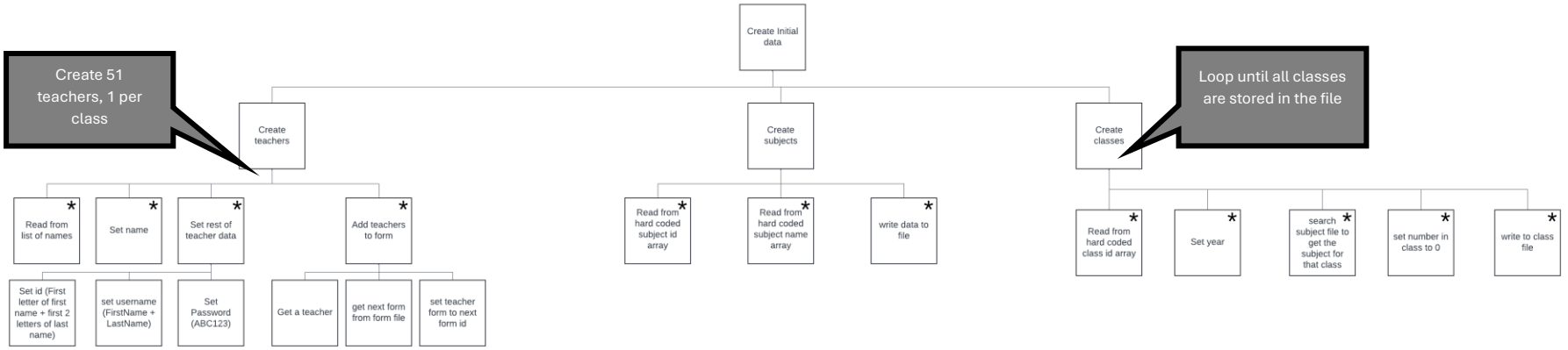
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

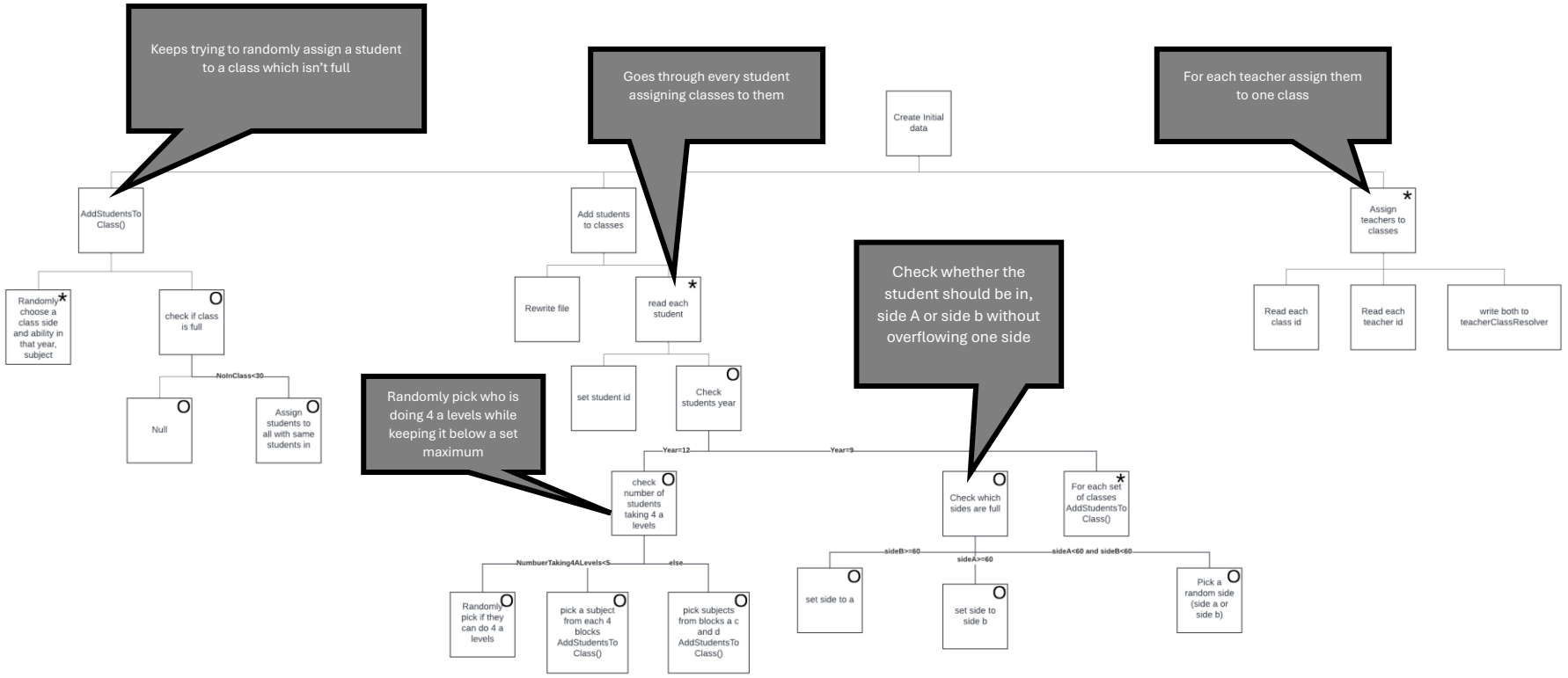
Create Files

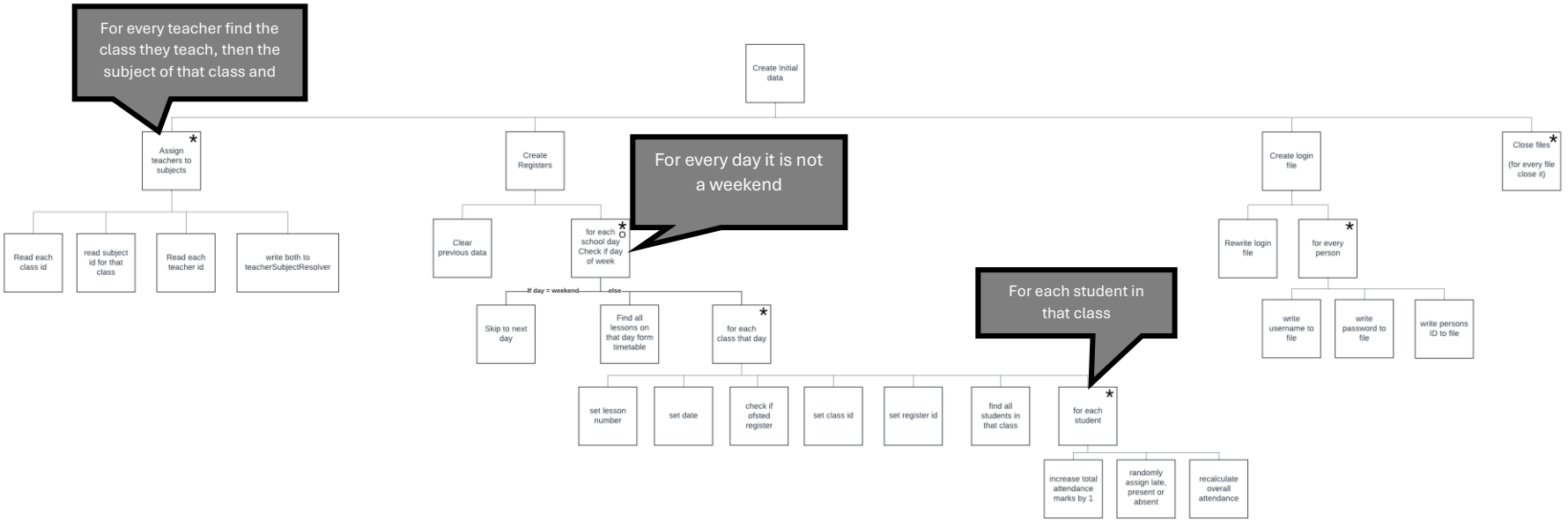
Top-Down Design - Create Files




Create Initial data
Create students
Every 30 students read next form in form file
Create 120 year 9 students as that is enough to perfectly fill every class
Create 20 year 12 students as that is enough to perfectly fill every class
Create year 9 students
Create year 12 students
get next form id every 30 students
Read from list of names
Set name
Set rest of student data
Change number of students in that form
initialize attendance data
Set id to number of students
set username (FirstName + LastName)
Set Password (ABC123)
Set year (9)
set form to current form id
increase number in form by 1
write data back to form file
Set attendance to 100
Set rest of the data to 0
Set year (12)


Create Initial data
Create 51 teachers, 1 per class
Create teachers
Create subjects
Create classes
Loop until all classes are stored in the file
Read from list of names
Set name
Set rest of teacher data
Add teachers to form
Read from hard coded subject id array
Read from hard coded subject name array
write data to file
Read from hard coded class id array
Set year
search subject file to get the subject for that class
set number in class to 0
write to class file
Set id (First letter of first name + first 2 letters of last name)
set username (FirstName + LastName)
Set Password (ABC123)
Get a teacher
get next form from form file
set teacher form to next form id


Keeps trying to randomly assign a student to a class which isn't full
Goes through every student assigning classes to them
For each teacher assign them to one class
Create Initial data
AddStudentsTo Class()
Add students to classes
Assign teachers to classes
Randomly choose a class side and ability in that year, subject
check if class is full
Rewrite file
read each student
Read each class id
Read each teacher id
write both to teacherClassResolver
NoInClass<30
Null
Assign students to all with same students in
set student id
Check students year
Check whether the student should be in, side A or side b without overflowing one side
Randomly pick who is doing 4 a levels while keeping it below a set maximum
Year=12
Year=9
check number of students taking 4 a levels
Check which sides are full
For each set of classes AddStudentsTo Class()
NumbuerTaking4ALevels<5
else
sideB>=60
sideA>=60
sideA<60 and sideB<60
Randomly pick if they can do 4 a levels
pick a subject from each 4 blocks AddStudentsTo Class()
pick subjects from blocks a c and d AddStudentsTo Class()
set side to a
set side to side b
Pick a random side (side a or side b)



Testing

Iteration 1

Pseudocode – Create Files

```
PROGRAM CreateFiles
  uses crt, sysutils, search, loading, DisplayErrorScrn, math
PROCEDURE initialiseFiles //assign places on drive to identifiers
    BEGIN
      percentComplete = 0.0 //initializing percentDone and total iterations
      total = 0
      //assigning a place on the drive to the files
      TimeTableTextFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\TimeTableText.txt')
                  TimeTableFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\TimeTable.dta')
                  total = total + 51

                  RegisterFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\Registers.dta')
                  total = total + 60

                  NameFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\N
                  StudentFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\Studen
                  total = total + 140

                  TeacherFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\Teachers.dta')
                  total = total + 51

                  classFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\classes.dta')
                  total = total + 51

                  subjectFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\subjects.dta')
                  total = total + 21

                  studentClassResolverFile =
openRead('C:\\ComputerScienceCoursework\\Program\\files\\studentClassResolver.dta')
                  total = total + 140

                  teacherSubjectResolverFile =
openRead('C:\\ComputerScienceCoursework\\Program\\files\\teacherSubjectResolver.dta')
                  total = total + 51

                  teacherClassResolverFile =
openRead('C:\\ComputerScienceCoursework\\Program\\files\\teacherClassResolver.dta')
                  total = total + 51

                  FormFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\forms.dta')
```

Declare all the units to use in this script

Set the loading variables up, the percent done and total tasks to do

Reference a place on the drive where the timetable is stored in a text file

Reference a place on the drive where a list of first and surnames are stored

Assign all variables a space on the drive

```
                total = total + 5

                AttendanceFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\attendanceFile.dta')

                LogInDetailsFile = openRead('C:\\ComputerScienceCoursework\\Program\\files\\LogInDetails.dta')

    total = total + 191
  END


PROCEDURE closeAllFiles
  BEGIN
    //closing all files to confirm all changes are fully written and complete
    TimeTableTextFile.close()
                TimeTableFile.close()
                RegisterFile.close()
                NameFile.close()
                StudentFile.close()
                TeacherFile.close()
                classFile.close()
                subjectFile.close()
                studentClassResolverFile.close()
                teacherSubjectResolverFile.close()
                teacherClassResolverFile.close()
                FormFile.close()
                AttendanceFile.close()
                LogInDetailsFile.close()
  END

PROCEDURE CreateTimeTable
  BEGIN
    reset(TimeTableTextFile
    REWRITE(TimeTableFile)
                //iterate UNTIL all data is extracted
    WHILE NOT EOF(TimeTableTextFile) DO
      BEGIN
        numberDone = numberDone +1
        percentComplete = numberDone/total
        READLINE(TimeTableTextFile,TimeTableInfoFromFile) //get the class id
        //assign it to the record variable
                    TimeTable.ClassId = TimeTableInfoFromFile
```

Close each file to ensure that all data is successfully written to the respective files so data integrity is kept

Setup files ready to be used in this procedure

Take all the data from the timetable text file

Read the class id into the program and store it in a record

Iterate until all days and time have been extracted from the text file into the record and written to the data file

```
                    //for when ever that class is
                    REPEAT
          READLINE(TimeTableTextFile,TimeTableInfoFromFile)
          TimeTable.LessonNo = StrToInt(TimeTableInfoFromFile)
          READLINE(TimeTableTextFile,TimeTableInfoFromFile)
          TimeTable.Day = StrToInt(TimeTableInfoFromFile)
          READLINE(TimeTableTextFile,TimeTableInfoFromFile)
          WRITE(TimeTableFile,TimeTable)
        UNTIL TimeTableInfoFromFile == 'Q' //REPEAT UNTIL all dates are read and stored in the .dta file
                                            FOR that class marked by a Q in the text file
      END
  END

PROCEDURE addForms
  BEGIN
    REWRITE(FormFile)
    FOR i = 0 to 3 DO //iterate FOR all 4, year 9 forms
      BEGIN
        numberDone = numberDone + 1
        percentComplete = numberDone / total

                    //create 4 unique FOR identifiers
        form.formID = '9' + 'AA' + Chr(i +65)
        form.NumberInForm = 0
        WRITE(FormFile, form) //push data to file
      END
    numberDone = numberDone + 1
    percentComplete = numberDone / total
    form.formID = '12' + 'AAA' //DO the same FOR 1 year 12 form
    form.NumberInForm = 0
    WRITE(FormFile,form)
    reset(FormFile) //set point to start of form file
    FOR i = 0 to 4 DO //FOR every form
      BEGIN
      READ(formFile,form)
      timetable.classid = form.formID //get form id into a timetable class id variable
        FOR a = 1 to 10 DO //add it to the file as lesson 0 FOR every morning in the 2 week timetable
          BEGIN
            timetable.LessonNo = 0
            timetable.Day = a
```

Create 4 year 9 forms with unique identifiers, and 0 kids in the class

Create 1 year 12 class with similar data, but altered form identifier

Write the when the forms are to the timetable file

```
            WRITE(TimeTableFile, timetable) //add new data to timetable file
          END
      END
  END

PROCEDURE createRegisters
  BEGIN
    //set up files and reset file pointers
    reset(TimeTableFile)
    reset(studentClassResolverFile)
    RESET(StudentFIle)
    reWRITE(RegisterFile)
    i = 0
    startDate = '13/09/2024'
    //populating the ARRAYs with the data from the files
    FOR i = 0 To 139 DO
      BEGIN
        READ(StudentFile,Student)
        studentFormIdArray[i] = Student.Form
        returnStudentFormIdArray[i] = -1
      END
    i = 0
    WHILE NOT(eof(TimeTableFile)) DO
      BEGIN
        READ(TimeTableFile,TimeTable)
        dayArray[i] = IntToStr(TimeTable.day)
        dayReturnArray[i] = -1
        i = i +1
      END
    i = 0
    WHILE NOT(eof(studentClassResolverFile)) DO
      BEGIN
        READ(studentClassResolverFile,studentClassResolver)
        studentClassResolverClassArray[i] = studentClassResolver.ClassID
        classReturnArray[i] = -1
        i = i +1
      END
    b = 0

    // FOR loop FOR number of days with registers
    FOR i = 0 to 29 DO
```

Set data file for use to create register files

Extract data from files into arrays which can be searched

Create 30 days of registers

```
        BEGIN
          //gets the number day of the week and corrects to monday being 1
          day = DayOfWeek(StrToDate(startDate)+b-1)
          IF (day == 7) OR (day == 6) THEN //IF weekend decrease FOR loop VAR by 1
            BEGIN
              i = i -1
            end
          ELSE
            BEGIN
              numberDone = numberDone +1
              percentComplete = numberDone / total
              //clear out ARRAY FOR searching FOR classes on that day
              FOR c = 0 TO 269 DO
                dayReturnArray[c] = -1
              Search.SearchFunctions('L',0,269,IntToStr(DayOfWeek(StrToDate(startDate)+b-
1)),dayArray,dayReturnArray,empty)
              a = 0
              while dayReturnArray[a] != -1 DO // FOR every class on that day
                BEGIN
                  //find and get the data about one class
                  SEEK(TimeTableFile,dayReturnArray[a])
                  READ(TimeTableFile, TimeTable)
                  //set register data
                  Register.LessonNo = TimeTable.LessonNo
                  Register.Date = StrToDate(startDate)+b
                  IF TimeTable.LessonNo == 4 THEN
                    Register.isOfstedRegister = true
                  ELSE
                    Register.isOfstedRegister = false
                  Register.classID = ''
                  Register.classID = TimeTable.classID
                  register.registerID = IntToStr(register.lessonNo) + register.classID + DateTimeToStr(register.date)
                  classId = format('%s',[Register.classID])
                  //clear out ARRAY FOR searching FOR students in that class
                  FOR c = 0 to 1384 DO
                    begin
                                  classReturnArray[c] = -1
                    end
                  Search.SearchFunctions('L',0,1384,Register.ClassID,studentClassResolverClassArray,classReturnArray,empty)
                  c=0
                  //FOR every student in that class
```

Find the day number of the week correcting it to between 1 and 7

Check if it is a weekend, if so decrease I by 1 so day is not counted towards total

Set up array for searching for all classes in current day

Search for all classes that day

For every class found on that day

Get day information and add that all to the register record

Get the class id and register id and write it to the record

Flush out previous data from search so it can be used to find all student in that class

Find all student in that class and iterate through each of them

```
        while classReturnArray[c] != -1 DO
          BEGIN
            //get the class with the studenet in
            SEEK(studentClassResolverFile, classReturnArray[c])
            READ(studentClassResolverFile, studentClassResolver)
            //get the student id
            studentRegisterMark.studentID = studentClassResolver.studentID
            //get the data in the attendance file
            SEEK(AttendanceFile,studentClassResolver.studentID)
            READ(AttendanceFile,Attendance)
            Attendance.Total = Attendance.Total +1
            //ranDOmly assign the student a mark
            ranDOmMark = RANDOM(100)
            IF ranDOmMark <= 90 THEN
              BEGIN
                studentRegisterMark.registerMark = 'P'
                Attendance.NumberPresent = Attendance.NumberPresent + 1
              end
            ELSE IF ranDOmMark <= 97 THEN
              BEGIN
                studentRegisterMark.registerMark = 'L'
                Attendance.NumberLate = Attendance.NumberLate + 1
              end
            ELSE
              BEGIN
                studentRegisterMark.registerMark = 'A'
                Attendance.NumberAbsent = Attendance.NumberAbsent + 1
              END
            //write data back to register file
            Register.registerMarks[c] = studentRegisterMark
            SEEK(AttendanceFile,FilePos(AttendanceFile)-1)
            Attendance.Attendance = (Attendance.NumberPresent / Attendance.Total)
            WRITE(AttendanceFile,Attendance)
            c = c +1
          END
        WRITE(RegisterFile,Register)

        a = a +1
      END
END
b = b +1
```

Get the students id in the 2d array of register marks

Change students total attendance marks

Randomly assign the student a register mark, present, late or absent and add it to the 2d array of register marks

Write the student id and mark array to the record 2d array of register marks

Recalculate students attendance

Rewrite students attendance data

Once all students in the class are accounted for, write register record to register file

```
        END


        b= 0
        FOR i = 0 to 29 DO //FOR loop FOR each form
          BEGIN
            //gets the number day of the week and corrects to monday being 0
            day = DayOfWeek(StrToDate(startDate)+b-1)
            IF (day == 7) OR (day == 6) THEN //IF weekend decrease FOR loop VAR by 1
              BEGIN
                i = i -1
              end
            ELSE
              BEGIN
                numberDone = numberDone +1
                percentComplete = numberDone / total
                //clear out ARRAY FOR searching FOR forms on that day
                FOR c = 0 TO 320 DO
                  dayReturnArray[c] = -1
                Search.SearchFunctions('L',269,320,IntToStr(DayOfWeek(StrToDate(startDate)+b-
1)),dayArray,dayReturnArray,empty)
                a = 0
                while dayReturnArray[a] != -1 DO// FOR every form on that day
                  BEGIN
                    SEEK(TimeTableFile,dayReturnArray[a])
                    READ(TimeTableFile, TimeTable)
                    IF TimeTable.LessonNo == 0 THEN
                      BEGIN
                        //set register data
                        Register.LessonNo = TimeTable.LessonNo
                        Register.Date = StrToDate(startDate)+b
                        Register.isOfstedRegister = true
                        Register.classID = ''
                        Register.classID = TimeTable.classID
                        register.registerID = IntToStr(register.lessonNo) + register.classID +
DateTimeToStr(register.date)
                        //clear out ARRAY FOR searching FOR students in that form
                        classId = format('%s',[Register.classID])
                        FOR c = 0 TO 139 DO
                          returnStudentFormIdArray[c] = -1
```

Repeat for forms

```
Search.SearchFunctions('L',0,139,Register.ClassID,studentFormIdArray,returnStudentFormIdArray,empty)
                        c = 0
                        //FOR every student in that form
                        while returnStudentFormIdArray[c] != -1 DO
                          BEGIN
                            //get the class with the studenet in
                            SEEK(StudentFile, returnStudentFormIdArray[c])
                            READ(StudentFile, Student)
                            //get the student id
                            studentRegisterMark.studentID = student.studentID
                            //get the data in the attendance file
                            SEEK(AttendanceFile,student.studentID)
                            READ(AttendanceFile,Attendance)
                            Attendance.Total = Attendance.Total +1
                            //ranDOmly assign the student a mark
                            ranDOmMark = RANDOM(100)
                            IF ranDOmMark <= 90 THEN
                              BEGIN
                                studentRegisterMark.registerMark = 'P'
                                Attendance.NumberPresent = Attendance.NumberPresent + 1
                              end
                            ELSE IF ranDOmMark <= 97 THEN
                              BEGIN
                                studentRegisterMark.registerMark = 'L'
                                Attendance.NumberLate = Attendance.NumberLate + 1
                              end
                            ELSE
                              BEGIN
                                studentRegisterMark.registerMark = 'A'
                                Attendance.NumberAbsent = Attendance.NumberAbsent + 1
                              END
                            //write data back to register file
                            Register.registerMarks[c] = studentRegisterMark
                            SEEK(AttendanceFile,FilePos(AttendanceFile)-1)
                            Attendance.Attendance = (Attendance.NumberPresent / Attendance.Total)
                            WRITE(AttendanceFile,Attendance)
                            c = c +1
                          END
                          WRITE(RegisterFile,Register)
```

```
                    END
                    a = a + 1
                END
            END
            b = b + 1
        END
    END


PROCEDURE addStudents
  VAR i : LONGINT
    ffname, fsname, fyear, fform, fattendance : STRING //set up variables to read from text file into
  BEGIN
    i = 0
    RESET(FormFile) //resets pointer in form file
    READ(formFIle, form) //gets the first form
    RESET(NameFile)
    REWRITE(StudentFile) //clears both studenet and attendance file
    REWRITE(AttendanceFile)
    FOR i = 0 to 119 DO //generate 120 year 9 students
      BEGIN
        IF (i mod 30 == 0) and (i != 0) THEN //changes the form every 30 students
          BEGIN
            seek(formFile, (FilePos(formFile)-1))
            WRITE(FormFile,form)
            READ(FormFile, form)
          END
        numberDone= numberDone +1 //update
        percentComplete = numberDone/total
        Student.StudentID = i //set studen
        READLN(NameFile, student.fname) //get a first and
        READLN(NameFile, student.sname)
        student.year = 9
        student.uname = student.fname+student.sname
        student.pword = 'ABC123'
        student.form = form.formid
        form.NumberInForm = form.NumberInForm + 1
        //set attendance data
        Attendance.studentId = i
        Attendance.Attendance = 1
        Attendance.Total = 0
```

Set up files for reading and writing to

Create 120 year 9 students enough to fill every class perfectly

Set student name to names from the list of names in the names text file

Write data back to form file and read the next form data

Set rest of the students personal data

Set student id in attendance file to the same as the student id to create a 1 to 1 link

Set rest of the attendance data

```
        Attendance.NumberPresent = 0
        Attendance.NumberLate = 0
        Attendance.NumberAbsent = 0
        WRITE(StudentFile,student) //write student and attendance data to their respective files
        WRITE(AttendanceFile, Attendance)
      END
    READ(FormFile, form) //get the year 12 form
    FOR i = 120 to 139 DO //generate 20 year 12 students
      BEGIN
        numberDone = numberDone +1 //update progress bar
        percentComplete = numberDone/total
        Student.StudentID = i //set student data
        READLN(NameFile, student.fname) //get a first and last name from a list of names
        READLN(NameFile, student.sname)
        student.year = 12
        student.uname = student.fname+student.sname
        student.pword = 'ABC123'
        form.NumberInForm = form.NumberInForm + 1
        student.form = form.formid
        //set attendance data
        Attendance.studentId = i
        Attendance.Attendance = 1
        Attendance.Total = 0
        Attendance.NumberPresent = 0
        Attendance.NumberLate = 0
        Attendance.NumberAbsent = 0
        WRITE(StudentFile,student) //append this record to the end of the student file
        WRITE(AttendanceFile, Attendance)
      END
      //write changes to form file
      seek(formfile, filepos(formFile)-1)
      WRITE(formfile,form)
  END

PROCEDURE addTeachers
  VAR i : LONGINT
    ffname, fsname, fform : STRING //declare all variables to read from files into them
  BEGIN
    RESET(FormFile)
    reWRITE(TeacherFile)
    FOR i = 0 TO 50 DO //create 51 teachers, 1 per class
```

Write both records to respective files

Create 20 year 12 students and set the same data for year 9 students

Create 51 teachers, 1 teacher per 1 class

Get names from names text file

```
        BEGIN
            numberDone = numberDone + 1
            percentComplete = numberDone / total
            READLN(NameFile, teacher.Fname) //read first and surname form names file
            READLN(NameFile, teacher.Sname)
            Teacher.TeacherID = teacher.FName[1]+teacher.SName[1]+teacher.SName[2] //create the teacher id as first
letter of first name and forst two letters of surname
            Teacher.uname = teacher.FName + teacher.SName
            Teacher.pword = 'ABC123'
            WRITE(TeacherFile,teacher)
        END
      //assign teachers to forms
      RESET(TeacherFile)
      FOR i = 0 to 4 DO
        BEGIN
          READ(TeacherFile,teacher)
          READ(FormFile,form)
          teacher.form = form.formID
          seek(TeacherFile,FilePos(TeacherFile)-1)
          WRITE(TeacherFile,teacher)
        END
    END


  PROCEDURE createLogInFile
      BEGIN
        RESET(StudentFile) //reset file pointers FOR both files
        RESET(TeacherFile)
        REWRITE(LogInDetailsFile)
        //extract login information and student id and write it to a file
        WHILE NOT EOF(StudentFile) DO
          BEGIN
            READ(StudentFile,student)
            LogInDetails.PersonID = Format('%'+'5'+'s',[IntToStr(student.StudentID)]) // format the student id as a
STRING
            LogInDetails.Username = student.Uname
            LogInDetails.Passwowrd = student.Pword
            WRITE(LogInDetailsFile,LogInDetails)
            numberDone = numberDone +1
            percentComplete = numberDone/total
          END
```

set teach id to first letter of first name + first 2 letters of surname

Set rest of the teacher data

Pick the first 4 teachers as form tutors assigning them to 1 form each

For every student

Set the username and password of that student and write to log in file

Convert the id of the student to a string without long decimal places

For each teacher

```
  WHILE NOT EOF(TeacherFile) DO
    BEGIN
      //extract login information and student id and write it to a file
      READ(TeacherFile,teacher)
      LogInDetails.PersonID = teacher.teacherID
      LogInDetails.Username = teacher.Uname
      LogInDetails.Passwowrd = teacher.Pword
      WRITE(LogInDetailsFile,LogInDetails)
      numberDone = numberDone +1
      percentComplete = numberDone/total
    END

END
```

Set the Teacher id, username and password of that student and write to log in file

```
  PROCEDURE createSubjects
    // create a constant ARRAY of all subjects and the subjects' id
    CONST subjID : ARRAY[0..20] OF STRING[3] = ('Cs','Ma','Mu','Enl','Gg','Ch','Ph','Bi','DT','Dr',
                                                'Lf','Rpe','Fr','Hi','Gm','Bs','Ar','Py','Pe','Sp','So')
          subjNm : ARRAY[0..20] OF STRING[20] = ('Computer
Science','Maths','Music','English','Geography','Chemistry','Physics','Biology','Design Technology','Drama',
                                               'Life Skills','Religion, Philosophy &
Ethics','French','History','German','Business Studies','Art','Physical Education','Spanish','Sociology')


    BEGIN
      //clear subject file
      REWRITE(SubjectFile)
      //FOR every subject write subject name and id to a file
      FOR i = 0 TO 20 DO
        BEGIN
          numberDone = numberDone +1
          percentComplete = numberDone / total
          with subject DO
            BEGIN
              subjectID = subjID[i]
              SubjectName = subjNm[i]
            END
            WRITE(SubjectFile, subject)
          END
      END

  PROCEDURE createClasses
    //create ARRAYs of all classes
    CONST yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph', '9AT2/Ph',
'9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph', '9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs
          yTwelveClassArray : ARRAY[0
('12A/Ph1','12A/Py1','12B/Ph1','12                                    2D/Cp1')
      BEGIN
        reWRITE(classFile)
        //FOR every year 9 class
```

Set constant array for subject ids and names

For each item in the arrays

Write subject name and subject id to subject file

For each year 9 class

Arrays of all year 9 classes in 1 array and all year 12 classes in the other

Set class id to next value in the list

```
FOR i = 0 TO 43 DO
  BEGIN
    numberDone = numberDone + 1
    percentComplete = numberDone/total
    //set year 9 class data
    sClass.ClassID = yNineClassArray[i]
    sClass.year = 9
    sClass.NoOfStudents = 0
    //generate the class id
    val(yNineClassArray[i][8],empty,ErrCode)
    IF ErrCode != 0 THEN
      BEGIN
        sClass.subjectID = yNineClassArray[i][6]+yNineClassArray[i][7]+yNineClassArray[i][8]
        val(sClass.subjectID[1],empty,ErrCode)
        IF ErrCode == 0 THEN
          BEGIN
            sClass.subjectid = yNineClassArray[i][4]+yNineClassArray[i][5]
          END
      end
    ELSE
      sClass.subjectID = yNineClassArray[i][6]+yNineClassArray[i][7]
    WRITE(classFile, sClass)
    delay(100)
  END
//generate year 12 classes
FOR i = 0 TO 6 DO
  BEGIN
    numberDone= numberDone +1
    percentComplete = numberDone/total
    sClass.ClassId = yTwelveClassArray[i]
    sClass.year = 12
    sClass.NoOfStudents = 0
    sClass.subjectID = yTwelveClassArray[i][5]+yTwelveClassArray[i][6]
    WRITE(classFile, sClass)
  END
reset(classFile)
FOR i = 0 to 50 DO
BEGIN
  READ(classFile, classArray[i])
  classIDArray[i] = classArray[i].classID
END
```

Set year and number of students in that class

Check if the 8th character is empty

If it is empty then the subject id is the last 3 characters

If 2nd character is a digit, if so it catches any year 12 classes in case of failure

Write data to file

Otherwise the subject id is the 6th and 7th characters from the class id

For every year 12 class

Set class id from year 12 class array

Set subject id from 5th and 6th digit from the

Create an array of all class ids for use in other procedures

Procedure to add the student to a class

```
      END


  PROCEDURE addRecordsStudentClassResolver(start, noOfAviablableClasses, maxStudents, ClasesToSortInto, year : LONGINT)
    CONST yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph', '9AT2/Ph',
'9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph', '9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs1', '9D/Mu1', '9D/Gm1')
          yTwelveClassArray : ARRAY[0..6] OF STRING =
('12A/Ph1','12A/Py1','12B/Ph1','12C/Cp1','12C/Py1','12D/Mu1','12D/Cp1')
    BEGIN
      RESET(classFile)
      REPEAT
        randomChoiceClass = RANDOM(noOfAviablableClasses)  //randomly picks classes from the number of available class
that student can be in
        IF year == 9 THEN
          Search.SearchFunctions('L',0,50,yNineClassArray[start+ranDOmChoiceClass],classIDArray,returnArray,empty)
//checks the year 9 to find the class chosen

        ELSE IF year == 12 THEN
            Search.SearchFunctions('L',0,50,yTwelveClassArray[start+ranDOmChoiceClass],classIDArray,returnArray,empty)
//checks the year 12 to find the class chosen
        ELSE
          DisplayErrorScrn.ErrScr
        SEEK(classFile, returnArray[0]) //find the class in the file
        READ(classFile, sClass)
        UNTIL sClass.NoOfStudents < maxStudents //only exit IF there is a place FOR the student in that class
```

Arrays of all classes

Randomly pick a class from available classes

Search the year parsed as a parameter for the class

Get the data about the class from the file

Repeat until it finds a class which is not full

```
        l = 0
    startPoint = returnArray[0]
    FOR l = 0 to (ClasesToSortInto * 2) -1 DO //FOR every class connceted to that one class
      BEGIN
        IF year == 9 THEN
          studentClassResolver.classID = yNineClassArray[start+ranDOmChoiceClass+l]
        ELSE IF year == 12 THEN
            studentClassResolver.classID = yTwelveClassArray[start+ranDOmChoiceClass+l]
        ELSE
          DisplayErrorScrn.ErrScr
        WRITE(studentClassResolverFile, studentClassResolver) //write each student and the class to the resolver file
        //update number of students in that class
        SEEK(classFile, startPoint + l)
        READ(classFile, sClass)
        SEEK(classFile, FilePos(classFile)-1)
        sClass.NoOfStudents = sClass.NoOfStudents + 1
        WRITE(classFile, sClass)
        l = l + 1
      END
  END

  PROCEDURE createClassStudentResolver
    CONST yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph', '9AT2/Ph',
'9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph', '9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs1', '9D/Mu1', '9D/Gm1')
          yTwelveClassArray : ARRAY[0..6] OF STRING =
('12A/Ph1','12A/Py1','12B/Ph1','12C/Cp1','12C/Py1','12D/Mu1','12D/Cp1')
    BEGIN
      RESET(StudentFile)
      RESET(classFile)
      REWRITE(studentClassResolverFile)
      //keep track of how many students on each side of the year
      numberInASide = 0
      numberInBSide = 0
            noTakingFourAlevels = 0

      randomSide = RANDOM(2)
```

For every class which contains the same mebers

Get each class id

Write the data of the student and the class that they should be in

Update number of students in that class

Arrays of all classes

Randomly pick a side

For every student

```
WHILE NOT(eof(StudentFile)) DO //FOR every student
  BEGIN
    numberDone = numberDone+1
    percentComplete = numberDone/total
    READ(StudentFile,student) //get the student data
    studentClassResolver.studentID = student.StudentID
    IF student.year == 9 THEN
      BEGIN
        //pick a ranDOm side without overflowing one side
        IF (numberInASide < 60) AND (numberInBSide < 60) THEN

          ranDOmSide = RANDOM(2)
        ELSE IF (numberInASide < 60) THEN
          ranDOmSide = 0
        ELSE IF (numberInBSide < 60) THEN
          ranDOmSide = 1
        IF ranDOmSide == 0 THEN //a or b side
          BEGIN
              //put students in each set of classes
              numberInASide = numberInASide + 1
              //Science
              addRecordsStudentClassResolver(0, 2, 30, 3, 9)
              //Maths
              addRecordsStudentClassResolver(6, 2, 30, 1, 9)
              //Englishs
              addRecordsStudentClassResolver(8, 2, 30, 3, 9)
              //Pe
              addRecordsStudentClassResolver(14, 2, 30, 1, 9)
          end
          ELSE IF ranDOmSide == 1 THEN //a or b side
            BEGIN
              //Science
              numberInBSide = numberInBSide + 1
              addRecordsStudentClassResolver(16, 2, 30, 3, 9)
              //Maths
              addRecordsStudentClassResolver(22, 2, 30, 1, 9)
              //Englishs
              addRecordsStudentClassResolver(24, 2, 30, 3, 9)
              //Pe
              addRecordsStudentClassResolver(30, 2, 30, 1, 9)
          end
```

Get the student data from the file

Set the student id in the student class resolver record

Pick a random side while making sure that each side always has 60 students in it

Class procedure with data to randomly pick classes to assign students to. And then write the data to the file for students in side a

Class procedure with data to randomly pick classes to assign students to. And then write the data to the file for students in side b

```
        ELSE
          BEGIN
            //error catching FOR ranDOm side assignment
            writeln('A: ',numberInASide, '  B: ',numberInBSide)
            Delay(3000)
            DisplayErrorScrn.ErrScr
          END
      //ranDOmly pick the optional subjects
      FOR a = 0 to 8 DO
        BEGIN
          addRecordsStudentClassResolver(32 + a, 4, 30, 1, 9)
          a = a + 3
        END
    end
  ELSE IF (student.year == 12) THEN // year 12
    BEGIN
      IF noTakingFourAlevels < 5 THEN
        noOfAlevels = RANDOM(10)
      IF noOfAlevels == 0 THEN //taking 4 a lvels
        BEGIN
          noTakingFourAlevels = noTakingFourAlevels + 1
          //block a
          addRecordsStudentClassResolver(0, 2, 20, 1, 12)

          //block b
          addRecordsStudentClassResolver(2, 1, 5, 1, 12)


          //block c
          addRecordsStudentClassResolver(3, 2, 20, 1, 12)


          //block d
          addRecordsStudentClassResolver(5, 2, 20, 1, 12)
        end
      ELSE
        BEGIN
          //block a
          addRecordsStudentClassResolver(0, 2, 20, 1, 12)

          //block c
```

Error catching for if sides don't match up

Iterate through to pick one optional subject from each block.

Making sure that only a max of 5 students can take 4 a levels

Class procedure with data to randomly pick classes to assign students to. And then write the data to the file for students taking 4 a levels. 1 subject per block

Class procedure with data to randomly pick classes to assign students to. And then write the data to the file for students taking 3 a levels. 1 subject per block

```
                addRecordsStudentClassResolver(3, 2, 20, 1, 12)


                //block d
                addRecordsStudentClassResolver(5, 2, 20, 1, 12)

            END
        END
    END
END

PROCEDURE assignTeacherToClass
  BEGIN
    reset(classFile)
    reset(teacherfile)
    REWRITE(teacherClassResolverFile)
    WHILE NOT(eof(classFile)) DO //FOR each teacher
      BEGIN
        numberDone = numberDone+1
        percentComplete = numberDone / total
        //assign one teacher per class
        READ(classFile,sClass)
        READ(teacherfile, teacher)
        teacherClassResolver.teacherID = teacher.teacherID
        teacherClassResolver.classID = sClass.classId
        WRITE(teacherClassResolverFile,teacherClassResolver)
      END
  END
```

For every teacher

Read next class

Read next teacher

Teacher class resolver record teachers id to teacher id read, and class id to class id read. Then write to file

```
PROCEDURE assignTeacherToSubject
  VAR a, b : LONGINT
  BEGIN
    RESET(TeacherFile)
    reset(classFile)
    REWRITE(teacherSubjectResolverFile)
    WHILE NOT EOF(TeacherFile) DO
      BEGIN
        numberDone = numberDone+ 1
        percentComplete = numberDone / total
        READ(TeacherFile,teacher)
        READ(classFile,sClass)
        //find the class the teacher teaches and gets the subject id of that class and puts it in a resolver file
        teacherSubjectResolver.teacherId = teacher.teacherID
        teacherSubjectResolver.subjectId = sClass.subjectID
        WRITE(teacherSubjectResolverFile, teacherSubjectResolver)
      END
    END
```

For each teacher

Read each teacher and class

Teacher subject resolver record teachers id to teacher id read, and subject id to class id subject if of the class that was read. Then write to file

```
BEGIN
  //generates a random seed
  randomize
  //run each PROCEDURE
  initialiseFiles
  loading.startLoading('P',percentComplete)
  CreateTimetable
  addForms
  addStudents
  addTeachers
  createSubjects
  createClasses
  createClassStudentResolver
  assignTeacherToClass
  assignTeacherToSubject
  createRegisters
  createLogInFile
  readln
  closeAllFiles
```

Generate a random seed for all randomly generated numbers

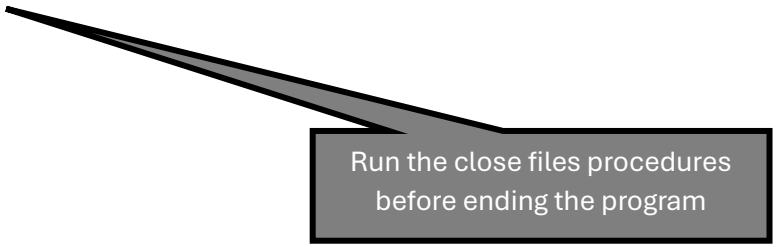
Initialize all starting data

Start the progress bar

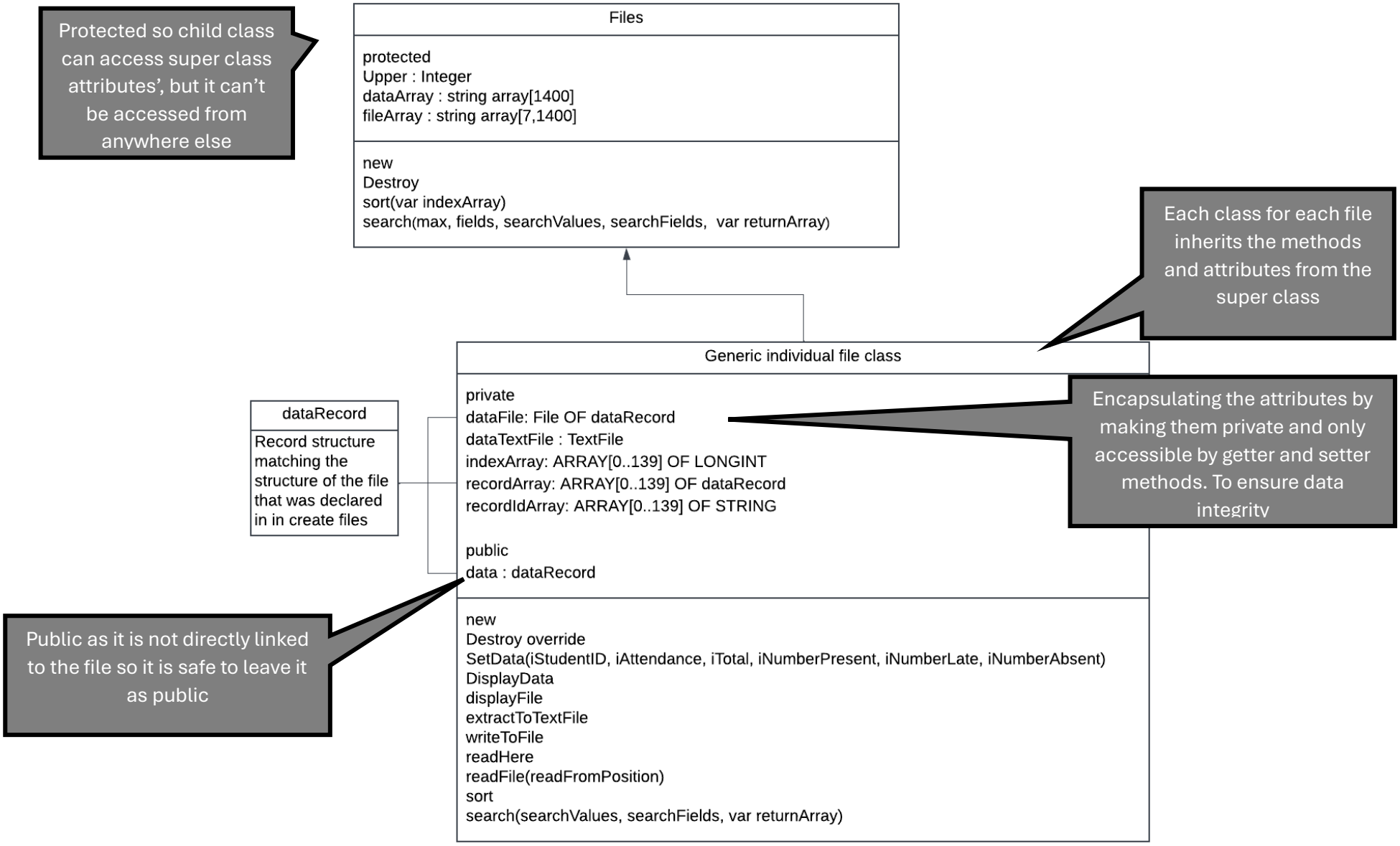
Call each procedure in turn to generate all the data

Pause so the programmer can check everything ran correctly

```
end.
```



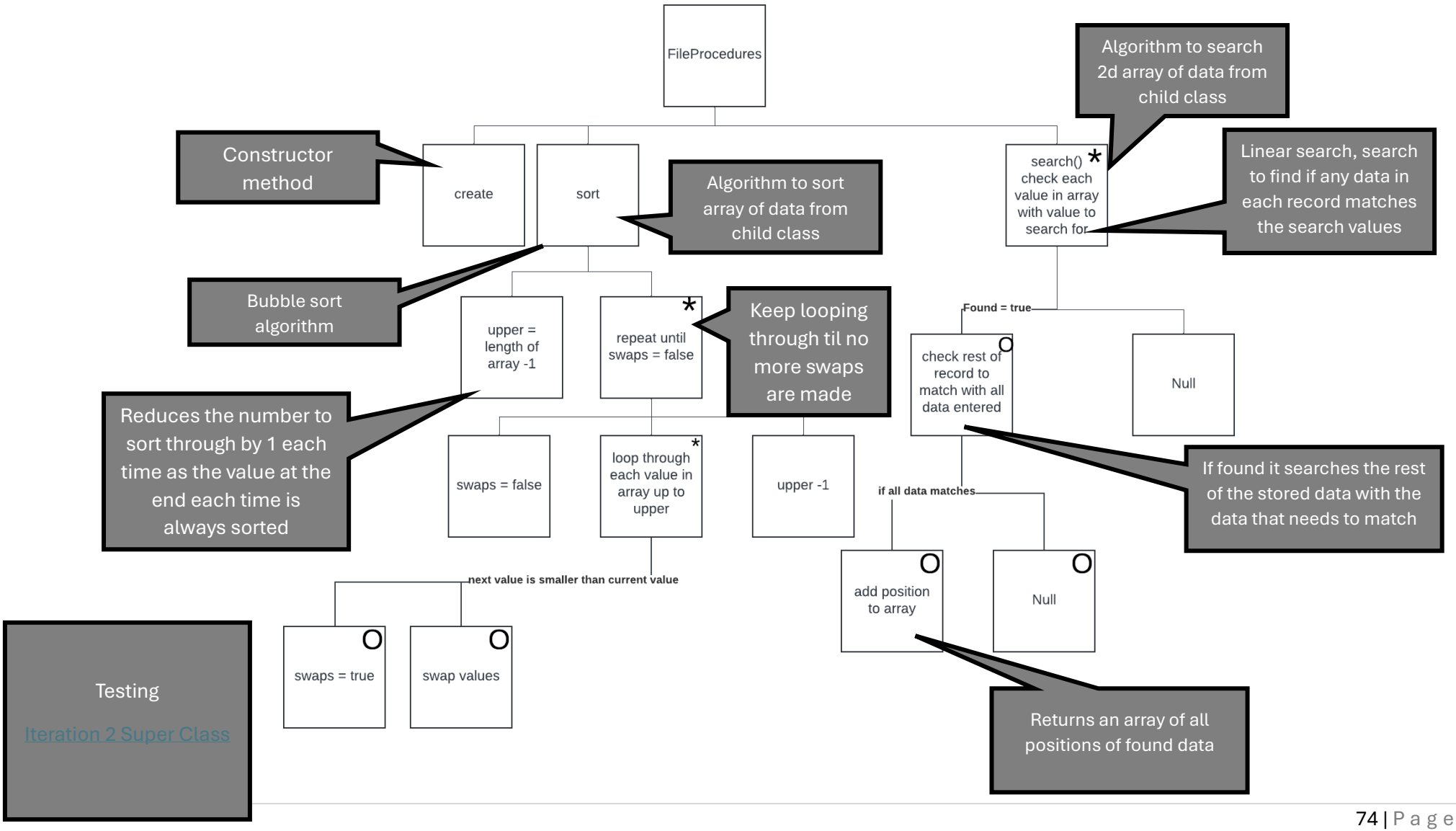
Class Diagram (Unified Modelling Language (UML))



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Files Super Class

Top-Down Design – Files Class



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Pseudocode – Files Class

```
unit FileProcedures
class Files
  protected upper
  protected array dataArray[1400]
  protected array fileArray[7,1400]

  protected procedure new()
  endprocedure
  protected procedure destroy()
  endprocedure

  // Bubble Sort Algorithm
  protected procedure sort(var indexArray)
    upper = upper - 1
    repeat
        Swaps = FALSE
        for i = 0 to upper do
            // Check if the current element is greater than the next element
            if dataArray[i] > dataArray[i + 1] then
                // Swap elements in dataArray
                TempStr = dataArray[i]
                dataArray[i] = dataArray[i + 1]
                dataArray[i + 1] = TempStr
                Swaps = TRUE

                // Swap corresponding elements in indexArray
                TempIndex = indexArray[i]
                indexArray[i] = indexArray[i + 1]
                indexArray[i + 1] = TempIndex
            endif
        next i
        upper = upper - 1
    until !Swaps OR upper < 0
  endprocedure

  // Linear Search Algorithm
```

Declare as protected so that only this class and child classes can access the attributes

Create constructor and destructor

Generic method to sort arrays of the files

Bubble sort

Checks the data from the data array

Swaps the data around

Swaps the indexes of each piece of data to keep track of changes to pass back

Arguments for length of array to search, number of fields, what value to search for in each field, which fields to be searched

```
  protected procedure sort(max, fields, searchValues, searchFields, var returnArray)
    a = -1
    returnIndex = 0
    // Check every value in the 2D array
    while a < max do
        a = a + 1
        // Check each item in the record
        for b = 0 to fields - 1 do
            data = toLower(fileArray[b][a])
            // Check if one piece of data matches
            if data == toLower(searchValues[0]) then
                found = TRUE
                // Check each piece of data in the record
                for c = 0 to LEN(searchFields) - 1 do
                    data = toLower(fileArray[searchFields[c]][a])
                    // If not all data matches, continue searching
                    if toLower(searchValues[c]) != data then
                        found = FALSE
                        break
                    endif
                next c
                // If all data matches, add index to returnArray
                if found == TRUE then
                    returnArray[returnIndex] = a
                    returnIndex = returnIndex + 1
                    break
                endif
            endif
        next b
    endwhile
  endprocedure
endclass
```

Only search the part of the 2d array that needs to be searched

Check each item in each record to see if one match is initially found

Tries to find any data in the record that doesn't match with the data that is being searched for

If any data doesn't stop searching that record and move on

If all the data matches write its index to the return array.

End search of that record

Child Class

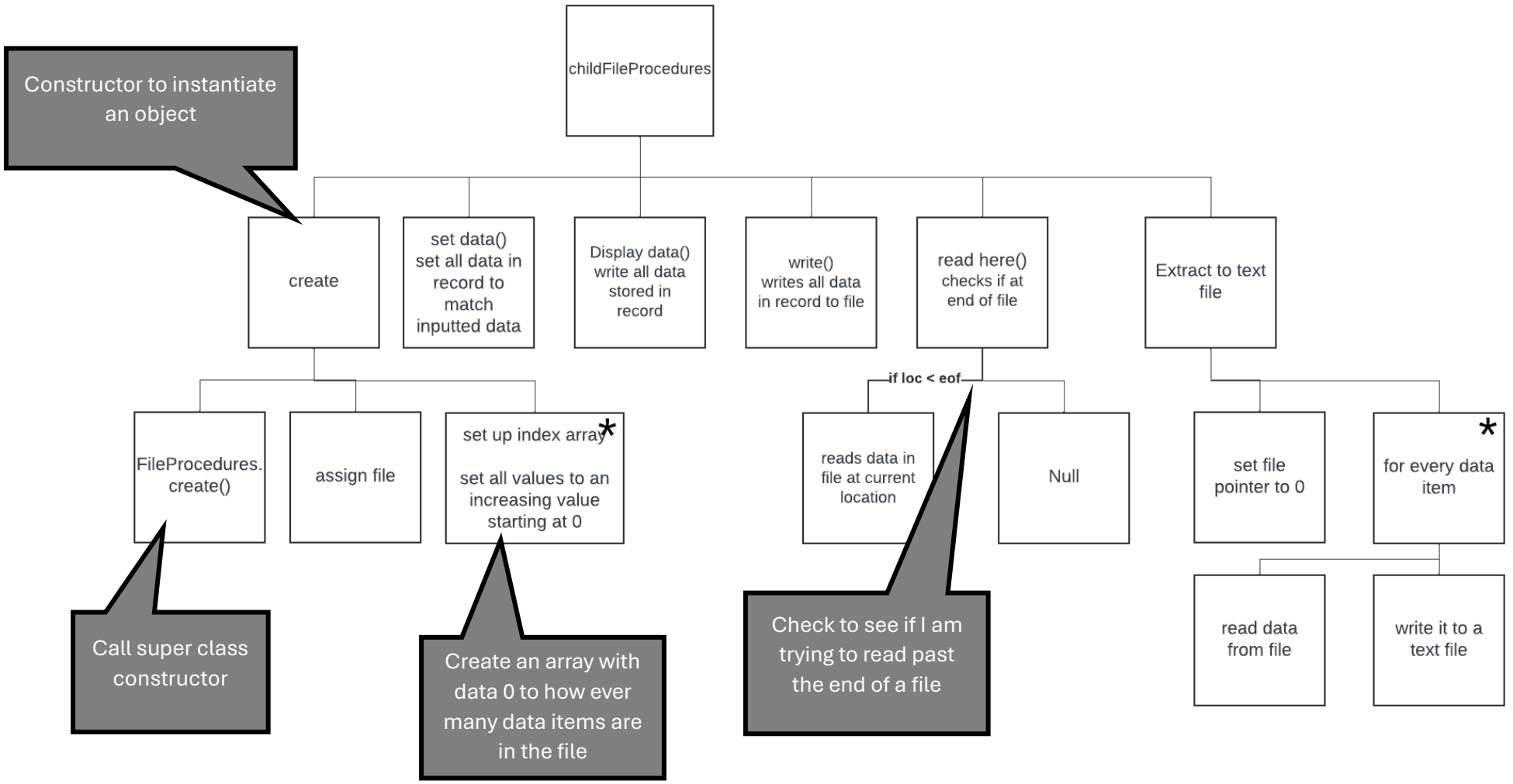
Explanation of design generalization

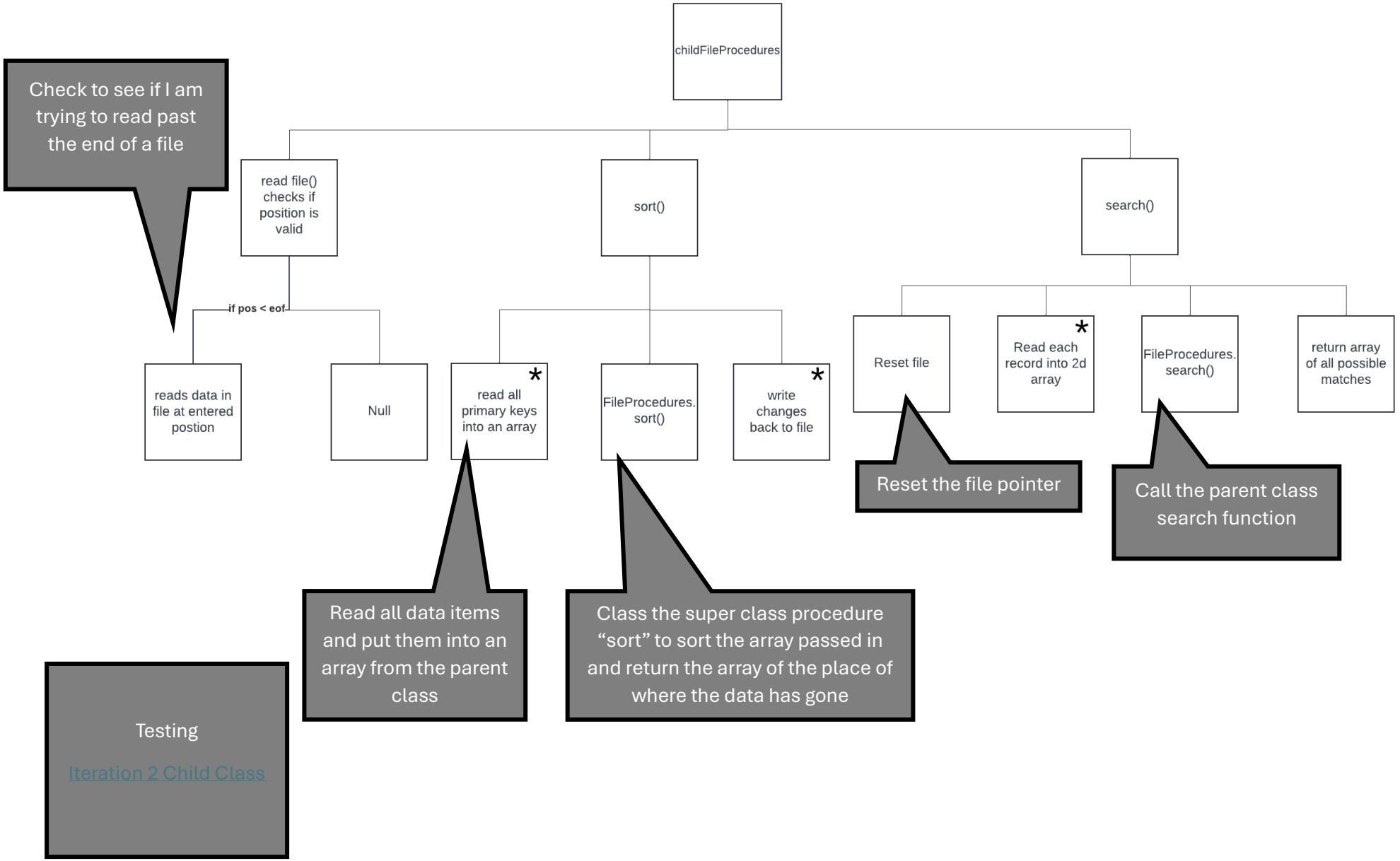
This is a general structure for the child class, which is specific to each file, but only differs from this general structure in these ways:

- The record structure the data is read into
- The specifics of how each record must be written to the screen and to the text file
- The data that is used as the field for sorting
- Casting for storing the data in the 2d array for searching

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Top-Down Design – Child Class




childFileProcedures
read file() checks if position is valid
sort()
search()
if pos < eof
reads data in file at entered postion
Null
read all primary keys into an array
*
FileProcedures. sort()
write changes back to file
*
Reset file
Read each record into 2d array
*
FileProcedures. search()
return array of all possible matches
Check to see if I am trying to read past the end of a file
Reset the file pointer
Call the parent class search function
Read all data items and put them into an array from the parent class
Class the super class procedure “sort” to sort the array passed in and return the array of the place of where the data has gone
Testing
Iteration 2 Child Class

Pseudocode – Child Class

```
unit generalChildClass

USES crt, sysutils, FileProcedures;

class childClass inherits Files

  private dataFile
  private dataTextFile
  private array indexArray[length of data file]
  private array recordArray[length of data file]

      public data = record(type in which is defined in the ERD)

      public procedure new
            super.new
            dataFile = openRead("fileLocation\\filename.dta")
            dataTextFile = openRead("fileLocation\\filenameTxtOut.txt")
            RESET(dataFile);
            RESET(dataFile);


            for i = 0 to (length of data file) do
                  indexArray[i] = i
            next i

      endproceudure

      public procedure Destroy
            dataFile.close()
            dataTextFile.close()
            super.Destroy
      endproceudure
```

Class definition inheriting methods and attributes from the file (super)(File Class Top Down Design) class

Declare attributes

Public so it can be access outside the class, but still encapsulated as the data in the file is protected by getter and setter methoids

Constructor

Assign files

Set up index array

Destructor to clear memory, must destroy the parent class as well and close the files to keep any changes that

```
public procedure SetData(each record field as parameter)
      data.field1 = arg1
      data.field2 = arg2
      data.field3 = arg3
      // Continue for other fields...
endproceudure

public procedure displayData
      print(data.field1)
      print(data.field2)
      print(data.field3)
      // Continue for other fields...
endproceudure

public procedure displayFile
      while !data.endOfFile do
            dataFile.read
            print(data.field1)
            print(data.field2)
            print(data.field3)
            // Continue for other fields...
      endwhile
endproceudure

public procedure extractToTextFile
      while !data.endOfFile do
            dataFile.read
            dataTextFile.writeLine(data.fi
            dataTextFile.writeLine(data.fi
            dataTextFile.writeLine(data.field3)
            // Continue for other fields...
      endwhile
endproceudure
```

Procedure to update the record

Procedure to write each file out

Procedure to display the whole file to the screen

Procedure to write all the data to a text file

```
public procedure writeToFile
      dataFile.write(data)
endproceudure

public procedure readHere
      if FilePosition(dataFile) < FileSize(dataFile) then
            data = dataFile.read()
      endif
endproceudure

public procedure readFile(readFromPosition)
      if readFromPosition < FileSize(dataFile) then
                  SEEK(dataFile,readFromPosition);
                  data = dataFile.read()
      endif
endproceudure

public procedure sort
      dataFile.reset()
      upper = numberOfRecordsInFile

      // Read data into arrays
      for a from 0 to (length of data file) do
                  dataFile.read(data)
                  recordArray[a] = data
                  dataArray[a] = data.fieldToSearch
      next a

      // Sort the data
      Files.sort(indexArray)

      // Write sorted data back to the file
      dataFile.reset()

      for a from 0 to (length of data file) do
                  dataFile.write(recordArray[indexArray[a]])
                  indexArray[a] = a
```

Write the data record to the file

Get the file data at the file pointers current location, if it is not at the end

Get the file data at the location from the parameter, if it is still in the file

Sets the upper limit for the array to be sorted

Get the data into 2 arrays. 1 to sort and the other as array of original data

Class the super sort procedure

Write the sorted data back by getting index of the original data in the sorted order to writing to the file

```
        next a
    endproceudure

     procedure classFile.search(searchValues, searchFields, returnArray)
    dataFile.reset()
    // Populate the fileArray with data and prepare returnArray
    for a from 0 to (length of data file) do
        dataFile.read(data)
        fileArray[0][a] = toLower(data.field1)
        fileArray[1][a] = str(data.field2)
        fileArray[2][a] = toLower(data.field3)
        fileArray[3][a] = str(data.field4)
        returnArray[a] = -1
    next a

    // Perform the search
    Files.search((length of data file) +1, 4, searchValues, searchFields, returnArray)
     endprocedure

endclass
```

Reset the index array

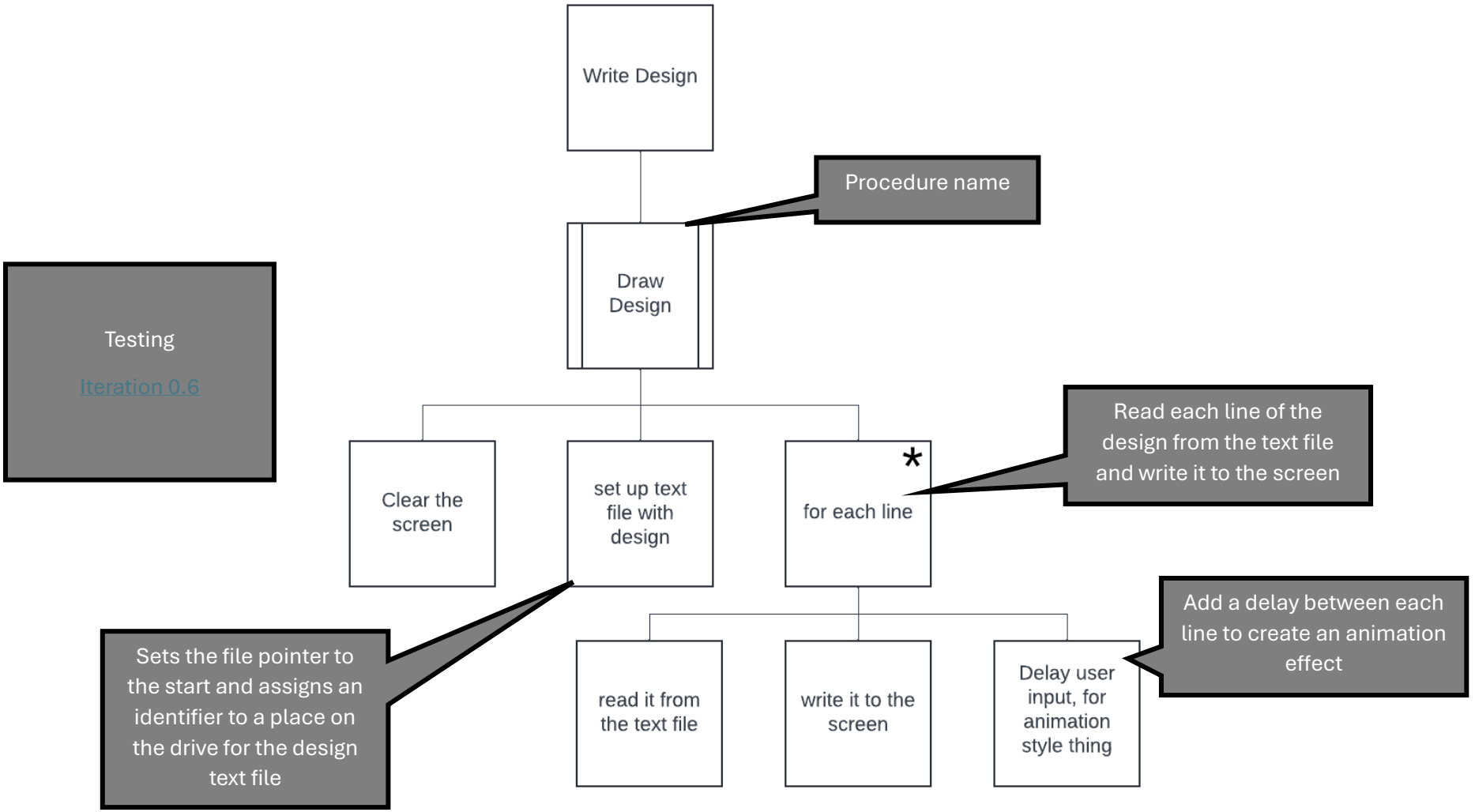
Sort all the data into string format and store it in a 2d array to be searched

Perform the search by calling the parent class search method

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Write Design

Top-Down Design – Write Design



Pseudocode – Write Design

```
Unit writeDesign

Uses crt, sysutils, vputils

procedure DrawDesign(path, delayWrite)
    // Initialize the file and screen
    HideCursor()
    ClearScreen()
    DesignTextFile = openRead(path)

    // Read and display each line with a delay
    for a from 0 to 28 do
        DesignTextFile.readLine(Character)
        print(Character)
        Delay(delayWrite)
    next a

    ShowCursor()
Endprocedure
```

Set up screen conditions and file location

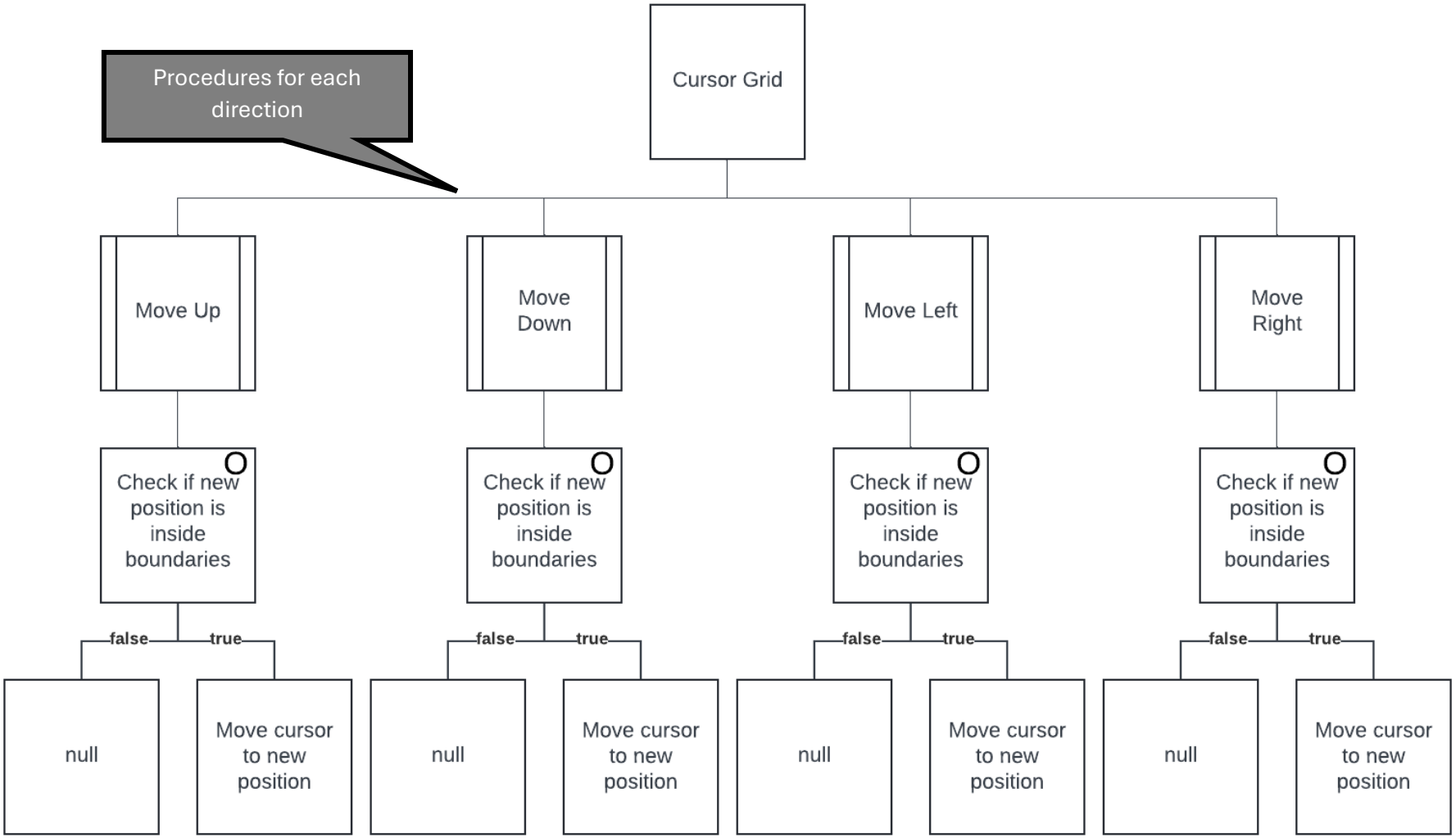
Write each line to the screen

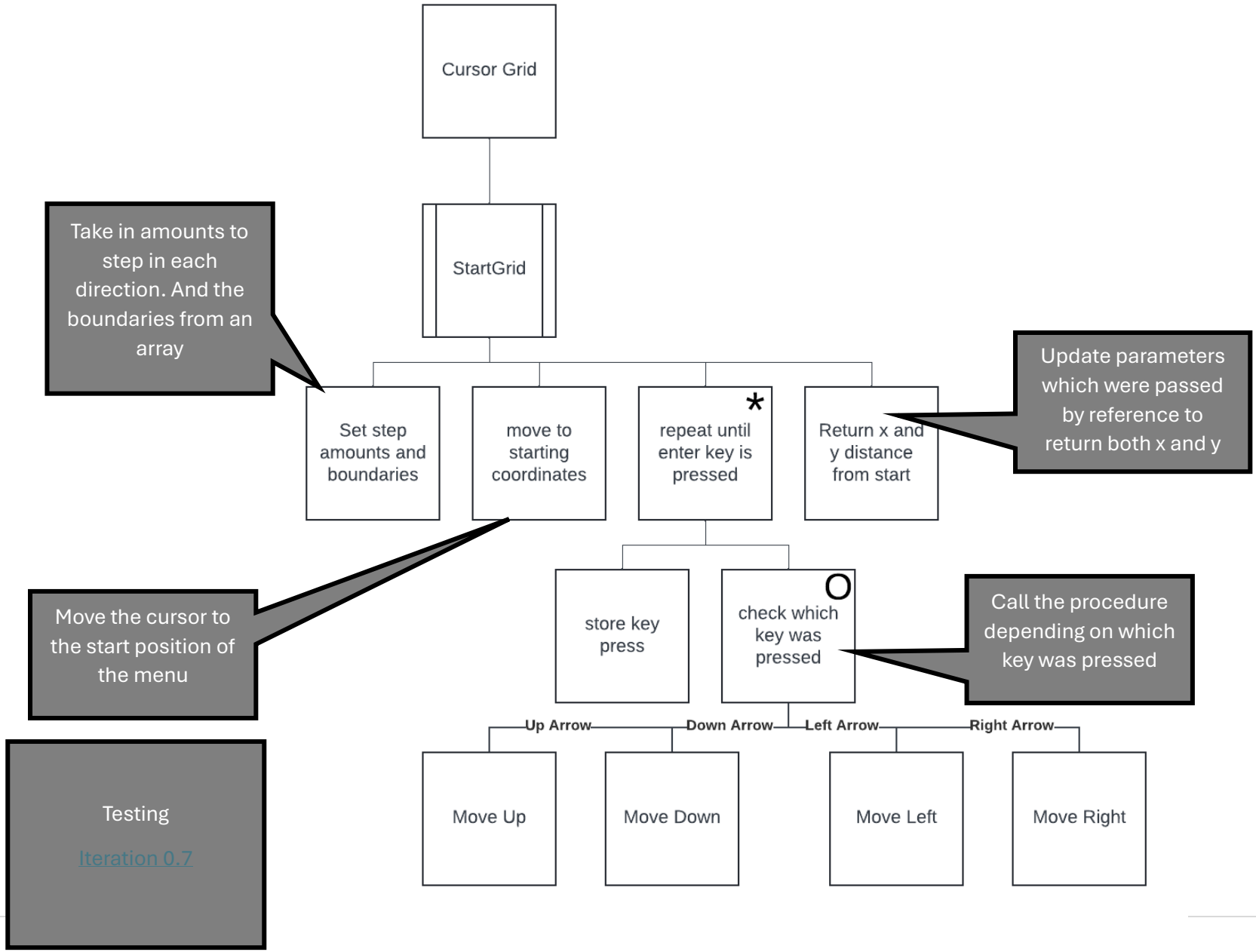
Delay for animation

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Cursor Grid

Top-Down Design – Cursor Grid




Cursor Grid
StartGrid
Take in amounts to step in each direction. And the boundaries from an array
Set step amounts and boundaries
move to starting coordinates
repeat until enter key is pressed
*
Return x and y distance from start
Update parameters which were passed by reference to return both x and y
Move the cursor to the start position of the menu
store key press
check which key was pressed
O
Call the procedure depending on which key was pressed
Up Arrow
Down Arrow
Left Arrow
Right Arrow
Move Up
Move Down
Move Left
Move Right
Testing
Iteration 0.7

Pseudocode – Cursor Grid

```
Unit CursorGrid

// Procedure to Move the Cursor Up
procedure moveUp
    if WhereY() - stepY >= minY then
        movePos = WhereY() - stepY
        GoToXY(WhereX(), movePos)
    endif
endprocedure

// Procedure to Move the Cursor Down
procedure moveDown
    if WhereY() + stepY <= maxY then
        movePos = WhereY() + stepY
        GoToXY(WhereX(), movePos)
    endif
endprocedure

// Procedure to Move the Cursor Left
procedure moveLeft
    if WhereX() - stepX >= minX then
        movePos = WhereX() - stepX
        GoToXY(movePos, WhereY())
    endif
endprocedure

// Procedure to Move the Cursor Right
procedure moveRight
    if WhereX() + stepX <= maxX then
        movePos = WhereX() + stepX
        GoToXY(movePos, WhereY())
    endif
endprocedure

// Procedure to Initialize and Control the Grid Movement
procedure startGrid(moveStepX, moveStepY, startX, startY, boundaries, REF x, REF y)
```

Validating if the direction the user wants to move the cursor is still within the menu

Validating if the direction the user wants to move the cursor is still within the menu

Validating if the direction the user wants to move the cursor is still within the menu

Validating if the direction the user wants to move the cursor is still within the menu

```
    stepX = moveStepX
    stepY = moveStepY
    minX = boundaries[0]
    minY = boundaries[1]
    maxX = boundaries[2]
    maxY = boundaries[3]
    GoToXY(startX, startY)
    Repeat //repeat until enter is pressed
        keyPress = readKey()
            switch keyPress
                case 'UpArrow':
                    moveUp()
                case 'DownArrow':
                    moveDown()
                case 'LeftArrow':
                    moveLeft()
                case 'RightArrow':
                    moveRight()
            endswitch
    until keyPress = CHR(13)
           //return end position
    if stepX != 0 then
       x = round((WhereX() - startX) / stepX)
            else
            x = startX
            endif

            if stepY != 0 then
            y = round((WhereY() - startY) / stepY)
            else
            y = startY
     endif
endprocedure
```

Set step controls and boundaries for the cursor

Move cursor to start position

Determines which arrow key was pressed

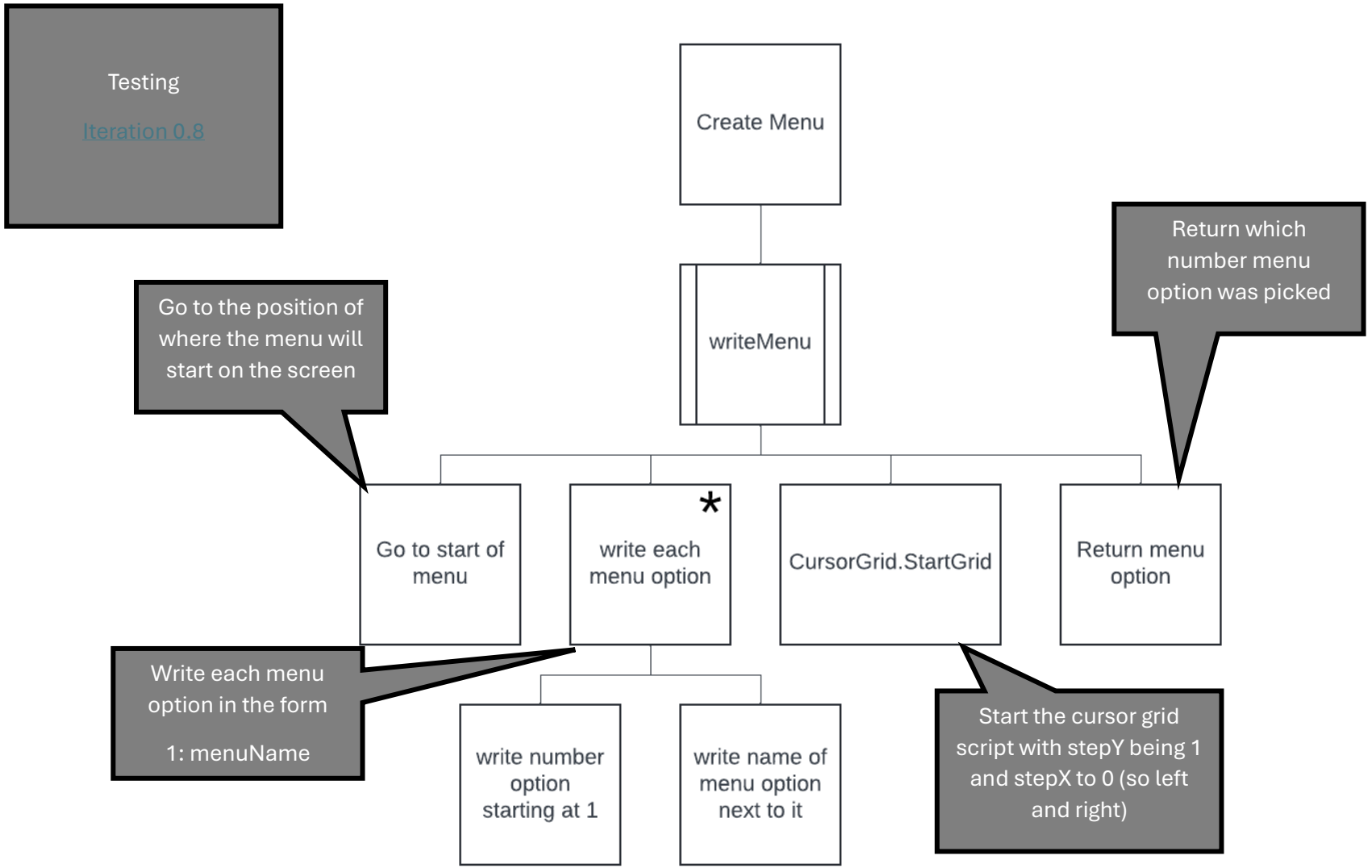
If the enter key is pressed end loop

Return the distance from the start as a number of steps from the start position

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Create Menu

Top-Down Design – Create Menu



Pseudocode – Create Menu

```
Unit createMenu
   // Function to Write and Handle a Menu
   function writeMenu(startPos, numberOfChoices, menuChoices, optionStart)
       HideCursor()


       // Display menu choices
       for a from 0 to numberOfChoices - 1 do
                 GoToXY(startPos[0], startPos[1]+a)
           print(a + 1 + ": " + menuChoices[a])
       endfor

       ShowCursor()
       Delay(100)

       // Start the grid for user selection
       CursorGrid.startGrid(0, 1, startPos[0], startPos[1] + optionStart,
                            [startPos[0], startPos[1], startPos[0], numberOfChoices + startPos[1] - 1],
                            x, y)

       ClearScreen()
       return optionStart + y
     endfunction

endunit
```

Go to start position

Write each menu option out

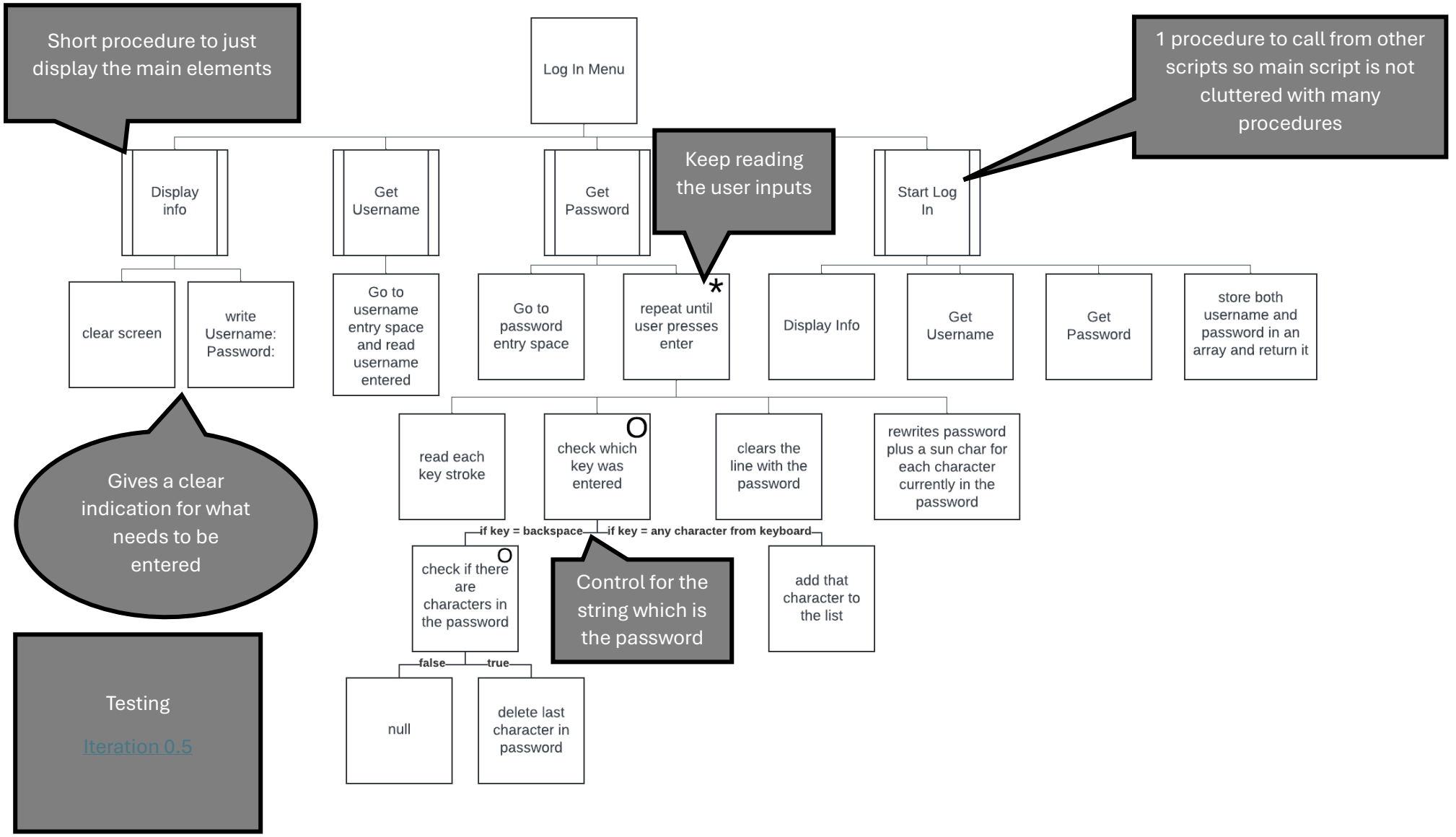
Start the cursor grid script

Return the option that was picked

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Log In Menu

Top-Down Design – Log in Menu



Pseudocode – Log in Menu

```
Unit LogInScreen
   // Procedure to Get the Username
   procedure GetUsername
       GoToXY(11, 1)
       Username = input() // Read the username input
   endprocedure

   // Procedure to Get the Password
   procedure GetPassword

       GoToXY(11, 2) // Move cursor to the password position
       i = 0
       Password = "" // Initialize password as an empty string
       HideCursor()

       // Loop to process key inputs until Enter is pressed
       repeat
           key = readKey()

           // If the key is Backspace and there are characters to delete
           if key = CHR(8) then
               if i > 0 then
                   Password = deleteChar(Password, i) // Remove the last character
                   i = i - 1
               endif
           // If the key is a valid printable character
           elseif (key >= CHR(33) and key < CHR(127) and key != CHR(13)) OR key = CHR(156) then
               Password = Password + key
               i = i + 1
           endif



           // Update the password line with masked characters
           GoToXY(1, 2)
```

Simple procedure to just read the user entered username

Create an empty string which is the current password

Reads each key press individually to compare each one and to track length so characters can be written to hide actual characters typed

Test if the backspace is

Make sure there are still characters in the password

Delete the last character in the password and decrease the length

Limits the user inputs to only certain characters and make sure its not “enter”. (See appendix (add part) for reference for all characters)

Rewrites the whole line with correct number of ☼ to hide the user entered password for security

```
            DelLine()
            write("Password: ")
            for a from 1 to i do
                write("☼") // Masked character representation
            next a
        until key = CHR(13) // Loop until Enter key is pressed

        writeln()
        ShowCursor()
    endprocedure

    // Procedure to Display Initial Information
    procedure DisplayInfo
        ClearScreen()
        writeLine("Username: ")
        writeLine("Password: ")
        GoToXY(11, 1) // Position cursor for username entry
    endprocedure

    // Main Login Procedure
    procedure startLogIn(userPassArrayr : byRef)
        DisplayInfo() // Show initial display
        GetUsername() // Capture the username
        GetPassword() // Capture the password
        ClearScreen() // Clear the screen for the next task
        userPassArrayr[0] = Username // Store the username in the array
        userPassArrayr[1] = Password // Store the password in the array
    endprocedure

endunit
```

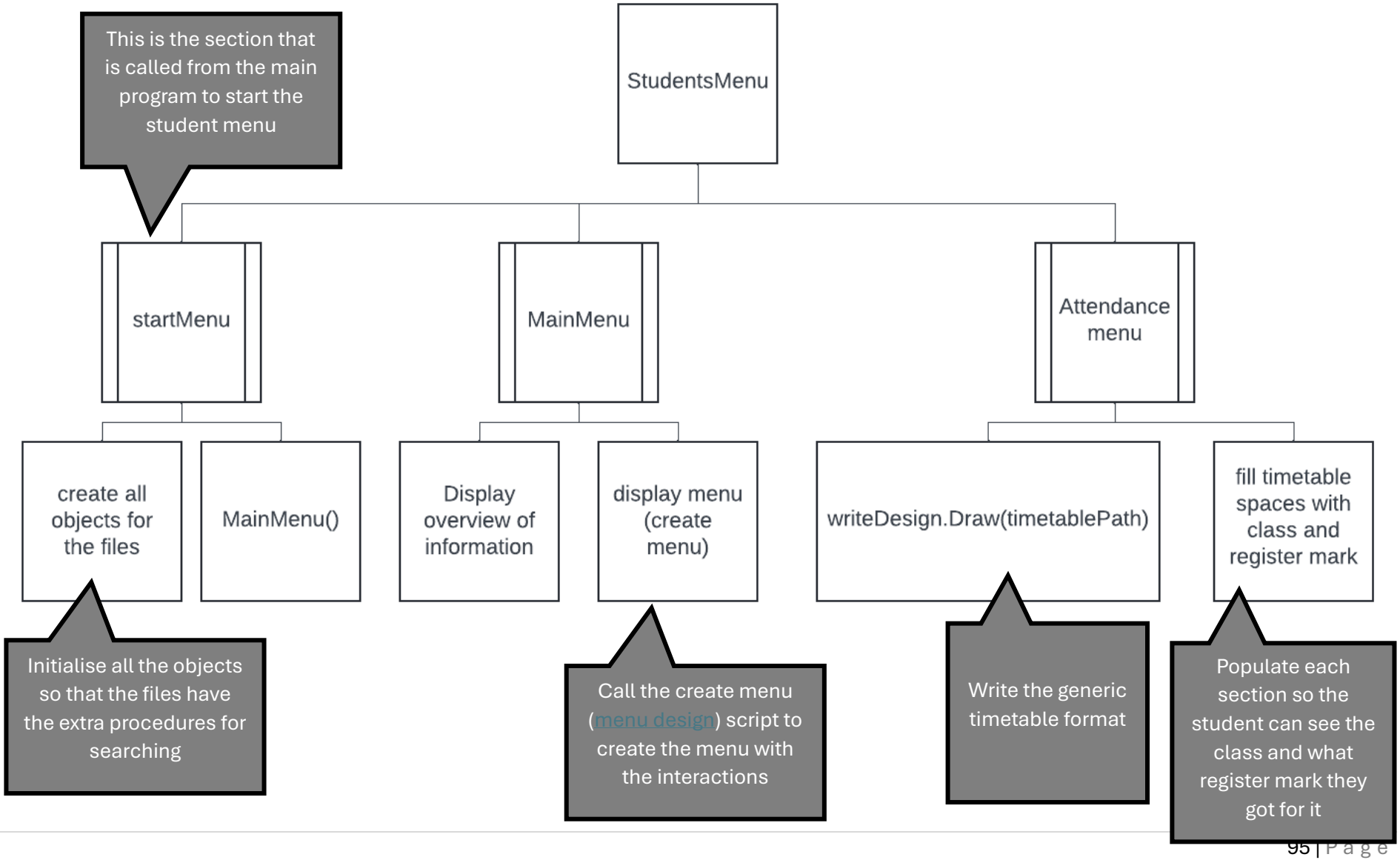
Writes the basic menu to the screen

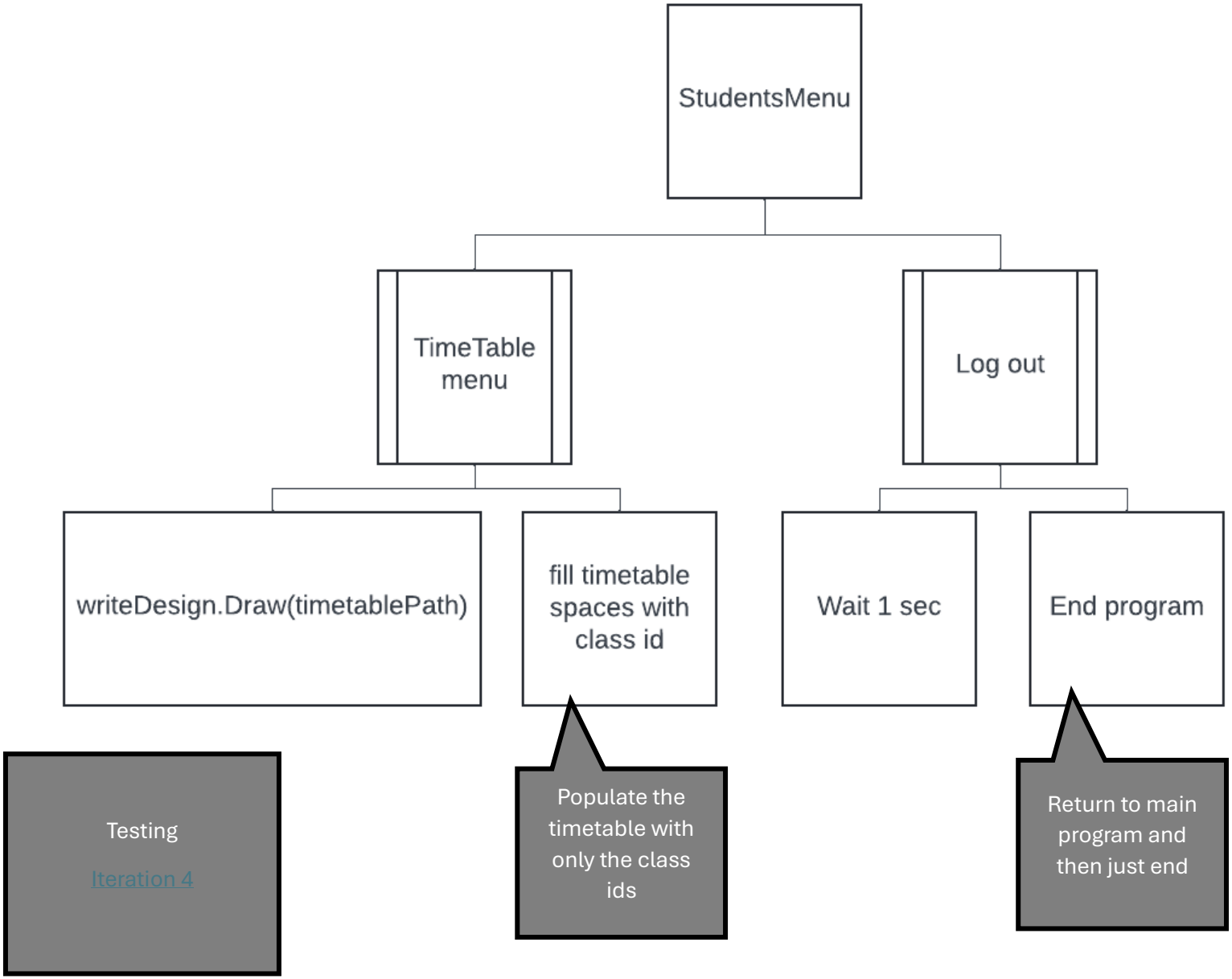
Immediate clear indication on what is happening and what the user needs to do

Call each procedure and return the entered data back to where it was called from

Student's Menu

Top-Down Design – Students Menu




StudentsMenu
TimeTable menu
Log out
writeDesign.Draw(timetablePath)
fill timetable spaces with class id
Wait 1 sec
End program
Testing
Iteration 4
Populate the timetable with only the class ids
Return to main program and then just end

Pseudocode – Students Menu

```
UNIT StudentsMenu

uses crt, sysutils, VPUtils, studentProcedures, timeTableProcedures,
       attendanceProcedures, studentClassProcedures, createMenu, writeDesign

// Procedure AttendanceMenu
procedure AttendanceMenu
    writeLine("Attendance")
    input()
endprocedure

// Procedure CompleteTimeTable
procedure CompleteTimeTable(inputDate)
    if animationTimeTable == TRUE then
        timeDelay = 100
    else
        timeDelay = 0
    endif

    writeDesign.DrawDesign("C:\\ComputerScienceCoursework\\Program\\files\\TimeTableLayout.txt", timeDelay)
    CurrentDate = inputDate
    day = DayOfWeek(CurrentDate - 1) - 1
    HideCursor()

    // Write dates with current date highlighted
    for a from 0 to 4 do
        GoToXY(5, 7 + a * 4)
        if CurrentDate - day + a == Date then
            TextColor(YELLOW)
        else
            TextColor(WHITE)
        endif
        write(DateToStr(CurrentDate - day + a))
        Delay(timeDelay)
    Next a
    TextColor(WHITE)
```

Procedure to show attendance, not enough time to complete

All sub programs references so that the routines can be used

Procedure to fill the empty timetable

Set the animation time if there should be an animation

Get the day in form as Monday is day 0

Write all the dates in the timetable highlighting all the current date in yellow

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

```
// Calculate if it is week A or week B
NumberOfDays = 0
while StrToDateTime("13/09/2024") + NumberOfDays != CurrentDate - day + a - 1 do
    NumberOfDays = NumberOfDays + 1
endwhile
NumberOfDays = NumberOfDays DIV 7
GoToXY(7, 2)
if NumberOfDays MOD 2 == 0 then
    week = "A"
else
    week = "B"
endif
write(week)

// Write lessons in timetable
stringStudentId = str(studentID)
studentClassC.search([stringStudentId], [1], studentClassReturnArray, TRUE)
b = 0

while studentClassReturnArray[b] != -1 do
    studentClassC.readFile(studentClassReturnArray[b])
    for a from 0 to 320 do
        timetableReturnArray[a] = -1
    next a
    timetable.search([studentClassC.data.ClassID], [2], timetableReturnArray, TRUE)
    c = 0

    while timetableReturnArray[c] != -1 do
        timetable.readFile(timetableReturnArray[c])


        if week = "A" then
            if timetable.data.day < 6 then
                GoToXY(10 * timetable.data.L                ble.data.Day - 1) + 7)
                write(Trim(timetable.data.ClassID))
                Delay(timeDelay)
```

Calculate if it is week a or b by finding number of weeks since start of year and seeing if it is an odd or even number

Find every class that the student is in

For each of those classes

Find every time that class in the timetable file

For each time the class is check If the current week is week a check to see if that class exists in week a

If it is week B then make sure that class at that time is in week B

Go to the correct space in the timetable and write it in

```
                endif
            else
                if timetable.data.day > 5 then
                    GoToXY(10 * timetable.data.LessonNo + 20, 4 * (timetable.data.Day - 6) + 7)
                    write(Trim(timetable.data.ClassID))
                    Delay(timeDelay)
                endif
            endif
            c = c + 1
        endwhile
        b = b + 1
    endwhile

    for a from 0 to 4 do
        GoToXY(21, 4 * a + 7)
        write(student.data.Form)
        Delay(timeDelay)
    Next a

    animationTimeTable = FALSE
endprocedure

// Procedure TimeTableMenu
procedure TimeTableMenu
    StartDate = Date
    CompleteTimeTable(StartDate)
    input()
endprocedure


// Procedure LogOutMenu
procedure LogOutMenu
    writeLine("Logging out")
    Timetable.free
    studentClassC.free
    student.free
    attendance.free
```

Write the form of that Student in

Set animation to false so it no longer animates the drawing

Pass the date the timetable should contain in case it needs to change from current date

Write logging out then wait a second as it will exit all so the program will terminate

Free all objects from memory so it closes without any incomplete changes in memory

```
    Delay(1000)
endprocedure

// Procedure Main Menu
procedure mainMenu
    GoToXY(0, 0)
    choice = 0

    // Get student and attendance data at the start
    student.readFile(studentID)
    attendance.readFile(studentID)

    // Main menu loop
    repeat
        ClearScreen()
        HideCursor()
        writeLine("Welcome " + student.data.fname + " " + student.data.sname)
        GoToXY(1, 3)
        writeLine("Attendance: " + format("%.2f", attendance.data.attendance * 100) + "%")

        choice = createMenu.writeMenu([1, 5], 3, mainMenuOptions, choice)

        switch choice
            case 0:
                AttendanceMenu()
            case 1:
                TimeTableMenu()
            case 2:
                LogOutMenu()
        endswitch
    until choice = 2
endprocedure

// Procedure Start Student
procedure startStudent(inputStudentId)
    animationTimeTable = TRUE

    for a from 0 to 1400 do
```

Main menu function while loops until logout is selected

Get the students main and attendance data before the loop so it is only retrieved once and only once

Write welcome message and attendance

Start the menu option with options declared at the start

Determine which choice was made and call appropriate routine

Set animation to true for the first time it is run

Pre populate this as the search will always yield the same result so no need to flush out previous data

```
        studentClassReturnArray[a] = -1
    next a

    timetable = TimeTableFile.create()
    studentClassC = StudentClassFile.create()
    studentID = inputStudentId
    attendance = AttendanceFile.create()
    student = StudentFile.create()

    mainMenu()
endprocedure

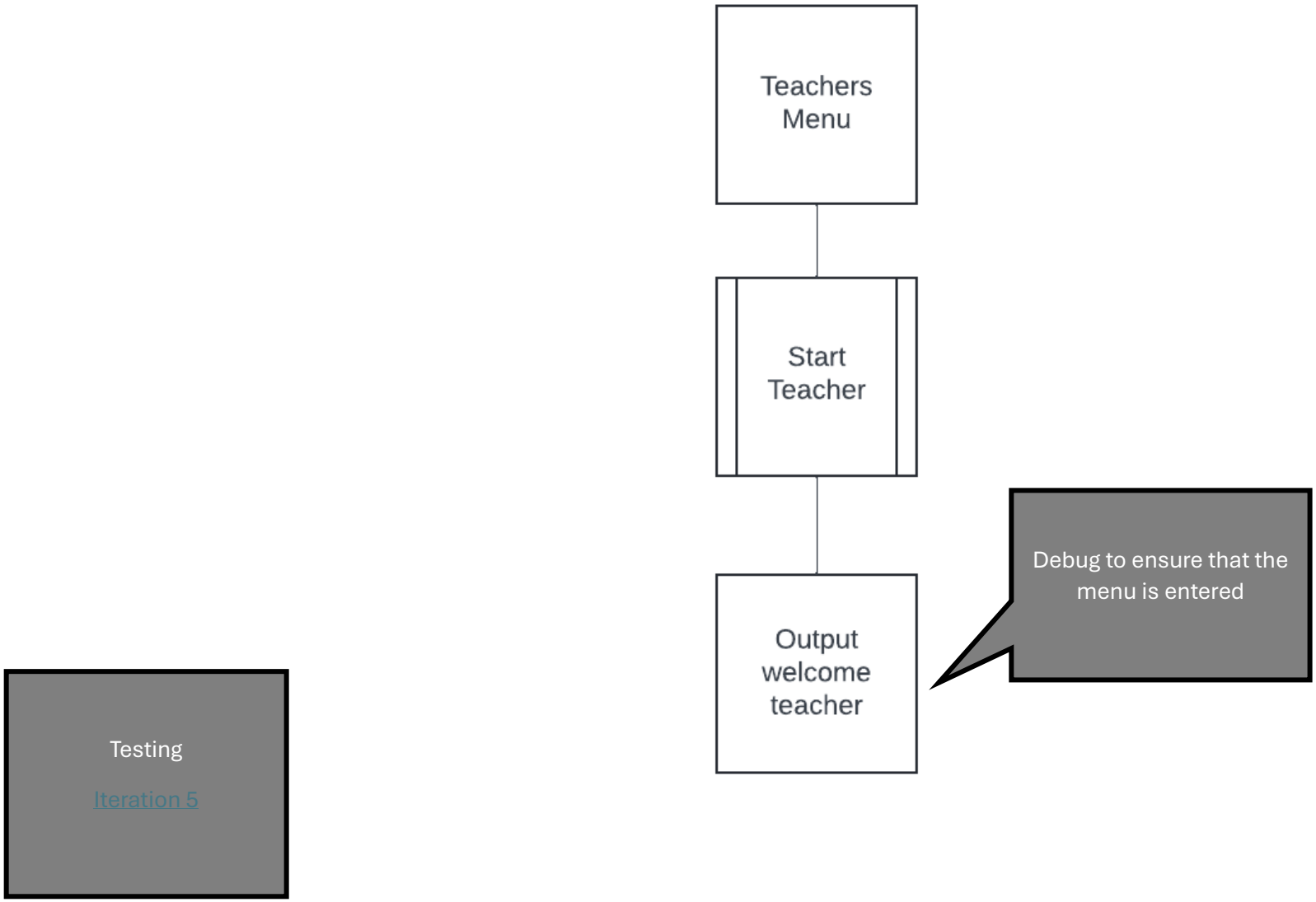
END UNIT
```

Instantiate all objects of files so these files can be used in the unit

Start the main menu

Teacher's Menu

Top-Down Design – Teachers Menu



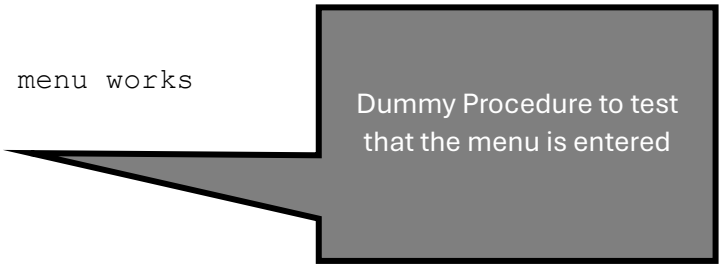
Pseudocode – Teachers Menu

```
UNIT TeachersMenu

IMPORT crt, sysutils

// Procedure: Start Teacher
procedure startTeacher
    // Debug message to confirm the menu works
    writeLine("Welcome teacher")
    input()
endprocedure

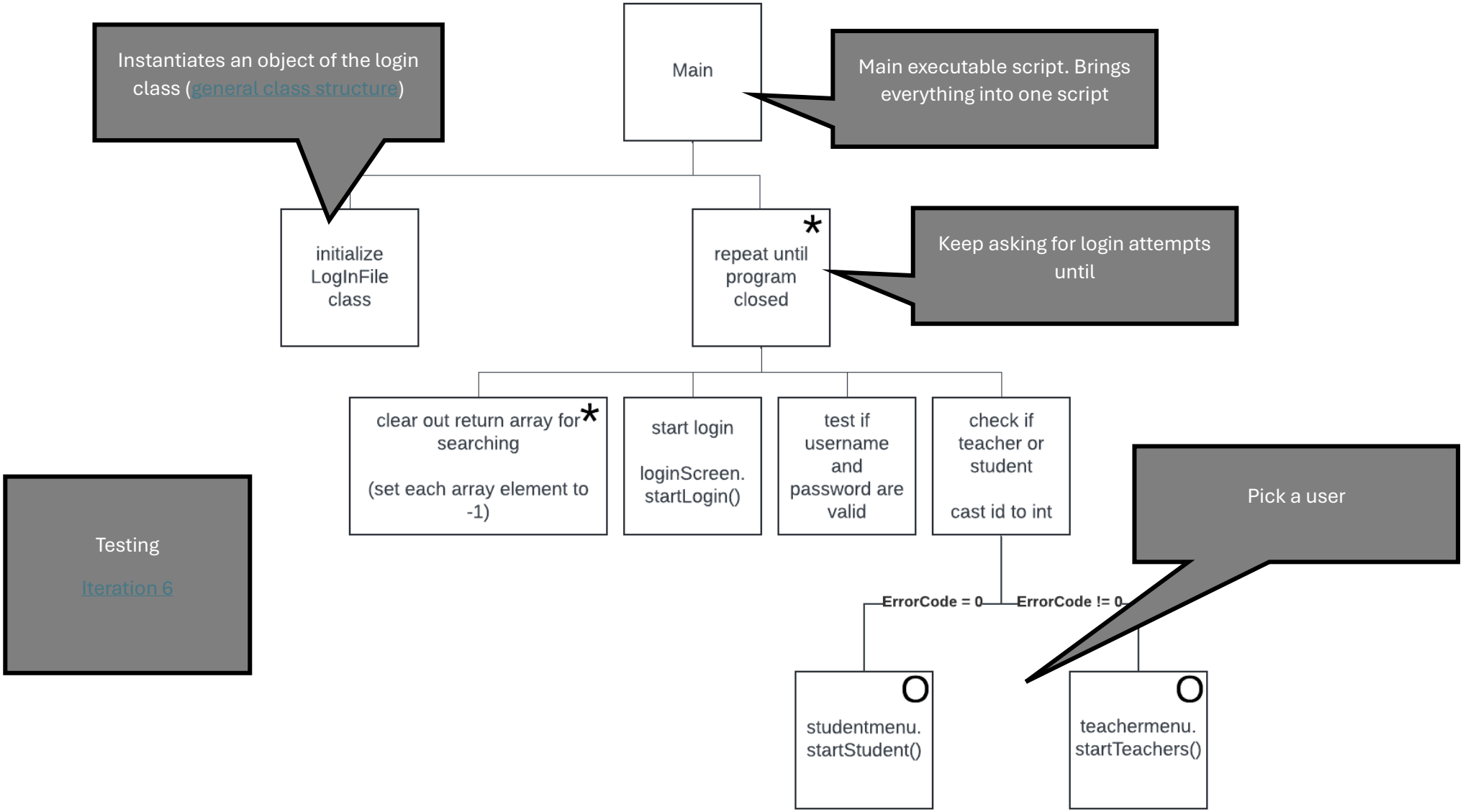
END UNIT
```



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Main

Top-Down Design – Main



Pseudocode – Main

```
function start() : STRING
    // Initialize login file
    logIn = LogInFile.create()
    logIn.sort()

           userIndex[i] = -1

    loggedIn = FALSE

    // Login loop
    while loggedIn = FALSE do
        ClearScreen()

        // Call login screen for user input
        LogInScreen.startLogIn(userDetailsArray)
        logIn.search(userDetailsArray, [1, 2], userIndex)

        // Simulate loading process
        loading.startLoading("L", percentDone)
        Delay(1000)
        loading.silentKillLoading()
        Delay(100)

        // Validate username and password
        if userIndex = -1 then
            writeLine("Username or Password is incorrect.")
            input() // Pause for feedback
        else
            loggedIn = TRUE

            // Retrieve user information
            logIn.readFile(userIndex)
            userId = Trim(logIn.data.PersonID)

            // Determine if the user is a teacher or student
            val(userId, intUserID, ErrCode)
```

Instantiates an object of the login class (general class structure)

Sets user position in file index to an impossible known value

Calls the log in screen procedure (Log In Procedure)

Search the log in file using the search procedure in the child class for the details entered to validate in the information entered is valid

Loading text for user friendliness

Checks to see if a position was found with the data entered

Gets the user id to test if it is a teacher or student

If it is not a number, then user is a teacher else a student

```
            if ErrCode != 0 then
                return "T" // Teacher
            else
                return "S" // Student
            endif
        endif
    endwhile

    // Free login object
    logIn.free()
endfunction

// Main Logic
userType = start()
if userType = "T" then
    // Call teacher menu
    TeachersMenu.startTeacher()
elseif userType = "S" then
    // Call student menu
    StudentsMenu.startStudent(intUserID)

endif
```

Delete the login file object to preserve memory

Checks which user was logged into and then starts the menu from the respective user

Usability Features

Feature	Description	Justification
Clear way to enter username and password	Clear box for the user to type in the username and then immediately type in the password	I decided to add this as students will be using the system, so it allows for simple and clear access
Hidden password on entry	The password is hidden while being typed	This is to keep accounts secure as a student should not be able to access a teachers account.
Easy keyboard navigation within the menus	Allows you to use the keyboard to navigate the menu without the cursor being able to leave the menu	This is so it is clear and more like other programmes that are being used. The ability to move the cursor through a menu and select something is more intuitive to the user than typing in numbers for menu options
Tabular designs	Displaying information in a table or in similar forms such as graphical.	This is to make the data more easily readable rather than just giving all the data in the list which is very hard to read with large amounts of data
Predictive searching	As you type to search for a student or something else it provides a list of everything that matches so far	I decided to add this so that even if the teacher doesn't know the students full name, they can start to search and then pick from a list rather than having to search specifically for one student
Colour coded attendance	In the overview of attendance for a form group the students will be sorted from lowest to highest attendance and colour coded to show if any actions should be taken	I made the choice to add this so that it is easier on the form tutor to find out who they need to talk to about their attendance rather than having to search through their entire form and being potentially being unclear on who they need to speak to

Data Dictionary

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
Any single character (i, j, d, a,...)	All	n/a	integer	local	None needed, iterative variable	These will be used as iterative variables for the sole purpose of counting loops	These are used to count for loops and to use a counting variable in while or repeat until loop
TimeTable	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Stores information about the timetable	Allows data about the timetable and when classes are to be retrieved
Day	CreateFiles.pas	Field	LONGINT	Global (TimeTable)	Taken from the text file of the timetable	A field within the timetable record	Allows classes to be on a day and a lesson number
LessonNo	CreateFiles.pas	Field	LONGINT	Global (TimeTable)	Taken from the text file of the timetable	A field within the timetable record	
ClassID	CreateFiles.pas	Field	STRING[8]	Global (TimeTable)	Taken from the text file of the timetable	A field within the timetable record	Stores information about which class is associated with the data
Form	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Stores information about the Forms	Allows data about the timetable and when classes are to be retrieved
FormID	CreateFiles.pas	Field	STRING[5]	Global (Form)	Must be a string of max length 5	A field within the form record	Stores the form id, a unique identifier for each form
NumberInForm	CreateFiles.pas	Field	LONGINT	Global (Form)	Must be an int ≥ 0	A field within the form record	Stores the number of students in the form for calculations

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
Register	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Stores the information about each register	Used to store information about each register allowing the data to be retrieved
RegisterID	CreateFiles.pas	Field	STRING[17]	Global (Register)	A combination of lesson Number Date and class ID	Stores the unique identifier for each register	This is sued to search uniquely for a specific register
LessonNo	CreateFiles.pas	Field	LONGINT	Global (Register)	Taken from the lesson number where that class resides at one time	Stores the lesson number in which that register was taken	Allows for registers to be searched by a time
Date	CreateFiles.pas	Field	TDateTime	Global (Register)	Program Generated	Stores the date on which the register was taken	
ClassID	CreateFiles.pas	Field	STRING[8]	Global (Register)	Must match the class id in which this register is for that class	Stores the Class ID for that register	Used to search for all registers for a specific class
RegisterMarks	CreateFiles.pas	Field	ARRAY[0..40] OF studentRegisterMarkRecord	Global (Register)	Program Generated	Used to store student and their assigned register marks	This is needed to calculate the attendance of each student
isOfstedRegister	CreateFiles.pas	Field	BOOLEAN	Global (Register)	Boolean	Stores whether that register should be sent to Ofsted	This is required by my client so I need a method to store it and be able to display it back
studentRegisterMark	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Stores the information about a single student and their register mark	Used to store information about students and their register marks. It is used in an array in the register file

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
studentID	CreateFiles.pas	Field	LONGINT	Global (studentRegisterMark)	Must match with the student id in which this register mark is attached to	Stores the student id	This is to relate a single student with a register mark
registerMark	CreateFiles.pas	Field	CHAR	Global (studentRegisterMark)	A character out of list (P,L,A) to indicate which register mark they got	Stores the register mark for that student	
Attendance	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Stores the information about each students attendance	Used to store information about each students attendance, allowing it to be retrieved.
StudentID	CreateFiles.pas	Field	LONGINT	Global (Attendance)	Must match with the student id in which this attendance record is attached to	Stores student id	This creates a 1 to 1 relation between the attendance file and student file to allow for easy searching between files
Attendance	CreateFiles.pas	Field	REAL	Global (Attendance)	A real number between 1 and 0 inclusive	Stores the attendance as a decimal	Used to display back to the student
Total	CreateFiles.pas	Field	LONGINT	Global (Attendance)	Must be an int ≥ 0	Total number of available register marks	This is to calculate the attendance
NumberPresent	CreateFiles.pas	Field	LONGINT	Global (Attendance)	Must be an int ≥ 0	Total number of present register marks	
NumberLate	CreateFiles.pas	Field	LONGINT	Global (Attendance)	Must be an int ≥ 0	Total number of late register marks	

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
NumberAbsent	CreateFiles.pas	Field	LONGINT	Global (Attendance)	Must be an int ≥ 0	Total number of absent register marks	This is to calculate the attendance
Student	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Store student data	Used to store information about each student
StudentID	CreateFiles.pas	Field	LONGINT	Global (Student)	Must be an int ≥ 0	Stores student id	Unique identifier so that each student can be searched for and found easily
FName	CreateFiles.pas	Field	STRING[20]	Global (Student)	Must be a string with max length 20	Stores first name of student	This is so a student can be searched for by first name
SName	CreateFiles.pas	Field	STRING[20]	Global (Student)	Must be a string with max length 20	Stores last name of student	This is so a student can be searched for by last name
Year	CreateFiles.pas	Field	LONGINT	Global (Student)	Must be an int, doesn't have to be bound but in terms of logic it should ≥ 0	Stores the student's year	This is so a student can be searched for by year
Form	CreateFiles.pas	Field	STRING[5]	Global (Student)	Must be a string matching to the form file	Stores the students form	This is so a student can be searched for by Form
Uname	CreateFiles.pas	Field	STRING[40]	Global (Student)	Must be a string with max length 40	Stores the student username	Allows the student to log in using a unique username
Pword	CreateFiles.pas	Field	STRING[20]	Global (Student)	Must be a string with max length 20	Stores the student password	Allows the student to log in using a unique password

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
Teacher	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Store teacher data	Used to store information about each teacher
TeacherID	CreateFiles.pas	Field	STRING[3]	Global (Teacher)	Must be a string with max length 3	Stores teacher id	Unique identifier so that each teacher can be searched for and found easily
fname	CreateFiles.pas	Field	STRING[20]	Global (Teacher)	Must be a string with max length 20	Stores first name of teacher	This is so a teacher can be searched for by first name
sname	CreateFiles.pas	Field	STRING[20]	Global (Teacher)	Must be a string with max length 20	Stores last name of teacher	This is so a teacher can be searched for by last name
form	CreateFiles.pas	Field	STRING[5]	Global (Teacher)	Must be a string matching to the form file	Stores the teacher form	This is so a teacher can be searched for by Form
Uname	CreateFiles.pas	Field	STRING[40]	Global (Teacher)	Must be a string with max length 40	Stores the teacher username	Allows the teacher to log in using a unique username
PWord	CreateFiles.pas	Field	STRING[20]	Global (Teacher)	Must be a string with max length 20	Stores the teacher password	Allows the teacher to log in using a unique password
studentClass	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Store class data	Used to store information about classes
ClassID	CreateFiles.pas	Field	STRING[8]	Global (studentClass)	Must be a string with max length 8	Stores class id	Unique identifier for each class so it can be found easily in the file

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
Year	CreateFiles.pas	Field	LONGINT	Global (studentClass)	Must be an int, doesn't have to be bound but in terms of logic it should ≥ 0	Stores the year of the class	So classes can be searched for by year
subjectID	CreateFiles.pas	Field	STRING[3]	Global (studentClass)	Must be a string matching a subject in the subject file	Stores the year of the subject associated with that class	So classes can be searched for by subject
NoOfStudents	CreateFiles.pas	Field	LONGINT	Global (studentClass)	Must be an int ≥ 0	Stores the number of students in that class	This keeps track of how many students are in the class and if it is the right number or too many
subject	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Stores information about the subjects	Allows retrieval of data from the subject file
SubjectID	CreateFiles.pas	Field	STRING[3]	Global (subject)	Must be a string with max length 3	Stores subject id	Allows for searching the subject file by id
SubjectName	CreateFiles.pas	Field	STRING[20]	Global (subject)	Must be a string with max length 20	Stores subject name	Associates a subject id with a name readable for students
studentClassResolver	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Stores which students are in which class	Allows the retrieval of data about which classes each student is in
StudentID	CreateFiles.pas	Field	LONGINT	Global (studentClassResolver)	Must be a string matching a student in the student file	Stores the student id	Combined allow for the link between students and a class

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
ClassID	CreateFiles.pas	Field	STRING[8]	Global (studentClassResolver)	Must be a string matching a class in the class file	Stores the Class ID	allowing the ability to read which class each student is in
teacherSubjectResolver	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Variable that the teacher subject record is read into	Allows the retrieval of data from the teacher subject resolver file
TeacherID	CreateFiles.pas	Field	STRING[3]	Global (teacherSubjectResolver)	Must be a string matching a teacher in the teacher file	Stores the teacher id	Associates with a specific teacher
SubjectID	CreateFiles.pas	Field	STRING[2]	Global (teacherSubjectResolver)	Must be a string matching a subject in the subject file	Stores the subject id	Associates with a specific subject
teacherClassResolver	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Variable that the teacher class record is read into	Allows the retrieval of data from the teacher class resolver file
TeacherID	CreateFiles.pas	Field	STRING[3]	Global (teacherClassResolver)	Must be a string matching a teacher in the teacher file	Stores the teacher id	Associates with a specific teacher
ClassID	CreateFiles.pas	Field	STRING[8]	Global (teacherClassResolver)	Must be a string matching a class in the class file	Stores the class id	Associates with a specific subject
LogInDetails	CreateFiles.pas	Record	-	Global	None, can't input into the record data structure	Variable that the login record is read into	Allows the retrieval of data from the login file

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
PersonID	CreateFiles.pas	Field	STRING[5]	Global (LogInDetails)	Must be a string or converted from an int to a string matching an id in either the student or teacher file	A variable that hold the id of that person whether it is a student id or teacher id	This is to relate the information of the person with their login credentials in a separate file
Username	CreateFiles.pas	Field	STRING[40]	Global (LogInDetails)	Must be a string matching the persons id and their username	Stores the persons username	Allows the person to log in using a unique username
Passwowrd	CreateFiles.pas	Field	STRING[20]	Global (LogInDetails)	Must be a string matching the persons id and their password	Stores the persons password	Allows the person to log in using a unique password
classIDArray	CreateFiles.pas	Array	ARRAY[0..50] OF STRING	Global	From File	An array that stores all the class ids	It is used in many procedures, so it is logical to create a global array of it
dayArray	CreateFiles.pas	Array	ARRAY[0..320] OF STRING	Local (createRegisters)	From File	Stores all the days in the timetable with 1 position in the array for each class and each lesson	This is so the timetable classes can be extracted by day number
dayReturnArray	CreateFiles.pas	Array	ARRAY[0..320] OF LONGINT	Local (createRegisters)	Program generated	Stores the search result of searching the day array	This returns the list of all classes on that day and their position in the file, so they can be used for each register on a day

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
studentClassResolverClassArray	CreateFiles.pas	Array	ARRAY[0..1384] OF STRING	Local (createRegisters)	From File	Stores all the class ids in the student class resolver in the same array	This is so you can search the array for all instances of a specific class and get every student in that class
classReturnArray	CreateFiles.pas	Array	ARRAY[0..1384] OF LONGINT	Local (createRegisters)	Program generated	Stores the result of the search of the student class resolver file	This is required to obtain all the students in the class for the register we are currently dealing with
studentFormIdArray	CreateFiles.pas	Array	ARRAY[0..139] OF STRING	Local (createRegisters)	From File	Stores all the form ids in the form file in the same array	This is so you can search the array for all instances of a specific form and get every student in that form
returnStudentFormIdArray	CreateFiles.pas	Array	ARRAY[0..139] OF LONGINT	Local (createRegisters)	Program generated	Stores the result of the search of the form file	This is required to obtain all the students in the form for the register we are currently dealing with
subjID	CreateFiles.pas	Array	ARRAY[0..20] OF STRING[3]	Local (createSubjects)	Programmer Set	An array of all subject ids	This is to create the initial write to file so this data must be hard coded in

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
subjNm	CreateFiles.pas	Array	ARRAY[0..20] OF STRING[20]	Local (createSubjects)	Programmer Set	An array of all subject names	This is to create the initial write to file so this data must be hard coded in
classArray	CreateFiles.pas	Array	ARRAY[0..50] OF classRecord	Local (createClasses)	From File	Array of all classes	This is so the class id array can be formed
returnArray	CreateFiles.pas	Array	ARRAY[0..50] OF LONGINT	Local (addRecordsStudentClassResolver, createClassStudentResolver)	Program generated	Stores the result of the search for a class to put a student in	This is to store the result of a search while searching for classes to put the students into
Super Class							
Files	FileProcedures.pas	Class	Files	Global	None, can't input into the class data structure	Stores the class identifier	This is to centralize the sort and search algorithms across all files into one script
upper	FileProcedures.pas	Attribute	LONGINT	Global (Files)	Programmer Set	Upper limit in the array to stop the sort algorithm at a certain point	This stops the program trying to read null data in the array and sorting that
dataArray	FileProcedures.pas	Attribute	ARRAY[0..1400] OF STRING	Global (Files)	Program generated	Array of the data primary keys for sorting	This is so the data can be sorted according to programmer set primary key

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
fileArray	FileProcedures.pas	Attribute	ARRAY[0..7,0..1400] OF STRING	Global (Files)	Program generated	Array of the file from the child class	This is the array to be searched and thus the data needs to be taken from the file and placed into an array
indexArray	FileProcedures.pas	Parameter	ARRAY OF LONGINT	Local (sort)	Program generated	Stores the new order for the index	This is so the data can be written back to the file sorted
TempIndex	FileProcedures.pas	Variable	LONGINT	Local (sort)	Program generated	temporary storage to swap index elements around	This is so data can be kept track of during the sort
TempStr	FileProcedures.pas	Variable	STRING	Local (sort)	Program generated	temporary storage to swap data elements around	This is to sort the data
Swaps	FileProcedures.pas	Variable	BOOLEAN	Local (sort)	Program generated	Keeps track of any swaps that happen in the bubble sort	Check to see if swaps are made as if no swaps that means data is stored
max	FileProcedures.pas	Parameter	LONGINT	Local (search)	Programmer Set	Max elements in array to search	This is so it doesn't read to far into the array as it will be empty and thus the program will break
fields	FileProcedures.pas	Parameter	LONGINT	Local (search)	Programmer Set	Stores number of fields to search through	This allows you to search a file with multiple criteria,

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
searchValues	FileProcedures.pas	Parameter	ARRAY OF STRING	Local (search)	List of inputs by programmer input or user input. Only constraint is it must be a string	The data to search for	specifying what and where the data should appear for more accurate results and not having to run the search many times
searchFields	FileProcedures.pas	Parameter	ARRAY OF BYTE	Local (search)	Programmer set	The fields to search for the given data in	
returnArray	FileProcedures.pas	Parameter	ARRAY OF LONGINT	Local (search)	Program generated	Array to return all positions of matching data	This so you can get a list of all a subset of data without having to run the search many times
switchLowerCase	FileProcedures.pas	Parameter	BOOLEAN	Local (search)	Programmer set	Switch to allow for forced lower case or not	This can allow case sensitive and case insensitive searches
found	FileProcedures.pas	Variable	BOOLEAN	Local (search)	Program generated	Stores whether a match has been found in the data	This is so that the program knows when it has found matching data and thus if to add current index to return array
returnIndex	FileProcedures.pas	Variable	LONGINT	Local (search)	Program generated	Counter for return array to next free location in return array	This is so no data is lost when it is being returned
data	FileProcedures.pas	Variable	STRING	Local (search)	Program generated	Stores data for current search value	This is so the data can be stored and checked against also allowing for the lowercase to only be set once

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
Generic Child Class							
genericClassFIle	genericClassFIle.pas	Class	genericClassFIle	Global	Class structure no data input	Stores the class identifier	This is so that each file can have the record specific code inside them while sharing main sort and search algorithms through inheritance
dataRecord	LogInProcedures.pas	Record	-	Global	None, can't input into the class data structure	Record to match the file type	So the data can be read from the file
Filed identifier for the record type matching the file	genericClassFIle.pas	Field	-	Global (dataRecord)	Validation depending on the field	Depends on the file and thus the record structure	So the data can be read from the file
dataFile	genericClassFIle.pas	Attribute	File OF dataRecord	Global (genericClassFIle)	Programmer set	A variable to store the location of the data file	This is so the program knows where the file is stored
dataTextFile	genericClassFIle.pas	Attribute	TextFile	Global (genericClassFIle)	Programmer set	A variable to store the location of the text file to write to	This is so the data can be viewed outside the program running
indexArray	genericClassFIle.pas	Attribute	ARRAY[0..190] OF LONGINT	Global (genericClassFIle)	Procedurally generated	Array to keep track of the indexes of the data	So when the data ids are sorted, the indexes of that data is also stored so that the data can be written back to the file successfully

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
recordArray	genericClassFIle.pas	Attribute	ARRAY[0..190] OF dataRecord	Global (genericClassFIle)	From File	Array of all records	Required to pass onto the sort and to write back into the file after the sort
data	genericClassFIle.pas	Attribute	dataRecord	Global (genericClassFIle)	From file	Store for a single record from the file	Allows the data from the file to be read into the program
searchValues	genericClassFIle.pas	Parameter	ARRAY OF STRING	Local (search)	List of inputs by programmer input or user input. Only constraint is it must be a string	The data to search for	This allows you to search a file with multiple criteria, specifying what and where the data should appear for more accurate results and not having to run the search many times
searchFields	genericClassFIle.pas	Parameter	ARRAY OF BYTE	Local (search)	Programmer set	The fields to search for the given data in	
returnArray	genericClassFIle.pas	Parameter	ARRAY OF LONGINT	Local (search)	Program generated	Array to return all positions of matching data	This so you can get a list of all a subset of data without having to run the search many times
switchLowerCase	genericClassFIle.pas	Parameter	BOOLEAN	Local (search)	Programmer set	Switch to allow for forced lower case or not	This can allow case sensitive and case insensitive searches
Main							
userType	Main.pas	Variable	STRING	Global	Program generated	Character to represent what user successfully logged in	This is so the code to pass over to the relevant script knows which user logged in

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
intUserID	Main.pas	Variable	LONGINT	Global	Program generated	Stores the user Id as an int	This is so it can be passed on to the student menu script
ErrCode	Main.pas	Variable	LONGINT	Local (start)	Program generated	A variable to test if the id is an int or not	This allows us to test if the string of the user id is an int or not allowing tests to see if the user is a teacher or student
userIndex	Main.pas	Variable	LONGINT	Local (start)	From File	Stores the output result of the search for credentials	This is so the program can identify whether the user entered credentials exist in the system
userId	Main.pas	Variable	STRING	Local (start)	From File	Stores the user id of the user found with matching credentials	This is so the user can then be filter out into either a teacher or student
userDetailsArray	Main.pas	Variable	ARRAY[0..1] OF STRING	Local (start)	Each element is a string so any input can be entered. But with the password only specific keys are allowed to be entered (all on the keyboard)	Stores the response from the user details that were input into the login screen	This is so the username and password that the user entered can be tested to see if they match with any in the files
logIn	Main.pas	Variable	logInFile	Local (start)	None needed as it's an object	Object for the login File class	This is required so that the methods from that class can be accessed and used

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
percentDone	Main.pas	Variable	REAL	Local (start)	Program generated	Filler for the loading procedure	A real variable is required to be passed into the loading procedure so an empty variable must be created to accommodate this
logedIn	Main.pas	Variable	BOOLEAN	Local (start)	Program generated	Stores if the user successfully logged into an account	This is so the user can keep trying to log in until they either close the system or got a successful log in
Student Menu							
studentID	StudentsMenu.pas	Variable	LONGINT	Global	From File	Stores the student id	This is so it can be used and searched for in other files
student	StudentsMenu.pas	Variable	StudentFile	Global	None needed as its an object	Object for the student File class	This is required so that the methods from that class can be accessed and used
attendance	StudentsMenu.pas	Variable	AttendanceFile	Global	None needed as its an object	Object for the attendance File class	This is required so that the methods from that class can be accessed and used
timetable	StudentsMenu.pas	Variable	TimeTableFile	Global	None needed as its an object	Object for the timetable class	This is required so that the methods from that class can be accessed and used

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
studentClassC	StudentsMenu.pas	Variable	studentClassFile	Global	None needed as its an object	Object for the studentClass File class	This is required so that the methods from that class can be accessed and used
studentClassReturnArray	StudentsMenu.pas	Variable	ARRAY[0..1400] OF LONGINT	Global	From File	Stores the results of searches to the studentClass file	This is needed so that if multiple occurrences are found they can be found and tested.
timetableReturnArray	StudentsMenu.pas	Variable	ARRAY[0..321] OF LONGINT	Global	From File	Stores the results of searches to the timetable file	This is needed so that if multiple occurrences are found they can be found and tested.
mainMenuOptions	StudentsMenu.pas	Variable	ARRAY[0..2] OF STRING	Global	Programmer set	Array of main menu options	This is the array which stores the names of the menu options for the student's main menu
animationTimeTable	StudentsMenu.pas	Variable	BOOLEAN	Global	Program generated	Stores whether the animation for the timetable should be executed	This is to stop the timetable animation
inputStudentId	StudentsMenu.pas	Parameter	LONGINT	Local (startStudent)	Parameter, check will happen earlier	Stores the student id passes form the main program	This saves us having to search for the student that was logged into and just allows direct reference to that place in the file to find the students info

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
inputDate	StudentsMenu.pas	Parameter	TDateTime	Local (CompleteTimeTable)	Parameter, check will happen earlier	Parameter to allow entering of start date for the timetable	This will allow any date to be inputted ant the timetable will start from there
NumberOfDays	StudentsMenu.pas	Variable	LONGINT	Local (CompleteTimeTable)	Program generated	Stores the number of days since the start of the school year	This is so we can calculate how many weeks it has been and thus calculate which week it is
timeDelay	StudentsMenu.pas	Variable	LONGINT	Local (CompleteTimeTable)	Programmer Set	Sets the delay in the writing to the screen	This is so that the animation time can be change and removed if needed
CurrentDate	StudentsMenu.pas	Variable	TDateTime	Local (CompleteTimeTable)	Program generated	Current date is the passed-on data from the procedure	This is required so that the data can be manipulated and restored in the same variable
week	StudentsMenu.pas	Variable	STRING	Local (CompleteTimeTable)	From File	Stores whether it is week a or b (1st or 2nd week in 2-week timetable)	This is so that it can be outputted to let the user know which week it is
stringStudentId	StudentsMenu.pas	Variable	STRING	Local (CompleteTimeTable)	From File	Stores the student id as a string	This is so it can be passed onto the search procedure
StartDate	StudentsMenu.pas	Variable	TDateTime	Local (TimeTableMenu)	Program generated	Stores the date to pass onto the writeTimeTable procedure	This is so that different dates can be entered rather than just the current date

Identifier	Module	Data Structure	Data Type	Scope	Validation	Explanation	Justification
choice	StudentsMenu.pas	Variable	LONGINT	Local (mainMenu)	Program generated	Stores the menu option chosen	This is required to store the menu option chosen and allows the program to be passed onto the relevant procedure

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Data Plan

Test 1

Test if a user can log into the system into the correct account with correct menu options. An example:

Username	RobertCole
Password	ABC123

This should log you into the account of Robert Cole and should display a student's menu

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test 2

Selecting a menu option in the either the student's menu or the teacher's menu should display the result of what should be in that menu. So if timetable is selected it should display the timetable of that person. For example, if Robert Cole was to look at his timetable it should look

Week B Date	Lesson	Form	1	2	3	4	5
06/01/2025		9AAA	9AAX/Enl	9AAY/Ma	9B/Gg1	9AAX/RPE	9AP2/Pe
07/01/2025		9AAA	9D/Bs1	9D/Bs1	9AT2/Ch	9AT2/Bi	9AT2/Ph
08/01/2025		9AAA	9AAY/Ma	9AAX/Enl	9AT2/Bi	9C/Py1	9C/Py1
09/01/2025		9AAA	9AT2/Ph	9AAY/Ma	9AT2/Ch	9AP2/Pe	9AAX/Enl
10/01/2025		9AAA	9AT2/Ph	9AP2/Pe	9B/Gg1	9B/Gg1	9AAX/Lf

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test 3

Inputting a timetable in a text file into the create files data will create all other files in relation to that new timetable. It will also convert the data from the text file into a .dta file which is stored as a series of records. For example if it input this data:

```
12D/Mu1
3
2

5
9
Q
12C/Cp1
4
1

4
10
Q
```

```
Class
Lesson Number
Day Number
Q - end of class section
```

This will be converted into records

Class	Lesson Number	Day number
12D/Mu1	3	2
12D/Mu1	5	9
12C/Cp1	4	1
12C/Cp1	4	10

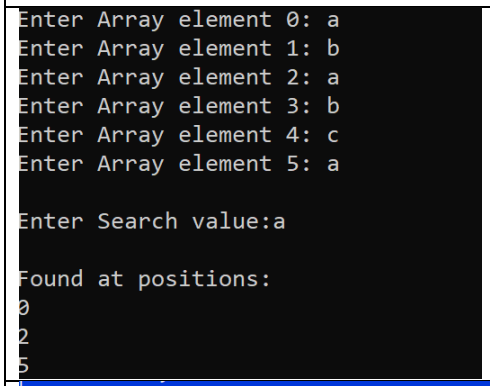
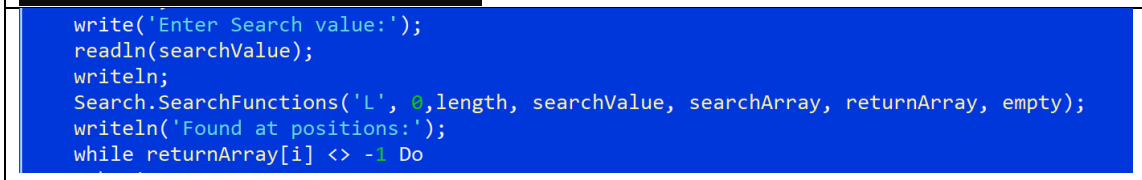
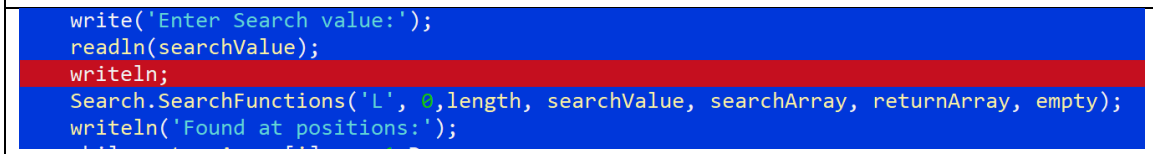
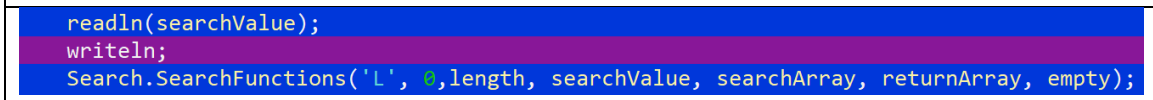
Post Development Data

The data that will be after the development is complete is very similar to the data above. I made sure at the start to create very life like data that can be used to accurately test my system. So, the data above will also be used during post development but with more data surrounding it such as a larger timetable, larger number of students. Potentially different ways to sort the students into classes, but most data will be similar.

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Iterations

Evidence Explanations

Evidence	Explanation
	Any black screen background with writing indicates the program running and a user having to interact with the program. Such as here I inputted the array elements. This will be what the user sees.
	Any blue backgrounds show evidence taken from the integrated development environment (IDE). This is done due to the way the program is written and that units cannot be run on their own. They need a program to run them so I will develop short scripts dedicated to running tests on the unit. The IDE view will be used to show evidence of what is run.
	Any code with a red bar represents that line having a breakpoint at that line, so during execution the program will pause there, this will be used to show execution of some units with no output and are just filler units to link two other units into one unit
	Code with a purple bar represents a hit breakpoint during execution. So the computer is currently paused just before executing this line but it has executed all previous code

``` ACa AdamCampbell ABC123 90 AgnusNielsen ABC123 50 AlfredoAllison ABC123 ARo AllisonRolling ABC123 15 AltonJoyner ABC123 ```		Any pieces of evidence looking like this comes form notepad and is the output of a file in a text form so I can check that it is working. All data will be explained

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Iteration 0 – Library Declarations

Aim

The aim of this iteration is to declare libraries which will be used throughout the rest of the program to shorten code in the main scripts that will be run. The user never sees this, but it will be useful for me as the programmer to get more done and keep code tidy and easy to read for any future developments.

Iteration 0.1 – Linear Search

Code

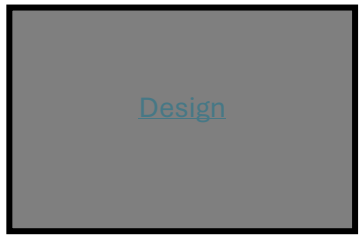
```
UNIT LinearSearch;
  INTERFACE
  USES CRT, SYSUTILS;

  PROCEDURE LinearSearchFunction(low, high : LONGINT; searchValue : STRING; UserArray : ARRAY OF STRING; var
returnArray : ARRAY OF INTEGER);
    VAR
      i, a, checkItem: LONGINT;

    IMPLEMENTATION

  PROCEDURE LinearSearchFunction(low, high : LONGINT; searchValue : STRING; UserArray : ARRAY OF STRING; var
returnArray : ARRAY OF INTEGER);
    VAR
      i, a, checkItem: LONGINT;
    BEGIN
      i := -1;
      a := 0;
     // iterate through each element until end of array
      REPEAT
        i := i+1;
       // add all positions for values matching the search value to a different array
        if userArray[i] = searchValue THEN
        begin
          returnArray[a] := i;
          a := a + 1;
        end;
      UNTIL (i = high);
    END;

  END.
```



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.1.1	Linear search unit, Iteration 0	To test the linear search works on small arrays of integers finding a singular value	Array = [5,4,3,2,1,0] Search value = 4	1	1	n/a

```
Enter Array element 0: 5
Enter Array element 1: 4
Enter Array element 2: 3
Enter Array element 3: 2
Enter Array element 4: 1
Enter Array element 5: 0

Enter Search value:4

Found at positions:
1
```

0.1.2	Linear search unit, Iteration 0	Testing if it can find the first value in the array of integers	Array = [5,4,3,2,1,0] Search value = 5	0	0	n/a

```
Enter Array element 0: 5
Enter Array element 1: 4
Enter Array element 2: 3
Enter Array element 3: 2
Enter Array element 4: 1
Enter Array element 5: 0

Enter Search value:5

Found at positions:
0
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.1.3	Linear search unit, Iteration 0	Testing if it can find the last value in the array of integers	Array = [5,4,3,2,1,0] Search value = 0	5	5	n/a

```
Enter Array element 0: 5
Enter Array element 1: 4
Enter Array element 2: 3
Enter Array element 3: 2
Enter Array element 4: 1
Enter Array element 5: 0

Enter Search value:0

Found at positions:
5
```

0.1.4	Linear search unit, Iteration 0	Testing to return null if no values found	Array = [5,4,3,2,1,0] Search value = 6	n/a	n/a	n/a

```
Enter Array element 0: 5
Enter Array element 1: 4
Enter Array element 2: 3
Enter Array element 3: 2
Enter Array element 4: 1
Enter Array element 5: 0

Enter Search value:6

Found at positions:
```

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.1.5	Linear search unit, Iteration 0	Testing to see if it gets all positions for all pieces of found integers	Array = [2,2,2,1,1,0] Search value = 1	3 4	3 4	n/a

```
Enter Array element 0: 2
Enter Array element 1: 2
Enter Array element 2: 2
Enter Array element 3: 1
Enter Array element 4: 1
Enter Array element 5: 0

Enter Search value:1

Found at positions:
3
4
```

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.1.6	Linear search unit, Iteration 0	Testing if it can find a string value in the array	Array = [a,b,c,d,e,f] Search value = c	2	2	n/a

```
Enter Array element 0: a
Enter Array element 1: b
Enter Array element 2: c
Enter Array element 3: d
Enter Array element 4: e
Enter Array element 5: f

Enter Search value:c

Found at positions:
2
```

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.1.7	Linear search unit, Iteration 0	Testing if it can find the first value in the array	Array = [a,b,c,d,e,f] Search value = a	0	0	n/a

```
Enter Array element 0: a
Enter Array element 1: b
Enter Array element 2: c
Enter Array element 3: d
Enter Array element 4: e
Enter Array element 5: f

Enter Search value:a

Found at positions:
0
```

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.1.8	Linear search unit, Iteration 0	Testing if it can find the last value in the array	Array = [a,b,c,d,e,f] Search value = f	5	5	n/a

```
Enter Array element 0: a
Enter Array element 1: b
Enter Array element 2: c
Enter Array element 3: d
Enter Array element 4: e
Enter Array element 5: f

Enter Search value:f

Found at positions:
5
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.1.9	Linear search unit, Iteration 0	Testing to return null if string is not found in array	Array = [a,b,c,d,e,f] Search value = g	n/a	n/a	n/a

```
Enter Array element 0: a
Enter Array element 1: b
Enter Array element 2: c
Enter Array element 3: d
Enter Array element 4: e
Enter Array element 5: f

Enter Search value:g

Found at positions:
```

0.1.10	Linear search unit, Iteration 0	Testing to see if it gets all positions for all pieces of found strings	Array = [a,b,a,b,c,a] Search value = a	0 2 5	0 2 5	n/a

```
Enter Array element 0: a
Enter Array element 1: b
Enter Array element 2: a
Enter Array element 3: b
Enter Array element 4: c
Enter Array element 5: a

Enter Search value:a

Found at positions:
0
2
5
```

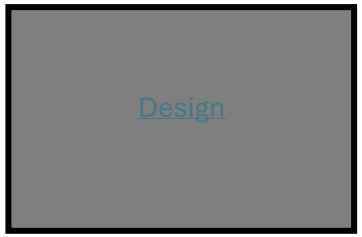
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.1.11	Linear search unit, Iteration 0	Testing if it can search based on different lengths	Array = [9,8,7,6,5,4,3,2,1,0] Search value = 8	1	1	n/a

```
Enter Array element 0: 9
Enter Array element 1: 8
Enter Array element 2: 7
Enter Array element 3: 6
Enter Array element 4: 5
Enter Array element 5: 4
Enter Array element 6: 3
Enter Array element 7: 2
Enter Array element 8: 1
Enter Array element 9: 0

Enter Search value:8

Found at positions:
1
```



Iteration 0.2 – Binary Search

Code

```
UNIT BinarySearch;
  INTERFACE

  USES CRT, SYSUTILS;

  PROCEDURE BinarySearchFunction(var UserArray, returnArray : ARRAY OF INTEGER; low, high, searchValue:
INTEGER);
    VAR
      i, mid : LONGINT;
      shouldEnd : BOOLEAN;

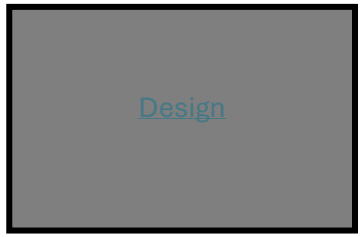
    IMPLEMENTATION

  PROCEDURE BinarySearchFunction(var UserArray, returnArray : ARRAY OF INTEGER; low, high, searchValue:
INTEGER);
    VAR
      i, mid : LONGINT;
      shouldEnd : BOOLEAN;
    BEGIN
      REPEAT
        mid := (low+high)DIV 2;
        IF UserArray[mid] <> searchValue THEN
          BEGIN
                IF UserArray[mid] < searchValue THEN
                  BEGIN
                    low := mid +1;
                  END
                ELSE
                  BEGIN
                    high := mid -1;
                  END;
              END
        ELSE
          BEGIN
            returnArray[0] := mid;
```

```
            shouldEnd := true;
          END;
      UNTIL (shouldEnd = true) OR (low > high);

    END;

  END.
```



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.2.1	Binary search unit, Iteration 0	To test the Binary search works on small arrays of integers finding a singular value	Array = [2,3,4,5,6,7] Search value = 4	2	2	n/a

```
Enter Array element 0: 2
Enter Array element 1: 3
Enter Array element 2: 4
Enter Array element 3: 5
Enter Array element 4: 6
Enter Array element 5: 7

Enter Search value:4

Found at positions:
2
```

0.2.2	Binary search unit, Iteration 0	Testing if it can find the first value in the array of integers	Array = [2,3,4,5,6,7] Search value = 2	0	0	n/a

```
Enter Array element 0: 2
Enter Array element 1: 3
Enter Array element 2: 4
Enter Array element 3: 5
Enter Array element 4: 6
Enter Array element 5: 7

Enter Search value:2

Found at positions:
0
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.2.3	Binary search unit, Iteration 0	Testing if it can find the last value in the array of integers	Array = [2,3,4,5,6,7] Search value = 7	7	5	n/a

```
Enter Array element 0: 2
Enter Array element 1: 3
Enter Array element 2: 4
Enter Array element 3: 5
Enter Array element 4: 6
Enter Array element 5: 7

Enter Search value:7

Found at positions:
5
```

0.2.4	Binary search unit, Iteration 0	Testing it correctly returning a null output if no value is found	Array = [2,3,4,5,6,7] Search value = 8	n/a	n/a	n/a

```
Enter Array element 0: 2
Enter Array element 1: 3
Enter Array element 2: 4
Enter Array element 3: 5
Enter Array element 4: 6
Enter Array element 5: 7

Enter Search value:8

Found at positions:
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.2.5	Binary search unit, Iteration 0	Testing it correctly returning a null output if no value is found	Array = [2,3,4,5,6,7] Search value = a	n/a	n/a	n/a

```
Enter Array element 0: 2
Enter Array element 1: 3
Enter Array element 2: 4
Enter Array element 3: 5
Enter Array element 4: 6
Enter Array element 5: 7

Enter Search value:a

Found at positions:
```

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.2.6	Binary search unit, Iteration 0	Testing if it can find a string value in the array	Array = [a,b,c,d,e,f] Search value = c	2	2	n/a

```
Enter Array element 0: a
Enter Array element 1: b
Enter Array element 2: c
Enter Array element 3: d
Enter Array element 4: e
Enter Array element 5: f

Enter Search value:c

Found at positions:
2
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.2.7	Binary search unit, Iteration 0	Testing if it can find the first value in the array	Array = [a,b,c,d,e,f] Search value = a	0	0	n/a

```
Enter Array element 0: a
Enter Array element 1: b
Enter Array element 2: c
Enter Array element 3: d
Enter Array element 4: e
Enter Array element 5: f

Enter Search value:a

Found at positions:
0
```

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.2.8	Binary search unit, Iteration 0	Testing if it can find the last value in the array	Array = [a,b,c,d,e,f] Search value = f	5	5	n/a

```
Enter Array element 0: a
Enter Array element 1: b
Enter Array element 2: c
Enter Array element 3: d
Enter Array element 4: e
Enter Array element 5: f

Enter Search value:f

Found at positions:
5
```

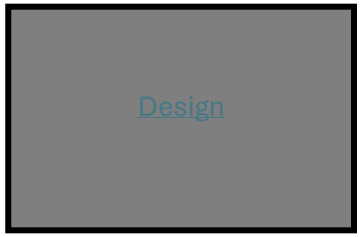
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.2.9	Binary search unit, Iteration 0	Testing if it can search based on different lengths	Array = [2,3,4,5,6,7,8,9] Search value = 8	6	6	n/a

```
Enter Array element 0: 2
Enter Array element 1: 3
Enter Array element 2: 4
Enter Array element 3: 5
Enter Array element 4: 6
Enter Array element 5: 7
Enter Array element 6: 8
Enter Array element 7: 9

Enter Search value:8

Found at positions:
6
```



Iteration 0.3 – Search Script

Code

```
UNIT Search;

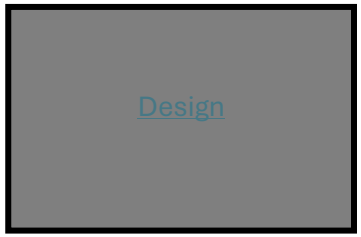
  INTERFACE
  USES CRT, SYSUTILS, BinarySearch, LinearSearch;

  PROCEDURE SearchFunctions(SearchType : CHAR; min, max : LONGINT; findValue : STRING; UserArray : ARRAY OF
STRING; var returnArray : ARRAY OF LONGINT);

  IMPLEMENTATION

  PROCEDURE SearchFunctions(SearchType : CHAR; min, max : LONGINT; findValue : STRING; UserArray : ARRAY OF
STRING; var returnArray : ARRAY OF LONGINT);
    BEGIN
      IF SearchType = 'L' THEN
       //call linear search and just pass through the data
        LinearSearch.LinearSearchFunction(min,max,FindValue,UserArray,returnArray);
      IF SearchType = 'B' THEN
       //call linear search and just pass through the data
        BinarySearch.BinarySearchFunction(min,max, FindValue,UserArray,returnArray);
    END;

  END.
```



Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.3.1	Searching Iteration 0	To test it enters the linear search script if a linear search is selected	L as the character in the call Breakpoint at the call to the linear search	Purple bar on the linear search function	Purple bar on the linear search function	None

```
writeln;
Search.SearchFunctions('L', 0,length, searchValue, searchArray, returnArray, empty);
writeln('Found at positions:');
```

```
PROCEDURE SearchFunctions(SearchType : CHAR; min, max : LONGINT; findValue : STRING; UserArray
  BEGIN
    IF SearchType = 'L' THEN
      LinearSearch.LinearSearchFunction(min,max,FindValue,UserArray,returnArray,numberDone);
    IF SearchType = 'B' THEN
      BinarySearch.BinarySearchFunction(min,max, FindValue,UserArray,returnArray,numberDone);
  END;
```

```
PROCEDURE SearchFunctions(SearchType : CHAR; min, max : LONGINT; findValue : STRING; UserArray
  BEGIN
    IF SearchType = 'L' THEN
      LinearSearch.LinearSearchFunction(min,max,FindValue,UserArray,returnArray,numberDone);
    IF SearchType = 'B' THEN
      BinarySearch.BinarySearchFunction(min,max, FindValue,UserArray,returnArray,numberDone);
  END;
```

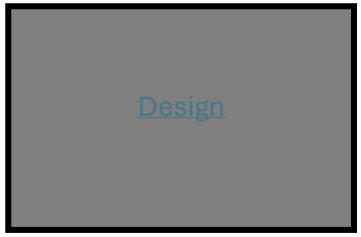
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.3.2	Searching Iteration 0	To test it enters the binary search script if a linear search is selected	B as the character in the call Breakpoint at the call to the binary search	Purple bar on the binary search function	Purple bar on the binary search function	None

```
writeln;
Search.SearchFunctions('B', 0,length, searchValue, searchArray, returnArray, empty);
writeln('Found at positions:');
```

```
PROCEDURE SearchFunctions(SearchType : CHAR; min, max : LONGINT; findValue : STRING; UserArray
  BEGIN
    IF SearchType = 'L' THEN
      LinearSearch.LinearSearchFunction(min,max,FindValue,UserArray,returnArray,numberDone);
    IF SearchType = 'B' THEN
      BinarySearch.BinarySearchFunction(min,max, FindValue,UserArray,returnArray,numberDone);
  END;
```

```
PROCEDURE SearchFunctions(SearchType : CHAR; min, max : LONGINT; findValue : STRING; UserArray
  BEGIN
    IF SearchType = 'L' THEN
      LinearSearch.LinearSearchFunction(min,max,FindValue,UserArray,returnArray,numberDone);
    IF SearchType = 'B' THEN
      BinarySearch.BinarySearchFunction(min,max, FindValue,UserArray,returnArray,numberDone);
  END;
```



Iteration 0.4 – Loading

Code

```
unit loading;
  interface

  uses sysutils, vputils, crt, math;
  //pointer to point to the value outside of this script
  type rPtr = ^real;

  //pre declaration of functions and procedures
  function loadingText(p : pointer) : LONGINT;
    var a : LONGINT;
  function loadingBar(p : pointer) : LONGINT;
    var percentComplete : REAL;
      b,c,completeBars,uncompleteBars : LONGINT;
  procedure StartLoading(loadingType : CHAR; var percentIn : real);
  procedure EndLoading;
  procedure silentKillLoading;

  implementation
  //the symbol for a spinning symbol
  const LoadingSymbols : ARRAY[1..4] OF CHAR = ('/',#196,'\','|');

  //local script variables
  var loadingDone : BOOLEAN;
    percentDone : real;
    percentPtr : rptr;
    startRunTime, endRunTime : TDateTime;

  //the basic loading text with the spinning line
  function loadingText(p : pointer) : LONGINT;
    var a : LONGINT;
    begin
      HideCursor;
      a := 1;
      ClrScr;
      while (not loadingDone) DO
```

```
        Begin
          //write loading and symbol from the array
          WRITELN('Loading... ' + LoadingSymbols[a]);
          //progress the loading symbol
          a := a+1;
          if a > 4 THEN
            a := 1;
          //every 0.1 secons update
          Delay(100);
          //rewrite the whole line
          GoToXY(1,1);
          DelLine;
        end;

      end;

  function loadingBar(p : pointer) : LONGINT;
    var percentComplete : REAL;
      b,c,d,completeBars,uncompleteBars : LONGINT;
    begin
      HideCursor;
      ClrScr;
      //sets up the empty progress bar
      Write('Progress ',#185);
      for c:= 0 TO 20 DO
        write(#176);
      write(#204);
      percentDone := 0.0*100;
      completeBars := 0;
```

```
    while not loadingDone DO
      begin
        //write the new percentage in
        gotoxy(33,1);
        Write(percentDone:4:0,'%');
        //gets the percent done from the main script by using a series of pointers
        //to point to a value outside this script so the procedure does not have to be recalled
        percentDone := (percentPtr^)*100;
        //calculate number of bars to fill
        completeBars := floor((percentPtr^)*20);
        gotoxy(11,1);
        //draw that number of bars
        for d := 0 to completeBars DO
          write(#219);
    end;
end;

procedure StartLoading(loadingType : CHAR; var percentIn : real);
  begin
    startRunTime := Time;
    loadingDone := false;
    //set a pointer to the memory address to the memory location of the percent
    //memory location passed in at the start
    percentPtr := @percentIn;

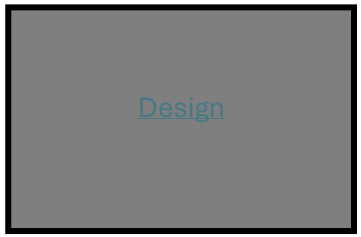
    //choose a loading type and start a new thread so it runs in parallel
    if loadingType = 'L' THEN
      VPBeginThread(loadingText,1,nil)
    else if loadingType = 'P' THEN
      VPBeginThread(loadingBar,1,nil);
  End;




procedure EndLoading;
  begin
    //kill the thread to stop it running
```

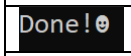
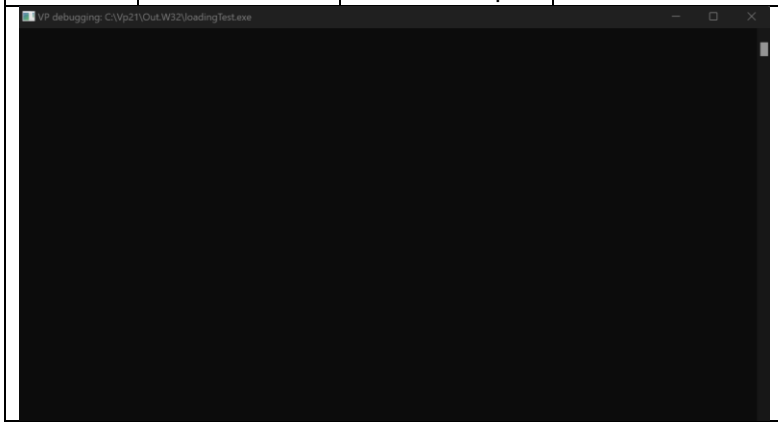
```
      KillThread(1);
      loadingDone := true;
      //show end messages
      WRITELN('Done!');
      endRunTime := Time;
      Delay(500);
      ClrScr;
      //state how long it took
      writeln('Time elapsed: ',TimeToStr(endRunTime - startRunTime));
      ShowCursor;
      end;

  procedure silentKillLoading;
    begin
      //stops the bar without displaying anything back to the user when done
      KillThread(1);
      loadingDone := true;
      ClrScr;
      ShowCursor;
    end;
  end.
```



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

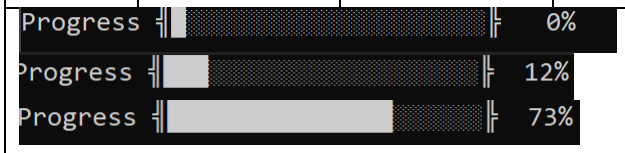
Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.4.1	Loading Iteration 0	To test the loading text with spinning symbol works	None needed to be entered	Loading...(with changing symbol next to it)	Correct result	n/a
						
0.4.2	Loading Iteration 0	To test the Done screen shows up when finished	None needed to be entered	Done! ☻	Done! ☻	n/a
						
0.4.3	Loading Iteration 0	To test the loading time is correct	Delay of 15 seconds used	Time elapsed: 00:00:15	Time elapsed: 00:00:15	n/a
						
0.4.4	Loading Iteration 0	Silent kill ends the loading with no output	No data needs to be entered	Blank screen	Blank screen	n/a
						

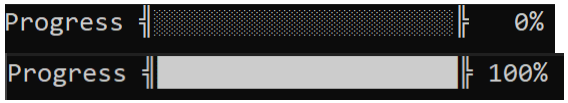
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

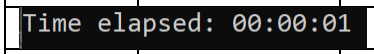
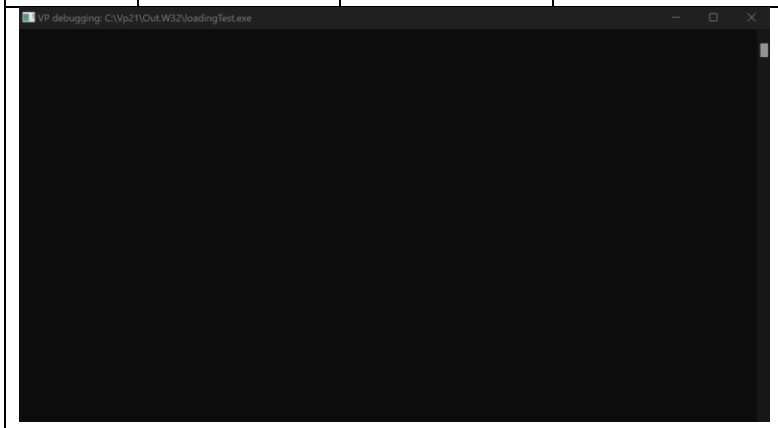
Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments

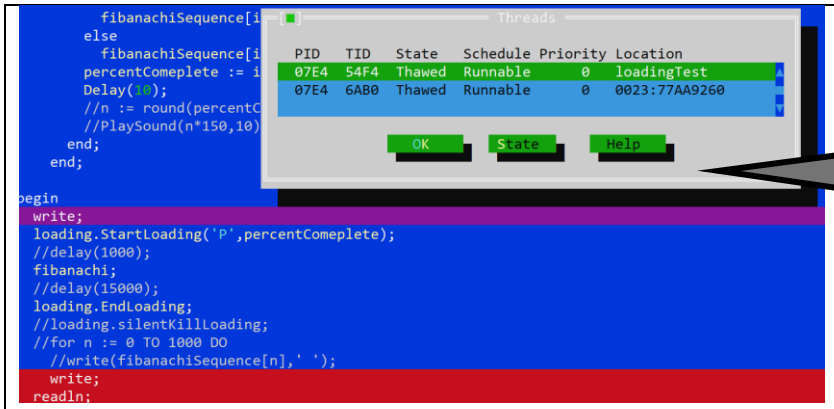
0.4.5	Loading Iteration 0	To test the progress bar works	A for looping counting up out of a total of 100	Progress
				0% (from 0 to 100 with correct bars)
					See below	There were 21 one spaces due to the for loop which sets the spaces going from 0 to 20 which is 21 numbers so this was changed to 19 for 20 blank spaces at the start.
						
						`for c:= 0 TO 19 DO`
						`write(#176);`
						
						Where the completed bars were being drawn in the for loop it starts from 0 to 0 which is still once to get it to run no times it needs to start at -1 and instead of that finishing at 20 it should also finish at 19. So it is now for d := o to completeBars**-1**.
						
						`for d := 0 to completeBars-1 DO`
						`write(#219);`
						
						The display for percent done now also needs to be moved closer as everything moved back
						
						`gotoxy(~~33~~→32,1);`



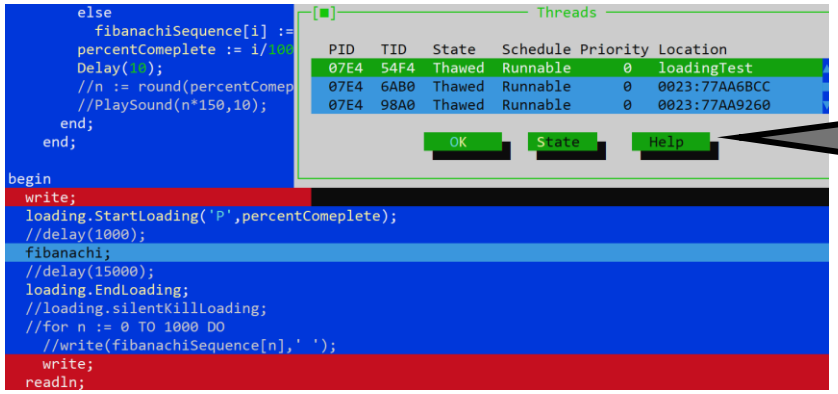
Amendments:



Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.4.6	Loading Iteration 1	To test if the progress bar ends properly	None needed	Time elapsed: (however long it took)	See below	n/a
						
0.4.7	Loading Iteration 1	Test if the progress bar ends using the silent kill	None needed	Nothing	Nothing	n/a
						
0.4.8	Loading Iteration 1	Test if the multi-threading is handled correctly, correctly starting and ending the threads	None needed	A number of threads before the loading script is called. An increase of 1 during the loading. And a decrease by 1 back to original number after loading is ended	See below	n/a



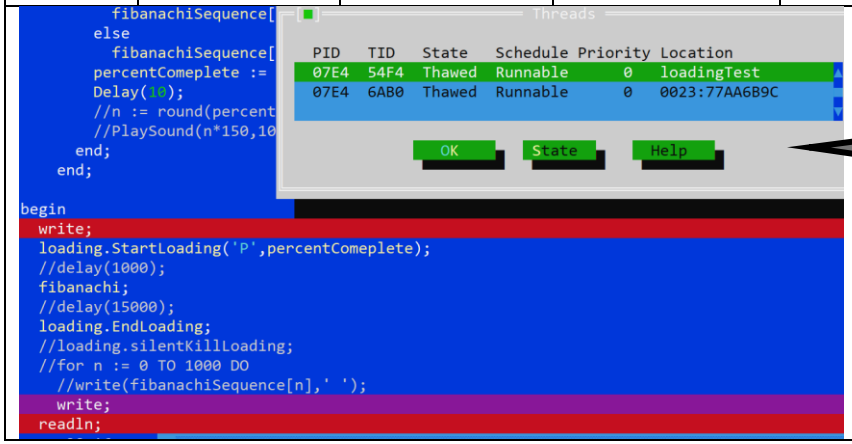
Two threads and before loading starts



Extra thread starts with the loading procedure

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments



Extra thread deleted after the end of the loading

Design

Iteration 0.5 – Log In

Code

```
unit LogInScreen;
  interface
  uses crt, sysutils, vputils; {importing built in units to help with the layout}

  TYPE userPassArray = ARRAY[0..1] OF STRING; {Use one data structure to return from the unit}

  VAR Username, Password : STRING; {Declaring variables}
    y : LONGINT;
    userPass : userPassArray;

  procedure GetUsername; {declaring all the procedures in this unit and any local variables and parameters
that they take}
  procedure GetPassword;
    var i, a : LONGINT;
    key : CHAR;
  procedure startLogIn(var userPassArrayr : ARRAY OF STRING);

  implementation

  PROCEDURE GetUsername; {The procedure that reads what the user types in for username}
      begin
        GotoXY(11,1);
        READLN(username);
      end;

  PROCEDURE GetPassword;
    var i,a : LONGINT;
    key : CHAR;
    begin
      GoToXY(11,2); {Moves the cursor to the correct position}
      i := 0;
      HideCursor; {Hides the cursor}
      password := ''; {sets the password to an empty string}
      repeat {repeat until enter is pressed to submit}
        key := readkey; {read the key that is pressed}
```

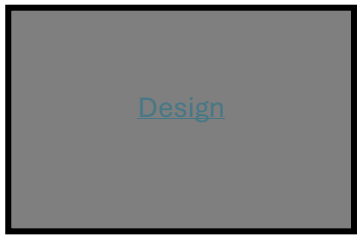
```
          if (key = #8) THEN {Check if its the back space key}
            begin
              if i > 0 THEN {check if the password has more than 0 characters in it}
                begin
                  Delete(password,i,1); {if so remove the last charachter in password}
                  i := i -1; {decrease the length of the password by 1}
                end;
                {else do nothing}
            end

          else if ((key >= #33) and (key < #127) and (key <> #13)) OR (key = #156) THEN {check if it is a
valid character and not a spcae}
            begin
              password := password + key; {add the key pressed to the password}
              i := i +1; {increase the length of password by 1}
            end;
          GoToXY(1,2); {go to the start of the password line}
          DelLine; {clears the line}
          Write('Password: '); {write the whole line again with the correct number of cogs}
          for a := 1 TO i DO
            write(#15);
    until key = #13;
      writeln;
    end;

  PROCEDURE DisplayInfo; {initial display of information that needs to be entered}
    Begin
      ClrScr;
      WRITELN('Username: ');
      WRITELN('Password: ');
      GoToXY(11,1); {go to the start of username}
    end;

  procedure startLogIn(var userPassArrayr : ARRAY OF STRING);
    begin
      DisplayInfo; {Display intial information}
      GetUsername; {get the username}
      GetPassword; {get the password}
```

```
      ClrScr; {clear the screen for the next task}
      userPassArrayr[0] := username; {put the username and password into the array to be editied}
      userPassArrayr[1] := password;
    end;
  end.
```



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.5.1	Login Iteration 0	To test if the password is hidden when typing	When it asks for username type bob then press enter. For password type abc123!"£	Username: bob Password: ☼☼☼☼☼☼☼☼☼☼	Username: bob Password: ☼☼☼☼☼☼☼☼☼☼	None Needed
Username: bob Password: **********						
0.5.2	Login Iteration 0	To test to see if the stored data is the same as the entered data	When it asks for username type bob then press enter. For password type abc123!"£	bob abc123!"£	bob abc123!"£	None Needed
bob abc123!"£						
0.5.2	Login Iteration 0	To test to see if the back space removes characters correctly in the hidden view	When it asks for username type bob then press enter. For password type a1!b2"c3£ Then press back space 3 times	Username: bob Password: ☼☼☼☼☼☼	Username: bob Password: ☼☼☼☼☼☼	None Needed
Username: bob Password: ******						
0.5.3	Login Iteration 0	To test to see if the backspace removes the characters from the stored data	When it asks for username type bob then press enter. For password type a1!b2"c3£ Then press back space 3 times	bob a1!b2"	bob a1!b2"	None Needed
bob a1!b2"						

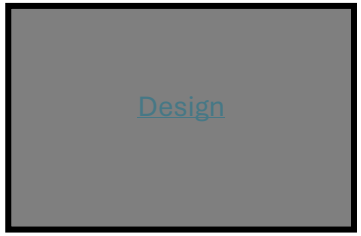
Design

Iteration 0.6 – Write Design

Code

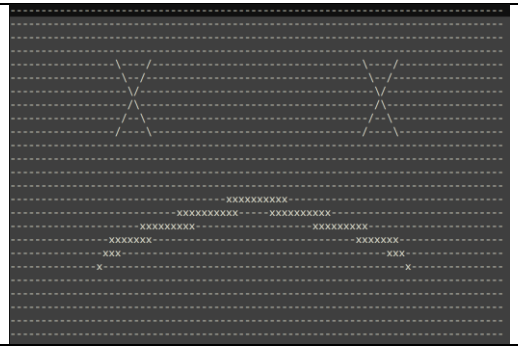
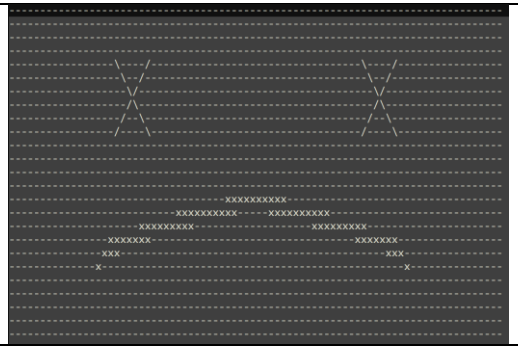
```
UNIT writeDesign;
  interface
  uses crt, sysutils, vputils;
  procedure DrawDesign(path: STRING; delayWrite : LONGINT);
  var Character : STRING;
      DesignTextFile : TextFile;
  implementation
  procedure DrawDesign(path: STRING; delayWrite : LONGINT);
    var a : LONGINT;
    begin
      HideCursor;
      ClrScr;
      //get the file to read from
      ASSIGN(DesignTextFile,path);
      reset(DesignTextFile);
      //read each line to a max of 28 (window size)
      for a := 0 to 28 DO
        begin
          readln(DesignTextFile, Character);
          //write each line
          writeln(Character);
          //delay for effects
          Delay(delayWrite);
        end;
      ShowCursor;

    end;
end.
```



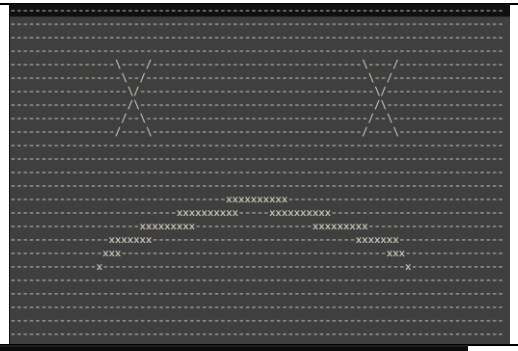
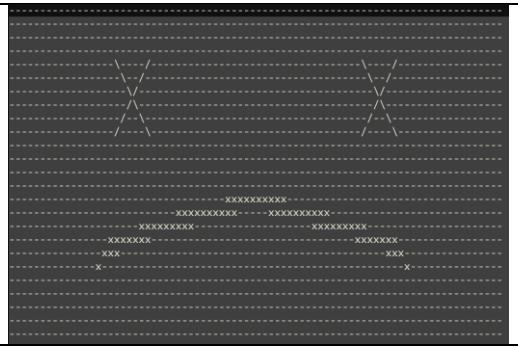
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

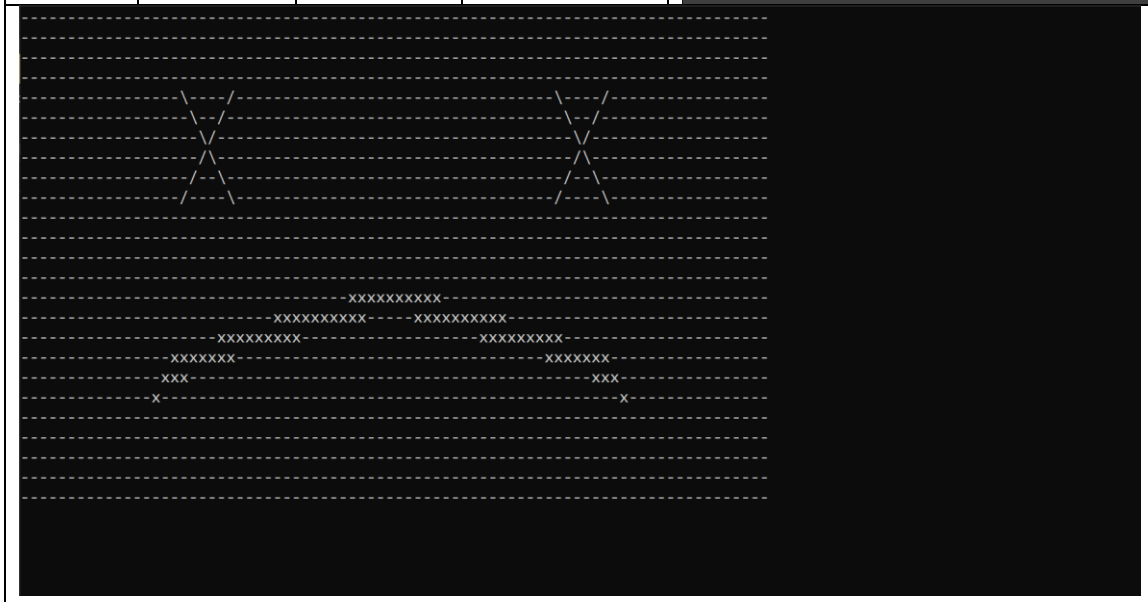
Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.6.1	Write Designs Iteration 0	This it to test if it works with full display	No data, just a text file with a design			n/a



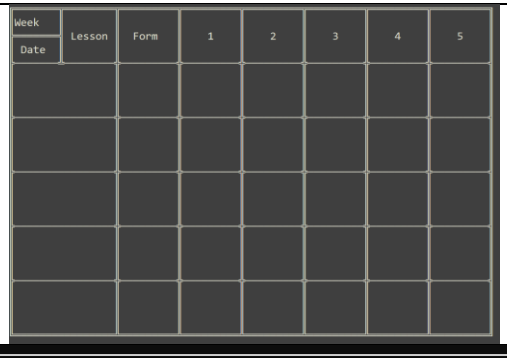
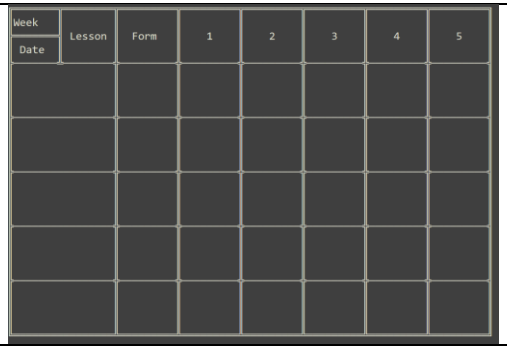
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

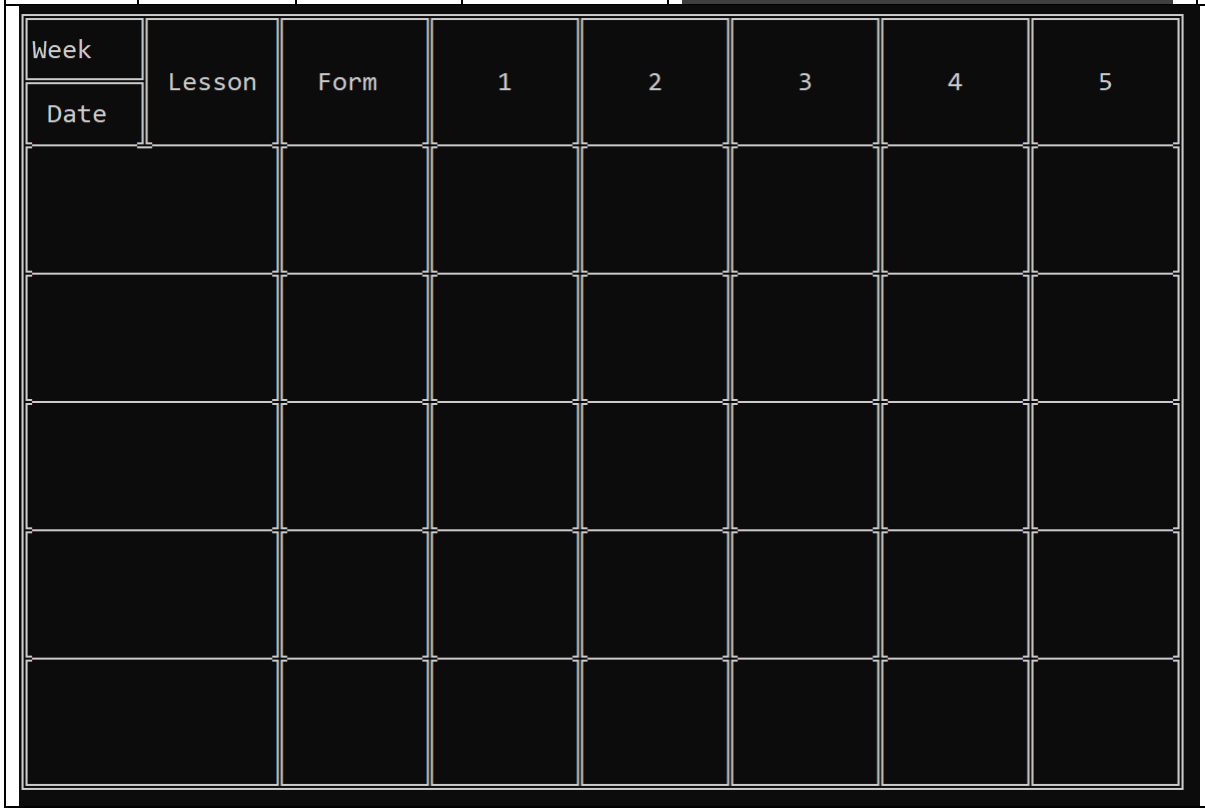
Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.6.2	Write Designs Iteration 0	This it to test if the animation feature works	Same file with a 100ms delay between each line			None

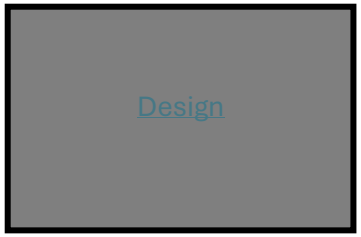


Mr Martin signature

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.6.3	Write Designs Iteration 0	Test if it works with something that has gaps in it	A different text file design			n/a





Iteration 0.7 – Cursor Grid

Code

```
UNIT CursorGrid;
  interface
  uses crt, sysutils, vputils;

  var stepx , stepY, movePos, minX, minY, maxX, maxY: LONGINT;

  procedure moveUp;
  procedure moveDown;
  procedure moveLeft;
  procedure moveRight;
  procedure startGrid(moveStepX, moveStepY, startX, startY : LONGINT; boundaries : ARRAY OF LONGINT; var x, y
: LONGINT);
  implementation
  procedure moveUp;
    begin
      //if the cursor will stay in the boundaries set
      if WhereY - stepy >= minY THEN
        begin
          movePos := WhereY - stepy;
          GoToXY(WhereX, movePos);
        end;
    end;

  procedure moveDown;
    begin
      //if the cursor will stay in the boundaries set
      if WhereY + stepy <= maxY THEN
        begin
          movePos := WhereY + stepy;
          GoToXY(WhereX, movePos);
        end;
    end;
  procedure moveLeft;
    begin
      //if the cursor will stay in the boundaries set
```

```
      if WhereX - stepx >= minX THEN
        begin
          movePos := WhereX - stepx;
          GoToXY(movePos, WhereY);
        end;
    end;

  procedure moveRight;
    begin
      //if the cursor will stay in the boundaries set
      if WhereX + stepx <= maxX THEN
        begin
          movePos := WhereX + stepX;
          GoToXY(movePos, WhereY);
        end;
    end;

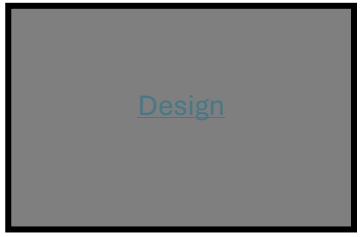
  procedure startGrid(moveStepX, moveStepY, startX, startY : LONGINT; boundaries : ARRAY OF LONGINT; var x, y
: LONGINT);
    var keyPress : char;
    begin
      stepX := moveStepX;
      stepY := moveStepY;
      minX := boundaries[0];
      minY := boundaries[1];
      maxX := boundaries[2];
      maxY := boundaries[3];
      GoToXY(startX,startY);


      //key checking key presses until enter is pressed
      repeat
        keyPress := readkey;
        if keyPress = #0 then //check if an extended key or function key
          begin
            //read its scan code to get which arrow key was pressed and call according procedure
            case readKey of
              'H' : moveUp;
```

```
              'P' : moveDown;
              'K' : moveLeft;
              'M' : moveRight;
            end;
          end;
      until keyPress = #13;
      //corrects the return to a choice of menu option rather than spaces moved
      if stepX <> 0 THEN//stops divide by 0 error
          x := Round((WhereX - startX) / stepX) //round to change float to int
      ELSE
          x := startX;

      if stepY <> 0 THEN//stops divide by 0 error
        y := round((WhereY - startY) / stepY) //round to change float to int
      ELSE
        y := startY;
    end;

  end.
```

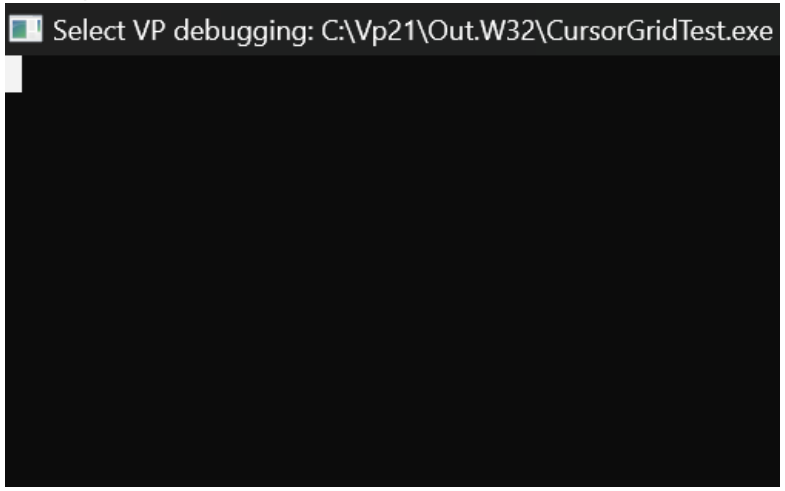


Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.7.1	Cursor Grid Iteration 0	To test if it can step the set amount	Set amount 2,2	Mouse only able to step 2 spaces in each direction	See below	n/a

Initial position



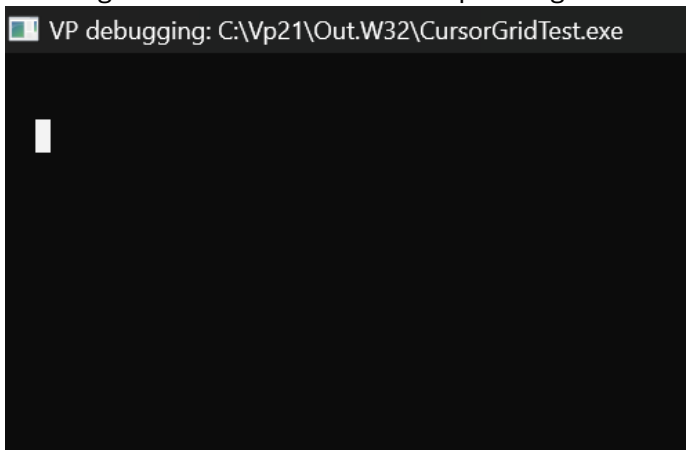
Press down once should lead to a step of 2 down

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments

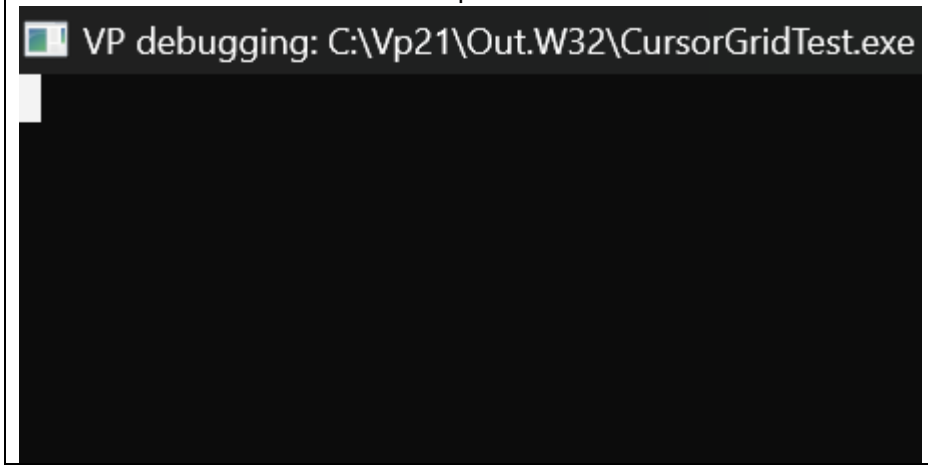


Press right once should lead to a step of 2 right



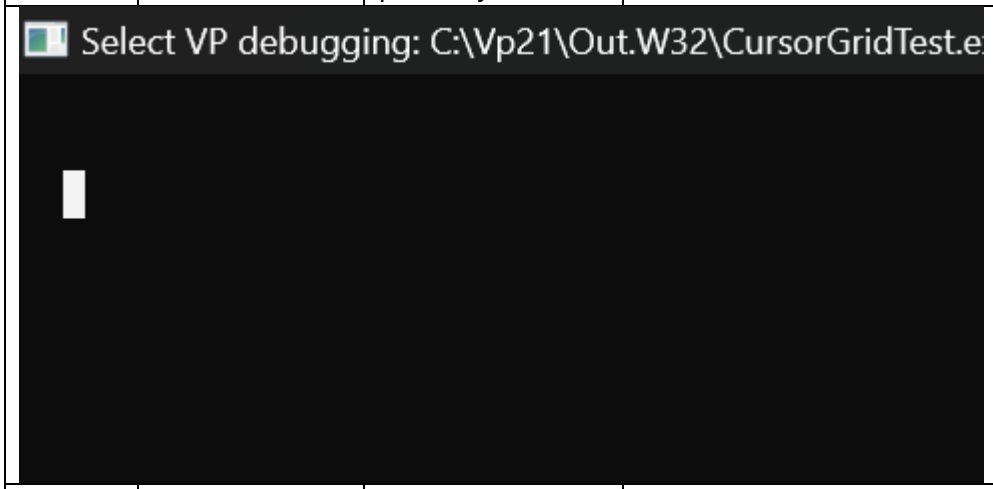
Press up once should lead to a step of 2 up

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
 Press left once should lead to a step of 2 left 						

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.7.2	Cursor Grid Iteration 0	To test if the boundary conditions will stop the cursor regardless of if it perfectly matches	Step 1, 2 Boundaries [1,1] and [3,4]	A max position of 3,3 before it stops it	See below	n/a



IT is currently at 3,3 and I can't move it further in any direction

0.7.3	Cursor Grid Iteration 0	TO test to see if it can start wherever	Start position at 5,4	The cursor starts at 5,4	See below	n/a



It is currently at position 5,4 which is where it starts

Design

Iteration 0.8 – Create Menu

Code

```
unit createMenu;
  interface
  uses crt, sysutils, CursorGrid, VPUtils;

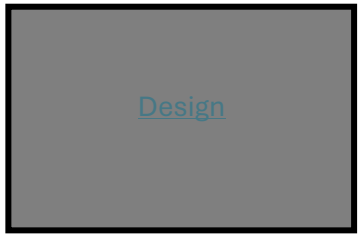
  function writeMenu(startPos : ARRAY OF LONGINT; numberOfChoices : LONGINT; menuChoices: ARRAY OF STRING;
optionStart : LONGINT) : LONGINT;

  implementation

  function writeMenu(startPos : ARRAY OF LONGINT; numberOfChoices : LONGINT; menuChoices: ARRAY OF STRING;
optionStart : LONGINT) : LONGINT;
    var a, x, y : LONGINT;
    begin
      HideCursor;

      //write each menu option
      for a := 0 TO numberOfChoices -1 DO
        begin
          //make sure it is alawys writing where it needs to
          GOTOXY(startPos[0],startPos[1]+a);
          writeln(a+1,': ',menuChoices[a]);
        end;
      ShowCursor;
      Delay(100);
      //start the cursor grid script with set boundary conditions and steps to allow control with arrow keys
      CursorGrid.startGrid(0,1,startPos[0],startPos[1]+optionStart,[startPos[0],startPos[1],startPos[0],number
OfChoices+startPos[1]-1],x,y);
      ClrScr;
      writeMenu := optionStart + y;
    end;

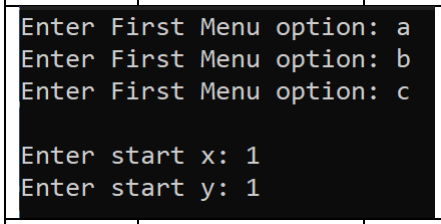
  end.
```

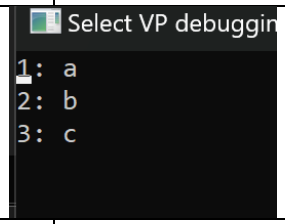


Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

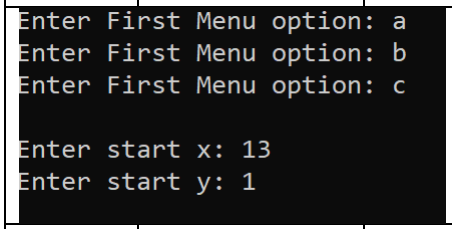
Testing

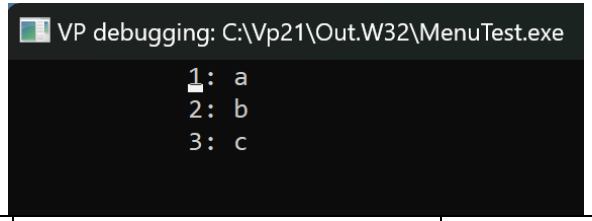
Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.8.1	Create menu Iteration 0	To see if the menu can write with correct format	Menu options = [a,b,c] Start position = [1,1] Start Menu option = 0 Number of menu options = 3	1. a 2. b 3. c	1. a 2. b 3. c	n/a



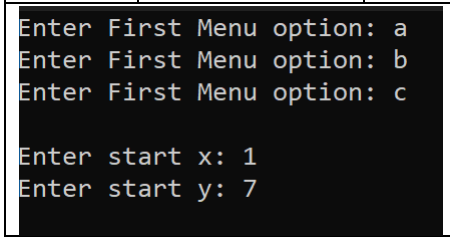


0.8.2.1	Create menu Iteration 0	To see if the menu can be written on any x	Menu options = [a,b,c] Start position = [13,1] Start Menu option = 0 Number of menu options = 3	1. a 2. b 3. c	1. a 2. b 3. c	n/a





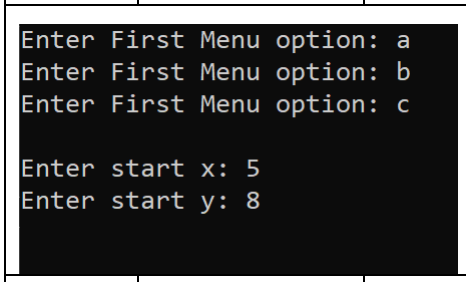
0.8.2.2	Create menu Iteration 0	To see if the menu can be written on any y	Menu options = [a,b,c] Start position = [1,7] Start Menu option = 0 Number of menu options = 3	1. a 2. b 3. c	1. a 2. b 3. c	n/a

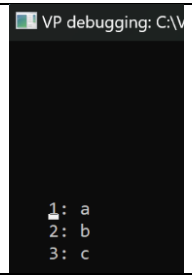




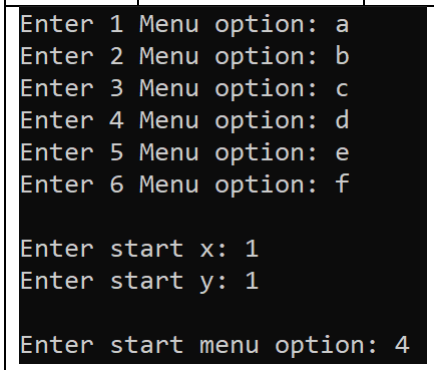
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

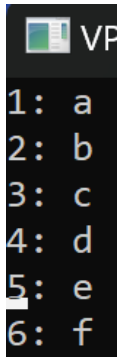
Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.8.2.2	Create menu Iteration 0	To see if the menu can be written on any x and y	Menu options = [a,b,c] Start position = [5,8] Start Menu option = 0 Number of menu options = 3	1. a 2. b 3. c	1. a 2. b 3. c	n/a





Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.8.3	Create menu Iteration 0	To see if the cursor can start on different values	Menu options = [a,b,c,d,e,f] Start position = [1,1] Start Menu option = 4 Number of menu options = 6	1. a 2. b 3. c 4. d 5. e 6. f	1. a 2. b 3. c 4. d 5. e 6. f	n/a



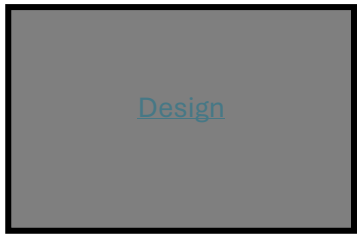


Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.8.4.1	Create menu Iteration 0	To see if it output the right result from which menu was selected	Menu options = [a,b,c,d,e,f] Start position = [1,1] Start Menu option = 0 Number of menu options = 6 Menu option to select = b	1	1	n/a
Menu option selected is number 1 which is b						
0.8.4.2	Create menu Iteration 0	To see if it output the right result from which menu was selected if it is first value	Menu options = [a,b,c,d,e,f] Start position = [1,1] Start Menu option = 0 Number of menu options = 6 Menu option to select = a	0	0	n/a
Menu option selected is number 0 which is a						
0.8.4.3	Create menu Iteration 0	To see if it output the right result from which menu was selected if it is last value	Menu options = [a,b,c,d,e,f] Start position = [1,1] Start Menu option = 0 Number of menu options = 6 Menu option to select = f	5	5	n/a
Menu option selected is number 5 which is f						
0.8.4.4	Create menu Iteration 0	Test to see if it works if offset in the y direction	Menu options = [a,b,c,d,e,f] Start position = [1,3] Start Menu option = 0 Number of menu options = 6 Menu option to select = c	2	2	n/a
Menu option selected is number 2 which is c						

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0.8.4.5	Create menu Iteration 0	Test to see if it works if offset in the x direction	Menu options = [a,b,c,d,e,f] Start position = [2,1] Start Menu option = 0 Number of menu options = 6 Menu option to select = d	3	3	n/a
`Menu option selected is number 3 which is d`						



Review

Due to this iteration being mostly background units designed and developed to be used by me as the programmer so most of this review will be a review from the programmer.

However, the following units my client did have comments on:

Loading: My client likes the idea of having a way to see that the program is actually doing something rather than crashed or stuck in an infinite loop.

Login: My client is happy with how it works and how the password is hidden while it is being typed in. She also likes how the character that is used to hide the password is not a character that can be easily typed in so she can use all special characters in her password without being restricted.

Cursor Grid: My client likes the ability to use the arrow keys to navigate through the menus as it feels more natural for her to use and it is more intuitive

From a programmer's perspective:

Iterations 0.1 to 0.3 allows me to search arrays whilst only calling one procedure making the code readability, it also means I only must include the search unit in the use instead of both binary and linear search.

The loading script can provide use information such as when a program breaks down, it its say at 56% I know where that is and can identify the issue faster.

The login script is very useful for me as the programmer as I can call start login from anywhere and it loads the login script and just returns the data that the user entered. This helps to keep the main program neater and more concise. This also allows for debugging of only the login script without having to go through the entire program

The write design script allows for easy graphical outputs. I can design an output in a text file and then just put the path into the procedure and it will be displayed

The cursor grid script allows for me to easily start a script which allows the use the arrow keys to navigate the screen and what is on the screen, and it returns the new position of the cursor so I can work out what was chosen

Iteration 0.8, the create menu script allows me as the programmer to pass an array of menu options that I want in the menu and it automatically writes all of them out and starts the cursor grid script to allows the user to navigate through the menu. This then returns only the menu option which was picked, which then allows me to pass that on to it relevant procedure.

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Iteration 1 – Creating Initial Data

Aim

The aim of this iteration is to generate data in which I can use to test the main program and prove and check that the main program is doing the correct task. Design the generation in a way that it is randomly generated so if any data gets corrupted, I can execute the program and have a whole new set of data to work with. I will set certain parts such as the timetable and I got a list of names from online which it reads from. These are kept the same so the data has a solid structure to build off and the rest of the data can be randomly generated without causing issues. I will keep the names the same so I always know a couple of users I can use as tests even if the rest of the data about them changes. I will also do this procedurally to allow me to generate large samples of data without having to manually input all the data into the file

Code

```
PROGRAM CreateFiles;
  uses crt, sysutils, search,sorting, loading, DisplayErrorScrn, math;

  type {declaring all record types}
    TimeTableRecord = RECORD
                        Day : LONGINT;
                        LessonNo : LONGINT;
                        ClassID : STRING[8];
                      end;
    FormRecord = RECORD
                        FormID : STRING[5];
                        NumberInForm : LONGINT;
                      end;

    AttendanceRecord = RECORD
                        StudentID : LONGINT;
                        Attendance : REAL;
                        Total : LONGINT;
                        NumberPresent : LONGINT;
                        NumberLate : LONGINT;
                        NumberAbsent : LONGINT;
                      end;

    StudentsRecord = RECORD
                        StudentID : LONGINT;
                        FName : STRING[20];
                        SName : String[20];
                        Year : LONGINT;
                        Form : STRING[5];
                        Uname : String[40];
                        Pword : String[20];
                      end;

    teacherRecord = RECORD
                        TeacherID : STRING[3];
                        fname : string[20];
                        sname : string[20];
                        form  : string[5];
                        Uname : STRING[40];
                        PWord : STRING[20];
```

```
                        end;

    classRecord = RECORD
                          ClassID : STRING[8];
                          Year : LONGINT;
                          subjectID : STRING[3];
                          NoOfStudents : LONGINT;
                        end;

    subjectRecord = RECORD
                          SubjectID : STRING[3];
                          SubjectName : STRING[20];
                        end;

    studentRegisterMarkRecord = RECORD
                          studentID : LONGINT;
                          registerMark : CHAR;
                        end;

    registerRecord = RECORD
                          RegisterID : STRING[17];
                          LessonNo : LONGINT;
                          Date : TDateTime;
                          ClassID : STRING[8];
                          RegisterMarks : ARRAY[0..40] OF studentRegisterMarkRecord;
                          isOfstedRegister : BOOLEAN;
                        end;

    studentClassResolverRecord = RECORD
                          StudentID : LONGINT;
                          ClassID : STRING[8];
                        end;
    teacherSubjectResolverRecord = RECORD
                          TeacherID : STRING[3];
                          SubjectID : STRING[2];
                        end;

    teacherClassResolverRecord = RECORD
                          TeacherID : STRING[3];
                          ClassID : STRING[6];
                        end;
```

```
    LogInDetailsRecord = RECORD
                          PersonID: STRING[5];
                          Username: STRING[40];
                          Passwowrd: String[20];
                         end;
  VAR {setting up all variables for records and files}
    numberDone, total : LONGINT;

    TimeTable : TimeTableRecord;
    TimeTableTextFile : TextFile;
    TimeTableFile : FILE OF TimeTableRecord;
    TimeTableInfoFromFile : STRING;

    Form : FormRecord;
    FormFile : FILE OF FormRecord;

    Register : registerRecord;
    studentRegisterMark : studentRegisterMarkRecord;
    RegisterFile : File OF registerRecord;

    Attendance : AttendanceRecord;
    AttendanceFile : FILE OF AttendanceRecord;
    StudentFName : STRING;
    NameFile : TextFile;
    Student : StudentsRecord;
    StudentFile : FILE OF StudentsRecord;

    TeacherFileFromFile : TextFile;
    TeacherFile : file of teacherRecord;
    TeacherFromFile : String;
    Teacher : teacherRecord;

    sClass : ClassRecord;
    classFile : FILE OF ClassRecord;

    subject : subjectRecord;
    subjectFile : FILE OF subjectRecord;

    studentClassResolver : studentClassResolverRecord;
    studentClassResolverFile : FILE OF studentClassResolverRecord;
```

```
    teacherSubjectResolver : teacherSubjectResolverRecord;
    teacherSubjectResolverFile : FILE OF teacherSubjectResolverRecord;

    teacherClassResolver : teacherClassResolverRecord;
    teacherClassResolverFile : FILE OF teacherClassResolverRecord;

    empty, ErrCode : LONGINT; {setting up other variables for functions that require an input}
    percentComplete : REAL; {variable for progress bar to track progress}

    LogInDetails: LogInDetailsRecord;
    LogInDetailsFile : FILE OF LogInDetailsRecord;

     {setting up common array across many functions}
    classIDArray : ARRAY[0..50] OF STRING;


  procedure initialiseFiles;
    begin
      percentComplete := 0.0; {initializing percentDone and total iterations}
      total := 0;
      {assigning a place on the drive to the files}
      ASSIGN(TimeTableTextFile,'C:\ComputerScienceCoursework\Program\files\TimeTableText.txt');
      ASSIGN(TimeTableFile,'C:\ComputerScienceCoursework\Program\files\TimeTable.dta');
      total:= total +51;

      ASSIGN(RegisterFile,'C:\ComputerScienceCoursework\Program\files\Registers.dta');
      total := total + 60;

      ASSIGN(NameFile,'C:\ComputerScienceCoursework\Program\files\Names.txt');
      ASSIGN(StudentFile,'C:\ComputerScienceCoursework\Program\files\Students.dta');
      total := total + 140;

      ASSIGN(TeacherFile,'C:\ComputerScienceCoursework\Program\files\Teachers.dta');
      total := total + 51;

      ASSIGN(classFile,'C:\ComputerScienceCoursework\Program\files\classes.dta');
      total := total + 51;

      ASSIGN(subjectFile,'C:\ComputerScienceCoursework\Program\files\subjects.dta');
      total := total + 21;
```

```
    ASSIGN(studentClassResolverFile,'C:\ComputerScienceCoursework\Program\files\studentClassResolver.dta');
    total := total + 140;

    ASSIGN(teacherSubjectResolverFile,'C:\ComputerScienceCoursework\Program\files\teacherSubjectResolver.dta');
    total:= total + 51;

    ASSIGN(teacherClassResolverFile,'C:\ComputerScienceCoursework\Program\files\teacherClassResolver.dta');
    total := total + 51;

    ASSIGN(FormFile,'C:\ComputerScienceCoursework\Program\files\forms.dta');
    total := total +5;

    ASSIGN(AttendanceFile,'C:\ComputerScienceCoursework\Program\files\attendanceFile.dta');

    ASSIGN(LogInDetailsFile,'C:\ComputerScienceCoursework\Program\files\LogInDetails.dta');
    total := total + 191;
  end;

procedure closeAllFiles;
  begin
    {closing all files to confirm all changes are fully written and complete}
    close(TimeTableTextFile);
    close(TimeTableFile);
    close(RegisterFile);
    close(NameFile);
    close(StudentFile);
    close(TeacherFile);
    close(classFile);
    close(subjectFile);
    close(studentClassResolverFile);
    close(teacherSubjectResolverFile);
    close(teacherClassResolverFile);
    close(FormFile);
    close(AttendanceFile);
    close(LogInDetailsFile);
  end;

procedure CreateTimeTable;

  begin
```

```
    reset(TimeTableTextFile); //set pointer to start of file
    rewrite(TimeTableFile); //clear all exiting data in timetable file
    while not EOF(TimeTableTextFile) do //iterate until all data is extracted
      begin
        numberDone := numberDone +1;
        percentComplete := numberDone/total;
        readln(TimeTableTextFile,TimeTableInfoFromFile); //get the class id
        TimeTable.ClassId := TimeTableInfoFromFile; //assign it to the record variable
        repeat
          readln(TimeTableTextFile,TimeTableInfoFromFile);
          TimeTable.LessonNo := StrToInt(TimeTableInfoFromFile);
          readln(TimeTableTextFile,TimeTableInfoFromFile);
          TimeTable.Day := StrToInt(TimeTableInfoFromFile);
          readln(TimeTableTextFile,TimeTableInfoFromFile);
          try
            write(TimeTableFile,TimeTable);
          finally
          end;
        until TimeTableInfoFromFile = 'Q'; {repeat until all dates are read and stored in the .dta file
                                            for that class marked by a Q in the text file}
      end;
  end;

procedure addForms;
  var a, i : LONGINT;
  begin
    REWRITE(FormFile); {clear all exisiting data in the form file}
    for i := 0 to 3 DO {iterate for all 4, year 9 forms}
      begin
        numberDone := numberDone + 1;
        percentComplete := numberDone / total;
        form.formID := '9' + 'AA' + Chr(i +65); {create 4 unique for identifiers}
        form.NumberInForm := 0; {set Number of kids in form to 0}
        write(FormFile, form); {push data to file}
      end;
    numberDone := numberDone + 1;
    percentComplete := numberDone / total;
    form.formID := '12' + 'AAA'; {do the same for 1 year 12 form}
    form.NumberInForm := 0;
    write(FormFile,form);
    reset(FormFile); {set point to start of form file}
```

```
    for i := 0 to 4 DO {for every form}
      begin
      read(formFile,form);
      timetable.classid := form.formID; {get form id into a timetable class id variable}
        for a := 1 to 10 DO {add it to the file as lesson 0 for every morning in the 2 week timetabe}
          begin
            timetable.LessonNo := 0;
            timetable.Day := a;
            write(TimeTableFile, timetable); {add new data to timetable file}
          end;
      end;
  end;

procedure createRegisters;
  var a,b,c,i,randomMark,day : LONGINT; {Initizlize iterative variables, arrays and other variables for this procedure}
      dayArray : ARRAY[0..320] OF STRING;
      dayReturnArray : ARRAY[0..320] OF LONGINT;
      studentClassResolverClassArray : ARRAY[0..1384] OF STRING;
      classReturnArray : ARRAY[0..1384] OF LONGINT;
      classId, startDate : STRING;
      studentFormIdArray : ARRAY[0..139] OF STRING;
      returnStudentFormIdArray : ARRAY[0..139] OF LONGINT;
      date : TDateTime;
  begin
    //set up files and reset file pointers
    reset(TimeTableFile);
    reset(studentClassResolverFile);
    RESET(StudentFIle);
    rewrite(RegisterFile);
    i := 0;
    startDate := '13/09/2024';
    //populating the arrays with the data from the files
    for i := 0 To 139 DO
      begin
        READ(StudentFile,Student);
        studentFormIdArray[i] := Student.Form;
        returnStudentFormIdArray[i] := -1;
      end;
    i := 0;
    while not(eof(TimeTableFile)) DO
      begin
```

```
          read(TimeTableFile,TimeTable);
          dayArray[i] := IntToStr(TimeTable.day);
          dayReturnArray[i] := -1;
          i := i +1;
        end;
      i := 0;
      while not(eof(studentClassResolverFile)) do
        begin
          read(studentClassResolverFile,studentClassResolver);
          studentClassResolverClassArray[i] := studentClassResolver.ClassID;
          classReturnArray[i] := -1;
          i := i +1;
        end;
      b := 0;

      // for loop for number of days with registers
      for i := 0 to 29 do
        begin
          //gets the number day of the week and corrects to monday being 0
          day := DayOfWeek(StrToDate(startDate)+b-1);
          if (day = 7) OR (day = 6) THEN //if weekend decrease for loop var by 1
            begin
              i := i -1;
            end
          else
            begin
              numberDone := numberDone +1;
              percentComplete := numberDone / total;
              //clear out array for searching for classes on that day
              for c := 0 TO 269 DO
                dayReturnArray[c] := -1;
              Search.SearchFunctions('L',0,269,IntToStr(DayOfWeek(StrToDate(startDate)+b-
1)),dayArray,dayReturnArray,empty);
              a := 0;
              while dayReturnArray[a] <> -1 DO // for every class on that day
                begin
                  //find and get the data about one class
                  SEEK(TimeTableFile,dayReturnArray[a]);
                  read(TimeTableFile, TimeTable);
                  //set register data
                  Register.LessonNo := TimeTable.LessonNo;
```

```
                Register.Date := StrToDate(startDate)+b;
                if TimeTable.LessonNo = 4 THEN
                  Register.isOfstedRegister := true
                else
                  Register.isOfstedRegister := false;
                Register.classID := '';
                Register.classID := TimeTable.classID;
                register.registerID := IntToStr(register.lessonNo) + register.classID + DateTimeToStr(register.date);
                classId := format('%s',[Register.classID]);
                //clear out array for searching for students in that class
                for c := 0 to 1384 DO
                  classReturnArray[c] := -1;
                  Search.SearchFunctions('L',0,1384,Register.ClassID,studentClassResolverClassArray,classReturnArray,em
pty);
                c:=0;
                //for every student in that class
                while classReturnArray[c] <> -1 DO
                  begin
                    //get the class with the studenet in
                    SEEK(studentClassResolverFile, classReturnArray[c]);
                    read(studentClassResolverFile, studentClassResolver);
                    //get the student id
                    studentRegisterMark.studentID := studentClassResolver.studentID;
                    //get the data in the attendance file
                    SEEK(AttendanceFile,studentClassResolver.studentID);
                    read(AttendanceFile,Attendance);
                    Attendance.Total := Attendance.Total +1;
                    //randomly assign the student a mark
                    randomMark := RANDOM(100);
                    if randomMark <= 90 then
                      begin
                        studentRegisterMark.registerMark := 'P';
                        Attendance.NumberPresent := Attendance.NumberPresent + 1;
                      end
                    else if randomMark <= 97 then
                      begin
                        studentRegisterMark.registerMark := 'L';
                        Attendance.NumberLate := Attendance.NumberLate + 1;
                      end
                    else
                      begin
```

```
                        studentRegisterMark.registerMark := 'A';
                        Attendance.NumberAbsent := Attendance.NumberAbsent + 1;
                      end;
                    //write data back to register file
                    Register.registerMarks[c] := studentRegisterMark;
                    SEEK(AttendanceFile,FilePos(AttendanceFile)-1);
                    Attendance.Attendance := (Attendance.NumberPresent / Attendance.Total);
                    write(AttendanceFile,Attendance);
                    c := c +1;
                  end;
                write(RegisterFile,Register);

                a := a +1;
              end;
          end;
          b := b +1;
      end;

      b:= 0;
      for i := 0 to 29 DO //for loop for each form
        begin
          //gets the number day of the week and corrects to monday being 0
          day := DayOfWeek(StrToDate(startDate)+b-1);
          if (day = 7) OR (day = 6) THEN //if weekend decrease for loop var by 1
            begin
              i := i -1;
            end
          else
            begin
              numberDone := numberDone +1;
              percentComplete := numberDone / total;
              //clear out array for searching for forms on that day
              for c := 0 TO 320 DO
                dayReturnArray[c] := -1;
              Search.SearchFunctions('L',269,320,IntToStr(DayOfWeek(StrToDate(startDate)+b-
1)),dayArray,dayReturnArray,empty);
              a := 0;
              while dayReturnArray[a] <> -1 DO// for every form on that day
                begin
                  SEEK(TimeTableFile,dayReturnArray[a]);
                  read(TimeTableFile, TimeTable);
```

```
                    if TimeTable.LessonNo = 0 THEN
                      begin
                        //set register data
                        Register.LessonNo := TimeTable.LessonNo;
                        Register.Date := StrToDate(startDate)+b;
                        Register.isOfstedRegister := true;
                        Register.classID := '';
                        Register.classID := TimeTable.classID;
                        register.registerID := IntToStr(register.lessonNo) + register.classID +
DateTimeToStr(register.date);
                        //clear out array for searching for students in that form
                        classId := format('%s',[Register.classID]);
                        for c := 0 TO 139 DO
                          returnStudentFormIdArray[c] := -1;
                        Search.SearchFunctions('L',0,139,Register.ClassID,studentFormIdArray,returnStudentFormIdArray,emp
ty);
                        c := 0;
                        //for every student in that form
                        while returnStudentFormIdArray[c] <> -1 DO
                          begin
                            //get the class with the studenet in
                            SEEK(StudentFile, returnStudentFormIdArray[c]);
                            read(StudentFile, Student);
                            //get the student id
                            studentRegisterMark.studentID := student.studentID;
                            //get the data in the attendance file
                            SEEK(AttendanceFile,student.studentID);
                            read(AttendanceFile,Attendance);
                            Attendance.Total := Attendance.Total +1;
                            //randomly assign the student a mark
                            randomMark := RANDOM(100);
                            if randomMark <= 90 then
                              begin
                                studentRegisterMark.registerMark := 'P';
                                Attendance.NumberPresent := Attendance.NumberPresent + 1;
                              end
                            else if randomMark <= 97 then
                              begin
                                studentRegisterMark.registerMark := 'L';
                                Attendance.NumberLate := Attendance.NumberLate + 1;
                              end
```

```
                            else
                              begin
                                studentRegisterMark.registerMark := 'A';
                                Attendance.NumberAbsent := Attendance.NumberAbsent + 1;
                              end;
                            //write data back to register file
                            Register.registerMarks[c] := studentRegisterMark;
                            SEEK(AttendanceFile,FilePos(AttendanceFile)-1);
                            Attendance.Attendance := (Attendance.NumberPresent / Attendance.Total);
                            write(AttendanceFile,Attendance);
                            c := c +1;
                          end;
                          write(RegisterFile,Register);

                      end;
                      a := a + 1;
                  end;
              end;
              b := b + 1;
          end;
    end;

  procedure addStudents;
    var i : LONGINT;
      ffname, fsname, fyear, fform, fattendance : STRING; {set up variables to read from text file into}
    begin
      i := 0;
      RESET(FormFile); {resets pointer in form file}
      read(formFIle, form); {gets the first form}
      RESET(NameFile);
      REWRITE(StudentFile); {clears both studenet and attendance file}
      REWRITE(AttendanceFile);
      for i := 0 to 119 DO {generate 120 year 9 students}
        begin
          if (i mod 30 = 0) and (i <> 0) THEN //changes the form every 30 students
            begin
              seek(formFile, (FilePos(formFile)-1));
              write(FormFile,form);
              READ(FormFile, form);
            end;
          numberDone:= numberDone +1; //update progress bar
```

```
      percentComplete := numberDone/total;
      Student.StudentID := i; //set student data
      READLN(NameFile, student.fname); //get a first and last name from a list of names
      READLN(NameFile, student.sname);
      student.year := 9;
      student.uname := student.fname+student.sname;
      student.pword := 'ABC123';
      student.form := form.formid;
      form.NumberInForm := form.NumberInForm + 1;
      //set attendance data
      Attendance.studentId := i;
      Attendance.Attendance := 1;
      Attendance.Total := 0;
      Attendance.NumberPresent := 0;
      Attendance.NumberLate := 0;
      Attendance.NumberAbsent := 0;
      write(StudentFile,student); //write student and attendance data to their respective files
      write(AttendanceFile, Attendance);
    end;
  read(FormFile, form); {get the year 12 form}
  for i := 120 to 139 DO {generate 20 year 12 students}
    begin
      numberDone := numberDone +1; //update progress bar
      percentComplete := numberDone/total;
      Student.StudentID := i; //set student data
      READLN(NameFile, student.fname); //get a first and last name from a list of names
      READLN(NameFile, student.sname);
      student.year := 12;
      student.uname := student.fname+student.sname;
      student.pword := 'ABC123';
      form.NumberInForm := form.NumberInForm + 1;
      student.form := form.formid;
      //set attendance data
      Attendance.studentId := i;
      Attendance.Attendance := 1;
      Attendance.Total := 0;
      Attendance.NumberPresent := 0;
      Attendance.NumberLate := 0;
      Attendance.NumberAbsent := 0;
      write(StudentFile,student); //append this record to the end of the student file
      write(AttendanceFile, Attendance);
```

```
        end;
        //write changes to form file
        seek(formfile, filepos(formFile)-1);
        write(formfile,form);
    end;

  procedure addTeachers;
    var i : LONGINT;
      ffname, fsname, fform : STRING; {declare all variables to read from files into them}
    begin
      RESET(FormFile);
      rewrite(TeacherFile);
      for i := 0 TO 50 DO //create 51 teachers, 1 per class
        begin
            numberDone := numberDone + 1;
            percentComplete := numberDone / total;
            READLN(NameFile, teacher.Fname); //read first and surname form names file
            READLN(NameFile, teacher.Sname);
            Teacher.TeacherID := teacher.FName[1]+teacher.SName[1]+teacher.SName[2]; //create the teacher id as first
letter of first name and forst two letters of surname
            Teacher.uname := teacher.FName + teacher.SName;
            Teacher.pword := 'ABC123';
            write(TeacherFile,teacher);
        end;
      //assign teachers to forms
      RESET(TeacherFile);
      for i := 0 to 4 DO
        begin
          read(TeacherFile,teacher);
          read(FormFile,form);
          teacher.form := form.formID;
          seek(TeacherFile,FilePos(TeacherFile)-1);
          write(TeacherFile,teacher);
        end;
    end;

  procedure createLogInFile;
      begin
        RESET(StudentFile); //reset file pointers for both files
        RESET(TeacherFile);
        REWRITE(LogInDetailsFile);
```

```
        //extract login information and student id and write it to a file
        while not EOF(StudentFile) do
          begin
            READ(StudentFile,student);
            LogInDetails.PersonID := IntToStr(student.StudentID); //convert student id from int to string
            LogInDetails.Username := student.Uname;
            LogInDetails.Passwowrd := student.Pword;
            write(LogInDetailsFile,LogInDetails);
            numberDone := numberDone +1;
            percentComplete := numberDone/total;
          end;
        while not EOF(TeacherFile) do
          begin
            //extract login information and student id and write it to a file
            READ(TeacherFile,teacher);
            LogInDetails.PersonID := teacher.teacherID;
            LogInDetails.Username := teacher.Uname;
            LogInDetails.Passwowrd := teacher.Pword;
            write(LogInDetailsFile,LogInDetails);
            numberDone := numberDone +1;
            percentComplete := numberDone/total;
          end;

      end;

  PROCEDURE createSubjects;
    // create a constant array of all subjects and the subjects' id
    const subjID : ARRAY[0..20] OF STRING[3] = ('Cs','Ma','Mu','Enl','Gg','Ch','Ph','Bi','DT','Dr',
                                                'Lf','Rpe','Fr','Hi','Gm','Bs','Ar','Py','Pe','Sp','So');
          subjNm : ARRAY[0..20] OF STRING[20] = ('Computer
Science','Maths','Music','English','Geography','Chemistry','Physics','Biology','Design Technology','Drama',
                                                'Life Skills','Religion, Philosophy &
Ethics','French','History','German','Business Studies','Art','Physcology','Physical Education','Spanish','Sociology');
    var i : LONGINT;
    begin
      //clear subject file
      REWRITE(SubjectFile);
      //for every subject write subject name and id to a file
      for i := 0 TO 20 do
        begin
          numberDone := numberDone +1;
```

```
          percentComplete := numberDone / total;
          with subject do
            begin
              subjectID := subjID[i];
              SubjectName := subjNm[i];
            end;
            WRITE(SubjectFile, subject);
          end;
      end;

  PROCEDURE createClasses;
    //create arrays of all classes
    const yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph', '9AT2/Ph',
'9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph', '9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs1', '9D/Mu1', '9D/Gm1');
          yTwelveClassArray : ARRAY[0..6] OF STRING =
('12A/Ph1','12A/Py1','12B/Ph1','12C/Cp1','12C/Py1','12D/Mu1','12D/Cp1');
      VAR i : LONGINT;
          classArray : ARRAY[0..50] OF classRecord;
      begin
        rewrite(classFile);
        //for every year 9 class
        for i := 0 TO 43 DO
          begin
            numberDone := numberDone + 1;
            percentComplete := numberDone/total;
            //set year 9 class data
            sClass.ClassID := yNineClassArray[i];
            sClass.year := 9;
            sClass.NoOfStudents := 0;
            //generate the class id
            val(yNineClassArray[i][8],empty,ErrCode);
            if ErrCode <> 0 THEN
              begin
                sClass.subjectID := yNineClassArray[i][6]+yNineClassArray[i][7]+yNineClassArray[i][8];
                val(sClass.subjectID[1],empty,ErrCode);
                if ErrCode = 0 THEN
                  begin
```

```
                  sClass.subjectid := yNineClassArray[i][4]+yNineClassArray[i][5];
                end;
              end
            else
              sClass.subjectID := yNineClassArray[i][6]+yNineClassArray[i][7];
            write(classFile, sClass);
            delay(100);
          end;
        //generate year 12 classes
        for i := 0 TO 6 DO
          begin
            numberDone:= numberDone +1;
            percentComplete := numberDone/total;
            sClass.ClassId := yTwelveClassArray[i];
            sClass.year := 12;
            sClass.NoOfStudents := 0;
            sClass.subjectID := yTwelveClassArray[i][5]+yTwelveClassArray[i][6];
            write(classFile, sClass);
          end;
        reset(classFile);
        for i := 0 to 50 do
        begin
          READ(classFile, classArray[i]);
          classIDArray[i] := classArray[i].classID;
        end;
      end;

  procedure addRecordsStudentClassResolver(start, noOfAviablableClasses, maxStudents, ClasesToSortInto, year : LONGINT);
    const yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph', '9AT2/Ph',
'9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph', '9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs1', '9D/Mu1', '9D/Gm1');
          yTwelveClassArray : ARRAY[0..6] OF STRING =
('12A/Ph1','12A/Py1','12B/Ph1','12C/Cp1','12C/Py1','12D/Mu1','12D/Cp1');
    VAR startPoint, randomChoiceClass, i,d,l : LONGINT;
        returnArray : ARRAY[0..50] OF LONGINT;
    begin
      RESET(classFile);
      d := 0;
```

```
      repeat
        d := d + 1;
        randomChoiceClass := RANDOM(noOfAviablableClasses);  //randomly picks classes from the number of available class
that student can be in
        if year = 9 THEN
          Search.SearchFunctions('L',0,50,yNineClassArray[start+randomChoiceClass],classIDArray,returnArray,empty)
//checks the year 9 to find the class chosen
        else if year = 12 THEN
            Search.SearchFunctions('L',0,50,yTwelveClassArray[start+randomChoiceClass],classIDArray,returnArray,empty)
//checks the year 12 to find the class chosen
        else
          DisplayErrorScrn.ErrScr;
        SEEK(classFile, returnArray[0]); //find the class in the file
        read(classFile, sClass);
        if d > 10000 THEN
          begin
            readln;
          end;
        until sClass.NoOfStudents < maxStudents; //only exit if there is a place for the student in that class
      l := 0;
      startPoint := returnArray[0];
      for l := 0 to (ClasesToSortInto * 2) -1 do //for every class connceted to that one class
        begin
          if year = 9 THEN
            studentClassResolver.classID := yNineClassArray[start+randomChoiceClass+l]
          else if year = 12 THEN
              studentClassResolver.classID := yTwelveClassArray[start+randomChoiceClass+l]
          else
            DisplayErrorScrn.ErrScr;
          write(studentClassResolverFile, studentClassResolver); //write each student and the class to the resolver file
          //update number of students in that class
          SEEK(classFile, startPoint + l);
          read(classFile, sClass);
          SEEK(classFile, FilePos(classFile)-1);
          sClass.NoOfStudents := sClass.NoOfStudents + 1;
          write(classFile, sClass);
          l := l + 1;
        end;
    end;
```

```
  procedure createClassStudentResolver;
    const yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph', '9AT2/Ph',
'9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph', '9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs1', '9D/Mu1', '9D/Gm1');
          yTwelveClassArray : ARRAY[0..6] OF STRING =
('12A/Ph1','12A/Py1','12B/Ph1','12C/Cp1','12C/Py1','12D/Mu1','12D/Cp1');
    var returnArray : ARRAY[0..51] OF LONGINT;
        studentYear,a, d, i, startPoint, noOfAlevels, noTakingFourAlevels, randClassNo, randomSide, numberInASide,
numberInBSide, randomChoiceClass : LONGINT;
        oldClass, newClass : STRING;

    begin
      RESET(StudentFile);
      RESET(classFile);
      rewrite(studentClassResolverFile);
      //keep track of how many students on each side of the year
      numberInASide := 0;
      numberInBSide := 0;
      randomSide := RANDOM(2);
      noTakingFourAlevels := 0;
      while not(eof(StudentFile)) do //for every student
        begin
          numberDone := numberDone+1;
          percentComplete := numberDone/total;
          READ(StudentFile,student); //get the student data
          studentClassResolver.studentID := student.StudentID;
          if student.year = 9 THEN
            begin
              //pick a random side without overflowing one side
              if (numberInASide < 60) AND (numberInBSide < 60) THEN
                randomSide := RANDOM(2)
              else if (numberInASide < 60) THEN
                randomSide := 0
              else if (numberInBSide < 60) THEN
                randomSide := 1;
              if randomSide = 0 THEN //a or b side
                begin
                    //put students in each set of classes
```

```
            numberInASide := numberInASide + 1;
            //Science
            addRecordsStudentClassResolver(0, 2, 30, 3, 9);
            //Maths
            addRecordsStudentClassResolver(6, 2, 30, 1, 9);
            //Englishs
            addRecordsStudentClassResolver(8, 2, 30, 3, 9);
            //Pe
            addRecordsStudentClassResolver(14, 2, 30, 1, 9);
          end
          else if randomSide = 1 THEN //a or b side
            begin
              //Science
              numberInBSide := numberInBSide + 1;
              addRecordsStudentClassResolver(16, 2, 30, 3, 9);
              //Maths
              addRecordsStudentClassResolver(22, 2, 30, 1, 9);
              //Englishs
              addRecordsStudentClassResolver(24, 2, 30, 3, 9);
              //Pe
              addRecordsStudentClassResolver(30, 2, 30, 1, 9);
          end
          else
            begin
              //error catching for random side assignment
              writeln('A: ',numberInASide, '  B: ',numberInBSide);
              Delay(3000);
              DisplayErrorScrn.ErrScr;
            end;
        //randomly pick the optional subjects
        for a := 0 to 8 do
          begin
            addRecordsStudentClassResolver(32 + a, 4, 30, 1, 9);
            a := a + 3;
          end;
      end
    else if (student.year = 12) THEN // year 12
      begin
        if noTakingFourAlevels < 5 THEN
          noOfAlevels := RANDOM(10);
        if noOfAlevels = 0 THEN //taking 4 a lvels
```

```
              begin
                noTakingFourAlevels := noTakingFourAlevels + 1;
                //block a
                addRecordsStudentClassResolver(0, 2, 20, 1, 12);

                //block b
                addRecordsStudentClassResolver(2, 1, 5, 1, 12);

                //block c
                addRecordsStudentClassResolver(3, 2, 20, 1, 12);

                //block d
                addRecordsStudentClassResolver(5, 2, 20, 1, 12);
              end
            else
              begin
                //block a
                addRecordsStudentClassResolver(0, 2, 20, 1, 12);

                //block c
                addRecordsStudentClassResolver(3, 2, 20, 1, 12);

                //block d
                addRecordsStudentClassResolver(5, 2, 20, 1, 12);

              end;
          end;
      end;
  end;

PROCEDURE assignTeacherToClass;
  begin
    reset(classFile);
    reset(teacherfile);
    REWRITE(teacherClassResolverFile);
    while not(eof(classFile)) DO //for each teacher
      begin
        numberDone := numberDone+1;
        percentComplete := numberDone / total;
        //assign one teacher per class
        read(classFile,sClass);
```

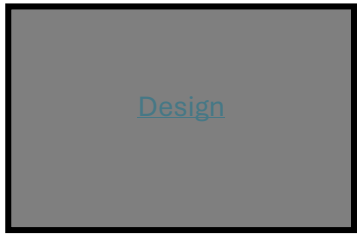
```
        read(teacherfile, teacher);
        teacherClassResolver.teacherID := teacher.teacherID;
        teacherClassResolver.classID := sClass.classId;
        write(teacherClassResolverFile,teacherClassResolver);
      end;
  end;

PROCEDURE assignTeacherToSubject;
  var a, b : LONGINT;
  begin
    RESET(TeacherFile);
    reset(classFile);
    REWRITE(teacherSubjectResolverFile);
    while not EOF(TeacherFile) DO
      begin
        numberDone := numberDone+ 1;
        percentComplete := numberDone / total;
        read(TeacherFile,teacher);
        read(classFile,sClass);
        //find the class the teacher teaches and gets the subject id of that class and puts it in a resolver file
        teacherSubjectResolver.teacherId := teacher.teacherID;
        teacherSubjectResolver.subjectId := sClass.subjectID;
        write(teacherSubjectResolverFile, teacherSubjectResolver);
      end;
    end;



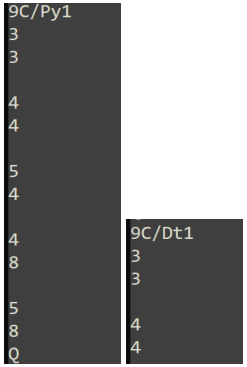

begin
  //generates a random seed
  randomize;
  //run each procedure
  initialiseFiles;
  loading.startLoading('P',percentComplete);
  CreateTimetable;
  addForms;
  addStudents;
  addTeachers;
  createSubjects;
```

```
    createClasses;
    createClassStudentResolver;
    assignTeacherToClass;
    assignTeacherToSubject;
    createRegisters;
    createLogInFile;
    readln;
    closeAllFiles;
  end.
```



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
1.1.1	Create Files Iteration 0	To test the program can read from the timetable I gave it into a .dta file with the correct format	 This is a small sample of the data entered	Records stored as 3 3 9C/Py1 4 4 9C/Py1 5 4 9C/Py1 4 8 9C/Py1 5 8 9C/Py1 3 3 9C/Dt1 4 4 9C/Dt1	See below	None

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
3 3 9C/Py1 4 4 9C/Py1 4 5 9C/Py1 8 4 9C/Py1 8 5 9C/Py1 3 3 9C/Dt1 4 4 9C/Dt1						

1.2.1	Create Files Iteration 0	Check the forms are created and populated correctly	Input data is the timetable file	9AAA
				30
				
				9AAB
				30
				
				9AAC
				30
				
				9AAD
				30
				
				12AAA
				20
					See below	Supposedly no students in form 9AAD. It was not writing the last form back to the file with the updated number in students.
						
						So after the students for loop add:
						`seek(formFile, (FilePos(formFile)-1));`
						`write(FormFile,form);`

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
1.2.2	Create Files Iteration 1	Check the forms are created and populated correctly	Input data is the timetable file	9AAA 30 9AAB 30 9AAC 30 9AAD 30 12AAA 20	See below	n/a

```
9AAA
30

9AAB
30

9AAC
30

9AAD
0

12AAA
20
```

```
9AAA
30

9AAB
30

9AAC
30

9AAD
30

12AAA
20
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
1.3.1	Create Files Iteration 1	Test the students are added correctly	Text file of names generated by another script	Student ID first name last name year Form Username: FirstName+Last Name Password : ABC123	See below	n/a

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
0 Robert Cole 9 9AAA RobertCole ABC123 1 Keith Holmes 9 9AAA KeithHolmes ABC123 2 Esther Braswell 9 9AAA EstherBraswell ABC123			120 Kari Wilson 12 12AAA KariWilson ABC123 121 Richard Shoulta 12 12AAA RichardShoulta ABC123 122 Frederic Saunders 12 12AAA FredericSaunders ABC123			

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
1.4.1	Create Files Iteration 1	Test the teachers are added correctly	Text file of names generated by another script	Teacher ID first name last name Form (might be blank) Username: FirstName+Last Name Password : ABC123	See below	n/a

```
AllisonRolling
ABC123

BCa
Barton
Campisi

BartonCampisi
ABC123

BTe
Bobby
Terry

BobbyTerry
ABC123

BWi
Brian
Wilson
9AAC
BrianWilson
ABC123
```

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
1.5.1	Create Files Iteration 1	Test the subjects are stored correctly	Array of all subject names And an array of all the subject id stored in the same order	Subject id Subject name	See below	n/a

```
Cs
Computer Science

Ma
Maths

Mu
Music

Enl
English

Gg
Geography
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
1.6.1	Create Files Iteration 1	Test that the classes are stored correctly	Array of year 9 classes And array of year 12 classes	Class ID Year Subject id Number of students (30 if year 9 class else a small number of year 12s)	See below	n/a

```
12D/Cp1
12
Cp
14

12D/Mu1
12
Mu
6

9AAX/Lf
9
Lf<NUL>
30

9AAX/Ma
9
Ma<NUL>
30
```

The nul characters represent an empty space in memory where the class id is too short in some places form a string of length 3 which is the max, so it leaves it blank. It is not an issue.

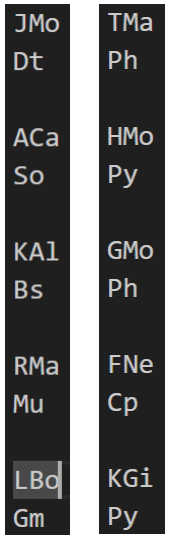
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

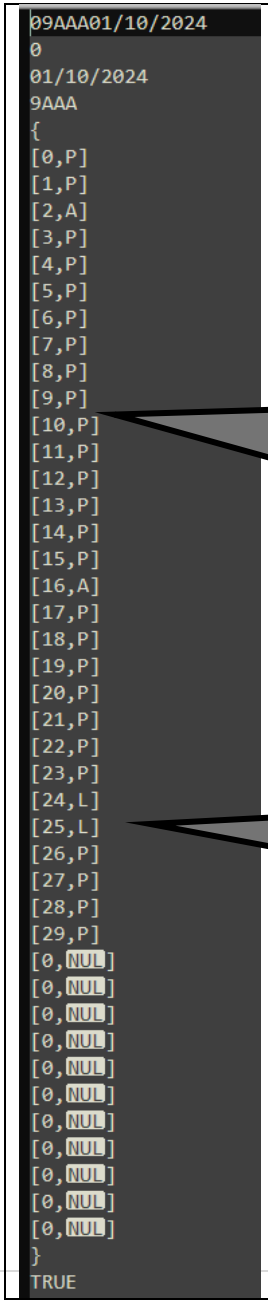
Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
1.7.1	Create Files Iteration 1	To test that students are correctly added to classes	Student data, and class data	Class ID Student ID	See below	n/a
9B/Fr155 9B/Fr165 9B/Fr166 9B/Fr167	9AT2/Bi116 9AT2/Ch1 9AT2/Ch9					
1.8.1	Create Files Iteration 1	To correctly assign teachers to classes	Teacher data and class data	Teacher ID Class ID With one teacher to each class	See below	n/a

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
JMo 9C/Dt1 ACa 9D/So1 KAl 9D/Bs1 RMa 9D/Mu1 LBo 9D/Gm1 TMa 12A/Ph1	HMo 12A/Py1 GMo 12B/Ph1 FNe 12C/Cp1 KGi 12C/Py1					
1.9.1	Create Files Iteration 1	To correctly assign teachers to subjects based on the class they are in	Teacher data, class data and subject data	Teacher ID subject ID With the teacher's class id matching up with the subject id	See below	n/a

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
 all this lines up with previous evidence with the classes						
1.10.1	Create Files Iteration 1	To test the registers are created correctly	The timetable needs to be entered along with the student's data, including attendance data. And the data about which student is in what class	Register id Lesson number Date Class id 2d array of student id and their register mark Is it an Ofsted register	See below	n/a



Register marks are distributed correctly with most being present, some being late and very few being absent

Correctly defines lesson 0 as an Ofsted register

[0,NUL] represents no students with no register mark as there are not anymore students in the

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
1.11.1	Create Files Iteration 1	To test the log in file is generated and stored correctly	Student and teacher data	Person ID (teacher/student) Username Password (ABC123)	See below	n/a

```
ACa
AdamCampbell
ABC123

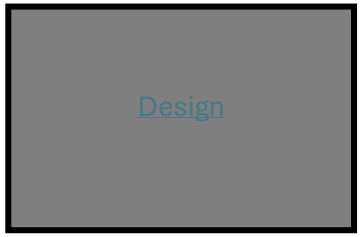
90
AgnusNielsen
ABC123

50
AlfredoAllison
ABC123

ARo
AllisonRolling
ABC123

15
AltonJoyner
ABC123

ALu
AnderaLueth
ABC123
```



Review

During this iteration I made a script which can generate a large set of data which can be used to test the rest of the system thoroughly with accurate data. This took a long time as I wanted to ensure high data integrity. I asked my client to look at the data to ensure the data I have got is adequate for the section of the school system that I am focusing on. Her thoughts on the data were that for a full school the data is a small amount but for testing purposes she said that if the solution works with this data it should work as an actual system with the full data.

Iteration 2 – Class Structure

Aim

In this iteration I aim to develop a method to generalize code that is used for each file. Such as sorting a file or searching it. I designed this using classes and inheritance to have the general search and sort algorithm in a parent class, while all of the file specific procedures and retrieving the data from the files is done in a child class specific to each file. I will only be testing one of the classes as they are all the same with the singular difference of the record in which they are read into and how that data is put into other structures due to different data types in the record and different number of fields.

Code

For testing I will use just one of the child classes, I choose the login file as it contains fewer fields allowing for easier test data, while still having to be correctly aligned with the student and teacher file

Super Class

```
unit FileProcedures;
//allows constructors
{$M+}

    interface

    USES crt, sysutils;

    TYPE
        //declare class structure
        Files = class
            protected
              upper : LONGINT;
              dataArray: ARRAY[0..1400] OF STRING;
              fileArray: ARRAY[0..7,0..1400] OF STRING;
            public
              constructor Create;
              destructor Destroy; override;
              procedure sort(var indexArray : ARRAY OF LONGINT);
              procedure search(max, fields : LONGINT; searchValues:ARRAY OF STRING; searchFields: ARRAY OF
BYTE; var returnArray : ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            end;

    IMPLEMENTATION




    constructor Files.Create;
      begin
      end;
```

```
    destructor Files.Destroy;
        begin
        end;

    //sort algorithm for the files
    procedure Files.sort(var indexArray : ARRAY OF LONGINT);
        var TempIndex, i, code, tmp1, tmp2 : LONGINT;
            TempStr : STRING;
            Swaps: BOOLEAN;
        begin
            //bubble sort
            upper := upper-1;
            REPEAT
                Swaps:= false;
                FOR i := 0 TO upper DO
                BEGIN
                    //checks done with the data array
                    IF dataArray[i] > dataArray[i+1] THEN
                    BEGIN
                        TempStr := dataArray[i];
                        dataArray[i] := dataArray[i+1];
                        dataArray[i+1] := TempStr;
                        Swaps := true;
                        //swap data in index array
                        TempIndex := indexArray[i];
                        indexArray[i] := indexArray[i+1];
                        indexArray[i+1] := TempIndex;
                    END;
                END;
                upper := upper -1;
            UNTIL (NOT Swaps) OR (upper < 0);
        end;

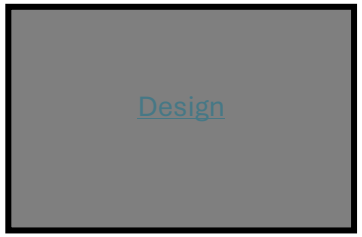
    //search algorithm
    procedure Files.search(max, fields : LONGINT; searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE;
var returnArray : ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
        var found : BOOLEAN;
            a, b, c, returnIndex : LONGINT;
```

```
        data : STRING;
    begin
        //linear search

        if switchLowerCase then
          begin
            for c := 0 to fields do
                begin
                    searchValues[c] := lowerCase(searchValues[c]);
                end;
          end
        else
          begin
            for c := 0 to fields do
                begin
                    searchValues[c] := searchValues[c];
                end;
          end;
        a := -1;
        returnIndex := 0;
        //check every value in the 2d array
        while a < max do
            begin
            a := a + 1;
            //check each item in the record
            for b := 0 TO (fields-1) DO
                begin
                  if switchLowerCase then
                    begin
                      data := LowerCase(fileArray[b,a]);
                    end
                  else
                    begin
                      data := fileArray[b,a];
                    end;
                  //check if one piece of data matches
                    if data = searchValues[0] THEN
                      begin
```

```
                        found := true;
                        //check each peice of data in record against data to search for
                        for c := 0 to (SizeOf(searchFields)-1) do
                            begin
                                if switchLowerCase then
                                  begin
                                    data := LowerCase(FileArray[searchFields[c],a]);
                                  end
                                else
                                  begin
                                    data := FileArray[searchFields[c],a];
                                  end;
                                //if not all data matches continue search and dont return result
                                if searchValues[c] <> data THEN
                                begin
                                    found := false;
                                    break;
                                end;
                            end;
                        //if all data matches add index to the array of found matches
                        if found = true THEN
                            begin
                                returnArray[returnIndex] := a;
                                returnIndex := returnIndex+1;
                                break
                            end;
                    end;
                end;
            end;
        end;

    END.
```



Child Class

```
unit LogInProcedures;
{$M+}
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        //record type for this file
        dataRecord = RECORD
                          PersonID: STRING[5];
                          Username: STRING[40];
                          Passwowrd: String[20];
                         end;

    type
        //declaration of class, attributes and methods
        logInFile = Class(Files)
              protected
                dataFile: File OF dataRecord;
                dataTextFile : TextFile;
                indexArray: ARRAY[0..190] OF LONGINT;
                recordArray: ARRAY[0..190] OF dataRecord;
              public
                data : dataRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(iPersonID: STRING; iUsername: STRING; iPasswowrd: String);
                procedure DisplayData;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
                procedure sort;
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray:
ARRAY OF LONGINT; switchLowerCase: BOOLEAN);
```

```
    end;

IMPLEMENTATION
//constructor and destructor
constructor logInFile.Create;
    var a : longint;
    begin
        inherited Create;
        ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\LogInDetails.dta');
        ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\logInOut.txt');
        RESET(dataFile);
        RESET(dataTextFile);
        for a := 0 TO 190 DO
          begin
            indexArray[a] := a;
          end;
    end;

destructor LogInFile.Destroy;
    begin
        close(dataFile);
        close(dataTextFile);
        inherited Destroy;
    end;

//update data attribute
procedure logInFile.SetData(iPersonID: STRING;
                      iUsername: STRING;
                      iPasswowrd: String);

    begin
        with data do
            begin
                PersonID := iPersonID;
                Username:=iUsername;
                Passwowrd:=iPasswowrd;
            end;
    end;
```

```
    //display data attribute
    procedure logInFile.DisplayData;
        begin
        with data do
            begin
                writeln(PersonID);
                writeln(Username);
                writeln(Passwowrd);
                writeln('--------------------------------------------------------------------------------');
            end;
        end;
    //write all data in file to text file
    procedure logInFile.extractToTextFile;
        begin
            reset(dataFile);
            rewrite(dataTextFile);
            while not eof(dataFile) DO
                begin
                    read(dataFile, data);
                    writeln(dataTextFile, data.PersonID);
                    writeln(dataTextFile, data.Username);
                    writeln(dataTextFile, data.Passwowrd);
                    writeln(dataTextFile, '');
                end;
        end;
    //write data attribute to file
    procedure logInFile.writeToFile;
        begin
            write(dataFile, data);
        end;

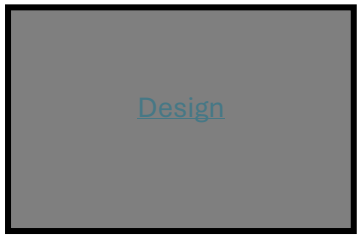
    //read data from file at current position
    procedure logInFile.readHere;
        begin
            if FIlePos(dataFile) < FileSize(dataFile) then
                begin
                    read(dataFile,data);
                end;
```

```
        end;
    //read data from file at specified position
    procedure logInFile.readFile(readFromPosition : LONGINT);
        begin
            if readFromPosition < FileSize(dataFile) then
                begin
                    SEEK(dataFile,readFromPosition);
                    read(dataFile,data);
                end;
        end;
    //sort the file
    procedure logInFile.sort;
        var a,b : LONGINT;
            maxStrLen : STRING;
        begin
          RESET(dataFile);
          upper:= 190;
          //extrat the data into an array
          for a := 0 TO 190 DO
            begin
              READ(dataFile,data);
              recordArray[a] := data;
              dataArray[a] := data.Username
            end;
        Files.sort(indexArray);
        RESET(dataFile);
        //write the data back to the file
        for a := 0 TO 190 DO
            begin
                WRITE(dataFile,recordArray[indexArray[a]]);
                indexArray[a] := a;
            end;

        end;

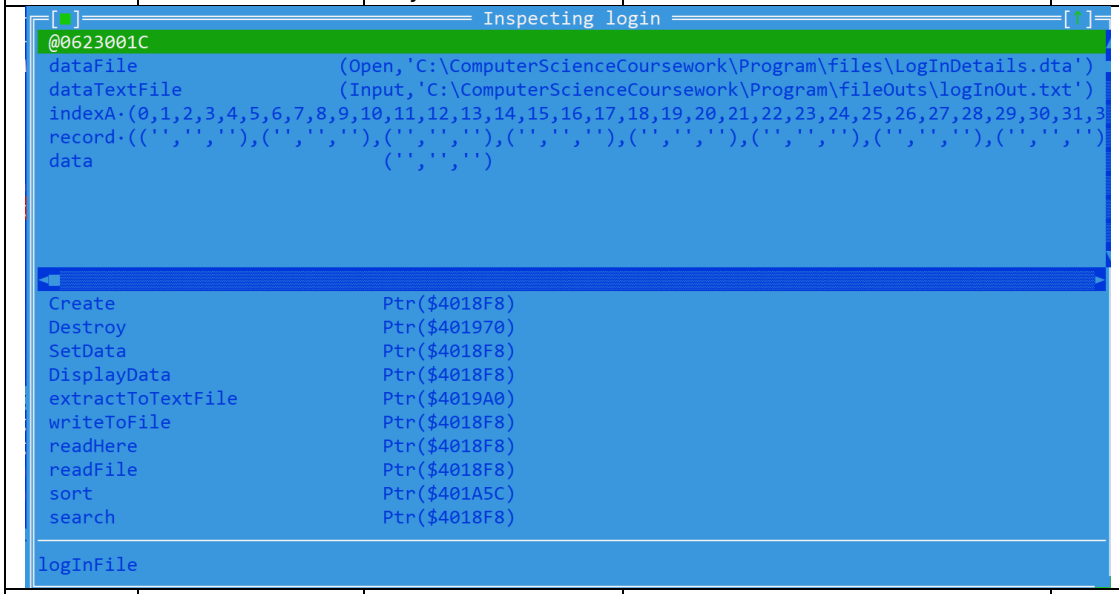
    //search the file
        procedure logInFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray
: ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
```

```
            var a: LONGINT;
            begin
                RESET(dataFile);
                for a := 0 TO 190 DO
                    begin
                        read(dataFile, data);
                        fileArray[0,a] := data.PersonID;
                        fileArray[1,a] := data.Username;
                        fileArray[2,a] := data.Passwowrd;
                    end;
                Files.search(190,3,searchValues, searchFields, returnArray, switchLowerCase);
            end;
    END.
```



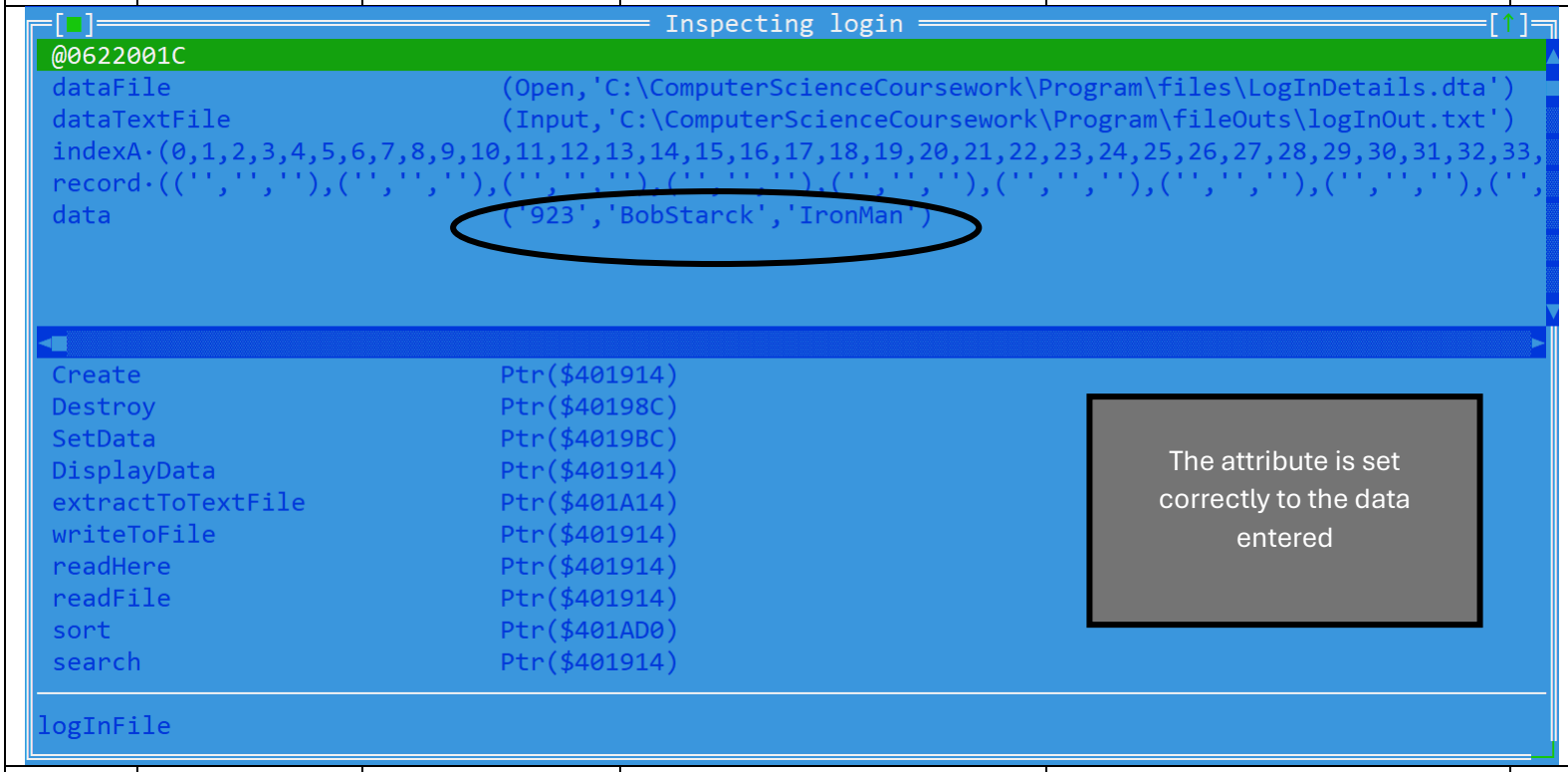
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
2.1.1	Class structure Iteration 0	To test if the object is created	None	Evidence of the object being created	See below	n/a
 Inspecting login @0623001C dataFile (Open,'C:\ComputerScienceCoursework\Program\files\LogInDetails.dta') dataTextFile (Input,'C:\ComputerScienceCoursework\Program\fileOuts\logInOut.txt') indexA·(0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,3 record·(('','',''),('','',''),('','',''),

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments



2.3.1	Class structure Iteration 0	To test if it correctly writes the data stored in the current record to the screen.	Person id: 923 Username: BobStarck Password: IronMan	The data should be outputted to the console With a method to break apart the record incase many are outputted	See below	n/a

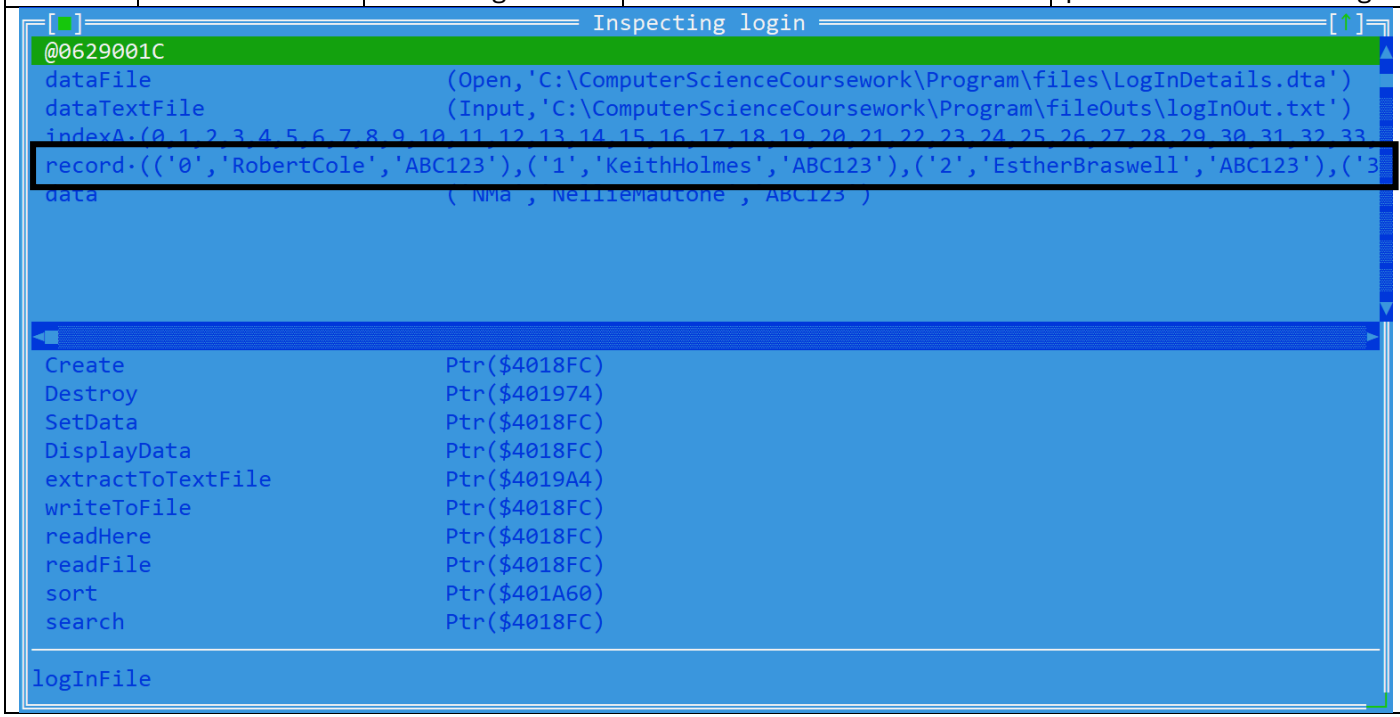
```
923
BobStarck
IronMan
--------------------------------------------------------------------------------
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
2.4.1	Class structure Iteration 0	Explanation of extract to file	n/a	n/a	n/a	n/a
The method extractToTextFile is a procedure used only for me as the programmer to debug the contents of the file as it is not possible to extract the data in any other way. Evidence of this working is used in the previous iteration and is how I got the data to use as evidence. Evidence						
2.5.1	Class structure Iteration 0	Testing to see if I can write back to the file. The data attribute should be written to the file	Person id: 923 Username: BobStarck Password: IronMan	The first set of data in the file should be the data that was just entered	See below	
923 BobStarck IronMan 1 KeithHolmes ABC123 2 EstherBraswell ABC123						
2.6.1	Class structure Iteration 0	Testing the read here method	None needed	1 KeithHolmes ABC123 He is the next person to be stored and after the write with BobStarck it should output him next	See Below	n/a
1 KeithHolmes ABC123						

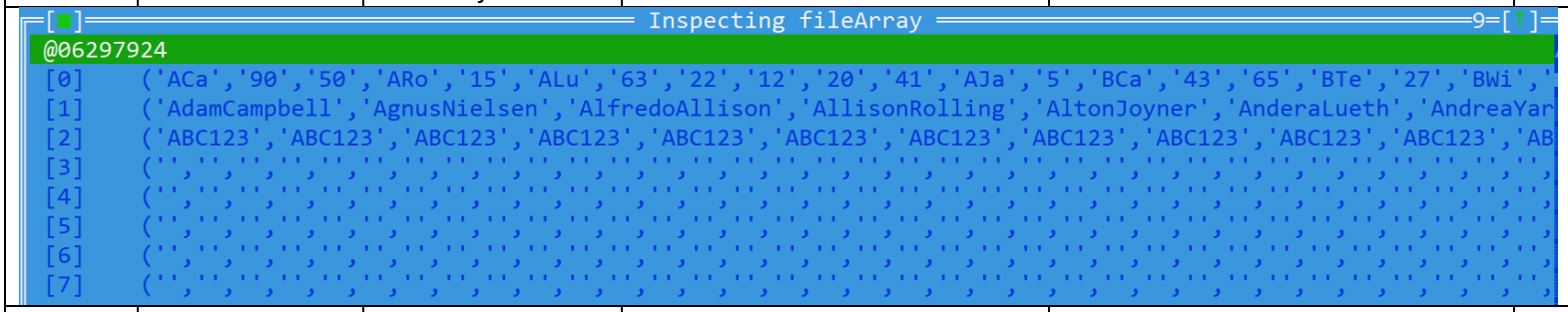
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
2.7.1	Class structure Iteration 0	To test that the read file from a given position works	Login.readFile(3)	3 RitaGamble ABC123	See below	n/a
3 RitaGamble ABC123						
3.8.1	Class structure Iteration 0	To test if the data is placed in the record array and the record id array for sorting	None	The attributes record array to contain all the records in the file and record id array to a single field for each record which will be passed to the search algorithm	See below	n/a



Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
3.9.1	Class structure Iteration 0	To test the file data is passed into the actual search algorithm correctly	None	A 2d array of all the data in the file		n/a



2D array storing all the data from the file

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
3.10.1	Class structure Iteration 0	To test if the data is sorted and written back to the file when complete	None	The data should be ordered by Username	See below	n/a

```
0
RobertCole
ABC123

1
KeithHolmes
ABC123

2
EstherBraswell
ABC123

3
RitaGamble
ABC123

4
HerbertGaston
ABC123
```

Data before sort

```
ACa
AdamCampbell
ABC123

90
AgnusNielsen
ABC123

50
AlfredoAllison
ABC123

ARo
AllisonRolling
ABC123

15
AltonJoyner
ABC123

ALu
AnderaLueth
ABC123
```

Data after sort, data is ordered by username (the second field)

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
3.11.1	Class structure Iteration 0	To test if it can find the first person in the file	Search data: RobertCole Field 1 for username	Found at positions: 0 0 RobertCole ABC123	See below	n/a

```
Enter search value: RobertCole
Enter search index: 1

Found at positions:
0
0
RobertCole
ABC123
--------------------------------------------------------------------------------
```

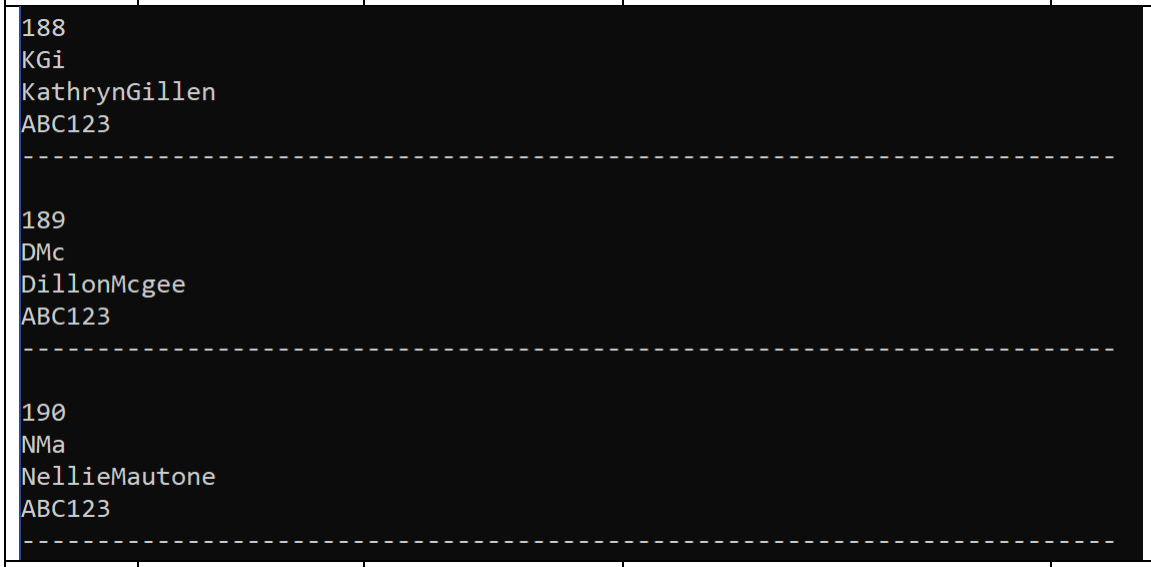
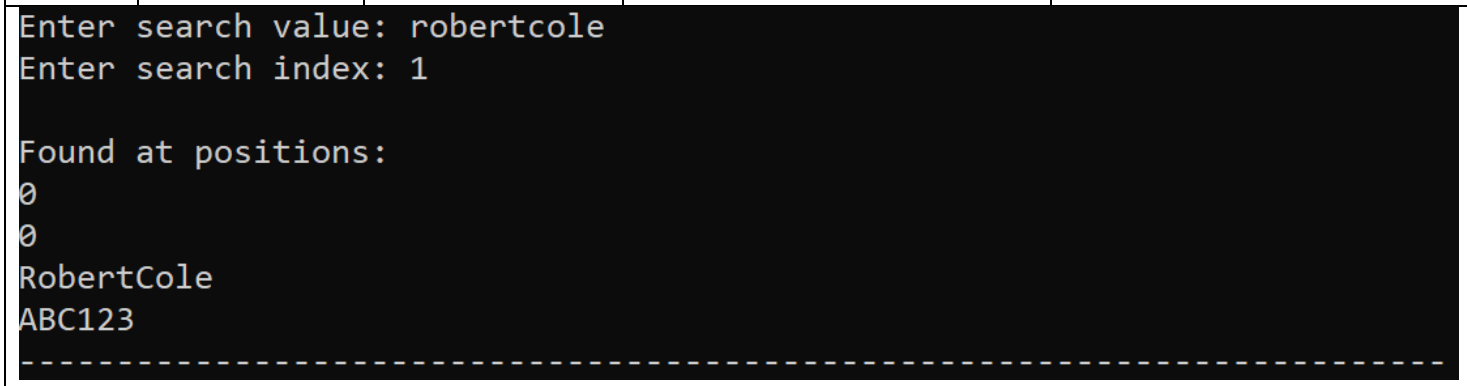
3.11.2	Class structure Iteration 0	To test if it can find the any person in the file	Search data: 8 Field 0 for person id	Found at positions: 8 8 MarylouiseBrooks ABC123	See below	n/a

```
Enter search value: 8
Enter search index: 0

Found at positions:
8
8
MarylouiseBrooks
ABC123
--------------------------------------------------------------------------------
```

3.11.3	Class structure Iteration 0	To test if it can find multiple person in the file	Search data: ABC123 Field 2 for password	Every person should be output once as all users have password ABC123	See below	n/a

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
				Displays all people as teachers are the last to get added and it ends at 190 people which is where it should end		
3.11.4.0	Class structure Iteration 0	TO test case sensitive searching	Search data : robertcole Field 1 for username	No people found	See below	Add a parameter to bypass force lower if the search needs to be case sensitive
						

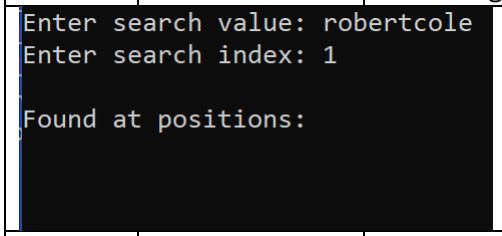
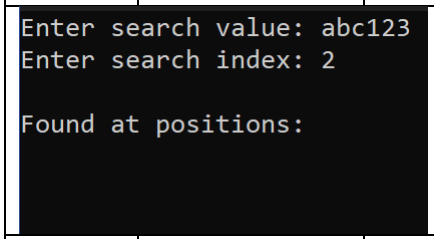
Fix:

```
procedure Files.search(max, fields : LONGINT; searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var
returnArray : ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
        var found : BOOLEAN;
            a, b, c, returnIndex : LONGINT;
            data : STRING;
        begin
            //linear search

            if switchLowerCase then
              begin
                for c := 0 to fields do
                    begin
                        searchValues[c] := lowerCase(searchValues[c]);
                    end;
              end
            else
              begin
                for c := 0 to fields do
                    begin
                        searchValues[c] := searchFields[c];
                    end;
              end;
            a := -1;
            returnIndex := 0;
            //check every value in the 2d array
            while a < max do
                begin
                a := a + 1;
                //check each item in the record
                for b := 0 TO (fields-1) DO
                    begin
                      if switchLowerCase then
                        begin
                          data := LowerCase(fileArray[b,a]);
                        end
                      else
```

```
                    begin
                      data := fileArray[b,a];
                    end;
                  //check if one piece of data matches
                    if data = searchValues[0] THEN
                      begin
                        found := true;
                        //check each peice of data in record against data to search for
                        for c := 0 to (SizeOf(searchFields)-1) do
                            begin
                                if switchLowerCase then
                                  begin
                                    data := LowerCase(FileArray[searchFields[c],a]);
                                  end
                                else
                                  begin
                                    data := FileArray[searchFields[c],a];
                                  end;
                                //if not all data matches continue search and dont return result
                                if searchValues[c] <> data THEN
                                begin
                                    found := false;
                                    break;
                                end;
                            end;
                        //if all data matches add index to the array of found matches
                        if found = true THEN
                            begin
                                returnArray[returnIndex] := a;
                                returnIndex := returnIndex+1;
                                break
                            end;
                      end;
                end;
        end;
    end;
```

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
3.11.4.1	Class structure Iteration 1	TO test case sensitive searching	Search data : robertcole Field 1 for username	No people found	See below	n/a
						
3.11.4.2	Class structure Iteration 1	To test case sensitive searching in a different field	Search data : abc123 Field 2 for password	No people found	See below	n/a
						
3.11.5	Class structure Iteration 1	Test if it can search using multiple pieces of information	Search data : RobertCole Field 1 for username Search data : ABC123 Field 2 for password	Found at positions: 0 0 RobertCole ABC123	See below	n/a

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments

```
Enter search value: RobertCole
Enter search index: 1

Enter search value: ABC123
Enter search index: 2

Found at positions:
0
0
RobertCole
ABC123
------------------------------------------------------------------
```

Review

This iteration is purely for use by the programmer. This class structure that I developed allows me to attach extra methods such as sorts and searches to each file allowing for more concise code in the main program. By using inheritance, I created the general sort and search algorithms and place them in a super class, and by creating a class for each file which inherits the methods of the super class each file has a sort and search algorithm attached to it. This will then allow me to just use the methods in the main script without having to write searches and sorts for each file in the main script.





Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Iteration 4 – Student's Menu

Aim

The aim in this iteration is to develop a menu in which the student can interact with the system and use it. This will be done using a series of menus which the student should be able to navigate using the keyboard arrow keys. This reduces the chance of the student being able to break the program by keeping the free data input sections to a minimum.

Code

```
UNIT StudentsMenu;
  interface
  uses crt, sysutils, VPUtils, studentProcedures, timeTableProcedures,
  attendancProcedures, studentClassProcedures,createMenu, writeDesign;

  //declaration of all procedures and functions
  procedure startStudent(inputStudentId : LONGINT);
  procedure CompleteTimeTable(inputDate : TDateTime);
  procedure TimeTableMenu;
  procedure AttendanceMenu;
  procedure LogOutMenu;
  procedure mainMenu;

  implementation
  //declare variables local to this unit
  var studentID : LONGINT;
      student : StudentFIle;
      attendance : AttendanceFile;
      timetable : TimeTableFile;
      studentClassC : studentClassFile;
      studentClassReturnArray : ARRAY[0..1400] OF LONGINT;
      timetableReturnArray: ARRAY[0..321] OF LONGINT;
      mainMenuOptions : ARRAY[0..2] OF STRING = ('Attendance','Timetable', 'Log Out');
      animationTimeTable : BOOLEAN;

  //Menu for viewing attendance
  procedure AttendanceMenu;
    begin
      Writeln('Attendance');
      readln;
    end;

  //fill timetable with the data
  procedure CompleteTimeTable(inputDate : TDateTime);
    var a, b, c, day, NumberOfDays, timeDelay : LONGINT;
        CurrentDate : TDateTime;
```

```
    week : STRING;
    stringStudentId : STRING;
    //dateArray : ARRAY OF STRING;
begin
  //draw basic timetable outline
  if animationTimeTable = true THEN
    begin
      timeDelay := 100;
    end
  else
    begin
      timeDelay := 0;
    end;
  writeDesign.DrawDesign('C:\ComputerScienceCoursework\Program\files\TimeTableLayout.txt', timeDelay);
  CurrentDate := inputDate;
  day := DayOfWeek(CurrentDate-2);
  HideCursor;
  //write the dates with current date highlighted
  for a := 0 to 4 DO
    begin
      GOTOXY(5,7+a*4);
      if CurrentDate-day+a = Date THEN
        TextColor(Yellow)
      else
        TextColor(White);
      write(DateToStr(CurrentDate-day+a));
      Delay(timeDelay);
    end;
  TextColor(White);
  //calcualte if it is week A or week B
  NumberOfDays := 0;
  while StrToDateTime('13/09/2024') + NumberOfDays <> CurrentDate-day+a-1 DO
    NumberOfDays := NumberOfDays +1;
  NumberOfDays := NumberOfDays DIV 7;
  GOTOXY(7,2);
  if (NumberOfDays mod 2 = 0 )THEN
    week := 'A'
  else
```

```
  week := 'B';
WRITE(week);
//write lessons in timetable
stringStudentId := IntToStr(studentID);
//get all classes which the student is in
studentClassC.search([stringStudentId],[1],studentClassReturnArray);
b := 0;
//for every class
while studentClassReturnArray[b] <> -1 DO
  begin
    //get the class id
    studentClassC.readFile(studentClassReturnArray[b]);
    for a := 0 to 320 DO
      timetableReturnArray[a] := -1;
    //find every time and day that class is on
    timetable.search([studentClassC.data.ClassID],[2], timetableReturnArray);
    c := 0;
    while timetableReturnArray[c] <> -1 DO
      begin
        //get the day and time for that one time that one class is
        timetable.readFile(timetableReturnArray[c]);
        //test for the current week
        if week = 'A' THEN
          begin
            //test if the class is on the correct week
            if timetable.data.day < 6 THEN
              begin
                //got to position and write class id
                GOTOXY((10*timetable.data.LessonNo+20),( 4*(timetable.data.Day-1)+7));
                WRITE(Trim(TimeTable.data.ClassID));
                Delay(timeDelay);
              end;
          end
        else
          begin
            //test if the class is on the correct week
            if timetable.data.day > 5 THEN
              begin
```

```
                    //got to position and write class id
                    GOTOXY((10*timetable.data.LessonNo+20),( 4*(timetable.data.Day-6)+7));
                    WRITE(Trim(TimeTable.data.ClassID));
                    Delay(timeDelay);
                  end;
              end;
            c := c + 1;
          end;
        b := b + 1;
      end;
    //write the form data in
    for a := 0 TO 4 DO
      begin
        GOTOXY(21, 4*a+7);
        write(student.data.Form);
        Delay(timeDelay);
      end;
      animationTimeTable := false;
  end;

//Draw and populate timetable
procedure TimeTableMenu;
  var StartDate : TDateTime;
  begin
    StartDate := Date;
    CompleteTimeTable(StartDate);
    readln;
  end;

procedure LogOutMenu;
  begin
    writeln('Logging out');
    Timetable.free;
    studentClassC.free;
    student.free;
    attendance.free;

    Delay(1000);
```

```
    end;

  //main menu
  procedure mainMenu;
    var choice : LONGINT;
    begin
      GOTOXY(0,0);
      choice := 0;
      //get student and attendance data at the start
      student.readFile(studentID);
      attendance.readFile(studentID);
      //main menu display until logout selected
      repeat
        ClrScr;
        HideCursor;
        //messages welcoming and displaying overall attendance
        writeln('welcome ', student.data.fname, ' ', student.data.sname, ' ');
        GOTOXY(1,3);
        writeln('Attendance: ', Format('%.2f', [attendance.data.attendance*100]),'%');

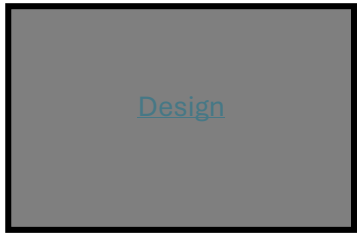
        //create the menu and start to control it
        choice := createMenu.writeMenu([1,5],3,mainMenuOptions, choice);
        //test for each menu choice
        case choice of
          0 : AttendanceMenu;
          1 : TimeTableMenu;
          2 : LogOutMenu;
        end;
      until choice = 2;

    end;

  //declaration of global variables
  procedure startStudent(inputStudentId : LONGINT);
    var a : LONGINT;
    begin
      animationTimeTable := true;
      //populate arrays with -1
```

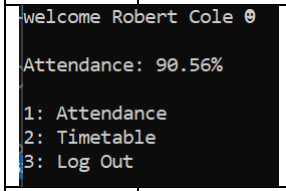
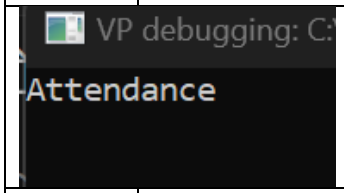
```
      for a := 0 TO 1400 DO
        studentClassReturnArray[a] := -1;
      for a := 0 to 320 DO
        begin
        timetableReturnArray[a] := -1;
        end;
      //declare file objects for this unit
      timetable := TimeTableFile.create;
      studentClassC := studentClassFile.create;
      studentID := inputStudentId;
      attendance := AttendanceFile.create;
      student := StudentFile.create;
      mainMenu;
    end;

  end.
```

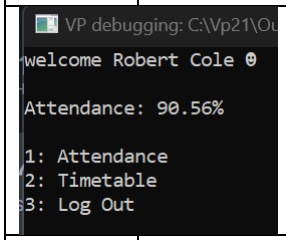


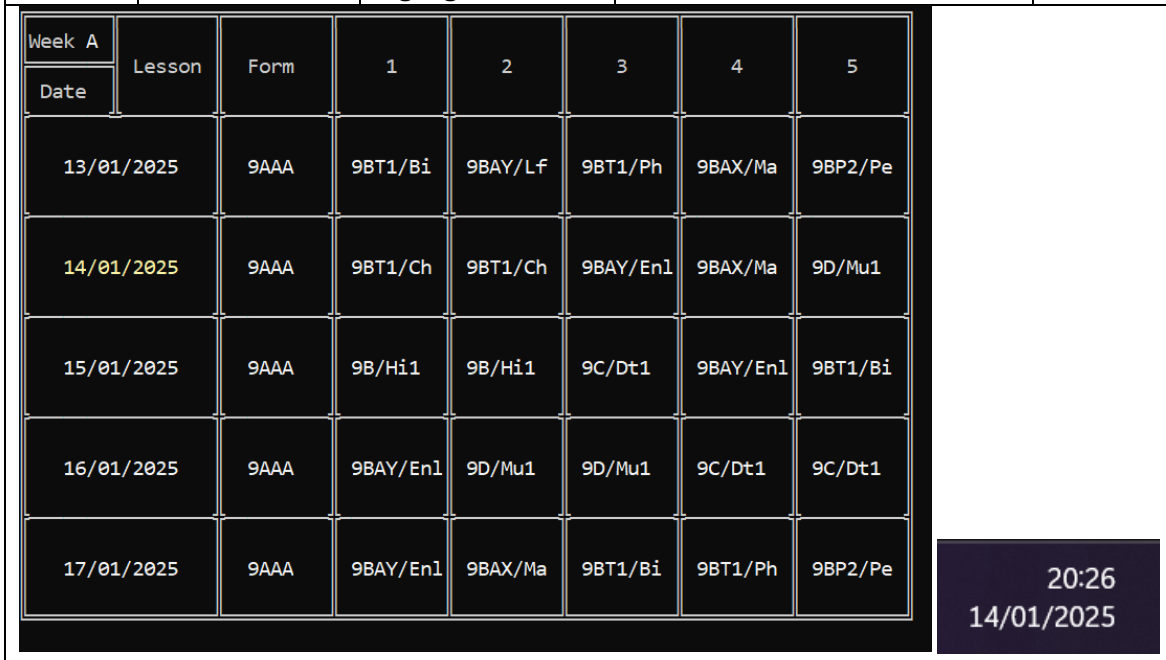
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
4.1.1	Student Menu Iteration 0	TO test if the student home screen is displayed	Username: RobertCole Password: ABC123	Welcome Robert Cole Attendance: 90% (as example) 1. Attendance 2. Timetable 3. Log Out	See below	n/a
						
4.2.1	Student Menu Iteration 0	To test if it opens the attendance menu	Username: RobertCole Password: ABC123 Then menu option 1	Attendance	See below	n/a
						
4.2.2	Student Menu Iteration 0	Testing if you can return to the main menu by any key	Username: RobertCole Password: ABC123 Then menu option 1 Any key press	Welcome Robert Cole Attendance: 90% (as example) 1. Attendance 2. Timetable 3. Log Out	See below	n/a

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
						
4.3.1	Student Menu Iteration 0	Testing if the timetable is correctly displayed in table format with current date highlighted	Username: RobertCole Password: ABC123 Then menu option 2 Any key press	Display of timetable with correct date highlighted and classes filled in	See below	n/a



Week A Date	Lesson	Form	1	2	3	4	5
13/01/2025		9AAA	9BT1/Bi	9BAY/Lf	9BT1/Ph	9BAX/Ma	9BP2/Pe
14/01/2025		9AAA	9BT1/Ch	9BT1/Ch	9BAY/Enl	9BAX/Ma	9D/Mu1
15/01/2025		9AAA	9B/Hi1	9B/Hi1	9C/Dt1	9BAY/Enl	9BT1/Bi
16/01/2025		9AAA	9BAY/Enl	9D/Mu1	9D/Mu1	9C/Dt1	9C/Dt1
17/01/2025		9AAA	9BAY/Enl	9BAX/Ma	9BT1/Bi	9BT1/Ph	9BP2/Pe

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

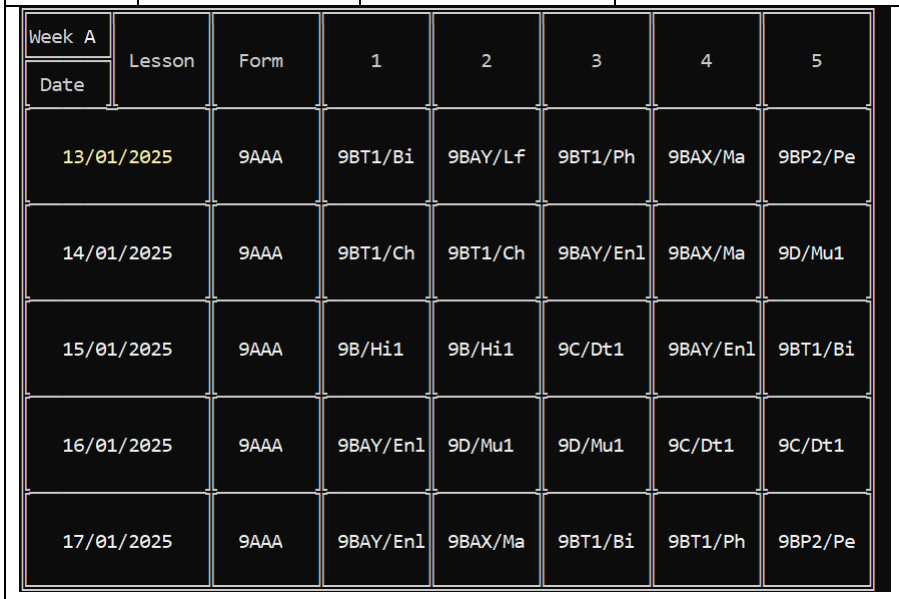
Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments

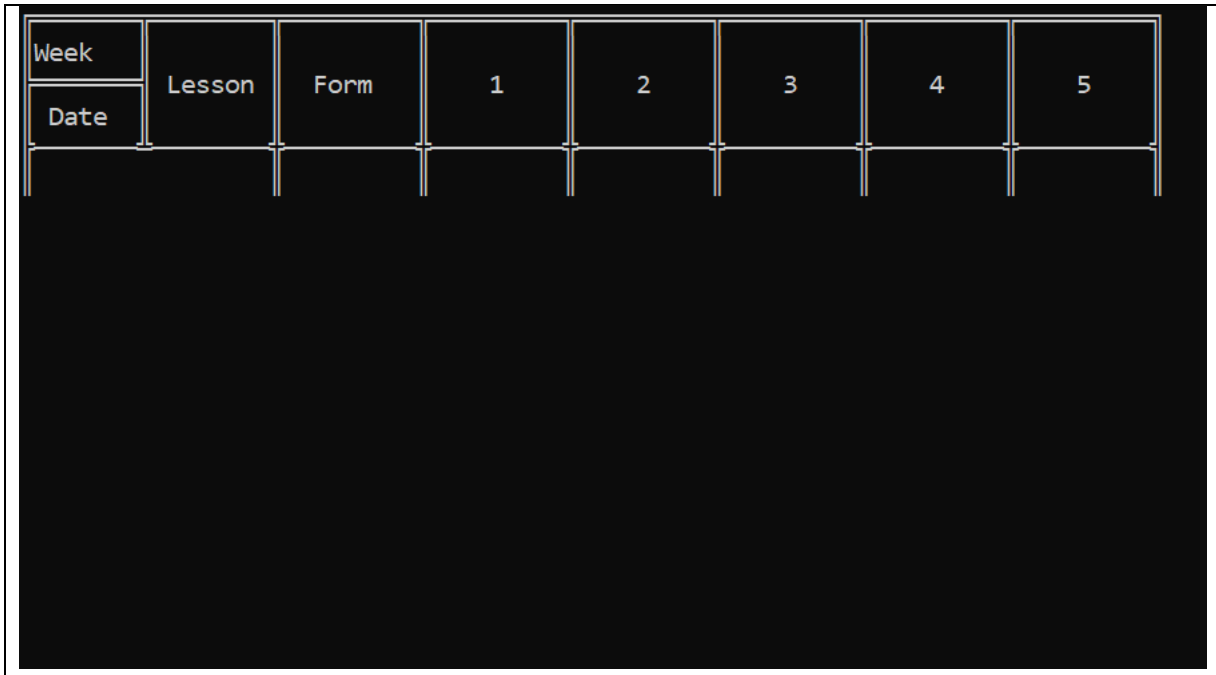
4.3.2	Student Menu Iteration 0	Testing if the program works on a Monday, the first day of a new week	Username: RobertCole
			Password: ABC123
			
			Then menu option 2
			
			Any key press
				The first date should say 13/0/2025 in yellow	It shows the previous week	Change
						`day := DayOfWeek(CurrentDate-2);`
						
						`day := DayOfWeek(CurrentDate-1)-1;`
						
						so instead of rolling the week back by one it adjusts it so that Monday is a 0 rather than sunday

Week B Date	Lesson	Form	1	2	3	4	5
06/01/2025		9AAA	9BAY/Enl	9BP2/Pe	9B/Hi1	9BAX/Ma	9BAY/Lf
07/01/2025		9AAA	9D/Mu1	9D/Mu1	9BAY/RPE	9BT1/Ch	9BT1/Ch
08/01/2025		9AAA	9BT1/Ph	9BAX/Ma	9BAY/Enl	9C/Dt1	9C/Dt1
09/01/2025		9AAA	9BAY/Enl	9BT1/Bi	9BAX/Ma	9BT1/Ph	9BP2/Pe
10/01/2025		9AAA	9BT1/Ph	9BAY/Enl	9B/Hi1	9B/Hi1	9BP2/Pe

4.3.2	Student Menu Iteration 1	Testing if the program works on a Monday, the first day of a new week	Username: RobertCole Password: ABC123 Then menu option 2 Any key press	The first date should say 13/0/2025 in yellow	See below	n/a

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
						
4.3.3	Student Menu Iteration 1	To test the animation runs the first time it is loaded and no others	Enter the timetable menu	Slowly entering the data to create an animation	See below	n/a



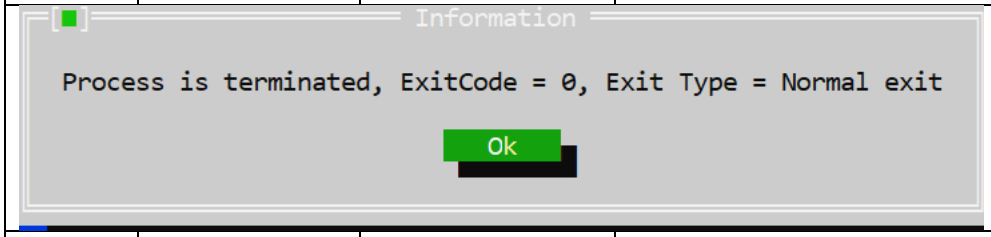
And then second time entering

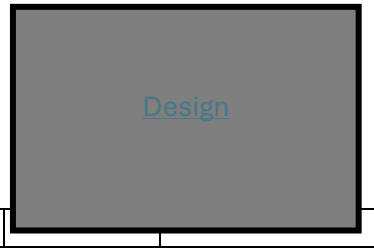
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments

Week A Date	Lesson	Form	1	2	3	4	5
13/01/2025		9AAA	9BT1/Bi	9BAY/Lf	9BT1/Ph	9BAX/Ma	9BP2/Pe
14/01/2025		9AAA	9BT1/Ch	9BT1/Ch	9BAY/Enl	9BAX/Ma	9D/Mu1
15/01/2025		9AAA	9B/Hi1	9B/Hi1	9C/Dt1	9BAY/Enl	9BT1/Bi
16/01/2025		9AAA	9BAY/Enl	9D/Mu1	9D/Mu1	9C/Dt1	9C/Dt1
17/01/2025		9AAA	9BAY/Enl	9BAX/Ma	9BT1/Bi	9BT1/Ph	9BP2/Pe

4.4.1	Student Menu Iteration 1	To test the log out works	Enter the menu logout	The program terminates	See below	n/a





Review

My client has provided quite a lot of useful feedback. She likes the layout of the main menu and how the information is presented nice and concisely, the screen is not too cluttered with many menus. They like the menu options being named appropriately. They really like the timetable formatting and how it is laid out. They also like the quality-of-life feature which highlights the current date in yellow. They enjoy the animation of the timetable drawing itself as it is “appealing to watch”, they also like that it only happens the first time instead of every time. They like the logout button which lets you know its logging out. They however would like me to finish the attendance form as that is not complete.

Iteration 5 – Teacher's Menu

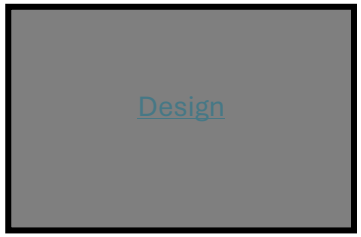
Aim

The aim of this iteration is to develop a menu for the teacher to interact with. It will have a similar design to the student's menu in the sense that it will be mostly controlled by menus and using the arrow keys in the menu to reduce the chance of an error by a mis tap.

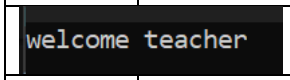
Code

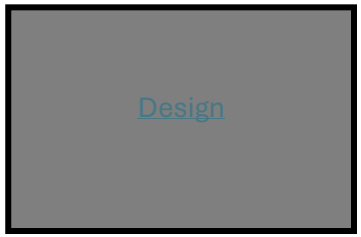
```
UNIT TeachersMenu;
  interface
  uses crt, sysutils;
  procedure startTeacher;
  implementation
  //main menu
  procedure startTeacher;
    begin
      //debug to prove it works
      writeln('welcome teacher');
      readln;
    end;

  end.
```



Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
5.1.1	Teachers Menu	Test if the menu is run	Logged into a teacher	Welcome teacher	See below	n/a
						



Review

My client didn’t have much to say about this other than its missing all the functionality.

Iteration 6 – Main Program

Aim

The aim of this iteration is to make a singular program which the client will run, and it will include all aspects of the solution. This keeps the program modular by only having a small section that needs to be tested. In this iteration only the login code is run and then it chooses another unit to go into based on whether a student or teacher logged in. Keeping each part of the solution separate allows for easier testing and making sure very unit works before integrating it.

Code

```
PROGRAM Main;
  //declare all units to be used
  USES Crt, Sysutils, LogInScreen, loading, LogInProcedures,
  TeachersMenu, StudentsMenu;
  var userType : STRING;
      intUserID : LONGINT;

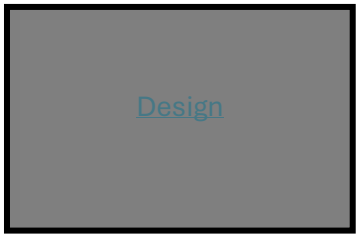
  //declare main program
  function start:STRING;
    var i, ErrCode, userIndex : LONGINT;
      userId : STRING;
      userDetailsArray : ARRAY[0..1] OF STRING;
      logIn : logInFile;
      percentDone : REAL;
      logedIn : BOOLEAN;
  begin
    //declare login file
    logIn := logInFile.create;
    logIn.sort;
    userIndex := -1;

    //while loop to keep running log in attempts
    logedIn := false;
    while logedIn = false do
      begin
        ClrScr;
        //login methods
        LogInScreen.startLogIn(userDetailsArray);
        logIn.search(userDetailsArray,[1,2], userIndex, false);
        loading.startLoading('L', percentDone);
        Delay(1000);
        loading.silentKillLoading;
        Delay(100);
        //test username and password are valid
        if userIndex = -1 THEN
          begin
```

```
            writeln('Username Or Password is incorrect.');
            readkey;
          end
        else
          begin
            logedIn := true;
            //get the user info
            login.readFile(userIndex);
            //test if the username and password belong to a teacher or student
            userId := Trim(login.data.PersonID);
            val(userid,intUserID,ErrCode);
            if ErrCode <> 0 THEN
              begin
                start := 'T';
              end
            else
              begin
                start := 'S';
              end;
          end;

      end;
      //delete login file object
      logIn.free;
  end;

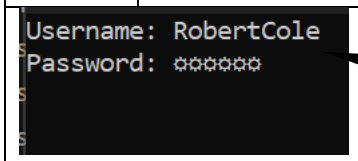
  begin
    //start login
    userType := start;
    if userType = 'T' THEN
      //teacher menu
      TeachersMenu.startTeacher
    else if userType = 'S' THEN
      //student menu
      StudentsMenu.startStudent(intUserID)
    end;
  end.
```



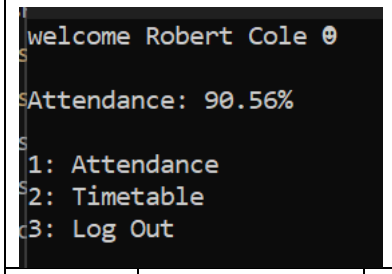
Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Testing

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
6.1.1	Main Iteration 0	To test if it passes correctly onto the student unit	Username: RobertCole Password: ABC123	Welcome Robert Cole Attendance: 90% (as example) 1. Attendance 2. Timetable 3. Log Outd	See below	n/a

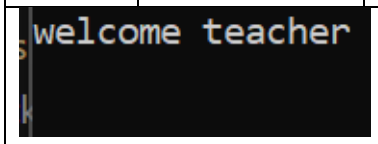


Evidence to show it brings up the login menu with data entered



Evidence of the student menu showing up after entering valid username and password

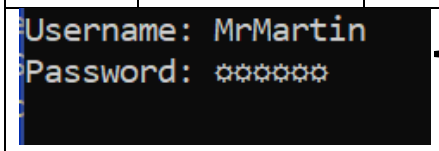
6.2.1	Main Iteration 0	To test if it passes correctly onto the teacher unit	Username: JessicaBowman Password: ABC123	Welcome Teacher	See below	n/a



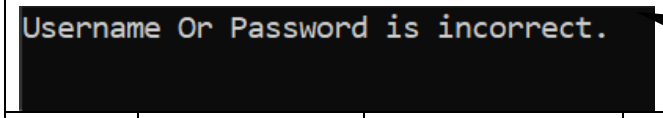
Entering a valid username and password for a teacher accounts logs you into the teacher account.

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Test Number	Program and Iteration	Purpose of test	Data to be entered	Expected result	Actual results	Amendments
6.3.1	Main Iteration 0	Test if incorrect username is entered then it requests the user to enter it again	Username: MrMartin Password: ABC123	Username Or Password is incorrect.	See below	n/a



Evidence of invalid user data being entered to test it rejects the entered data



Evidence of it rejecting the information

Design

Review

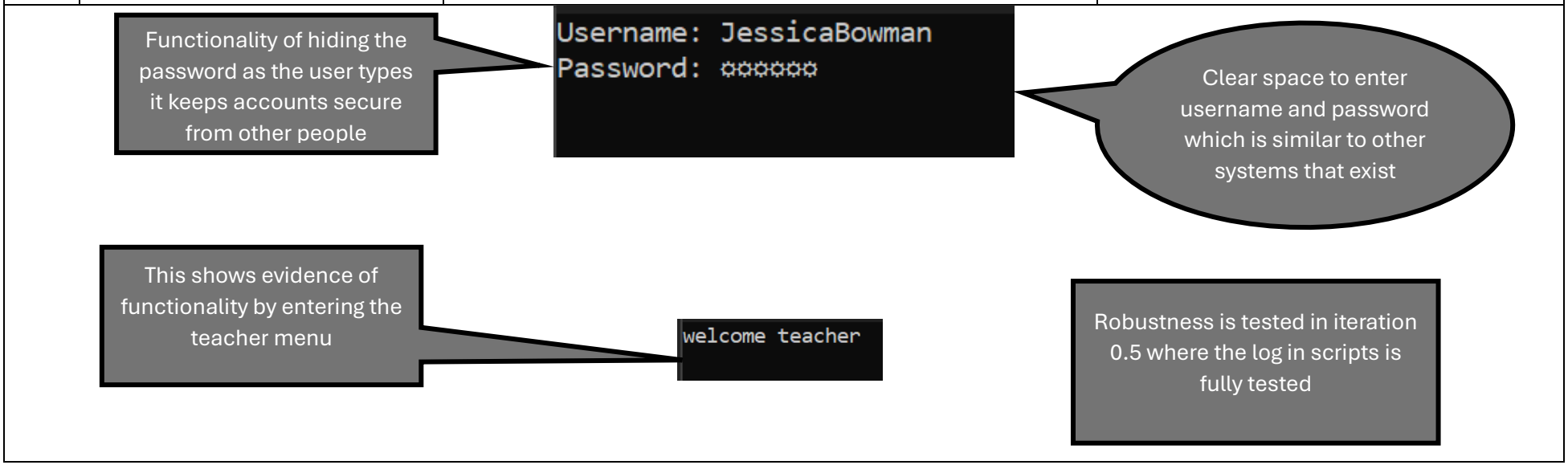
My client made most of her comments in the review of iteration 0 about the login screen. But she likes the loading bit before you go into the actual menu.

Evaluation

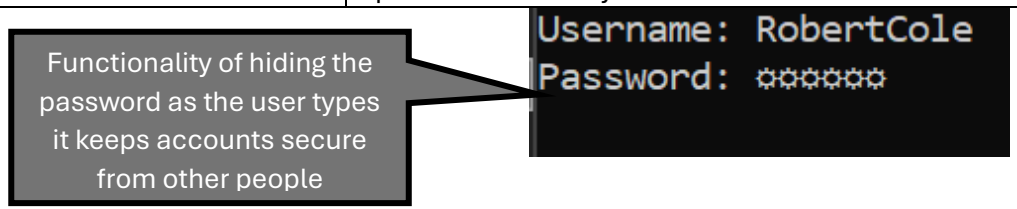
Testing to Inform Evaluation

For post development testing, I have put together a selection of tasks that the user could want to do whilst using the solution. I have documented these tasks with a test plan below. The data that I selected to test the scenario has been included during the iterative development stage. All the data that will be used here has been tested independently and all subset of the task has been tested independently as well.

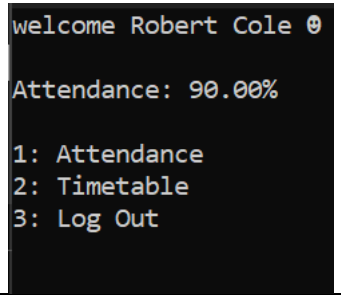
Test	Scenario	Justification	Data Used
1	Teacher wants to log into the system	All teacher options and data access should not be accessible by students. So, by having an account system with secure log ins stops students being able to access data they should not be able to access	Username: JessicaBowman Password: ABC123



2	Student wants to log into the system	The system contains information about the student that the student will want access to. Such as being able to view their timetable or attendance. So, by having an account system and a log in system it allows for students to log into their own space and view only their data	Username: RobertCole Password: ABC123



Functionality of hiding the password as the user types it keeps accounts secure from other people

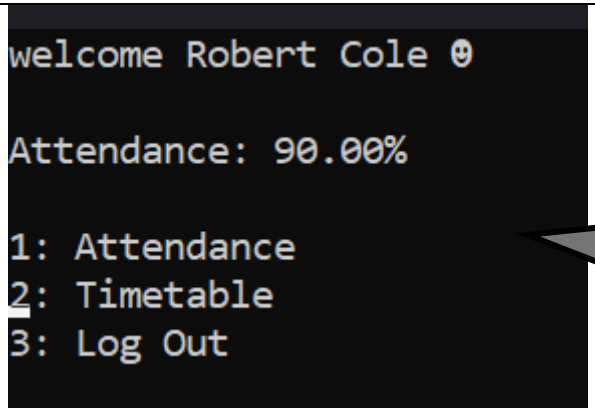


This shows evidence of functionality by entering the student menu

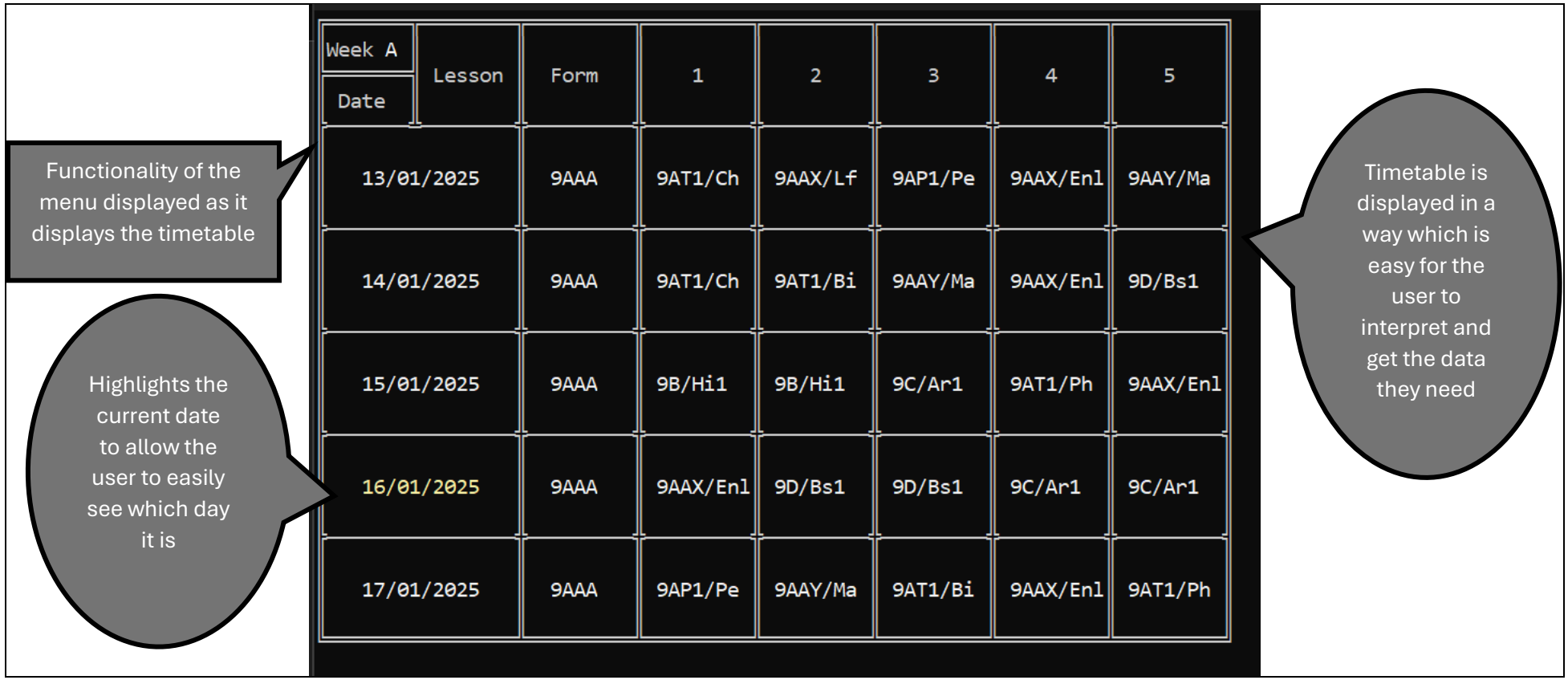
Robustness is tested in iteration 0.5 where the log in scripts is fully tested

3	Students wanting to be able to view their timetable	Each student needs to know which class they have an when it is so that they can cross reference that with their attendance marks or if they need to know what lessons they have and when.	Username: RobertCole Password: ABC123 Select menu option 2

The cursor is constrained to the menu options demonstrating robustness of the solution



Menu controlled by the arrow keys and then pressing enter to select making it more user friendly and convenient to use



Week A Date	Lesson	Form	1	2	3	4	5
13/01/2025		9AAA	9AT1/Ch	9AAX/Lf	9AP1/Pe	9AAX/Enl	9AAY/Ma
14/01/2025		9AAA	9AT1/Ch	9AT1/Bi	9AAY/Ma	9AAX/Enl	9D/Bs1
15/01/2025		9AAA	9B/Hi1	9B/Hi1	9C/Ar1	9AT1/Ph	9AAX/Enl
16/01/2025		9AAA	9AAX/Enl	9D/Bs1	9D/Bs1	9C/Ar1	9C/Ar1
17/01/2025		9AAA	9AP1/Pe	9AAY/Ma	9AT1/Bi	9AAX/Enl	9AT1/Ph

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Success of the Solution

In creating a solution for my client, the following success criteria identified below were met. However, in developing the solution there were many complex success criteria that had to be met by the programmer to start working on the client success criteria

Evaluation of solution against success criteria for client

Success Criteria (Feature)	Description	Measurable Criterion	Criteria met?	Evidence of test data	Future Development
Log in page	This will ask for a username and password, and depending on the status logins in depends on the tasks offered to the user	This is measurable as it should either out put a menu for a teacher or a menu for the student. When a teacher logs in the functions will be: View Attendance Take a Register And if it was a student that logs in the list of functions will be: Attendance attendance percent: ...% Timetable	This criterion is partially met the login works for both student and teachers. And the student menu is complete with a slight redesign of the UI. But there is no teacher menu	Iterations 0.5 and 6 shows the log in section working. Iteration 4 shows the students menu. And iteration 5 shows that there is no teacher menu yet. This evidence shows that the criterion was partially met as some of the procedures were completed but not all.	. The menu design will be like the students though and can be easily completed using the create menu script demonstrated in iteration 0.8
Display register	This will display a list of student's names in that class with a choice of marks next to each student name	This is measurable as it should return all the students in the class with the options to mark them here or not This will display the names in a format of Name1 - [present] [absent with reason] [absent no reason] [late] Name2 - [present] [absent with reason] [absent no reason] [late]... And there will be the same number of names as students in that class which can be check by outputting the number of students in that class from the class file. With a mark all as present, mark all as absent and a submit button at the bottom	This criterion was not met.	There is no evidence to show as no part of this section has been developed yet.	For future development the craeteMenu script could be modified to allow for horizontal menus in different designs

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

<table>
<tr><th>Success Criteria (Feature)</th><th>Description</th><th>Measurable Criterion</th><th>Criteria met?</th><th>Evidence of test data</th><th>Future Development</th></tr>
<tr><td>Select different register marks</td><td>This will allow the user to move the cursor between each student and each option and selecting the option just by moving on to the next student. And them submitting by pressing enter</td><td>The end display will look like
Name1 - [present] [absent with reason] [absent no reason] [late]
Name2 - [present] [absent with reason] [absent no reason] [late]
Name3 - [present] [absent with reason] [absent no reason] [late]...
[Mark All Present]
[Mark All Absent]
[Submit Register]</td><td>This criterion was not met</td><td>There is no evidence to support any of this being done</td><td>A menu linking script could be used along with the cursor grid script to control the cursor in such a way that allows you to select each different option for each student. Or you can run it in a while loop storing the results in a 2d array corresponding to the position selected and looping until submit register is pressed</td></tr>
<tr><td>Display teacher timetable</td><td>This will display the teachers timetable and highlight the current lesson that it is. The timetable will be displayed with day along the top with date above that and then lesson numbers down the sides</td><td>
<table>
<tr><th></th><th>01/07/24</th><th>02/07/2024</th><th>03/07/2024</th><th>04/07/2024</th><th>05/07/2024</th></tr>
<tr><td>1</td><td>9AT1/Ch</td><td>9AT2/Ch</td><td>9AT1/Ch</td><td>9BT1/Ch</td><td></td></tr>
<tr><td>2</td><td>9AT1/Ch</td><td>9AT1/Bi</td><td></td><td>9BT1/Ch</td><td></td></tr>
<tr><td>3</td><td></td><td>9AT1/Bi</td><td>9BT1/Bi</td><td></td><td>9AT1/Ch</td></tr>
<tr><td>4</td><td>9BT1/Ch</td><td></td><td>9AT1/Bi</td><td>9BT2/Bi</td><td></td></tr>
<tr><td>5</td><td>9BT2/Bi</td><td></td><td></td><td>9AT2/Ch</td><td></td></tr>
</table>
The data will look something like this with classes and the highlighted one is the current lesson that they need to do the register for</td><td>This criterion was partially met. The timetable was redesigned to having the dates down the side and the lesson number on the top. Also, the timetable was never complete for the teacher but is very similar to the student one.</td><td>Evidence in Iteration 0.6 shows that the generic timetable can be written out. Then evidence in iteration 4 for the students shows that code has been developed to fill the timetable with data. This shows that it is partially met</td><td></td></tr>
</table>

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Success Criteria (Feature)	Description	Measurable Criterion	Criteria met?	Evidence of test data	Future Development
Search for student	There will be a search bar provided that can be typed into and will display all students which are close to what they have typed in, they can then navigate down to each student and then select the student	Search for name: R Robert Cole Rita Gamble Rose Roberts Robert Flores... This will refine as more letters are typed in, the selected one can then be selected by pressing enter and you will enter the overview for that student	This criterion has not been met	No evidence	For future development you can do a repeat until loop with a read key for the user typing in. Then using a string subset search, search student file for that substring. Then write all results storing in an array that resets itself each time a key is pressed. If the Down key is pressed then convert the list of names into a menu which the user can then select from

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Success Criteria (Feature)	Description	Measurable Criterion	Criteria met?	Evidence of test data	Future Development
Display form attendance	This will display a list of all students in that teachers form with their attendance next to their name and colour coded to highlight the students that need talking to. This will also be ordered from lowest attendance to highest attendance	Wendell Hulslander – 75% Marylouise Brooks – 83% Rose Roberts – 92% Tracy Boyd – 95% Velma Morris – 99.8% This is what an example set of data should look like when looking through the teachers for attendance	This criterion is not met	There is no evidence for any section of this.	For future development a search can be done on the students file to get all the students in that form and then an insertion sort can be done to sort the students by attendance with lowest attendance first with the student id attached to the attendance using parallel arrays. This array can then be iterated through with a case statement to control the colour of the text to write all students and their attendance

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

<table>
<tr><th>Success Criteria (Feature)</th><th>Description</th><th>Measurable Criterion</th><th>Criteria met?</th><th>Evidence of test data</th><th>Future Development</th></tr>
<tr>
<td>attendance certificate</td>
<td>This will display all information about the student's attendance. Including percentages and a table of where they missed lesson.</td>
<td>Name: Wendell Hulslander
Attendance: 75%
Authorised absences attendance: 89%
Unauthorised absences: 14
Lates: 12
<table>
<tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr>
<tr><td>05/07/24</td><td>/\</td><td>L</td><td>A</td><td>L</td><td>/\</td></tr>
<tr><td>04/07/24</td><td>L</td><td>A</td><td>A</td><td>/\</td><td>/\</td></tr>
<tr><td>03/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>L</td></tr>
<tr><td>02/07/24</td><td>L</td><td>/\</td><td>A</td><td>/\</td><td>/\</td></tr>
<tr><td>01/07/24</td><td>L</td><td>L</td><td>A</td><td>/\</td><td>A</td></tr>
</table>
...
IT should show the students name, attendance, attendance with authorised absences, unauthorised absences, and late marks. It should then also have a table and all marks in that year.</td>
<td>This criterion was partially met, all the independent sections of this can be easily achieved. The table can be easily written using writeDesign. The filling for the table is already done and the information at the top is just read and write from the file</td>
<td>In the iteration 4 there is demonstration of a timetable being filled in. There is also evidence of being able to read and write attendance data. And iteration 4 shows an example of writing out the timetable. These pieces of evidence show that some of this is done just scattered in other places</td>
<td>For future development reads and writes from the attendance file can display the information at the top. writeDesign can be used to draw the table. A similar algorithm to the one that fills out the students timetable can be used for the attendance marks. And then using a cursor grid you can use the output of that to change the start date in the timetable by a week at a time</td>
</tr>
<tr>
<td>Attendance timetable (teacher)</td>
<td>This will provide a timetable for the student with all register marks for a ser period</td>
<td>Name: Erica Hernandez
...
<table>
<tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr>
<tr><td>05/07/24</td><td>/\</td><td>L</td><td>A</td><td>L</td><td>/\</td></tr>
<tr><td>04/07/24</td><td>L</td><td>A</td><td>/\</td><td>/\</td><td>/\</td></tr>
<tr><td>03/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>L</td></tr>
<tr><td>02/07/24</td><td>L</td><td>/\</td><td>A</td><td>/\</td><td>/\</td></tr>
<tr><td>01/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>A</td></tr>
</table>
This is a display of all register marks between two time periods set by the teacher when entering this view</td>
<td>This criterion was not met</td>
<td>There is no evidence for anything similar to this except the timetable layout shown in iteration 4. Showing that this criterion was not met due to it lacking the critical information</td>
<td>This would be a copy of what the student would see, so I would make a procedure in writeDesigns to write the attendance data starting on a certain date for a week</td>
</tr>
</table>

Success Criteria (Feature)	Description	Measurable Criterion	Criteria met?	Evidence of test data	Future Development
Attendance overview (teacher)	This will bring up a display giving the name of the student, overall attendance, Attendance with authorised absences, Unauthorised absences, and late marks	Name: Ann Lavoie Attendance: 85% Authorised absences attendance: 93% Unauthorised absences: 4 Lates: 2	This criterion is not met	No evidence	For future development, after selecting Attendance overview in the students' options after the teacher searched for that student. Just read the attendance data for that student and write it out
Attendance overview (student)	This will give the student a quick overview of all their attendance statistics, which is the same as what the teacher will see. It will display overall attendance, attendance with authorised absences, Unauthorised absences, and late marks	Attendance: 89% Authorised absences attendance: 96.4% Unauthorised absences: 1 Lates: 0	This criterion is partially met. In the main menu for the students, that students overall attendance is displayed at the top.	Evidence for this is shown in iteration 4. There is no entire overview for the students attendance. But in the main menu it does show overall attendance. This shows that some of the data can be outputted thus proving the criterion was partially met	For future development a sub menu in the students attendance menu, then another option called "attendance overview" which then reads from the attendance file and displays the information out.

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

<table>
<tr><th>Success Criteria (Feature)</th><th>Description</th><th>Measurable Criterion</th><th>Criteria met?</th><th>Evidence of test data</th><th>Future Development</th></tr>
<tr><td>Attendance timetable (student)</td><td>This will give the student a week-by-week view of all their register marks over the entire year which they can cycle through to see when they have missed lessons or were marked as late</td><td>This will display a 5-day week from Monday to Friday with lesson numbers as headings and it will display all the register marks that have been marked down that week. So, from my example data one example could look like this.

<table><tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td>01/07/24</td><td>/\</td><td>L</td><td>A</td><td>/\</td><td>/\</td></tr><tr><td>02/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td></tr><tr><td>03/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>L</td></tr><tr><td>04/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td></tr><tr><td>05/07/24</td><td>/\</td><td>/\</td><td>/\</td><td>/\</td><td>A</td></tr></table></td><td>This criterion was not met</td><td>No evidence</td><td>For future development reads and writes from the attendance file can display the information at the top. writeDesign can be used to draw the table. A similar algorithm to the one that fills out the students timetable can be used for the attendance marks. And then using a cursor grid you can use the output of that to change the start date in the timetable by a week at a time</td></tr>
<tr><td>Timetable (student)</td><td>This will give the student a week-by-week view of their timetable and their classes. This is so they can cross reference what register marks they got for which class</td><td>This will display a 5-day week from Monday to Friday with lesson numbers as headings and it will display the class that the student has at that time. So, from my example data one example could look like this.

<table><tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td>01/07/24</td><td>9AT1/Bi</td><td>9AT1/Bi</td><td>9D/Dt1</td><td>9D/Dt1</td><td>9AAX/Ma</td></tr><tr><td>02/07/24</td><td>9AAX/Ma</td><td>9ABY/En</td><td>9B/Fr1</td><td>9C/Pe1</td><td>9C/Pe1</td></tr><tr><td>03/07/24</td><td>9BAY/Lf</td><td>9A/Dr1</td><td>9ABY/En</td><td>9B/Fr1</td><td>9B/Fr1</td></tr><tr><td>04/07/24</td><td>9D/Dt1</td><td>9AAX/Ma</td><td>9A/Dr1</td><td>9AT1/Bi</td><td>9ABY/En</td></tr><tr><td>05/07/24</td><td>9ABY/En</td><td>9C/Pe1</td><td>9D/Dt1</td><td>9AAX/Ma</td><td>9A/Dr1</td></tr></table></td><td>This criterion has been fully met. In the students menu they can select timetable to view their timetable for that week.</td><td>This is shown in iteration 4 which shows evidence of the students timetable being displayed in the correct format. Which proves this to be fully met by outputting the same result</td><td>n/a</td></tr>
</table>

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Evaluation of solution against success criteria for programmer

Success Criteria (Feature)	Description	Measurable Criterion	Criteria met?	Evidence of test data	Future Developments
Developing a data set with as much realism as possible	Develop a set a of data that mimics a real set of data from a school. This data set is also very large to see if my solution can cope with large amounts of data like it will have to in school	The data should match what my client has access to and uses for the registration system. So, it is based on client feedback. There should also be large amount data, which is more than 100 students, full timetable with multiple class on at each time	When the CreateFiles.pas A series of files is created with all the data that is needed for the system to operate. I asked my client to review the data, and she said that this data is adequate for the system	Iteration 1 shows evidence of all the data being generated correctly for each file which then got approved by my client	n/a
Developing a general sort and search algorithm for the files	Each file should be able to be sorted and searched. Since .dta file are stored using records you have to read each file into a different variable with the structure that matches that of the file being read	This is measurable by having a single search and sort algorithm that each file can use to search and sort itself.	This criterion was also fully met. The class structure that I developed for each file uses inheritance to have a generic search and sort algorithm which each child class having specific methods to extract the data from the files	Iteration 2 shows the development of my class structure for the files. It also shows the sorts and searchers working showing the criterion is fully met.	n/a

Success Criteria (Feature)	Description	Measurable Criterion	Criteria met?	Evidence of test data	Future Developments
Developing a module to control the cursor	Due to the solution containing controls with the cursor it was necessary to develop a universal script which could be used to control the cursor in many ways such as bounding conditions, different step sizes and starting positions.	With a single call, using parameters to control step size, boundaries and start position the script should allow the user to control the cursor for as long as they need until the press enter to submit. The script should then return the new position of the cursor for further handling	This criterion was fully met. When the script runs it allows full control of the cursor using the arrow keys and exits when you press enter	Iteration 0.7 for the cursor control. And to prove it returns the correct position there is evidence in iteration 0.8 with a menu to help prove it. The evidence shows the success of this criterion by demonstrating that the user can control the cursor.	n/a
Develop a method to design an interface in a text file and then have that written to the screen	Due to the solution containing needing to contain many tables and other graphical forms it would be simpler to design an outline of the structure in a text file and have that written to the screen to then be filled with data	Design a method to take a text file or file path to a text file and write out the design inside the text file with a method to add delays into the output to add an animation effect	This criterion was fully met. When I call the writeDesign script passing in a path to a text file, it will read that and display it to the screen	Iteration 0.6 shows evidence for creating a UI procedure. This shows this by running different designs through it proving it can do any text file thus meaning the criterion	n/a

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Evidence of usability features

Usability feature	Description	Evidence	Success of usability feature
Clear way to enter username and password	Clear box for the user to type in the username and then immediately type in the password	Evidence of this is shown in iteration 0.5 which is where I test in depth the login screen	This was very effective as a useability feature due to the design of it mimicking most other forms of login making it intuitive to use and operate.
Hidden password on entry	The password is hidden while being typed	Evidence for this is also shown in iteration 0.5	This was also very effective as a usability feature as it hides the password so other people cannot log into your account
Easy keyboard navigation within the menus	Allows you to use the keyboard to navigate the menu without the cursor being able to leave the menu	Evidence for this is shown in iteration 0.7 where I test the menu creation and the cursor moving functions which is tested in more depth in iteration 0.6	This was partially effective as a usability feature as while it is nice and intuitive to use the most common method of navigation is the mouse making it only partially effective
Tabular designs	Displaying information in a table or in similar forms such as graphical.	Evidence of this is shown in both iteration 0.6 and iteration 4 which also shows evidence of the table being completed	This was a very effective usability feature as the most common way of represent data is in a table due to it being easy to read and interpret for everyone making this a highly effective usability feature
Predictive searching	As you type to search for a student or something else it provides a list of everything that matches so far	There is no evidence for this feature as I ran out of time to develop it	This is not a very effective usability feature as it doesn't exist so it can not be used machining it ineffective
Colour coded attendance	In the overview of attendance for a form group the students will be sorted from lowest to highest attendance and colour coded to show if any actions should be taken	There is no evidence for this feature as I ran out of time to develop it	This is not a very effective usability feature as it doesn't exist so it can not be used machining it ineffective

Maintenance and Future Development

Future Development of Usability features

Usability feature	Description	Future development plan
Predictive searching	As you type to search for a student or something else it provides a list of everything that matches so far	I plan to develop this in such a way where you can press the down arrow key, and it swaps to you selecting from the list of names rather then you having to press enter and then it brings you to a new page of options before resetting. I also need to adjust some parts in in the cursor grid or redesign it to act more like a switchboard for key binds where you can match any key to a list of functions.
Colour coded attendance	In the overview of attendance for a form group the students will be sorted from lowest to highest attendance and colour coded to show if any actions should be taken	I plan to actually develop it so it can actually be used as a feature.

Maintenance issues and limitations

The way in which I designed the solution is modular by nature, almost everything is contained in another unit (another script which hold procedures) making it very modular and easy to expand on. The file class structure while modular and new child classes can easily be created for new files by copying and pasting the existing ones and just changing the record type. However, I use static array types to reference the data in the file which only during testing is going to be of a fixed size and in deployment will be constantly changing. This does cause limitations with expandability of the files but in future development a linked list could be used instead of an array so the data can be stored in a dynamic structure rather than a static one which causes limitations. I also use linear searches for all of my searches, which for some cases is necessary as multiple pieces of data can match the search criteria, but as the data size grows this will take longer and longer to complete, so in the future some searches can be binary searches to cut down on search time

Appendix A – Final Code

```
unit attendancProcedures;
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        dataRecord = RECORD
                          StudentID : LONGINT;
                          Attendance : REAL;
                          Total : LONGINT;
                          NumberPresent : LONGINT;
                          NumberLate : LONGINT;
                          NumberAbsent : LONGINT;
                        end;

    type
        AttendanceFile = Class(Files)
            private
                dataFile: File OF dataRecord;
                dataTextFile : TextFile;
                indexArray: ARRAY[0..139] OF LONGINT;
                recordArray: ARRAY[0..139] OF dataRecord;
            public
                data : dataRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(iStudentID : LONGINT; iAttendance : REAL; iTotal : LONGINT; iNumberPresent :
LONGINT; iNumberLate : LONGINT; iNumberAbsent : LONGINT);
                procedure DisplayData;
                procedure displayFile;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
                procedure sort;
```

```
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray: ARRAY
OF LONGINT; switchLowerCase: BOOLEAN);

        end;

    IMPLEMENTATION

    constructor AttendanceFile.Create;
        var a : longint;
        begin
            inherited Create;
            ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\attendanceFile.dta');
            ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\attendanceOut.txt');
            RESET(dataFile);
            RESET(dataTextFile);
            for a := 0 TO 139 DO
              begin
                indexArray[a] := a;
              end;
        end;

    destructor AttendanceFile.Destroy;
        begin
            close(dataFile);
            close(dataTextFile);
            inherited Destroy;
        end;

    procedure AttendanceFile.SetData(iStudentID : LONGINT; iAttendance : REAL; iTotal : LONGINT; iNumberPresent :
LONGINT; iNumberLate : LONGINT; iNumberAbsent : LONGINT);

        begin
            with data do
                begin
                    StudentID := iStudentID;
                    Attendance := iAttendance;
                    Total := iTotal;
                    NumberPresent := iNumberPresent;
                    NumberLate := iNumberLate;
```

```
                NumberAbsent := iNumberAbsent;
            end;
    end;



procedure AttendanceFile.DisplayData;
    begin
    with data do
        begin
            writeln(StudentID);
            writeln(Attendance);
            writeln(Total);
            writeln(NumberPresent);
            writeln(NumberLate);
            writeln(NumberAbsent);
            writeln('--------------------------------------------------------------------------');
        end;
    end;

procedure AttendanceFile.displayFile;
    var a : LONGINT;
    begin
        reset(dataFile);
        while not (eof(dataFile)) do
            begin
                READ(dataFile,data);
                with data do
                    begin
                        writeln(StudentID);
                        writeln(Attendance);
                        writeln(Total);
                        writeln(NumberPresent);
                        writeln(NumberLate);
                        writeln(NumberAbsent);
                        writeln('--------------------------------------------------------------------------');
                    end;
            end;
    end;
```

```
procedure AttendanceFile.extractToTextFile;
    begin
        reset(dataFile);
        rewrite(dataTextFile);
        while not eof(dataFile) do
            begin
                read(dataFile, data);
                writeln(dataTextFile, data.StudentID);
                writeln(dataTextFile, data.Attendance);
                writeln(dataTextFile, data.Total);
                writeln(dataTextFile, data.NumberPresent);
                writeln(dataTextFile, data.NumberLate);
                writeln(dataTextFile, data.NumberAbsent);
                writeln(dataTextFile, '');
            end;
    end;

procedure AttendanceFile.writeToFile;
    begin
        write(dataFile, data);
    end;


procedure AttendanceFile.readHere;
    begin
        if FIlePos(dataFile) < FileSize(dataFile) then
            begin
                read(dataFile,data);
            end;
    end;

procedure AttendanceFile.readFile(readFromPosition : LONGINT);
    begin
        if readFromPosition < FileSize(dataFile) then
            begin
                SEEK(dataFile,readFromPosition);
                read(dataFile,data);
            end;
```

```
        end;

    procedure AttendanceFile.sort;
        var a,b : LONGINT;
            maxStrLen : STRING;
        begin
          RESET(dataFile);
          upper:= 139;
          for a := 0 TO 139 DO
            begin
              READ(dataFile,data);
              recordArray[a] := data;
              dataArray[a] := Format('%'+'3'+'s',[IntToStr(data.StudentID)]);
            end;
        Files.sort(indexArray);
        RESET(dataFile);
        for a := 0 TO 139 DO
            begin
                WRITE(dataFile,recordArray[indexArray[a]]);
                indexArray[a] := a;
            end;

        end;

        procedure AttendanceFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray
: ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            var a: LONGINT;
            begin
                RESET(dataFile);
                for a := 0 TO 139 DO
                    begin
                        read(dataFile, data);
                        Str(data.StudentID, fileArray[0,a]);
                        Str(data.Attendance, fileArray[1,a]);
                        Str(data.Total, fileArray[2,a]);
                        Str(data.NumberPresent, fileArray[3,a]);
                        Str(data.NumberLate, fileArray[4,a]);
                        Str(data.NumberAbsent, fileArray[5,a]);
                    end;
```

```
            Files.search(139,6,searchValues, searchFields, returnArray, switchLowerCase);
        end;


END.
```

```
UNIT BinarySearch;
  INTERFACE

  USES CRT, SYSUTILS;


  PROCEDURE BinarySearchFunction(low, high : LONGINT; searchValue : STRING; UserArray : ARRAY OF STRING; var
returnArray : ARRAY OF INTEGER; var numberDone : LONGINT);
    VAR
      i, mid : LONGINT;
      shouldEnd : BOOLEAN;

    IMPLEMENTATION

  PROCEDURE BinarySearchFunction(low, high : LONGINT; searchValue : STRING; UserArray : ARRAY OF STRING; var
returnArray : ARRAY OF INTEGER; var numberDone : LONGINT);
    VAR
      i, mid : LONGINT;
      shouldEnd : BOOLEAN;
    BEGIN
      REPEAT
        mid := (low+high)DIV 2;
        IF UserArray[mid] <> searchValue THEN
          BEGIN
                IF UserArray[mid] < searchValue THEN
                  BEGIN
                    low := mid +1;
                  END
                ELSE
                  BEGIN
                    high := mid -1;
                  END;
              END
        ELSE
          BEGIN
            returnArray[0] := mid;
            shouldEnd := true;
          END;
      UNTIL (shouldEnd = true) OR (low > high);
```

```
    END;

  END.
```

```
UNIT BubbleSort;
  INTERFACE

  USES CRT, SYSUTILS;


  PROCEDURE BubbleSortFunction(VAR UserArray : ARRAY OF STRING; low, upper: LONGINT);

    VAR
      i : LONGINT;
      Temp : STRING;
      Swaps : BOOLEAN;

    IMPLEMENTATION

  PROCEDURE BubbleSortFunction(VAR UserArray : ARRAY OF STRING; low, upper: LONGINT);
    BEGIN
      upper := upper-1;
      REPEAT
        Swaps:= false;
        FOR i := 0 TO upper DO
          BEGIN
            IF UserArray[i] > UserArray[i+1] THEN
              BEGIN
                Temp := UserArray[i];
                UserArray[i] := UserArray[i+1];
                UserArray[i+1] := Temp;
                Swaps := true;
              END;
          END;
        upper := upper -1;
      UNTIL (NOT Swaps) OR (upper < 0);
    END;

  END.
```

```
unit classProcedures;
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        dataRecord = RECORD
                          ClassID : STRING[8];
                          Year : LONGINT;
                          subjectID : STRING[3];
                          NoOfStudents : LONGINT;
                        end;

    type
        classFile = Class(Files)
            private
                dataTextFile : TextFile;
                dataFile: File OF dataRecord;
                indexArray: ARRAY[0..51] OF LONGINT;
                recordArray: ARRAY[0..51] OF dataRecord;
            public
                data : dataRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(iClassID : STRING; iYear : LONGINT; isubjectID : STRING; iNoOfStudents :
LONGINT);
                procedure DisplayData;
                procedure displayFile;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
                procedure sort;
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray: ARRAY
OF LONGINT; switchLowerCase: BOOLEAN);

        end;

    IMPLEMENTATION
```

```
constructor classFile.Create;
    var a : longint;
    begin
        inherited Create;
        ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\classes.dta');
        ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\classesOut.txt');
        RESET(dataFile);
        RESET(dataTextFile);
        for a := 0 TO 51 DO
          begin
            indexArray[a] := a;
          end;
    end;

destructor classFile.Destroy;
    begin
        close(dataFile);
        close(dataTextFile);
        inherited Destroy;
    end;

procedure classFile.SetData(iClassID : STRING; iYear : LONGINT; isubjectID : STRING; iNoOfStudents : LONGINT);
    begin
        with data do
            begin
                ClassID := iClassID;
                Year := iYear;
                subjectID := isubjectID;
                NoOfStudents := iNoOfStudents;
            end;
    end;



procedure classFile.DisplayData;
    begin
    with data do
        begin
```

```
                writeln(ClassID);
                writeln(Year);
                writeln(subjectID);
                writeln(NoOfStudents);
                writeln('--------------------------------------------------------------------------------');
            end;
        end;

    procedure classFile.displayFile;
        var a : LONGINT;
        begin
            reset(dataFile);
            while not (eof(dataFile)) do
                begin
                    READ(dataFile,data);
                    with data do
                        begin
                            writeln(ClassID);
                            writeln(Year);
                            writeln(subjectID);
                            writeln(NoOfStudents);
                            writeln('--------------------------------------------------------------------------------');
                        end;
                end;
        end;

    procedure classFile.extractToTextFile;
        begin
            reset(dataFile);
            rewrite(dataTextFile);
            while not eof(dataFile) DO
                begin
                    read(dataFile, data);
                    writeln(dataTextFile, data.ClassID);
                    writeln(dataTextFile, data.Year);
                    writeln(dataTextFile, data.subjectID);
                    writeln(dataTextFile, data.NoOfStudents);
                    writeln(dataTextFile, '');
                end;
```

```
        end;

    procedure classFile.writeToFile;
        begin
            write(dataFile, data);
        end;


    procedure classFile.readHere;
        begin
            if FIlePos(dataFile) < FileSize(dataFile) then
                begin
                    read(dataFile,data);
                end;
        end;

    procedure classFile.readFile(readFromPosition : LONGINT);
        begin
            if readFromPosition < FileSize(dataFile) then
                begin
                    SEEK(dataFile,readFromPosition);
                    read(dataFile,data);
                end;
        end;

    procedure classFile.sort;
        var a,b : LONGINT;
            maxStrLen : STRING;
        begin
          RESET(dataFile);
          upper:= 51;
          for a := 0 TO 51 DO
            begin
              READ(dataFile,data);
              recordArray[a] := data;
              dataArray[a] := Format('%'+'8'+'s',[(data.classID)]);
            end;
        Files.sort(indexArray);
        RESET(dataFile);
```

```
        for a := 0 TO 51 DO
            begin
                WRITE(dataFile,recordArray[indexArray[a]]);
                indexArray[a] := a;
            end;

        end;

        procedure classFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray :
ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            var a: LONGINT;
            begin
                RESET(dataFile);
                for a := 0 TO 51 DO
                    begin
                        read(dataFile, data);
                        fileArray[0,a] := LowerCase(data.classID);
                        Str(data.Year, fileArray[1,a]);
                        fileArray[2,a] := LowerCase(data.subjectID);
                        Str(data.NoOfStudents, fileArray[3,a]);
                        returnArray[a] := -1;
                    end;
                Files.search(51,4,searchValues, searchFields, returnArray, switchLowerCase);
            end;


    END.
```

```
PROGRAM CreateFiles;
  uses crt, sysutils, search,sorting, loading, DisplayErrorScrn, math;

  type {declaring all record types}
    TimeTableRecord = RECORD
                        Day : LONGINT;
                        LessonNo : LONGINT;
                        ClassID : STRING[8];
                      end;
    FormRecord = RECORD
                        FormID : STRING[5];
                        NumberInForm : LONGINT;
                      end;

    AttendanceRecord = RECORD
                        StudentID : LONGINT;
                        Attendance : REAL;
                        Total : LONGINT;
                        NumberPresent : LONGINT;
                        NumberLate : LONGINT;
                        NumberAbsent : LONGINT;
                      end;


    StudentsRecord = RECORD
                        StudentID : LONGINT;
                        FName : STRING[20];
                        SName : String[20];
                        Year : LONGINT;
                        Form : STRING[5];
                        Uname : String[40];
                        Pword : String[20];
                      end;


    teacherRecord = RECORD
                        TeacherID : STRING[3];
                        fname : string[20];
                        sname : string[20];
```

```
                        form  : string[5];
                        Uname : STRING[40];
                        PWord : STRING[20];
                      end;

    classRecord = RECORD
                        ClassID : STRING[8];
                        Year : LONGINT;
                        subjectID : STRING[3];
                        NoOfStudents : LONGINT;
                      end;

    subjectRecord = RECORD
                        SubjectID : STRING[3];
                        SubjectName : STRING[20];
                      end;

    studentRegisterMarkRecord = RECORD
                        studentID : LONGINT;
                        registerMark : CHAR;
                      end;

    registerRecord = RECORD
                        RegisterID : STRING[17];
                        LessonNo : LONGINT;
                        Date : TDateTime;
                        ClassID : STRING[8];
                        RegisterMarks : ARRAY[0..40] OF studentRegisterMarkRecord;
                        isOfstedRegister : BOOLEAN;
                      end;

    studentClassResolverRecord = RECORD
                        StudentID : LONGINT;
                        ClassID : STRING[8];
                      end;
    teacherSubjectResolverRecord = RECORD
                        TeacherID : STRING[3];
                        SubjectID : STRING[2];
                      end;
```

```
    teacherClassResolverRecord = RECORD
                           TeacherID : STRING[3];
                           ClassID : STRING[8];
                         end;

    LogInDetailsRecord = RECORD
                           PersonID: STRING[5];
                           Username: STRING[40];
                           Passwowrd: String[20];
                         end;
  VAR {setting up all variables for records and files}
    numberDone, total : LONGINT;

    TimeTable : TimeTableRecord;
    TimeTableTextFile : TextFile;
    TimeTableFile : FILE OF TimeTableRecord;
    TimeTableInfoFromFile : STRING;

    Form : FormRecord;
    FormFile : FILE OF FormRecord;

    Register : registerRecord;
    studentRegisterMark : studentRegisterMarkRecord;
    RegisterFile : File OF registerRecord;


    Attendance : AttendanceRecord;
    AttendanceFile : FILE OF AttendanceRecord;
    StudentFName : STRING;
    NameFile : TextFile;
    Student : StudentsRecord;
    StudentFile : FILE OF StudentsRecord;

    TeacherFileFromFile : TextFile;
    TeacherFile : file of teacherRecord;
    TeacherFromFile : String;
    Teacher : teacherRecord;
```

```
    sClass : ClassRecord;
    classFile : FILE OF ClassRecord;

    subject : subjectRecord;
    subjectFile : FILE OF subjectRecord;

    studentClassResolver : studentClassResolverRecord;
    studentClassResolverFile : FILE OF studentClassResolverRecord;

    teacherSubjectResolver : teacherSubjectResolverRecord;
    teacherSubjectResolverFile : FILE OF teacherSubjectResolverRecord;

    teacherClassResolver : teacherClassResolverRecord;
    teacherClassResolverFile : FILE OF teacherClassResolverRecord;

    empty, ErrCode : LONGINT; {setting up other variables for functions that require an input}
    percentComplete : REAL; {variable for progress bar to track progress}

    LogInDetails: LogInDetailsRecord;
    LogInDetailsFile : FILE OF LogInDetailsRecord;

     {setting up common array across many functions}
    classIDArray : ARRAY[0..50] OF STRING;



  procedure initialiseFiles;
    begin
      percentComplete := 0.0; {initializing percentDone and total iterations}
      total := 0;
      {assigning a place on the drive to the files}
      ASSIGN(TimeTableTextFile,'C:\ComputerScienceCoursework\Program\files\TimeTableText.txt');
      ASSIGN(TimeTableFile,'C:\ComputerScienceCoursework\Program\files\TimeTable.dta');
      total:= total +51;

      ASSIGN(RegisterFile,'C:\ComputerScienceCoursework\Program\files\Registers.dta');
      total := total + 60;

      ASSIGN(NameFile,'C:\ComputerScienceCoursework\Program\files\Names.txt');
```

```
    ASSIGN(StudentFile,'C:\ComputerScienceCoursework\Program\files\Students.dta');
    total := total + 140;

    ASSIGN(TeacherFile,'C:\ComputerScienceCoursework\Program\files\Teachers.dta');
    total := total + 51;

    ASSIGN(classFile,'C:\ComputerScienceCoursework\Program\files\classes.dta');
    total := total + 51;

    ASSIGN(subjectFile,'C:\ComputerScienceCoursework\Program\files\subjects.dta');
    total := total + 21;

    ASSIGN(studentClassResolverFile,'C:\ComputerScienceCoursework\Program\files\studentClassResolver.dta');
    total := total + 140;

    ASSIGN(teacherSubjectResolverFile,'C:\ComputerScienceCoursework\Program\files\teacherSubjectResolver.dta');
    total:= total + 51;

    ASSIGN(teacherClassResolverFile,'C:\ComputerScienceCoursework\Program\files\teacherClassResolver.dta');
    total := total + 51;

    ASSIGN(FormFile,'C:\ComputerScienceCoursework\Program\files\forms.dta');
    total := total +5;

    ASSIGN(AttendanceFile,'C:\ComputerScienceCoursework\Program\files\attendanceFile.dta');

    ASSIGN(LogInDetailsFile,'C:\ComputerScienceCoursework\Program\files\LogInDetails.dta');
    total := total + 191;
  end;


procedure closeAllFiles;
  begin
    {closing all files to confirm all changes are fully written and complete}
    close(TimeTableTextFile);
    close(TimeTableFile);
    close(RegisterFile);
    close(NameFile);
    close(StudentFile);
```

```
    close(TeacherFile);
    close(classFile);
    close(subjectFile);
    close(studentClassResolverFile);
    close(teacherSubjectResolverFile);
    close(teacherClassResolverFile);
    close(FormFile);
    close(AttendanceFile);
    close(LogInDetailsFile);
  end;

procedure CreateTimeTable;

  begin
    reset(TimeTableTextFile); //set pointer to start of file
    rewrite(TimeTableFile); //clear all exiting data in timetable file
    while not EOF(TimeTableTextFile) do //iterate until all data is extracted
      begin
        numberDone := numberDone +1;
        percentComplete := numberDone/total;
        readln(TimeTableTextFile,TimeTableInfoFromFile); //get the class id
        TimeTable.ClassId := TimeTableInfoFromFile; //assign it to the record variable
        repeat
          readln(TimeTableTextFile,TimeTableInfoFromFile);
          TimeTable.LessonNo := StrToInt(TimeTableInfoFromFile);
          readln(TimeTableTextFile,TimeTableInfoFromFile);
          TimeTable.Day := StrToInt(TimeTableInfoFromFile);
          readln(TimeTableTextFile,TimeTableInfoFromFile);
          try
            write(TimeTableFile,TimeTable);
          finally
          end;
        until TimeTableInfoFromFile = 'Q'; {repeat until all dates are read and stored in the .dta file
                                            for that class marked by a Q in the text file}
      end;
  end;

procedure addForms;
  var a, i : LONGINT;
```

```
    begin
      REWRITE(FormFile); {clear all exisiting data in the form file}
      for i := 0 to 3 DO {iterate for all 4, year 9 forms}
        begin
          numberDone := numberDone + 1;
          percentComplete := numberDone / total;
          form.formID := '9' + 'AA' + Chr(i +65); {create 4 unique for identifiers}
          form.NumberInForm := 0; {set Number of kids in form to 0}
          write(FormFile, form); {push data to file}
        end;
      numberDone := numberDone + 1;
      percentComplete := numberDone / total;
      form.formID := '12' + 'AAA'; {do the same for 1 year 12 form}
      form.NumberInForm := 0;
      write(FormFile,form);
      reset(FormFile); {set point to start of form file}
      for i := 0 to 4 DO {for every form}
        begin
        read(formFile,form);
        timetable.classid := form.formID; {get form id into a timetable class id variable}
          for a := 1 to 10 DO {add it to the file as lesson 0 for every morning in the 2 week timetabe}
            begin
              timetable.LessonNo := 0;
              timetable.Day := a;
              write(TimeTableFile, timetable); {add new data to timetable file}
            end;
        end;
    end;

  procedure createRegisters;
    var a,b,c,i,randomMark,day : LONGINT; {Initizlize iterative variables, arrays and other variables for this
procedure}
        dayArray : ARRAY[0..320] OF STRING;
        dayReturnArray : ARRAY[0..320] OF LONGINT;
        studentClassResolverClassArray : ARRAY[0..1384] OF STRING;
        classReturnArray : ARRAY[0..1384] OF LONGINT;
        classId, startDate : STRING;
        studentFormIdArray : ARRAY[0..139] OF STRING;
        returnStudentFormIdArray : ARRAY[0..139] OF LONGINT;
```

```
    date : TDateTime;
  begin
    //set up files and reset file pointers
    reset(TimeTableFile);
    reset(studentClassResolverFile);
    RESET(StudentFIle);
    rewrite(RegisterFile);
    i := 0;
    startDate := '13/09/2024';
    //populating the arrays with the data from the files
    for i := 0 To 139 DO
      begin
        READ(StudentFile,Student);
        studentFormIdArray[i] := Student.Form;
        returnStudentFormIdArray[i] := -1;
      end;
    i := 0;
    while not(eof(TimeTableFile)) DO
      begin
        read(TimeTableFile,TimeTable);
        dayArray[i] := IntToStr(TimeTable.day);
        dayReturnArray[i] := -1;
        i := i +1;
      end;
    i := 0;
    while not(eof(studentClassResolverFile)) do
      begin
        read(studentClassResolverFile,studentClassResolver);
        studentClassResolverClassArray[i] := studentClassResolver.ClassID;
        classReturnArray[i] := -1;
        i := i +1;
      end;
    b := 0;

    // for loop for number of days with registers
    for i := 0 to 29 do
      begin
        //gets the number day of the week and corrects to monday being 0
        day := DayOfWeek(StrToDate(startDate)+b-1);
```

```
          if (day = 7) OR (day = 6) THEN //if weekend decrease for loop var by 1
            begin
              i := i -1;
            end
          else
            begin
              numberDone := numberDone +1;
              percentComplete := numberDone / total;
              //clear out array for searching for classes on that day
              for c := 0 TO 269 DO
                dayReturnArray[c] := -1;
              Search.SearchFunctions('L',0,269,IntToStr(DayOfWeek(StrToDate(startDate)+b-
1)),dayArray,dayReturnArray,empty);
              a := 0;
              while dayReturnArray[a] <> -1 DO // for every class on that day
                begin
                  //find and get the data about one class
                  SEEK(TimeTableFile,dayReturnArray[a]);
                  read(TimeTableFile, TimeTable);
                  //set register data
                  Register.LessonNo := TimeTable.LessonNo;
                  Register.Date := StrToDate(startDate)+b;
                  if TimeTable.LessonNo = 4 THEN
                    Register.isOfstedRegister := true
                  else
                    Register.isOfstedRegister := false;
                  Register.classID := '';
                  Register.classID := TimeTable.classID;
                  register.registerID := IntToStr(register.lessonNo) + register.classID +
DateTimeToStr(register.date);
                  classId := format('%s',[Register.classID]);
                  //clear out array for searching for students in that class
                  for c := 0 to 1384 DO
                    classReturnArray[c] := -1;

Search.SearchFunctions('L',0,1384,Register.ClassID,studentClassResolverClassArray,classReturnArray,empty);
                  c:=0;
                  //for every student in that class
                  while classReturnArray[c] <> -1 DO
```

```
            begin
              //get the class with the studenet in
              SEEK(studentClassResolverFile, classReturnArray[c]);
              read(studentClassResolverFile, studentClassResolver);
              //get the student id
              studentRegisterMark.studentID := studentClassResolver.studentID;
              //get the data in the attendance file
              SEEK(AttendanceFile,studentClassResolver.studentID);
              read(AttendanceFile,Attendance);
              Attendance.Total := Attendance.Total +1;
              //randomly assign the student a mark
              randomMark := RANDOM(100);
              if randomMark <= 90 then
                begin
                  studentRegisterMark.registerMark := 'P';
                  Attendance.NumberPresent := Attendance.NumberPresent + 1;
                end
              else if randomMark <= 97 then
                begin
                  studentRegisterMark.registerMark := 'L';
                  Attendance.NumberLate := Attendance.NumberLate + 1;
                end
              else
                begin
                  studentRegisterMark.registerMark := 'A';
                  Attendance.NumberAbsent := Attendance.NumberAbsent + 1;
                end;
              //write data back to register file
              Register.registerMarks[c] := studentRegisterMark;
              SEEK(AttendanceFile,FilePos(AttendanceFile)-1);
              Attendance.Attendance := (Attendance.NumberPresent / Attendance.Total);
              write(AttendanceFile,Attendance);
              c := c +1;
            end;
          write(RegisterFile,Register);

          a := a +1;
        end;
    end;
```

```
            b := b +1;
        end;


        b:= 0;
        for i := 0 to 29 DO //for loop for each form
          begin
            //gets the number day of the week and corrects to monday being 0
            day := DayOfWeek(StrToDate(startDate)+b-1);
            if (day = 7) OR (day = 6) THEN //if weekend decrease for loop var by 1
              begin
                i := i -1;
              end
            else
              begin
                numberDone := numberDone +1;
                percentComplete := numberDone / total;
                //clear out array for searching for forms on that day
                for c := 0 TO 320 DO
                  dayReturnArray[c] := -1;
                Search.SearchFunctions('L',269,320,IntToStr(DayOfWeek(StrToDate(startDate)+b-
1)),dayArray,dayReturnArray,empty);
                a := 0;
                while dayReturnArray[a] <> -1 DO// for every form on that day
                  begin
                    SEEK(TimeTableFile,dayReturnArray[a]);
                    read(TimeTableFile, TimeTable);
                    if TimeTable.LessonNo = 0 THEN
                      begin
                        //set register data
                        Register.LessonNo := TimeTable.LessonNo;
                        Register.Date := StrToDate(startDate)+b;
                        Register.isOfstedRegister := true;
                        Register.classID := '';
                        Register.classID := TimeTable.classID;
                        register.registerID := IntToStr(register.lessonNo) + register.classID +
DateTimeToStr(register.date);
                        //clear out array for searching for students in that form
                        classId := format('%s',[Register.classID]);
```

```
                        for c := 0 TO 139 DO
                          returnStudentFormIdArray[c] := -1;

Search.SearchFunctions('L',0,139,Register.ClassID,studentFormIdArray,returnStudentFormIdArray,empty);
                        c := 0;
                        //for every student in that form
                        while returnStudentFormIdArray[c] <> -1 DO
                          begin
                            //get the class with the studenet in
                            SEEK(StudentFile, returnStudentFormIdArray[c]);
                            read(StudentFile, Student);
                            //get the student id
                            studentRegisterMark.studentID := student.studentID;
                            //get the data in the attendance file
                            SEEK(AttendanceFile,student.studentID);
                            read(AttendanceFile,Attendance);
                            Attendance.Total := Attendance.Total +1;
                            //randomly assign the student a mark
                            randomMark := RANDOM(100);
                            if randomMark <= 90 then
                              begin
                                studentRegisterMark.registerMark := 'P';
                                Attendance.NumberPresent := Attendance.NumberPresent + 1;
                              end
                            else if randomMark <= 97 then
                              begin
                                studentRegisterMark.registerMark := 'L';
                                Attendance.NumberLate := Attendance.NumberLate + 1;
                              end
                            else
                              begin
                                studentRegisterMark.registerMark := 'A';
                                Attendance.NumberAbsent := Attendance.NumberAbsent + 1;
                              end;
                            //write data back to register file
                            Register.registerMarks[c] := studentRegisterMark;
                            SEEK(AttendanceFile,FilePos(AttendanceFile)-1);
                            Attendance.Attendance := (Attendance.NumberPresent / Attendance.Total);
                            write(AttendanceFile,Attendance);
```

```
                        c := c +1;
                      end;
                      write(RegisterFile,Register);

                  end;
                  a := a + 1;
              end;
          end;
          b := b + 1;
      end;
  end;


procedure addStudents;
  var i : LONGINT;
    ffname, fsname, fyear, fform, fattendance : STRING; {set up variables to read from text file into}
  begin
    i := 0;
    RESET(FormFile); {resets pointer in form file}
    read(formFIle, form); {gets the first form}
    RESET(NameFile);
    REWRITE(StudentFile); {clears both studenet and attendance file}
    REWRITE(AttendanceFile);
    for i := 0 to 119 DO {generate 120 year 9 students}
      begin
        if (i mod 30 = 0) and (i <> 0) THEN //changes the form every 30 students
          begin
            seek(formFile, (FilePos(formFile)-1));
            write(FormFile,form);
            READ(FormFile, form);
          end;
        numberDone:= numberDone +1; //update progress bar
        percentComplete := numberDone/total;
        Student.StudentID := i; //set student data
        READLN(NameFile, student.fname); //get a first and last name from a list of names
        READLN(NameFile, student.sname);
        student.year := 9;
        student.uname := student.fname+student.sname;
        student.pword := 'ABC123';
```

```
      student.form := form.formid;
      form.NumberInForm := form.NumberInForm + 1;
      //set attendance data
      Attendance.studentId := i;
      Attendance.Attendance := 1;
      Attendance.Total := 0;
      Attendance.NumberPresent := 0;
      Attendance.NumberLate := 0;
      Attendance.NumberAbsent := 0;
      write(StudentFile,student); //write student and attendance data to their respective files
      write(AttendanceFile, Attendance);
    end;
  seek(formFile, (FilePos(formFile)-1));
  write(FormFile,form);
  read(FormFile, form); {get the year 12 form}
  for i := 120 to 139 DO {generate 20 year 12 students}
    begin
      numberDone := numberDone +1; //update progress bar
      percentComplete := numberDone/total;
      Student.StudentID := i; //set student data
      READLN(NameFile, student.fname); //get a first and last name from a list of names
      READLN(NameFile, student.sname);
      student.year := 12;
      student.uname := student.fname+student.sname;
      student.pword := 'ABC123';
      form.NumberInForm := form.NumberInForm + 1;
      student.form := form.formid;
      //set attendance data
      Attendance.studentId := i;
      Attendance.Attendance := 1;
      Attendance.Total := 0;
      Attendance.NumberPresent := 0;
      Attendance.NumberLate := 0;
      Attendance.NumberAbsent := 0;
      write(StudentFile,student); //append this record to the end of the student file
      write(AttendanceFile, Attendance);
    end;
    //write changes to form file
    seek(formfile, filepos(formFile)-1);
```

```
        write(formfile,form);
    end;

  procedure addTeachers;
    var i : LONGINT;
      ffname, fsname, fform : STRING; {declare all variables to read from files into them}
    begin
      RESET(FormFile);
      rewrite(TeacherFile);
      for i := 0 TO 50 DO //create 51 teachers, 1 per class
        begin
            numberDone := numberDone + 1;
            percentComplete := numberDone / total;
            READLN(NameFile, teacher.Fname); //read first and surname form names file
            READLN(NameFile, teacher.Sname);
            Teacher.TeacherID := teacher.FName[1]+teacher.SName[1]+teacher.SName[2]; //create the teacher id as
first letter of first name and forst two letters of surname
            Teacher.uname := teacher.FName + teacher.SName;
            Teacher.pword := 'ABC123';
            write(TeacherFile,teacher);
        end;
      //assign teachers to forms
      RESET(TeacherFile);
      for i := 0 to 4 DO
        begin
          read(TeacherFile,teacher);
          read(FormFile,form);
          teacher.form := form.formID;
          seek(TeacherFile,FilePos(TeacherFile)-1);
          write(TeacherFile,teacher);
        end;
    end;


  procedure createLogInFile;
      begin
        RESET(StudentFile); //reset file pointers for both files
        RESET(TeacherFile);
        REWRITE(LogInDetailsFile);
```

```
        //extract login information and student id and write it to a file
        while not EOF(StudentFile) do
          begin
            READ(StudentFile,student);
            //LogInDetails.PersonID := Format('%'+'5'+'s',[IntToStr(student.StudentID)]); // format the student id
as a string
            LogInDetails.PersonID := IntToStr(student.StudentID);
            LogInDetails.Username := student.Uname;
            LogInDetails.Passwowrd := student.Pword;
            write(LogInDetailsFile,LogInDetails);
            numberDone := numberDone +1;
            percentComplete := numberDone/total;
          end;
        while not EOF(TeacherFile) do
          begin
            //extract login information and student id and write it to a file
            READ(TeacherFile,teacher);
            LogInDetails.PersonID := teacher.teacherID;
            LogInDetails.Username := teacher.Uname;
            LogInDetails.Passwowrd := teacher.Pword;
            write(LogInDetailsFile,LogInDetails);
            numberDone := numberDone +1;
            percentComplete := numberDone/total;
          end;

      end;


  PROCEDURE createSubjects;
    // create a constant array of all subjects and the subjects' id
    const subjID : ARRAY[0..20] OF STRING[3] = ('Cs','Ma','Mu','Enl','Gg','Ch','Ph','Bi','DT','Dr',
                                                'Lf','Rpe','Fr','Hi','Gm','Bs','Ar','Py','Pe','Sp','So');
          subjNm : ARRAY[0..20] OF STRING[20] = ('Computer
Science','Maths','Music','English','Geography','Chemistry','Physics','Biology','Design Technology','Drama',
                                                'Life Skills','Religion, Philosophy &
Ethics','French','History','German','Business Studies','Art','Physcology','Physical
Education','Spanish','Sociology');
    var i : LONGINT;
    begin
```

```
      //clear subject file
      REWRITE(SubjectFile);
      //for every subject write subject name and id to a file
      for i := 0 TO 20 do
        begin
          numberDone := numberDone +1;
          percentComplete := numberDone / total;
          with subject do
            begin
              subjectID := subjID[i];
              SubjectName := subjNm[i];
            end;
            WRITE(SubjectFile, subject);
          end;
      end;

  PROCEDURE createClasses;
    //create arrays of all classes
    const yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph',
'9AT2/Ph', '9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph',
'9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs1', '9D/Mu1', '9D/Gm1');
          yTwelveClassArray : ARRAY[0..6] OF STRING =
('12A/Ph1','12A/Py1','12B/Ph1','12C/Cp1','12C/Py1','12D/Mu1','12D/Cp1');
      VAR i : LONGINT;
          classArray : ARRAY[0..50] OF classRecord;
      begin
        rewrite(classFile);
        //for every year 9 class
        for i := 0 TO 43 DO
          begin
            numberDone := numberDone + 1;
            percentComplete := numberDone/total;
            //set year 9 class data
            sClass.ClassID := yNineClassArray[i];
            sClass.year := 9;
```

```
            sClass.NoOfStudents := 0;
            //generate the class id
            val(yNineClassArray[i][8],empty,ErrCode);
            if ErrCode <> 0 THEN
              begin
                sClass.subjectID := yNineClassArray[i][6]+yNineClassArray[i][7]+yNineClassArray[i][8];
                val(sClass.subjectID[1],empty,ErrCode);
                if ErrCode = 0 THEN
                  begin
                    sClass.subjectid := yNineClassArray[i][4]+yNineClassArray[i][5];
                  end;
              end
            else
              sClass.subjectID := yNineClassArray[i][6]+yNineClassArray[i][7];
            write(classFile, sClass);
            //delay(100);
          end;
        //generate year 12 classes
        for i := 0 TO 6 DO
          begin
            numberDone:= numberDone +1;
            percentComplete := numberDone/total;
            sClass.ClassId := yTwelveClassArray[i];
            sClass.year := 12;
            sClass.NoOfStudents := 0;
            sClass.subjectID := yTwelveClassArray[i][5]+yTwelveClassArray[i][6];
            write(classFile, sClass);
          end;
        reset(classFile);
        for i := 0 to 50 do
        begin
          READ(classFile, classArray[i]);
          classIDArray[i] := classArray[i].classID;
        end;
      end;

  procedure addRecordsStudentClassResolver(start, noOfAviablableClasses, maxStudents, ClasesToSortInto, year :
LONGINT);
```

```
    const yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph',
'9AT2/Ph', '9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph',
'9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs1', '9D/Mu1', '9D/Gm1');
          yTwelveClassArray : ARRAY[0..6] OF STRING =
('12A/Ph1','12A/Py1','12B/Ph1','12C/Cp1','12C/Py1','12D/Mu1','12D/Cp1');
    VAR startPoint, randomChoiceClass, i,d,l : LONGINT;
        returnArray : ARRAY[0..50] OF LONGINT;
    begin
      RESET(classFile);
      d := 0;
      repeat
        d := d + 1;
        randomChoiceClass := RANDOM(noOfAviablableClasses);  //randomly picks classes from the number of available
class that student can be in
        if year = 9 THEN
          Search.SearchFunctions('L',0,50,yNineClassArray[start+randomChoiceClass],classIDArray,returnArray,empty)
//checks the year 9 to find the class chosen
        else if year = 12 THEN

Search.SearchFunctions('L',0,50,yTwelveClassArray[start+randomChoiceClass],classIDArray,returnArray,empty) //checks
the year 12 to find the class chosen
        else
          DisplayErrorScrn.ErrScr;
        SEEK(classFile, returnArray[0]); //find the class in the file
        read(classFile, sClass);
        if d > 10000 THEN
          begin
            readln;
          end;
        until sClass.NoOfStudents < maxStudents; //only exit if there is a place for the student in that class
      l := 0;
      startPoint := returnArray[0];
      for l := 0 to (ClasesToSortInto * 2) -1 do //for every class connceted to that one class
        begin
          if year = 9 THEN
```

```
            studentClassResolver.classID := yNineClassArray[start+randomChoiceClass+l]
          else if year = 12 THEN
              studentClassResolver.classID := yTwelveClassArray[start+randomChoiceClass+l]
          else
            DisplayErrorScrn.ErrScr;
          write(studentClassResolverFile, studentClassResolver); //write each student and the class to the resolver
file
          //update number of students in that class
          SEEK(classFile, startPoint + l);
          read(classFile, sClass);
          SEEK(classFile, FilePos(classFile)-1);
          sClass.NoOfStudents := sClass.NoOfStudents + 1;
          write(classFile, sClass);
          l := l + 1;
        end;
    end;

  procedure createClassStudentResolver;
    const yNineClassArray : ARRAY[0..43] OF STRING = ('9AT1/Ch', '9AT2/Ch', '9AT1/Bi', '9AT2/Bi', '9AT1/Ph',
'9AT2/Ph', '9AAX/Ma', '9AAY/Ma', '9AAX/Enl', '9AAY/Enl', '9AAX/Lf', '9AAY/Lf',
          '9AAX/RPE', '9AAY/RPE', '9AP1/Pe', '9AP2/Pe', '9BT1/Ch', '9BT2/Ch', '9BT1/Bi', '9BT2/Cp', '9BT1/Ph',
'9BT2/Ph',
          '9BAX/Ma', '9BAY/Ma', '9BAX/Enl', '9BAY/Enl', '9BAX/Lf', '9BAY/Lf', '9BAX/RPE', '9BAY/RPE', '9BP1/Pe',
'9BP2/Pe', '9B/Fr1', '9B/Sp1', '9B/Gg1', '9B/Hi1', '9C/Dr1', '9C/Ar1', '9C/Py1',
          '9C/Dt1', '9D/So1', '9D/Bs1', '9D/Mu1', '9D/Gm1');
          yTwelveClassArray : ARRAY[0..6] OF STRING =
('12A/Ph1','12A/Py1','12B/Ph1','12C/Cp1','12C/Py1','12D/Mu1','12D/Cp1');
    var returnArray : ARRAY[0..51] OF LONGINT;
        studentYear,a, d, i, startPoint, noOfAlevels, noTakingFourAlevels, randClassNo, randomSide, numberInASide,
numberInBSide, randomChoiceClass : LONGINT;
        oldClass, newClass : STRING;

    begin
      RESET(StudentFile);
      RESET(classFile);
      rewrite(studentClassResolverFile);
      //keep track of how many students on each side of the year
      numberInASide := 0;
      numberInBSide := 0;
```

```
randomSide := RANDOM(2);
noTakingFourAlevels := 0;
while not(eof(StudentFile)) do //for every student
  begin
    numberDone := numberDone+1;
    percentComplete := numberDone/total;
    READ(StudentFile,student); //get the student data
    studentClassResolver.studentID := student.StudentID;
    if student.year = 9 THEN
      begin
        //pick a random side without overflowing one side
        if (numberInASide < 60) AND (numberInBSide < 60) THEN
          randomSide := RANDOM(2)
        else if (numberInASide < 60) THEN
          randomSide := 0
        else if (numberInBSide < 60) THEN
          randomSide := 1;
        if randomSide = 0 THEN //a or b side
          begin
              //put students in each set of classes
              numberInASide := numberInASide + 1;
              //Science
              addRecordsStudentClassResolver(0, 2, 30, 3, 9);
              //Maths
              addRecordsStudentClassResolver(6, 2, 30, 1, 9);
              //Englishs
              addRecordsStudentClassResolver(8, 2, 30, 3, 9);
              //Pe
              addRecordsStudentClassResolver(14, 2, 30, 1, 9);
          end
          else if randomSide = 1 THEN //a or b side
            begin
              //Science
              numberInBSide := numberInBSide + 1;
              addRecordsStudentClassResolver(16, 2, 30, 3, 9);
              //Maths
              addRecordsStudentClassResolver(22, 2, 30, 1, 9);
              //Englishs
              addRecordsStudentClassResolver(24, 2, 30, 3, 9);
```

```
            //Pe
            addRecordsStudentClassResolver(30, 2, 30, 1, 9);
          end
          else
            begin
              //error catching for random side assignment
              writeln('A: ',numberInASide, '  B: ',numberInBSide);
              Delay(3000);
              DisplayErrorScrn.ErrScr;
            end;
        //randomly pick the optional subjects
        for a := 0 to 8 do
          begin
            addRecordsStudentClassResolver(32 + a, 4, 30, 1, 9);
            a := a + 3;
          end;
      end
    else if (student.year = 12) THEN // year 12
      begin
        if noTakingFourAlevels < 5 THEN
          noOfAlevels := RANDOM(10);
        if noOfAlevels = 0 THEN //taking 4 a lvels
          begin
            noTakingFourAlevels := noTakingFourAlevels + 1;
            //block a
            addRecordsStudentClassResolver(0, 2, 20, 1, 12);

            //block b
            addRecordsStudentClassResolver(2, 1, 5, 1, 12);


            //block c
            addRecordsStudentClassResolver(3, 2, 20, 1, 12);


            //block d
            addRecordsStudentClassResolver(5, 2, 20, 1, 12);
          end
        else
```

```
                begin
                  //block a
                  addRecordsStudentClassResolver(0, 2, 20, 1, 12);

                  //block c
                  addRecordsStudentClassResolver(3, 2, 20, 1, 12);


                  //block d
                  addRecordsStudentClassResolver(5, 2, 20, 1, 12);

                end;
            end;
        end;
    end;

  PROCEDURE assignTeacherToClass;
    begin
      reset(classFile);
      reset(teacherfile);
      REWRITE(teacherClassResolverFile);
      while not(eof(classFile)) DO //for each teacher
        begin
          numberDone := numberDone+1;
          percentComplete := numberDone / total;
          //assign one teacher per class
          read(classFile,sClass);
          read(teacherfile, teacher);
          teacherClassResolver.teacherID := teacher.teacherID;
          teacherClassResolver.classID := sClass.classId;
          write(teacherClassResolverFile,teacherClassResolver);
        end;
    end;

  PROCEDURE assignTeacherToSubject;
    var a, b : LONGINT;
    begin
      RESET(TeacherFile);
      reset(classFile);
```

```
    REWRITE(teacherSubjectResolverFile);
    while not EOF(TeacherFile) DO
      begin
        numberDone := numberDone+ 1;
        percentComplete := numberDone / total;
        read(TeacherFile,teacher);
        read(classFile,sClass);
        //find the class the teacher teaches and gets the subject id of that class and puts it in a resolver file
        teacherSubjectResolver.teacherId := teacher.teacherID;
        teacherSubjectResolver.subjectId := sClass.subjectID;
        write(teacherSubjectResolverFile, teacherSubjectResolver);
      end;
    end;



begin
  //generates a random seed
  randomize;
  //run each procedure
  initialiseFiles;
  loading.startLoading('P',percentComplete);
  CreateTimetable;
  addForms;
  addStudents;
  addTeachers;
  createSubjects;
  createClasses;
  createClassStudentResolver;
  assignTeacherToClass;
  assignTeacherToSubject;
  createRegisters;
  createLogInFile;
  closeAllFiles;
  loading.EndLoading;
  readln;
end.
```

```
UNIT CursorGrid;
  interface
  uses crt, sysutils, vputils;

  var stepx , stepY, movePos, minX, minY, maxX, maxY: LONGINT;

  procedure moveUp;
  procedure moveDown;
  procedure moveLeft;
  procedure moveRight;
  procedure startGrid(moveStepX, moveStepY, startX, startY : LONGINT; boundaries : ARRAY OF LONGINT; var x, y :
LONGINT);

  implementation
  procedure moveUp;
    begin
      //if the cursor will stay in the boundaries set
      if WhereY - stepy >= minY THEN
        begin
          movePos := WhereY - stepy;
          GoToXY(WhereX, movePos);
        end;
    end;

  procedure moveDown;
    begin
      //if the cursor will stay in the boundaries set
      if WhereY + stepy <= maxY THEN
        begin
          movePos := WhereY + stepy;
          GoToXY(WhereX, movePos);
        end;
    end;

  procedure moveLeft;
    begin
      //if the cursor will stay in the boundaries set
      if WhereX - stepx >= minX THEN
        begin
```

```
          movePos := WhereX - stepx;
          GoToXY(movePos, WhereY);
        end;
    end;

  procedure moveRight;
    begin
      //if the cursor will stay in the boundaries set
      if WhereX + stepx <= maxX THEN
        begin
          movePos := WhereX + stepX;
          GoToXY(movePos, WhereY);
        end;
    end;

  procedure startGrid(moveStepX, moveStepY, startX, startY : LONGINT; boundaries : ARRAY OF LONGINT; var x, y :
LONGINT);
    var keyPress : char;
    begin
      stepX := moveStepX;
      stepY := moveStepY;
      minX := boundaries[0];
      minY := boundaries[1];
      maxX := boundaries[2];
      maxY := boundaries[3];
      GoToXY(startX,startY);
      //key checking key presses until enter is pressed
      repeat
        keyPress := readkey;
        if keyPress = #0 then //check if an extended key or function key
          begin
            case readKey of //read its scan code to get which arrow key was pressed and call according procedure
              'H' : moveUp;
              'P' : moveDown;
              'K' : moveLeft;
              'M' : moveRight;
            end;
          end;
      until keyPress = #13;
```

```
      //corrects the return to a choice of menu option rather than spaces moved
      if stepX <> 0 THEN//stops divide by 0 error
          x := Round((WhereX - startX) / stepX) //round to change float to int
      ELSE
          x := startX;

      if stepY <> 0 THEN//stops divide by 0 error
        y := round((WhereY - startY) / stepY) //round to change float to int
      ELSE
        y := startY;
    end;

  end.
```

```
unit FileProcedures;

//allows constructors
{$M+}

    interface

    USES crt, sysutils;

    TYPE
        //declare class structure
        Files = class
            protected
              upper : LONGINT;
              dataArray: ARRAY[0..1400] OF STRING;
              fileArray: ARRAY[0..7,0..1400] OF STRING;
            public
              constructor Create;
              destructor Destroy; override;
              procedure sort(var indexArray : ARRAY OF LONGINT);
              procedure search(max, fields : LONGINT; searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE;
var returnArray : ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            end;

    IMPLEMENTATION

    constructor Files.Create;
      begin
      end;

    destructor Files.Destroy;
        begin
        end;

    //sort algorithm for the files
    procedure Files.sort(var indexArray : ARRAY OF LONGINT);
        var TempIndex, i, code, tmp1, tmp2 : LONGINT;
            TempStr : STRING;
            Swaps: BOOLEAN;
        begin
```

```
            //bubble sort
            upper := upper-1;
            REPEAT
                Swaps:= false;
                FOR i := 0 TO upper DO
                BEGIN
                    //checks done with the data array
                    IF dataArray[i] > dataArray[i+1] THEN
                    BEGIN
                        TempStr := dataArray[i];
                        dataArray[i] := dataArray[i+1];
                        dataArray[i+1] := TempStr;
                        Swaps := true;
                        //swap data in index array
                        TempIndex := indexArray[i];
                        indexArray[i] := indexArray[i+1];
                        indexArray[i+1] := TempIndex;
                    END;
                END;
                upper := upper -1;
            UNTIL (NOT Swaps) OR (upper < 0);
        end;

    //search algorithm
    procedure Files.search(max, fields : LONGINT; searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var
returnArray : ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
        var found : BOOLEAN;
            a, b, c, returnIndex : LONGINT;
            data : STRING;
        begin
            //linear search
            //check if the search is case sensitive
            if switchLowerCase then
              begin
                searchValues[0] := LowerCase(searchValues[0]);
              end;
            else
              begin
                searchValues[0] := searchValues[0];
```

```
  end;
a := -1;
returnIndex := 0;
//check every value in the 2d array
while a < max do
    begin
    a := a + 1;
    //check each item in the record
    for b := 0 TO (fields-1) DO
        begin
          if switchLowerCase then
            begin
              data := LowerCase(fileArray[b,a]);
            end
          else
            begin
              data := fileArray[b,a];
            end;
          //check if one piece of data matches
            if data = searchValues[0] THEN
              begin
                found := true;
                //check each peice of data in record against data to search for
                for c := 0 to (SizeOf(searchFields)-1) do
                    begin
                        if switchLowerCase then
                          begin
                            data := LowerCase(FileArray[searchFields[c],a]);
                          end
                        else
                          begin
                            data := FileArray[searchFields[c],a];
                          end;
                        //if not all data matches continue search and dont return result
                        if searchValues[c] <> data THEN
                        begin
                            found := false;
                            break;
                        end;
```

```
                            end;
                          //if all data matches add index to the array of found matches
                          if found = true THEN
                              begin
                                  returnArray[returnIndex] := a;
                                  returnIndex := returnIndex+1;
                                  break
                              end;
                        end;
                end;
            end;
        end;

    END.
```

```
UNIT InsertionSort;
  interface
  uses crt, sysutils;

  PROCEDURE InsertionSortFunction(VAR UserArray : ARRAY OF STRING; low, upper: LONGINT);
    VAR
      i,position : LONGINT;
      temp : STRING;

    IMPLEMENTATION

  PROCEDURE InsertionSortFunction(VAR UserArray : ARRAY OF STRING; low, upper: LONGINT);
    BEGIN
        FOR i := 0 TO upper DO
          BEGIN
            temp := UserArray[i];
            position := i;
            while (position >= 0) AND (userArray[position-1] > temp) DO
              begin
                userarray[position] := userarray[position-1];
                position := position -1;
              end;
            userarray[position] := temp;
          end;
    end;
  end.
```

```
UNIT LinearSearch;
  INTERFACE

  USES CRT, SYSUTILS;


  PROCEDURE LinearSearchFunction(low, high : LONGINT; searchValue : STRING; UserArray : ARRAY OF STRING; var
returnArray : ARRAY OF INTEGER; var numberDone : LONGINT);
    VAR
      i, a,checkItem: LONGINT;

    IMPLEMENTATION

  PROCEDURE LinearSearchFunction(low, high : LONGINT; searchValue : STRING; UserArray : ARRAY OF STRING; var
returnArray : ARRAY OF INTEGER; var numberDone : LONGINT);
    VAR
      i, a,checkItem: LONGINT;
    BEGIN
      i := -1;
      a := 0;
      REPEAT
        i := i+1;
        numberDone := numberDone +1;
        if userArray[i] = searchValue THEN
        begin
          returnArray[a] := i;
          a := a + 1;
          //writeln('confirm');
        end;
      UNTIL (i = high);
    END;

  END.
```

```
unit loading;
  interface

  uses sysutils, vputils, crt, math;
  //pointer to point to the value outside of this script
  type rPtr = ^real;


  //pre declaration of functions and procedures
  function loadingText(p : pointer) : LONGINT;
    var a : LONGINT;
  function loadingBar(p : pointer) : LONGINT;
    var percentComplete : REAL;
      b,c,completeBars,uncompleteBars : LONGINT;
  procedure StartLoading(loadingType : CHAR; var percentIn : real);
  procedure EndLoading;
  procedure silentKillLoading;

  implementation
  //the symbol for a spinning symbol
  const LoadingSymbols : ARRAY[1..4] OF CHAR = ('/',#196,'\','|');

  //local script variables
  var loadingDone : BOOLEAN;
    percentDone : real;
    percentPtr : rptr;
    startRunTime, endRunTime : TDateTime;

  //the basic loading text with the spinning line
  function loadingText(p : pointer) : LONGINT;
    var a : LONGINT;
    begin
      HideCursor;
      a := 1;
      ClrScr;
      while (not loadingDone) DO
        Begin
          //write loading and symbol from the array
          WRITELN('Loading... ' + LoadingSymbols[a]);
```

```
          //progress the loading symbol
          a := a+1;
          if a > 4 THEN
            a := 1;
          //every 0.1 secons update
          Delay(100);
          //rewrite the whole line
          GoToXY(1,1);
          DelLine;
        end;

      end;

  function loadingBar(p : pointer) : LONGINT;
    var percentComplete : REAL;
      b,c,d,completeBars,uncompleteBars : LONGINT;
    begin
      HideCursor;
      ClrScr;
      //sets up the empty progress bar
      Write('Progress ',#185);
      for c:= 0 TO 19 DO
        write(#176);
      write(#204);
      percentDone := 0.0*100;
      completeBars := 0;
      while not LoadingDone DO
        begin
          //write the new percentage in
          gotoxy(32,1);
          Write(percentDone:4:0,'%');
          //gets the percent done from the main script by using a series of pointers
          //to point to a value outside this script so the procedure does not have to be recalled
          percentDone := (percentPtr^)*100;
          //calculate number of bars to fill
          completeBars := floor((percentPtr^)*20);
          gotoxy(11,1);
          //draw that number of bars
          for d := 0 to completeBars-1 DO
```

```
            begin
              write(#219);
            end;
      end;
  end;

  procedure StartLoading(loadingType : CHAR; var percentIn : real);
    begin
      startRunTime := Time;
      loadingDone := false;
      //set a pointer to the memory address to the memory location of the percent
      //memory location passed in at the start
      percentPtr := @percentIn;

      //choose a loading type and start a new thread so it runs in parallel
      if loadingType = 'L' THEN
        VPBeginThread(loadingText,1,nil)
      else if loadingType = 'P' THEN
        VPBeginThread(loadingBar,1,nil);
    End;

  procedure EndLoading;
    begin
      //kill th thread to stop it running
      KillThread(1);
      loadingDone := true;
      //show end messages
      WRITELN('Done!');
      endRunTime := Time;
      Delay(500);
      ClrScr;
      //state how long it took
      writeln('Time elapsed: ',TimeToStr(endRunTime - startRunTime));
      ShowCursor;
      end;

  procedure silentKillLoading;
    begin
      //stops the bar without displaying anything back to the user when done
```

```
      KillThread(1);
      loadingDone := true;
      ClrScr;
      ShowCursor;
    end;
  end.
```

```
unit LogInProcedures;
{$M+}
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        //record type for this file
        dataRecord = RECORD
                           PersonID: STRING[5];
                           Username: STRING[40];
                           Passwowrd: String[20];
                          end;

    type
        //declaration of class, attributes and methods
        logInFile = Class(Files)
              protected
                dataFile: File OF dataRecord;
                dataTextFile : TextFile;
                indexArray: ARRAY[0..190] OF LONGINT;
                recordArray: ARRAY[0..190] OF dataRecord;
              public
                data : dataRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(iPersonID: STRING; iUsername: STRING; iPasswowrd: String);
                procedure DisplayData;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
                procedure sort;
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray: ARRAY
OF LONGINT; switchLowerCase: BOOLEAN);
        end;

    IMPLEMENTATION
    //constructor and destructor
```

```
constructor logInFile.Create;
    var a : longint;
    begin
        inherited Create;
        ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\LogInDetails.dta');
        ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\logInOut.txt');
        RESET(dataFile);
        RESET(dataTextFile);
        for a := 0 TO 190 DO
          begin
            indexArray[a] := a;
          end;
    end;

destructor LogInFile.Destroy;
    begin
        close(dataFile);
        close(dataTextFile);
        inherited Destroy;
    end;

//update data attribute
procedure logInFile.SetData(iPersonID: STRING;
                    iUsername: STRING;
                    iPasswowrd: String);

    begin
        with data do
            begin
                PersonID := iPersonID;
                Username:=iUsername;
                Passwowrd:=iPasswowrd;
            end;
    end;
//display data attribute
procedure logInFile.DisplayData;
    begin
    with data do
        begin
```

```
                writeln(PersonID);
                writeln(Username);
                writeln(Passwowrd);
                writeln('-------------------------------------------------------------------------------');
            end;
        end;
    //write all data in file to text file
    procedure logInFile.extractToTextFile;
        begin
            reset(dataFile);
            rewrite(dataTextFile);
            while not eof(dataFile) DO
                begin
                    read(dataFile, data);
                    writeln(dataTextFile, data.PersonID);
                    writeln(dataTextFile, data.Username);
                    writeln(dataTextFile, data.Passwowrd);
                    writeln(dataTextFile, '');
                end;
        end;
    //write data attribute to file
    procedure logInFile.writeToFile;
        begin
            write(dataFile, data);
        end;

    //read data from file at current position
    procedure logInFile.readHere;
        begin
            if FIlePos(dataFile) < FileSize(dataFile) then
                begin
                    read(dataFile,data);
                end;
        end;
    //read data from file at specified position
    procedure logInFile.readFile(readFromPosition : LONGINT);
        begin
            if readFromPosition < FileSize(dataFile) then
                begin
```

```
                    SEEK(dataFile,readFromPosition);
                    read(dataFile,data);
                end;
        end;
    //sort the file
    procedure logInFile.sort;
        var a,b : LONGINT;
            maxStrLen : STRING;
        begin
          RESET(dataFile);
          upper:= 190;
          //extrat the data into an array
          for a := 0 TO 190 DO
            begin
              READ(dataFile,data);
              recordArray[a] := data;
              dataArray[a] := data.Username
            end;
        Files.sort(indexArray);
        RESET(dataFile);
        //write the data back to the file
        for a := 0 TO 190 DO
            begin
                WRITE(dataFile,recordArray[indexArray[a]]);
                indexArray[a] := a;
            end;

        end;
    //search the file
        procedure logInFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray :
ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            var a: LONGINT;
            begin
                RESET(dataFile);
                for a := 0 TO 190 DO
                    begin
                        read(dataFile, data);
                        fileArray[0,a] := data.PersonID;
                        fileArray[1,a] := data.Username;
```

```
                    fileArray[2,a] := data.Passwowrd;
                end;
            Files.search(190,3,searchValues, searchFields, returnArray, switchLowerCase);
        end;
END.
```

```
unit LogInScreen;
  interface
  uses crt, sysutils, vputils; {importing built in units to help with the layout}

  TYPE userPassArray = ARRAY[0..1] OF STRING; {Use one data structure to return from the unit}

  VAR Username, Password : STRING; {Declaring variables}
    y : LONGINT;
    userPass : userPassArray;

  procedure GetUsername; {declaring all the procedures in this unit and any local variables and parameters that
they take}
  procedure GetPassword;
    var i, a : LONGINT;
    key : CHAR;
  procedure startLogIn(var userPassArrayr : ARRAY OF STRING);


  implementation

  PROCEDURE GetUsername; {The procedure that reads what the user types in for username}
      begin
        GotoXY(11,1);
        READLN(username);
      end;

  PROCEDURE GetPassword;
    var i,a : LONGINT;
    key : CHAR;
    begin
      GoToXY(11,2); {Moves the cursor to the correct position}
      i := 0;
      HideCursor; {Hides the cursor}
      password := ''; {sets the password to an empty string}
      repeat {repeat until enter is pressed to submit}
        key := readkey; {read the key that is pressed}
          if (key = #8) THEN {Check if its the back space key}
            begin
              if i > 0 THEN {check if the password has more than 0 characters in it}
```

```
                begin
                  Delete(password,i,1); {if so remove the last charachter in password}
                  i := i -1; {decrease the length of the password by 1}
                end;
                {else do nothing}
            end

          else if ((key >= #33) and (key < #127) and (key <> #13)) OR (key = #156) THEN {check if it is a valid
character and not a spcae}
            begin
              password := password + key; {add the key pressed to the password}
              i := i +1; {increase the length of password by 1}
            end;
          GoToXY(1,2); {go to the start of the password line}
          DelLine; {clears the line}
          Write('Password: '); {write the whole line again with the correct number of cogs}
          for a := 1 TO i DO
            write(#15);
    until key = #13;
      writeln;
    end;


  PROCEDURE DisplayInfo; {initial display of information that needs to be entered}
    Begin
      ClrScr;
      WRITELN('Username: ');
      WRITELN('Password: ');
      GoToXY(11,1); {go to the start of username}
    end;

  procedure startLogIn(var userPassArrayr : ARRAY OF STRING);
    begin
      DisplayInfo; {Display intial information}
      GetUsername; {get the username}
      GetPassword; {get the password}
      ClrScr; {clear the screen for the next task}
      userPassArrayr[0] := username; {put the username and password into the array to be editied}
      userPassArrayr[1] := password;
```

```
    end;
  end.
```

```
PROGRAM Main;
  //declare all units to be used
  USES Crt, Sysutils, LogInScreen, loading, LogInProcedures,
  teacherProcedures, TeachersMenu, StudentsMenu, waitTime;
  var userType : STRING;
      intUserID : LONGINT;


  //declare main program
  function start:STRING;
    var i, ErrCode, attempts, userIndex : LONGINT;
      userId : STRING;
      userDetailsArray : ARRAY[0..1] OF STRING;
      logIn : logInFile;
      teacher : TeacherFile;
      percentDone : REAL;
      logedIn : BOOLEAN;
  begin
    //declare login file
    logIn := logInFile.create;
    logIn.sort;
    userIndex := -1;
    //delete in write up (debug)
    teacher := teacherFile.create;
    teacher.readHere;
    teacher.DisplayData;
    readln;

    attempts := 0;
    //while loop to keep running log in attempts
    logedIn := false;
    while logedIn = false do
      begin
        ClrScr;
        //login methods
        LogInScreen.startLogIn(userDetailsArray);
        logIn.search(userDetailsArray,[1,2], userIndex, false);
        loading.startLoading('L', percentDone);
        Delay(1000);
```

```
        loading.silentKillLoading;
        Delay(100);
        //test username and password are valid
        if userIndex = -1 THEN
          begin
            writeln('Username Or Password is incorrect.');
            readkey;
          end
        else
          begin
            logedIn := true;
            //get the user info
            login.readFile(userIndex);
            //test if the username and password belong to a teacher or student
            userId := Trim(login.data.PersonID);
            val(userid,intUserID,ErrCode);
            if ErrCode <> 0 THEN
              begin
                start := 'T';
              end
            else
              begin
                start := 'S';
              end;
          end;
        attempts := attempts + 1;
        case attempts of
          3 : waitTime.waitForTime('Too many attempts', 0);
          5 : waitTime.waitForTime('Too many attempts', 1);
          7 : waitTime.waitForTime('Too many attempts', 2);
          9 : waitTime.waitForTime('Too many attempts', 3);
          10 : waitTime.waitForTime('Too many attempts', 4);
          11 : waitTime.waitForTime('Too many attempts', 5);
          12 : start := '';
        end;
      end;
      //delete login file object
      logIn.free;
  end;
```

```
begin
  //start login
  userType := start;
  if userType = 'T' THEN
    //teacher menu
    TeachersMenu.startTeacher
  else if userType = 'S' THEN
    //student menu
    StudentsMenu.startStudent(intUserID)
  else
    writeln('Too many incorrect attempts :(');
end.
```

```
unit registerProcedures;
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        studentRegisterMarkRecord = RECORD
                          studentID : LONGINT;
                          registerMark : CHAR;
                        end;
        dataRecord = RECORD
                          RegisterID : STRING[17];
                          LessonNo : LONGINT;
                          Date : TDateTime;
                          ClassID : STRING[8];
                          RegisterMarks : ARRAY[0..40] OF studentRegisterMarkRecord;
                          isOfstedRegister : BOOLEAN;
                        end;

    type
        registerFile = Class(Files)
            private
                dataFile: File OF dataRecord;
                dataTextFile : TextFile;
                indexArray: ARRAY[0..1000] OF LONGINT;
                recordArray: ARRAY[0..1000] OF dataRecord;
                studentRegisterMark : studentRegisterMarkRecord;
            public
                data : dataRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(iRegisterID : STRING; iLessonNo : LONGINT; iDate : TDateTime; iClassID : STRING;
iRegisterMarks : ARRAY OF studentRegisterMarkRecord; iisOfstedRegister : BOOLEAN);
                procedure DisplayData;
                procedure displayFile;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
```

```
                procedure sort;
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray: ARRAY
OF LONGINT; switchLowerCase: BOOLEAN);

        end;

    IMPLEMENTATION

    constructor registerFile.Create;
        var a : longint;
        begin
            inherited Create;
            ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\Registers.dta');
            ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\regOut.txt');
            RESET(dataFile);
            RESET(dataTextFile);
            for a := 0 TO 1000 DO
              begin
                indexArray[a] := a;
              end;
        end;

    destructor registerFile.Destroy;
        begin
            close(dataFile);
            close(dataTextFile);
            inherited Destroy;
        end;

    procedure registerFile.SetData(iRegisterID : STRING; iLessonNo : LONGINT; iDate : TDateTime; iClassID : STRING;
iRegisterMarks : ARRAY OF studentRegisterMarkRecord; iisOfstedRegister : BOOLEAN);
        begin
            with data do
                begin
                    RegisterID := iRegisterID;
                    LessonNo := iLessonNo;
                    Date := iDate;
                    ClassID := iClassID;
                    //RegisterMarks := iRegisterMarks;
```

```
                isOfstedRegister := iisOfstedRegister;
            end;
    end;



procedure registerFile.DisplayData;
  var i : LONGINT;
    begin
        READ(dataFile,data);
        with data do
            begin
                writeln(RegisterID);
                writeln(LessonNo);
                writeln(Date);
                writeln(ClassID);
                writeln('{');
                for i := 0 TO 40 DO
                    begin
                        studentRegisterMark := RegisterMarks[i];
                        writeln(studentRegisterMark.studentID);
                        writeln(studentRegisterMark.RegisterMark);
                    end;
                writeln('}');
                writeln(isOfstedRegister);
                writeln('--------------------------------------------------------------------------------');
            end;
    end;

procedure registerFile.displayFile;
    var a, i : LONGINT;
    begin
        reset(dataFile);
        rewrite(dataTextFile);
        while not (eof(dataFile)) do
            begin
                 READ(dataFile,data);
                with data do
                    begin
```

```
                writeln(RegisterID);
                writeln(LessonNo);
                writeln(Date);
                writeln(ClassID);
                writeln('{');
                for i := 0 TO 40 DO
                    begin
                        studentRegisterMark := data.RegisterMarks[i];
                        writeln(studentRegisterMark.studentID);
                        writeln(studentRegisterMark.RegisterMark);
                    end;
                writeln('}');
                writeln(isOfstedRegister);
                writeln('--------------------------------------------------------------------------');
            end;

        end;
    end;

procedure registerFile.extractToTextFile;
    var i : LONGINT;
    begin
        reset(dataFile);
        rewrite(dataTextFile);
        while not eof(dataFile) do
            begin
                read(dataFile, data);
                writeln(dataTextFile, data.RegisterID);
                    writeln(dataTextFile, IntToStr(data.LessonNo));
                    writeln(dataTextFile, DateTimeToStr(data.Date));
                    writeln(dataTextFile, data.classID);
                    writeln(dataTextFile, '{');
                    for i := 0 TO 40 DO
                        begin
                            studentRegisterMark := data.RegisterMarks[i];
                            write(dataTextFile, '[');
                            write(dataTextFile, studentRegisterMark.studentID);
                            write(dataTextFile, ',');
                            write(dataTextFile, studentRegisterMark.RegisterMark);
```

```
                            writeln(dataTextFile, ']');
                        end;
                    writeln(dataTextFile, '}');
                    writeln(dataTextFile, data.isOfstedRegister);
            end;
    end;

procedure registerFile.writeToFile;
    begin
        write(dataFile, data);
    end;


procedure registerFile.readHere;
    begin
        if FIlePos(dataFile) < FileSize(dataFile) then
            begin
                read(dataFile,data);
            end;
    end;

procedure registerFile.readFile(readFromPosition : LONGINT);
    begin
        if readFromPosition < FileSize(dataFile) then
            begin
                SEEK(dataFile,readFromPosition);
                read(dataFile,data);
            end;
    end;

procedure registerFile.sort;
    var a,b : LONGINT;
        maxStrLen : STRING;
    begin
      RESET(dataFile);
      upper:= 1000;
      for a := 0 TO 1000 DO
        begin
          READ(dataFile,data);
```

```
              recordArray[a] := data;
              dataArray[a] := Format('%'+'20'+'s',[(data.RegisterID)]);
            end;
        Files.sort(indexArray);
        RESET(dataFile);
        for a := 0 TO 1000 DO
            begin
                WRITE(dataFile,recordArray[indexArray[a]]);
                indexArray[a] := a;
            end;

        end;

        procedure registerFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray :
ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            var a: LONGINT;
            begin
                RESET(dataFile);
                for a := 0 TO 1000 DO
                    begin
                        read(dataFile, data);
                        fileArray[0,a] := data.RegisterID;
                        Str(data.LessonNo, fileArray[1,a]);
                        fileArray[2,a] := DateTimeToStr(data.Date);
                        fileArray[3,a] := data.classID;
                        returnArray[a] := -1;
                    end;
                Files.search(1000,4,searchValues, searchFields, returnArray, switchLowerCase);
            end;

    END.
```

```
UNIT Search;


  INTERFACE
  USES CRT, SYSUTILS, BinarySearch, LinearSearch;

  PROCEDURE SearchFunctions(SearchType : CHAR; min, max : LONGINT; findValue : STRING; UserArray : ARRAY OF STRING;
var returnArray : ARRAY OF LONGINT; var numberDone : LONGINT);

  IMPLEMENTATION

  PROCEDURE SearchFunctions(SearchType : CHAR; min, max : LONGINT; findValue : STRING; UserArray : ARRAY OF STRING;
var returnArray : ARRAY OF LONGINT; var numberDone : LONGINT);
    BEGIN
      IF SearchType = 'L' THEN
        LinearSearch.LinearSearchFunction(min,max,FindValue,UserArray,returnArray,numberDone);
      IF SearchType = 'B' THEN
        BinarySearch.BinarySearchFunction(min,max, FindValue,UserArray,returnArray,numberDone);
    END;

  END.
```

```
unit Sorting;
  interface
  USES CRT, SYSUTILS, BubbleSort,InsertionSort;

  procedure sortFunction(sortType : CHAR; var UserArray : ARRAY OF STRING; lower, upper : LONGINT);

  IMPLEMENTATION

  PROCEDURE sortFunction(sortType : CHAR; var UserArray : ARRAY OF STRING; lower, upper : LONGINT);
    begin
      if sortType = 'I' THEN
        InsertionSort.InsertionSortFunction(UserArray, lower, upper)
      else if sortType = 'B' THEN
        BubbleSort.BubbleSortFunction(UserArray, lower, upper)
      else
        writeln('You are an idot. Put I or B.');
    end;

  END.
```

```
unit studentClassProcedures;
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        dataRecord = RECORD
                          StudentID : LONGINT;
                          ClassID : STRING[8];
                        end;

    type
        studentClassFile = Class(Files)
            private
                dataTextFile : TextFile;
                dataFile: File OF dataRecord;
                indexArray: ARRAY[0..1384] OF LONGINT;
                recordArray: ARRAY[0..1384] OF dataRecord;
            public
                data : dataRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(iStudentID : LONGINT; iClassID : STRING);
                procedure DisplayData;
                procedure displayFile;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
                procedure sort;
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray: ARRAY
OF LONGINT; switchLowerCase: BOOLEAN);

        end;

    IMPLEMENTATION

    constructor studentClassFile.Create;
        var a : longint;
```

```
    begin
        inherited Create;
        ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\studentClassResolver.dta');
        ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\outStudentClass.txt');
        RESET(dataFile);
        RESET(dataTextFile);
        for a := 0 TO 1384 DO
          begin
            indexArray[a] := a;
          end;
    end;

destructor studentClassFile.Destroy;
    begin
        close(dataFile);
        close(dataTextFile);
        inherited Destroy;
    end;

procedure studentClassFile.SetData(iStudentID : LONGINT; iClassID : STRING);
    begin
        with data do
            begin
                ClassID := iClassID;
                StudentID := iStudentID;
            end;
    end;



procedure studentClassFile.DisplayData;
    begin
            writeln(data.ClassID);
            writeln(data.StudentID);
            writeln('-------------------------------------------------------------------------');
    end;

procedure studentClassFile.displayFile;
    var a : LONGINT;
```

```
        begin
            reset(dataFile);
            rewrite(dataTextFile);
            while not (eof(dataFile)) do
                begin
                    READ(dataFile,data);
                    with data do
                        begin
                            writeln(ClassID);
                            writeln(StudentID);
                            writeln('--------------------------------------------------------------------------------');
                        end;
                    writeln(dataTextFile, data.ClassID);
                    writeln(dataTextFile, IntToStr(data.StudentID));
                end;
            close(dataTextFile);
        end;

    procedure studentClassFile.extractToTextFile;
        begin
            reset(dataFile);
            rewrite(dataTextFile);
            while not eof(dataFile) do
                begin
                    read(dataFile, data);
                    writeln(dataTextFile, data.ClassID);
                    writeln(dataTextFile, data.StudentID);
                    writeln(dataTextFile, '');
                end;
        end;

    procedure studentClassFile.writeToFile;
        begin
            write(dataFile, data);
        end;


    procedure studentClassFile.readHere;
        begin
```

```
            if FIlePos(dataFile) < FileSize(dataFile) then
                begin
                    read(dataFile,data);
                end;
        end;

    procedure studentClassFile.readFile(readFromPosition : LONGINT);
        begin
            if readFromPosition < FileSize(dataFile) then
                begin
                    SEEK(dataFile,readFromPosition);
                    read(dataFile,data);
                end;
        end;

    procedure studentClassFile.sort;
        var a,b : LONGINT;
            maxStrLen : STRING;
        begin
          RESET(dataFile);
          upper:= 1384;
          for a := 0 TO 1384 DO
            begin
              READ(dataFile,data);
              recordArray[a] := data;
              dataArray[a] := Format('%'+'8'+'s',[(data.classID)]);
            end;
        Files.sort(indexArray);
        RESET(dataFile);
        for a := 0 TO 1384 DO
            begin
                WRITE(dataFile,recordArray[indexArray[a]]);
                indexArray[a] := a;
            end;

        end;

        procedure studentClassFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var
returnArray : ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
```

```
            var a: LONGINT;
            begin
                RESET(dataFile);
                a := 0;
                while not EOF(dataFile) DO
                    begin
                        read(dataFile, data);
                        fileArray[0,a] := data.classID;
                        Str(data.StudentID, fileArray[1,a]);
                        returnArray[a] := -1;
                        a := a + 1;
                    end;
                Files.search(a+1,2,searchValues, searchFields, returnArray, switchLowerCase);
            end;
    END.
```

```
unit studentProcedures;
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        StudentsRecord = RECORD
                           StudentID : LONGINT;
                           FName : STRING[20];
                           SName : String[20];
                           Year : LONGINT;
                           Form : STRING[5];
                           Uname : String[40];
                           Pword : String[20];
                         end;

    type
        StudentFile = Class(Files)
            private
                dataFile: File OF StudentsRecord;
                dataTextFile : TextFile;
                indexArray: ARRAY[0..139] OF LONGINT;
                recordArray: ARRAY[0..139] OF StudentsRecord;
            public
                data : StudentsRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(StudentId : LONGINT; FirstName: STRING; SurName : STRING; SYear : LONGINT; SForm
: STRING; SUname : String; SPword : String);
                procedure DisplayData;
                procedure displayFile;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
                procedure sort;
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray: ARRAY
OF LONGINT; switchLowerCase: BOOLEAN);
```

```
    end;

IMPLEMENTATION

constructor StudentFile.Create;
    var a : longint;
    begin
        inherited Create;
        ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\Students.dta');
        ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\outStudents.txt');
        RESET(dataFile);
        RESET(dataTextFile);
        for a := 0 TO 139 DO
          begin
            indexArray[a] := a;
          end;
    end;

destructor StudentFile.Destroy;
    begin
        close(dataFile);
        close(dataTextFile);
        inherited Destroy;
    end;

procedure StudentFile.SetData(StudentId : LONGINT;
                      FirstName : STRING;
                      SurName : STRING;
                      SYear : LONGINT;
                      SForm : STRING;
                      SUname : String;
                      SPword : String);

    begin
        with data do
            begin
                StudentID := StudentId;
                FName := FirstName;
                SName := SurName;
```

```
                Year := SYear;
                Form := SForm;
                Uname := SUname;
                Pword := SPword;
            end;
    end;

procedure StudentFile.DisplayData;
    begin
    with data do
        begin
            writeln(StudentID);
            writeln(FName);
            writeln(SName);
            writeln(Year);
            writeln(Form);
            writeln(Uname);
            writeln(PWord);
            writeln('--------------------------------------------------------------------------------');
        end;
    end;

procedure StudentFile.displayFile;
    var a : LONGINT;
    begin
        reset(dataFile);
        while not (eof(dataFile)) do
            begin
                READ(dataFile,data);
                with data do
                    begin
                        writeln(StudentID);
                        writeln(FName);
                        writeln(SName);
                        writeln(Year);
                        writeln(Form);
                        writeln(Uname);
                        writeln(PWord);
                        writeln('--------------------------------------------------------------------------------');
```

```
                    end;
                end;
        end;

    procedure StudentFile.extractToTextFile;
        begin
            reset(dataFile);
            rewrite(dataTextFile);
            while not eof(dataFile) do
                begin
                    read(dataFile, data);
                    writeln(dataTextFile, data.StudentID);
                    writeln(dataTextFile, data.FName);
                    writeln(dataTextFile, data.SName);
                    writeln(dataTextFile, data.Year);
                    writeln(dataTextFile, data.Form);
                    writeln(dataTextFile, data.Uname);
                    writeln(dataTextFile, data.PWord);
                    writeln(dataTextFile, '');
                end;
        end;

    procedure StudentFile.writeToFile;
        begin
            write(dataFile, data);
        end;


    procedure StudentFile.readHere;
        begin
            if FIlePos(dataFile) < FileSize(dataFile) then
                begin
                    read(dataFile,data);
                end;
        end;

    procedure StudentFile.readFile(readFromPosition : LONGINT);
        begin
            if readFromPosition < FileSize(dataFile) then
```

```
                begin
                    SEEK(dataFile,readFromPosition);
                    read(dataFile,data);
                end;
        end;

    procedure StudentFile.sort;
        var a,b : LONGINT;
            maxStrLen : STRING;
        begin
          RESET(dataFile);
          upper:= 139;
          for a := 0 TO 139 DO
            begin
              READ(dataFile,data);
              recordArray[a] := data;
              maxStrLen :='3';
              dataArray[a] := Format('%'+maxStrLen+'s',[IntToStr(data.StudentID)]);
            end;
        Files.sort(indexArray);
        RESET(dataFile);
        for a := 0 TO 139 DO
            begin
                WRITE(dataFile,recordArray[indexArray[a]]);
                indexArray[a] := a;
            end;

        end;

        procedure StudentFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray :
ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            var a: LONGINT;
            begin
                RESET(dataFile);
                for a := 0 TO 139 DO
                    begin
                        read(dataFile, data);
                        Str(data.StudentID, fileArray[0,a]);
                        fileArray[1,a] := data.FName;
```

```
                    fileArray[2,a] := data.SName;
                    Str(data.Year, fileArray[3,a]);
                    fileArray[4,a] := data.Form;
                    fileArray[5,a] := data.Uname;
                    fileArray[6,a] := data.PWord;
                    returnArray[a] := -1;
                end;
            Files.search(139,7,searchValues, searchFields, returnArray, switchLowerCase);
        end;


END.
```

```
UNIT StudentsMenu;
  interface
  uses crt, sysutils, VPUtils, studentProcedures, timeTableProcedures,
  attendancProcedures, studentClassProcedures,createMenu, writeDesign;

  //declaration of all procedures and functions
  procedure startStudent(inputStudentId : LONGINT);
  procedure CompleteTimeTable(inputDate : TDateTime);
  procedure TimeTableMenu;
  procedure AttendanceMenu;
  procedure LogOutMenu;
  procedure mainMenu;


  implementation
  //declare variables local to this unit
  var studentID : LONGINT;
      student : StudentFIle;
      attendance : AttendanceFile;
      timetable : TimeTableFile;
      studentClassC : studentClassFile;
      studentClassReturnArray : ARRAY[0..1400] OF LONGINT;
      timetableReturnArray: ARRAY[0..321] OF LONGINT;
      mainMenuOptions : ARRAY[0..2] OF STRING = ('Attendance','Timetable', 'Log Out');
      animationTimeTable : BOOLEAN;

  //Menu for viewing attendance
  procedure AttendanceMenu;
    begin
      Writeln('Attendance');
      readln;
    end;

  //fill timetable with the data
  procedure CompleteTimeTable(inputDate : TDateTime);
    var a, b, c, day, NumberOfDays, timeDelay : LONGINT;
        CurrentDate : TDateTime;
        week : STRING;
        stringStudentId : STRING;
```

```
    //dateArray : ARRAY OF STRING;
  begin
    //draw basic timetable outline
    if animationTimeTable = true THEN
      begin
        timeDelay := 100;
      end
    else
      begin
        timeDelay := 0;
      end;
    writeDesign.DrawDesign('C:\ComputerScienceCoursework\Program\files\TimeTableLayout.txt', timeDelay);
    CurrentDate := inputDate;
    day := DayOfWeek(CurrentDate-1)-1;
    HideCursor;
    //write the dates with current date highlighted
    for a := 0 to 4 DO
      begin
        GOTOXY(5,7+a*4);
        if CurrentDate-day+a = Date THEN
          TextColor(Yellow)
        else
          TextColor(White);
        write(DateToStr(CurrentDate-day+a));
        Delay(timeDelay);
      end;
    TextColor(White);
    //calcualte if it is week A or week B
    NumberOfDays := 0;
    while StrToDateTime('13/09/2024') + NumberOfDays <> CurrentDate-day+a-1 DO
      NumberOfDays := NumberOfDays +1;
    NumberOfDays := NumberOfDays DIV 7;
    GOTOXY(7,2);
    if (NumberOfDays mod 2 = 0 )THEN
      week := 'A'
    else
      week := 'B';
    WRITE(week);
    //write lessons in timetable
```

```
stringStudentId := IntToStr(studentID);
//get all classes which the student is in
studentClassC.search([stringStudentId],[1],studentClassReturnArray, true);
b := 0;
//for every class
while studentClassReturnArray[b] <> -1 DO
  begin
    //get the class id
    studentClassC.readFile(studentClassReturnArray[b]);
    for a := 0 to 320 DO
      timetableReturnArray[a] := -1;
    //find every time and day that class is on
    timetable.search([studentClassC.data.ClassID],[2], timetableReturnArray, true);
    c := 0;
    while timetableReturnArray[c] <> -1 DO
      begin
        //get the day and time for that one time that one class is
        timetable.readFile(timetableReturnArray[c]);
        //test for the current week
        if week = 'A' THEN
          begin
            //test if the class is on the correct week
            if timetable.data.day < 6 THEN
              begin
                //got to position and write class id
                GOTOXY((10*timetable.data.LessonNo+20),( 4*(timetable.data.Day-1)+7));
                WRITE(Trim(TimeTable.data.ClassID));
                Delay(timeDelay);
              end;
          end
        else
          begin
            //test if the class is on the correct week
            if timetable.data.day > 5 THEN
              begin
                //got to position and write class id
                GOTOXY((10*timetable.data.LessonNo+20),( 4*(timetable.data.Day-6)+7));
                WRITE(Trim(TimeTable.data.ClassID));
                Delay(timeDelay);
```

```
                    end;
                end;
              c := c + 1;
            end;
          b := b + 1;
        end;
      //write the form data in
      for a := 0 TO 4 DO
        begin
          GOTOXY(21, 4*a+7);
          write(student.data.Form);
          Delay(timeDelay);
        end;
        animationTimeTable := false;
    end;

  //Draw and populate timetable
  procedure TimeTableMenu;
    var StartDate : TDateTime;
    begin
      StartDate := Date;
      CompleteTimeTable(StartDate);
      readln;
    end;

  procedure LogOutMenu;
    begin
      writeln('Logging out');
      Timetable.free;
      studentClassC.free;
      student.free;
      attendance.free;
      Delay(1000);
    end;

  //main menu
  procedure mainMenu;
    var choice : LONGINT;
    begin
```

```
      GOTOXY(0,0);
      choice := 0;
      //get student and attendance data at the start
      student.readFile(studentID);
      attendance.readFile(studentID);
      //main menu display until logout selected
      repeat
        ClrScr;
        HideCursor;
        //messages welcoming and displaying overall attendance
        writeln('welcome ', student.data.fname, ' ', student.data.sname, ' ');
        GOTOXY(1,3);
        writeln('Attendance: ', Format('%.2f', [attendance.data.attendance*100]),'%');

        //create the menu and start to control it
        choice := createMenu.writeMenu([1,5],3,mainMenuOptions, choice);
        //test for each menu choice
        case choice of
          0 : AttendanceMenu;
          1 : TimeTableMenu;
          2 : LogOutMenu;
        end;
      until choice = 2;

    end;


  //declaration of global variables
  procedure startStudent(inputStudentId : LONGINT);
    var a : LONGINT;
    begin
      animationTimeTable := true;
      //populate arrays with -1
      for a := 0 TO 1400 DO
        studentClassReturnArray[a] := -1;
      //declare file objects for this unit
      timetable := TimeTableFile.create;
      studentClassC := studentClassFile.create;
      studentID := inputStudentId;
```

```
      attendance := AttendanceFile.create;
      student := StudentFile.create;
      mainMenu;
    end;

  end.
```

```
unit teacherProcedures;
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        TeacherRecord = RECORD
                          TeacherID : STRING[3];
                          FName : string[20];
                          SName : string[20];
                          Form  : string[5];
                          Uname : STRING[40];
                          PWord : STRING[20];
                        end;

    type
        TeacherFile = Class(Files)
            private
                dataFile: File OF TeacherRecord;
                dataTextFile : TextFile;
                indexArray: ARRAY[0..50] OF LONGINT;
                recordArray: ARRAY[0..50] OF TeacherRecord;
            public
                data : TeacherRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(TeacherID : LONGINT; FirstName: STRING; SurName : STRING; SForm : STRING; SUname
: String; SPword : String);
                procedure DisplayData;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
                procedure sort;
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray: ARRAY
OF LONGINT; switchLowerCase: BOOLEAN);

        end;
```

```
IMPLEMENTATION

constructor TeacherFile.Create;
    var a : longint;
    begin
        inherited Create;
        ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\Teachers.dta');
        ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\outTeachers.txt');
        RESET(dataFile);
        RESET(dataTextFile);
        for a := 0 TO 139 DO
          begin
            indexArray[a] := a;
          end;
    end;

destructor TeacherFile.Destroy;
    begin
        close(dataFile);
        close(dataTextFile);
        inherited Destroy;
    end;

procedure TeacherFile.SetData(TeacherID : LONGINT;
                      FirstName : STRING;
                      SurName : STRING;
                      SForm : STRING;
                      SUname : String;
                      SPword : String);

    begin
        with data do
            begin
                TeacherID := TeacherID;
                FName := FirstName;
                SName := SurName;
                Form := SForm;
                Uname := SUname;
                Pword := SPword;
```

```
            end;
        end;

    procedure TeacherFile.DisplayData;
        begin
        with data do
            begin
                writeln(TeacherID);
                writeln(FName);
                writeln(SName);
                writeln(Form);
                writeln(Uname);
                writeln(PWord);
                writeln('--------------------------------------------------------------------------------');
            end;
        end;

    procedure TeacherFile.extractToTextFile;
        begin
            reset(dataFile);
            rewrite(dataTextFile);
            while not eof(dataFile) do
                begin
                    read(dataFile, data);
                    writeln(dataTextFile, data.TeacherID);
                    writeln(dataTextFile, data.FName);
                    writeln(dataTextFile, data.SName);
                    writeln(dataTextFile, data.Form);
                    writeln(dataTextFile, data.Uname);
                    writeln(dataTextFile, data.PWord);
                    writeln(dataTextFile, '');
                end;
        end;

    procedure TeacherFile.writeToFile;
        begin
            write(dataFile, data);
        end;
```

```
procedure TeacherFile.readHere;
    begin
        if FilePos(dataFile) < FileSize(dataFile) then
            begin
                read(dataFile,data);
            end;
    end;

procedure TeacherFile.readFile(readFromPosition : LONGINT);
    begin
        if readFromPosition < FileSize(dataFile) then
            begin
                SEEK(dataFile,readFromPosition);
                read(dataFile,data);
            end;
    end;

procedure TeacherFile.sort;
    var a,b : LONGINT;
        maxStrLen : STRING;
    begin
      RESET(dataFile);
      upper:= 50;
      for a := 0 TO 50 DO
        begin
          READ(dataFile,data);
          dataArray[a] := data.teacherID;
          recordArray[a] := data;
        end;
    Files.sort(indexArray);
    RESET(dataFile);
    for a := 0 TO 50 DO
        begin
            WRITE(dataFile,recordArray[indexArray[a]]);
            indexArray[a] := a;
        end;

    end;
```

```
        procedure TeacherFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray :
ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            var a: LONGINT;
            begin
                RESET(dataFile);
                for a := 0 TO 50 DO
                    begin
                        read(dataFile, data);
                        fileArray[0,a] := data.TeacherID;
                        fileArray[1,a] := data.FName;
                        fileArray[2,a] := data.SName;
                        fileArray[4,a] := data.Form;
                        fileArray[5,a] := data.Uname;
                        fileArray[6,a] := data.PWord;
                        returnArray[a] := -1;
                    end;
                Files.search(50,7,searchValues, searchFields, returnArray, switchLowerCase);
            end;


    END.
```

```
UNIT TeachersMenu;

  interface
  uses crt, sysutils;
  procedure startTeacher;
  implementation
  //main menu
  procedure startTeacher;
    begin
      //debug to prove it works
      writeln('welcome teacher');
      readln;
    end;

  end.
```

```
unit timeTableProcedures;
  interface

    USES crt, sysutils, FileProcedures;

    TYPE
        TimeTableRecord = RECORD
                            Day : LONGINT;
                            LessonNo : LONGINT;
                            ClassID : STRING[8];
                          end;

    type
        TimeTableFile = Class(Files)
            private
                dataFile: File OF TimeTableRecord;
                dataTextFile : TextFile;
                indexArray: ARRAY[0..320] OF LONGINT;
                recordArray: ARRAY[0..320] OF TimeTableRecord;
            public
                data : TimeTableRecord;
                constructor Create;
                destructor Destroy; override;
                procedure SetData(iDay : LONGINT;
                          iLessonNo : LONGINT;
                          iClassID : STRING);
                procedure DisplayData;
                procedure displayFile;
                procedure extractToTextFile;
                procedure writeToFile;
                procedure readHere;
                procedure readFile(readFromPosition : LONGINT);
                procedure sort;
                procedure search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray: ARRAY
OF LONGINT; switchLowerCase: BOOLEAN);

        end;

    IMPLEMENTATION
```

```
constructor TimeTableFile.Create;
    var a : longint;
    begin
        inherited Create;
        ASSIGN(dataFile,'C:\ComputerScienceCoursework\Program\files\TimeTable.dta');
        ASSIGN(dataTextFile,'C:\ComputerScienceCoursework\Program\fileOuts\outTimeTable.txt');
        RESET(dataFile);
        RESET(dataTextFile);
        for a := 0 TO 319 DO
          begin
            indexArray[a] := a;
          end;
    end;

destructor TimeTableFile.Destroy;
    begin
        close(dataFile);
        close(dataTextFile);
        inherited Destroy;
    end;

procedure TimeTableFile.SetData(iDay : LONGINT;
                    iLessonNo : LONGINT;
                    iClassID : STRING);

    begin
        with data do
            begin
                Day := iDay;
                LessonNo := iLessonNo;
                ClassID := iClassID;
            end;
    end;

procedure TimeTableFile.DisplayData;
    begin
    with data do
        begin
```

```
                writeln(day);
                writeln(lessonNo);
                writeln(ClassID);
                writeln('-------------------------------------------------------------------------------');
            end;
        end;

    procedure TimeTableFile.extractToTextFile;
        begin
            reset(dataFile);
            rewrite(dataTextFile);
            while not eof(dataFile) do
                begin
                    read(dataFile, data);
                    writeln(dataTextFile, data.day);
                    writeln(dataTextFile, data.lessonNo);
                    writeln(dataTextFile, data.ClassID);
                    writeln(dataTextFile, '');
                end;
        end;

    procedure TimeTableFile.displayFile;
        var a : LONGINT;
        begin
            reset(dataFile);
            while not (eof(dataFile)) do
                begin
                    READ(dataFile,data);
                    with data do
                        begin
                            writeln(day);
                            writeln(lessonNo);
                            writeln(ClassID);
                            writeln('-------------------------------------------------------------------------------');
                        end;
                end;
        end;

    procedure TimeTableFile.writeToFile;
```

```
        begin
            write(dataFile, data);
        end;


    procedure TimeTableFile.readHere;
        begin
            if FIlePos(dataFile) < FileSize(dataFile) then
                begin
                    read(dataFile,data);
                end;
        end;

    procedure TimeTableFile.readFile(readFromPosition : LONGINT);
        begin
            if readFromPosition < FileSize(dataFile) then
                begin
                    SEEK(dataFile,readFromPosition);
                    read(dataFile,data);
                end;
        end;

    procedure TimeTableFile.sort;
        var a,b : LONGINT;
            maxStrLen : STRING;
        begin
          RESET(dataFile);
          upper:= 319;
          for a := 0 TO 319 DO
            begin
              READ(dataFile,data);
              recordArray[a] := data;
              dataArray[a] := IntToStr(data.day);
            end;
        Files.sort(indexArray);
        RESET(dataFile);
        for a := 0 TO 319 DO
            begin
                WRITE(dataFile,recordArray[indexArray[a]]);
```

```
                indexArray[a] := a;
            end;

        end;

        procedure TimeTableFile.search(searchValues:ARRAY OF STRING; searchFields: ARRAY OF BYTE; var returnArray :
ARRAY OF LONGINT; switchLowerCase : BOOLEAN);
            var a: LONGINT;
            begin
                RESET(dataFile);
                for a := 0 TO 319 DO
                    begin
                        read(dataFile, data);
                        if a > 310 THEN
                          write;
                        Str(data.day, fileArray[0,a]);
                        Str(data.LessonNo, fileArray[1,a]);
                        fileArray[2,a] := data.ClassID;
                        returnArray[a] := -1;
                    end;
                Files.search(319,3,searchValues, searchFields, returnArray, switchLowerCase);
            end;


    END.
```

```
UNIT waitTime;

  interface
  uses crt, sysutils, vputils, math;

  procedure waitForTime(message : STRING; timeIndex : LONGINT);
    var a : LONGINT;
  implementation

  var waitArray : ARRAY[0..5] OF LONGINT = (30, 60, 300, 600, 1800, 3600);

  procedure waitForTime(message : String; timeIndex : LONGINT);
    var a : LONGINT;
    begin
      HideCursor;
      ClrScr;
      Writeln(message, ' please wait');
      for a := 0 TO waitArray[timeIndex] do
        begin

          if Ceil((waitArray[timeIndex]-a)/60) >= 10 then
            begin
              GOTOXY(1,2);
              write(Ceil((waitArray[timeIndex]-a)/60), ' minutes');
            end
          else if Ceil((waitArray[timeIndex]-a)/60) > 1 then
            begin
              GOTOXY(1,2);
              write('0');
              write(Ceil((waitArray[timeIndex]-a)/60), ' minutes');
            end
          else if waitArray[timeIndex]-a >= 10 then
            begin
              GOTOXY(1,2);
              DelLine;
              write(waitArray[timeIndex]-a, ' seconds');
            end
          else
            begin
```

```
              GOTOXY(1,2);
              DelLine;
              write('0');
              write(waitArray[timeIndex]-a, ' seconds');
            end;
          Delay(1000);
        end;
    end;

  end.
```

```
UNIT writeDesign;
  interface
  uses crt, sysutils, vputils;
  procedure DrawDesign(path: STRING; delayWrite : LONGINT);
  var Character : STRING;
      DesignTextFile : TextFile;
  implementation
  procedure DrawDesign(path: STRING; delayWrite : LONGINT);
    var a : LONGINT;
    begin
      HideCursor;
      ClrScr;
      //get the file to read from
      ASSIGN(DesignTextFile,path);
      reset(DesignTextFile);
      //read each line to a max of 28 (window size)
      for a := 0 to 28 DO
        begin
          readln(DesignTextFile, Character);
          //write each line
          writeln(Character);
          //delay for effects
          Delay(delayWrite);
        end;
      ShowCursor;

    end;
end.
```

Extra Projects

https://github.com/GreenFrog-Hub?tab=repositories

Name: Robert Starck Candidate Number: 9260 Centre Number: 52304

Project 1 – Neural Network training using backpropagation

Goal:

To design and code a program which can take a set of points and plot a line of degree one less than the number of points. So, for 4 points a cubic of form $ax^3 + bx^2 + cx + d = 0$ will be plotted to the points. The goal is to get this line as close as possible to all the points.

Design:

The only method to achieve this result and calculate suitable coefficients is to guess, many times, and calculate how wrong you are. The method of improving this is by making an initial guess and calculating how wrong you are, then based on how wrong you are, then calculating how to tweak each of the coefficients to get closer to the desired function.

This can be accomplished using a neural network and then backpropagating to find out how to tweak each coefficient to get closer to the solution.

The neural networks will take all the coefficients and data point coordinates as inputs. This network will then output the total loss, the sum of the squared difference from the curve to the data points.

Code:

```
import numpy as np
from tqdm import tqdm
from time import sleep
import random

class Network:
    def __init__(self, data, passes, learnRate):
        self.data = data
        self.dataX = np.zeros(len(self.data))
        self.dataY = np.zeros(len(self.data))
        self.coef = np.random.randint(-300,300,len(data))
        #self.coef = np.random.randn(2)
        #self.coef[0] = random.randint(-600,600)

        self.passes = passes
        for i in range(len(self.data)):
            self.dataX.put(i, self.data[i][0])
            self.dataY.put(i, self.data[i][1])
        self.passes = passes
        self.learnRate = learnRate

        self.graph = []




    def train(self):
        self.losses = []
        self.deltaLoss = 5
        self.prevLoss = 0
        for i in tqdm(range(self.passes)):
            self.grad = np.zeros(len(self.data))
            self.delatY = np.zeros(len(self.data))
            self.dataYHat = np.zeros(len(self.data))
            self.forwardPass()
            self.backPass()
            self.tweak()
            self.deltaLoss = self.totalLoss - self.prevLoss
            self.prevLoss = self.totalLoss
        # self.MinLoss = self.totalLoss
        # if self.deltaLoss > 10000:
        #     self.exitCondition = 1
        # else:
        #     self.exitCondition = 0

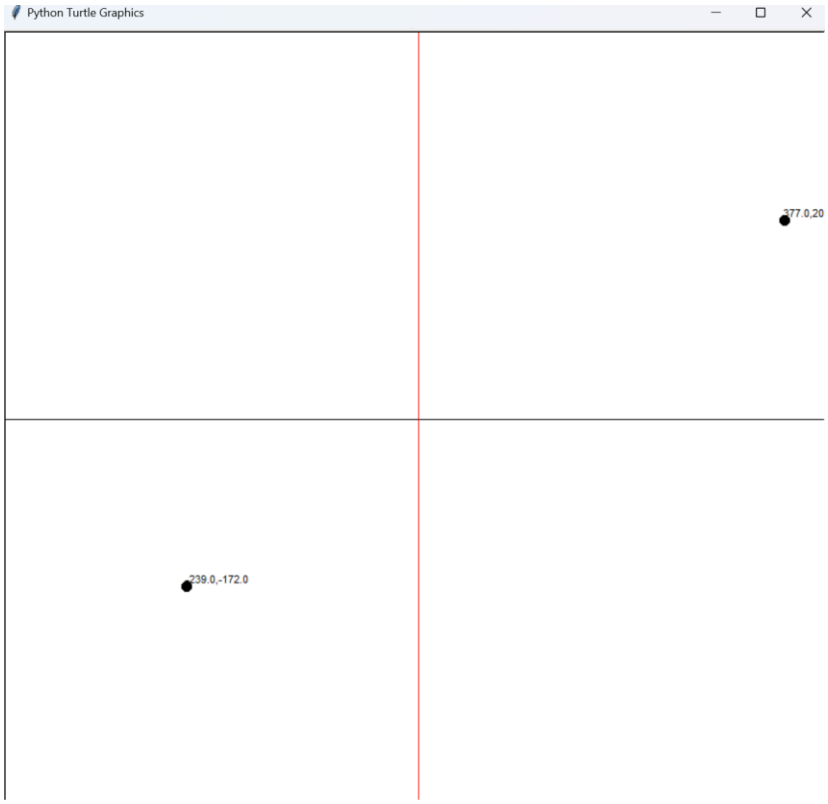
    def forwardPass(self):
        for i in range(len(self.dataX)):
            self.yHat = np.dot(self.coef, self.dataX[i])
            self.dataYHat.put(i, self.yHat)
        self.delatY = self.dataYHat- self.dataY
        self.totalLoss = np.sum(self.delatY**2)



    def backPass(self):
        self.grad = 2*self.delatY * self.dataX

    def tweak(self):
        mag = np.linalg.norm(self.grad)
        self.coef = self.coef - self.learnRate *self.grad
```

Testing:





This with 1000000 passes estimated these coefficients which relates to y = -4.267E+03 X + 7.1966E-1. Which results in the red line which is visibly wrong. This is due to the backpropagation algorithm being wrong.

I then spent 5 days redoing it 5 times and ended up with this algorithm

Code:

```
import numpy as np
from tqdm import tqdm
from time import sleep
import random
import turtle

class Network:
    def __init__(self, data, passes, learnRate):
        self.data = data
        self.dataXs = []
        self.dataX = np.zeros(len(self.data))
        self.dataY = np.zeros(len(self.data))
        self.coef = np.random.randn(len(self.data))

        self.passes = passes
        for i in range(len(self.data)):
            self.dataX = np.zeros(len(self.data))
            self.dataX.put(0, 1)
            for j in range(1, len(self.data)):
                self.dataX.put(j, self.data[i][0]**j)
            self.dataXs.append(self.dataX)
            self.dataY.put(i, self.data[i][1])
        self.passes = passes
        self.learnRate = learnRate

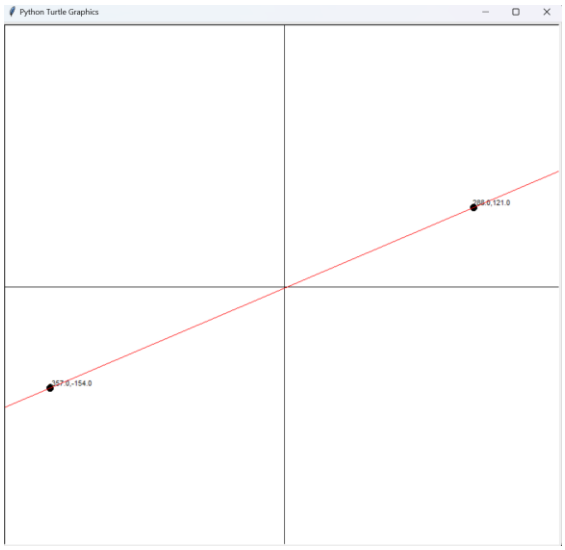
    def train(self):
        self.losses = np.zeros(self.passes)
        self.iteration = np.zeros(self.passes)
        for i in tqdm(range(self.passes)):
            self.grad = np.zeros(len(self.data))
            self.deltaYHat = np.zeros(len(self.data))
            self.dataYHat = np.zeros(len(self.data))
            self.lossVector = np.zeros(len(self.data))
            self.forwardPass()
            self.backPass()
            self.tweak()
            self.losses.put(i, self.loss*1000)
            self.iteration.put(i, i)

    def forwardPass(self):
        for i in range(len(self.data)):
            self.dataYHat.put(i, np.dot(self.dataXs[i],self.coef))
            self.deltaYHat.put(i, self.dataYHat[i]-self.dataY[i])
            self.lossVector.put(i, self.deltaYHat[i]**2)
        self.loss = np.sum(self.lossVector)

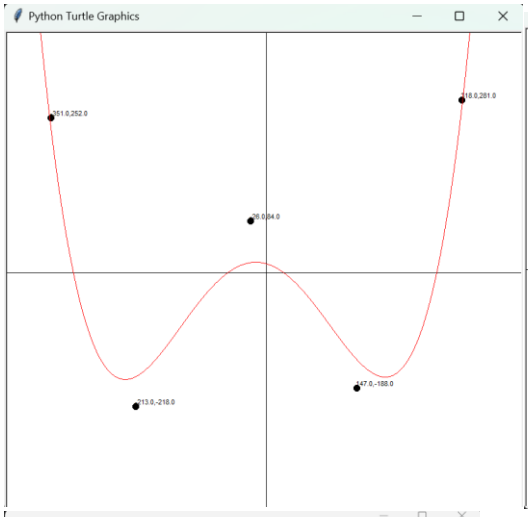
    def backPass(self):
        for i in range(len(self.data)):
            yVector = np.zeros(len(self.data))
            for j in range(len(self.data)):
                yVector.put(j, self.deltaYHat[i])
            self.grad += 2*yVector*self.dataXs[i]

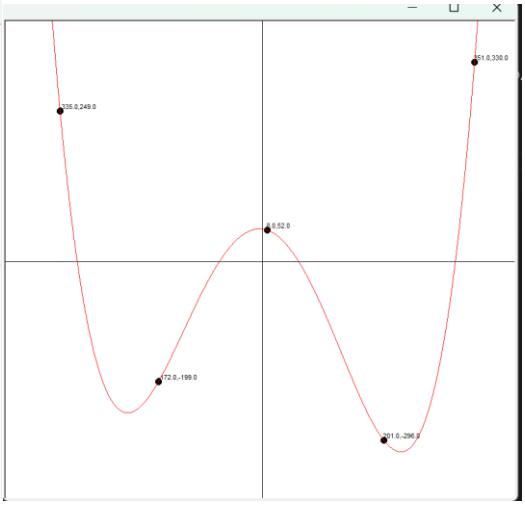
    def tweak(self):
        self.coef = self.coef - self.learnRate * self.grad
```

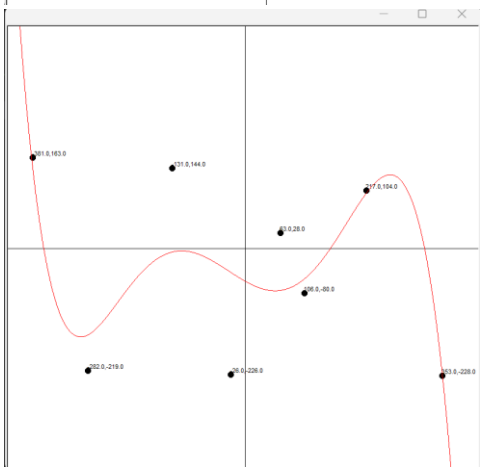
Testing:



This leads to this results which is very good for 2 points







As shown above with a higher number of points results tend to vary a lot more, but the curve still roughly matches to the curves. This issue can be solved by running more iterations allowing it longer to train and find better values which more accurately fit the data.

Project 2 – Genetic algorithm

Goal: To get a population of agents to navigate an environment which contains obstacles. The agent should not need any prior knowledge of the environment, such as position of obstacles or position of end goal. They should also move by taking steps in any direction with no confinement to direction and how big the step is.

Design: This objective can be achieved by creating a population of agents, dots in this scenario, with a series of forces in an array which is the substitute for a brain. Each frame the agent will apply the next force in brain. To train the agents faster the fitness function, a solid definition for how "good" each agent is, is important. The fitness functions I designed is as follows. If the agent does not make it to the end goal its fitness is $\frac{1}{|distance\ to\ end\ goal|^2}$, if they do make it to the end goal there fitness is equal to $1 + \frac{10000}{number\ of\ steps^2}$, this makes sure that if an agent reaches the goal it has a much higher chance of being bread to make the next generation. This ensures that good genes (moves) are passed on, so the population increases over generations. It is also important to not always choose just the agent with the lowest fitness as there can sometimes be other methods which are eventually faster but need more time to develop. Another important factor is to always keep the champion, the best performing agent, of the previous generation this stops the population regressing and unlearning moves.

Code:

Brain

```
import numpy as np
from random import random, uniform
import math

class Brain:
    def __init__(self, size):
        self.size = size
        self.directions = []

        self.step = 0
        self.randomize()

    def randomize(self):
        for i in range(0, self.size):
            self.randAng= uniform(0,2*math.pi)
            self.direction = []
            self.direction.append(math.cos(self.randAng))
            self.direction.append(math.sin(self.randAng))
            self.directions.append(self.direction.copy())


    def clone(self):
        self.clone = Brain(self.size)
        self.clone.directions = self.directions
        return self.clone

    def mutate(self, change):
        self.mutationRate = 0.03
        for i in range(0,self.size):
            self.rand = uniform(0,1)
            if self.rand < self.mutationRate:
                self.randAng= uniform(0,2*math.pi)
                self.direction = []
                self.direction.append(math.cos(self.randAng))
                self.direction.append(math.sin(self.randAng))
                self.directions[i] = self.direction.copy()
```

Agent Script

```
import pyglet as pg
import numpy as np
import brain
import math
from random import uniform

class Dots:
    def __init__(self, width, height, startPos, goal, maxSteps, walls):
        self.width = width
        self.height = height
        self.pos = np.copy(startPos)
        self.vel = np.zeros(2)
        self.acc = np.zeros(2)
        self.maxSteps = maxSteps

        self.sprite = pg.shapes.Circle(self.pos[0], self.pos[1], 2,
color=(255,255,255))
        self.brain = brain.Brain(self.maxSteps)

        self.reachedGoal = False
        self.dead = False
        self.hitWall = False

        self.goal = goal

        self.fitness = 0

        self.walls = walls


    def calcFitness(self):
        if self.reachedGoal == True:
            self.fitness = 1+(10000/self.brain.step**2)
        else:
            self.distanceToGoal = math.sqrt((self.goal[0]-self.pos[0])**2
+ (self.goal[1]-self.pos[1])**2)
            self.fitness = 1/(self.distanceToGoal)


    def move(self):
        if len(self.brain.directions) >self.brain.step:
            self.acc = self.direction =
np.asarray(self.brain.directions[self.brain.step])
            self.brain.step += 1
            self.vel += self.acc
        else:
            self.vel = 0
        self.mag = np.linalg.norm(self.vel)
        if self.mag > 5:
            for i in range(0, len(self.vel)):
                self.vel[i] = self.vel[i]/(self.mag/5)
        self.pos += self.vel
        self.sprite.x = self.pos[0]
        self.sprite.y = self.pos[1]
```

```
    def update(self):
        if (self.pos[0] < 2 or self.pos[1] < 2 or self.pos[0] >
self.width-2 or self.pos[1] > self.height-2):
            self.dead = True
        elif math.sqrt((self.goal[0]-self.pos[0])**2 + (self.goal[1]-
self.pos[1])**2) < 5:
            self.reachedGoal = True
        elif len(self.brain.directions) <= self.brain.step:
            self.dead = True
        for i in self.walls:
            if i.checkCollision(self.pos[0], self.pos[1]):
                self.dead = True
                self.hitWall = True
                break
        if self.dead != True and self.reachedGoal != True:
            self.move()

        self.sprite.draw()

    def breed(self):
        self.babyBrain = brain.Brain(self.maxSteps)
        for i in range(0, len(self.brain.directions)):
            self.babyBrain.directions[i] = self.brain.directions[i]
        return self.babyBrain

    def setBestDot(self):
        self.sprite.color = (255,255,0)
        self.sprite.radius = 6
```

```
Population control script
from Dots import Dots
from random import random, uniform
import numpy as np

class Population:
    def  __init__(self, size, width, height, startPos,goal, max,change,
walls):
        self.size = size
        self.width = width
        self.height = height
        self.goal = goal
        self.startPos = np.copy(startPos)
        self.dotArray = []
        self.gen = 0
        self.maxSteps = max
        self.change = change
        self.minStep=10000
        self.walls = walls
        self.timesNoDotMadeIt = 0
        for i in range(0, self.size):
            self.dotArray.append(Dots(width, height, self.startPos, goal,
self.maxSteps, self.walls))

    def updateDots(self):
        for i in range(0, len(self.dotArray)):
            if self.dotArray[i].brain.step > self.minStep:
                self.dotArray[i].dead = True
            else:
                self.dotArray[i].update()


    def allDead(self):
        for i in range(0, len(self.dotArray)):
            if self.dotArray[i].dead == False and
self.dotArray[i].reachedGoal == False:
                return False
        return True

    def clacFitness(self):
        for i in range(0, len(self.dotArray)):
            self.dotArray[i].calcFitness()
            if self.gen > 50 and self.dotArray[i].hitWall:
                self.dotArray[i].fitness = self.dotArray[i].fitness/5


    def naturalSelection(self):
        self.calcFitnessSum()
        self.newBrainArray = []

        self.gen += 1

        for i in range(0, len(self.dotArray)-1):

            #select parent
            self.parent = self.selectParent()
```

```
            #make baby
            self.newBrainArray.append(self.parent.breed())

        self.getBestDot()
        self.newBrainArray.append(self.bestDot.breed())

        if self.numberOfDotsMakingit() > 0:
            self.timesNoDotMadeIt = 0
        else:
            if self.gen > 50:
                if self.numberOfDotsMakingit() == 0:
                    self.timesNoDotMadeIt += 1
                if self.timesNoDotMadeIt > 5:
                    self.maxSteps += 5
        self.dotArray = []
        for i in range(0, self.size):
            self.dotArray.append(Dots(self.width, self.height,
self.startPos, self.goal, self.maxSteps, self.walls))
            self.dotArray[i].brain = self.newBrainArray[i]
        self.dotArray[self.size-1].setBestDot()

    def calcFitnessSum(self):
        self.totalFitness = 0
        for i in range(0, len(self.dotArray)):
            self.totalFitness += self.dotArray[i].fitness * 100


    def selectParent(self):
        randomChoice = uniform(0, self.totalFitness)

        runningSum = 0
        for i in range(0, len(self.dotArray)):
            runningSum += self.dotArray[i].fitness*100
            if runningSum > randomChoice:
                return self.dotArray[i]


    def mutateDemBabies(self):
        for i in range(0, len(self.dotArray)-1):
            self.dotArray[i].brain.mutate(self.change)


    def getBestDot(self):
        max = 0
        maxIndex = 0
        for i in range(0, len(self.dotArray)):
            currentDot = self.dotArray[i]
            if self.dotArray[i].fitness > max:
                max = self.dotArray[i].fitness
                maxIndex = i

        self.bestDot = self.dotArray[maxIndex]

        if self.bestDot.reachedGoal == True:
            self.minStep = self.bestDot.brain.step
```

```
    def numberOfDotsMakingit(self):
        numberMadeIt = 0
        for i in range(0, len(self.dotArray)):
            if self.dotArray[i].reachedGoal:
                numberMadeIt += 1
        return numberMadeIt
```

Testing:

In the first test the dots never actually made it. This is due to the dots having to move too far around the second wall and thus further away from the goal thus giving them a lower fitness. This is an inherit issue with the algorithm, if the path a single dot has to take is too long and too estrange from the path the probability of it being taken is so low that it will never happen. This can be solved by adding points where if the dots make it there they gain an extra reward. This will allow the user to assist the ai to find a path or the path that the user wants the ai to take.

In the second training the ai makes it with out too much difficulty. It did need extra steps originally to reach the goal, which is more beneficial as it knows where to go and then optimises the route which is faster for the ai to do.

Project 3 – Reinforcement Learning using Q learning

Goal: To train an agent to navigate an environment based on a punishment reward system.

Design: The environment will be a grid in which the agent can move up down left or right in. This environment will contain obstacles which reset the agent on collision, a start points which will always be top left and a goal which is always bottom right. A table of all possible states and all possible actions from each state must be stored with a value determining how good that action from that state is. This table will have all values initially set to 0. The reward for each action will be as follows, moving to a blank square yields a reward of -10, to encourage the agent to get to the goal faster. Hitting an obstacle will yield a reward of -1000 to heavily discourage hitting an obstacle. Then reaching the goal yields a reward of 10000. The agent will make a decision on the action by using the epsilon-greedy policy, which is at the start all the decisions made are random but as time goes on the agent will pick more of the highest valued action until it always picks the best action which should lead it to the goal. The way it calculates the value for the action in that state uses Q Learning formula. This is done by taking the previous value for that action and adding, the reward of taking that action + the maximum action from the previous state multiplied by the discount factor and subtracted by the action just taken all multiplied by the learning rate. After enough actions the agent will update the table with values that if followed will lead the agent to the goal.

Code:

Main:

```
import pyglet as pg
from pyglet.window import key, mouse
import cell, agent, random
from time import sleep
import Training

# Create a window
window = pg.window.Window(width = 1040, height = 1040)

maze = ["WWWWWWWWWWWWWWWWWWWW",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                  W",
        "W                 GW",
        "WWWWWWWWWWWWWWWWWWWW",]

tileArray=[]
for i in range(0,len(maze)):
        for j in range(0,len(maze[i])):
            tileArray.append(cell.cell(maze[i][j],j,i))

start = False
@window.event
def on_mouse_press(x, y, button, modifiers):
    xCoord = int(x//52)
    yCoord = int((1040-y)//52)
    if maze[yCoord][xCoord] != "W" and maze[yCoord][xCoord] != "G" and
start == False:
        if button == mouse.LEFT:
            s = list(maze[yCoord])
            s[xCoord] = "O"
            maze[yCoord] = "".join(s)

            tileArray[yCoord*20 + xCoord] = cell.cell("O", xCoord,
yCoord)
        elif button == mouse.RIGHT:
            s = list(maze[yCoord])
            s[xCoord] = " "
            maze[yCoord] = "".join(s)
```

```
            tileArray[yCoord*20 + xCoord] = cell.cell(" ", xCoord,
yCoord)
        elif button == mouse.MIDDLE:
            s = list(maze[yCoord])
            s[xCoord] = "S"
            maze[yCoord] = "".join(s)

            tileArray[yCoord*20 + xCoord] = cell.cell("S", xCoord,
yCoord)

#lermin = agent.Agent(grid=maze)
lermins = []
for i in range(0,250):
    lermins.append(agent.Agent(grid=maze))

QLearn = Training.train(10000, tileArray)


@window.event
def on_key_press(symbol, modifiers):
    global start
    if symbol == key.Q:
        QLearn.printQTable()
    elif symbol == key.ENTER:
        start = True


def draw(dt):
    window.clear()
    for i in tileArray:
        i.update()
    if start:
        for i in range(0,len(lermins)):
            action = 0
            action = QLearn.chooseAction(lermins[i].getPos())
            if action == 1:
                lermins[i].moveRight()
            elif action == 2:
                lermins[i].moveLeft()
            elif action == 3:
                lermins[i].moveUp()
            elif action == 4:
                lermins[i].moveDown()
            elif action > 4:
                print("shit")
            if action != -1:
                QLearn.updateQTable(lermins[i].getPos())
            else:
                lermins[i].setPos(1,1)
                QLearn.nextEpisode()
            if tileArray[lermins[i].getPos()].cellType == "G":
                lermins[i].setPos(1,1)
            elif tileArray[lermins[i].getPos()].cellType == "O":
                lermins[i].setPos(1,1)
            lermins[i].update()

pg.clock.schedule_interval(draw, 1/1200)
```

```
pg.app.run()
```

Training:

```
import numpy as np
import random

class train:
    def __init__(self, maxEpochs, tileArray):
        self.states = 324
        self.actions = 4
        self.goalState = 324
        self.QTable = np.zeros((self.states,self.actions))

        self.discount = 0.95
        self.learnRate = 0.85

        self.maxExplorationProb = 1
        self.minExplorationProb = 0.00
        self.decayFactor = 0.01

        self.maxEpochs = maxEpochs
        self.epoch = 0
        self.episode = 0
        self.steps = 0

        self.tileArray = tileArray


    def printQTable(self):
        print(self.QTable)

    def nextEpisode(self):
        self.episode +=1

        #print(self.episode,"-",self.steps)
        self.steps=0

    def chooseAction(self, playerPos):
        self.action = 0
        self.prevPlayerPos = playerPos
        self.gridPos = []
        self.gridPos.append(playerPos%20)
        self.gridPos.append(playerPos//20)
        self.currentState = self.gridPos[0]-1 + (self.gridPos[1]-1) * 18
        explorationProb = self.minExplorationProb +
(self.maxExplorationProb - self.minExplorationProb)* np.exp(-
self.decayFactor*self.episode)
        self.steps += 1
        if random.uniform(0,1) < explorationProb:
            self.action = random.randint(1,self.actions)
        else:
            self.action = np.argmax(self.QTable[self.currentState])+1
        if self.steps > 256:
            self.action = -1
        return self.action

    def updateQTable(self, playerPos):
        self.prevPos = self.currentState
        self.gridPos = []
```

```
        self.gridPos.append(playerPos%20)
        self.gridPos.append(playerPos//20)
        self.currentState = self.gridPos[0]-1 + (self.gridPos[1]-1) * 18
        if self.tileArray[playerPos].cellType == "G":
            self.reward = 100000
            self.nextEpisode()
        elif self.tileArray[playerPos].cellType == " ":
            self.reward = -10
        elif self.tileArray[playerPos].cellType == "S":
            self.reward = 4
        elif self.tileArray[playerPos].cellType == "O":
            self.reward = -1000
            self.nextEpisode()
        else:
            self.reward = 0
            print("whoops")
        self.QTable[self.prevPos][self.action-1] =
self.QTable[self.prevPos][self.action-1] + \
            self.learnRate * (self.reward + self.discount *
np.max(self.QTable[self.currentState]) -
self.QTable[self.prevPos][self.action-1])
        self.tileArray[self.prevPlayerPos].weight =
np.average(self.QTable[self.prevPos])
```

Testing:

In the first training session, there are 10 ais all training a hive mind to learn the environment faster and thus they find a path faster. However, this quite slow and is sped up but 5 times in the footage.

In the next training I added 190 ai so there are now 200 and they can learn the area much faster and thus find an optimal route faster. However, in this example it doesn't find and optimal route as the small chance that it never travels along that route. This could be done by adjusting the hyperparameter so that the chance it takes a random route is increased and thus it will explore routes like that and thus will always find the optimal path

In the next example it is the same setup, but it does find the optimal path

In the final example I added obstacles in which the ai also must avoid. If the ai hits a spike it is reset. With 200 ai all working together they can really easily find the path to the goal. It learns that hitting the spike is bad and thus tends to do that less and follows the path that will always lead it to the goal